



Universidad Veracruzana

Facultad de Estadística e Informática
Región Xalapa



Licenciatura en
Ingeniería de
Software

GUÍA

práctica de **Jenkins** para
el despliegue automatizado
de software

De la cruz García Ángel de Jesús y
Suárez Santiago Tristán Eduardo

• **2025** •

Universidad Veracruzana

Facultad de Estadística e Informática
Licenciatura en Ingeniería de Software
Región Xalapa
2025

Guía práctica de Jenkins para el despliegue automatizado de software

Autores

De la cruz García Ángel de Jesús y
Suárez Santiago Tristán Eduardo

Dirección

Domínguez Isidro Saul y
Pérez Arriaga Juan Carlos

Contenido

1	Introducción	1
1.1	Objetivo	2
1.2	Alcance y público objetivo	2
1.3	Contexto de uso	3
2	Jenkins	5
2.1	Herramientas de Automatización de CI/CD	6
2.2	¿Qué es Jenkins?	7
2.3	Jenkins Como Herramienta de Automatización de CI/CD	8
2.4	Cuestionario sobre Jenkins	10
3	Jobs	14
3.1	¿Qué es un Job en Jenkins?	15
3.2	Tipos de <i>jobs</i> comunes	16
3.3	Cuestionario sobre <i>jobs</i> en Jenkins	17
4	Introducción a los <i>pipelines</i>	19
4.1	Concepto general de <i>pipeline</i>	20
4.2	<i>Pipelines</i> de despliegue	21
4.3	Etapas comunes de los <i>pipelines</i> de despliegue	24
4.4	<i>Pipelines</i> en Jenkins	29
4.4.1	Concepto de <i>pipeline</i> en Jenkins	29
4.4.2	Estilos para definir <i>pipelines</i> en Jenkins	33
4.4.3	Estructura general de los <i>pipelines</i> declarativos	41
4.4.4	Directivas esenciales de los <i>pipelines</i> declarativos	47
5	Jenkins en entornos Linux	57
5.1	Comandos en Linux esenciales para el uso de Jenkins	58
5.2	Requisitos Técnicos de un Servidor de Jenkins	62
5.3	Instalación Paso a Paso de un Servidor Jenkins en Ubuntu Server	66
5.3.1	Preparación del entorno	67

5.3.2	Configuración de la máquina virtual	68
5.3.3	Instalación de JAVA (Eclipse Temurin) en Ubuntu Server.	70
5.3.4	Instalación de Jenkins en Ubuntu Server	73
5.3.5	Personalización inicial del entorno de Jenkins (opcional)	75
5.3.6	Ingreso a Jenkins por primera vez.....	79
6	Conociendo la Interfaz Gráfica de Jenkins	86
6.1	Gestión de Jenkins desde la Interfaz	87
6.1.1	<i>System</i>	90
6.1.2	<i>Tools</i>	91
6.1.3	<i>Plugins</i>	92
6.1.4	<i>Nodes</i>	93
6.1.5	<i>Credentials</i>	94
6.1.6	<i>Users</i>	95
6.2	Creación de Jobs desde la interfaz.....	98
7	Uso de credenciales en Jenkins	105
7.1	¿Qué es una credencial en Jenkins?	106
7.2	¿Cómo se usan las credenciales en Jenkins?	108
8	Plugins.....	118
8.1	¿Qué es un <i>plugin</i> ?.....	119
8.2	¿Cómo integrar <i>plugins</i> en Jenkins?	122
8.2.1	¿Cómo acceder a los <i>plugins</i> ?.....	122
8.2.2	¿Cómo instalar un <i>plugin</i> ?	124
8.2.3	¿Cómo actualizar un <i>plugin</i> ?	127
8.2.4	Gestión de los <i>plugins</i>	128
8.3	Jenkins y GitHub	131
9	Creación de pipelines	137
9.1	¿Cómo ejecutar <i>pipelines</i> ?	138
9.2	¿Cómo recuperar un <i>pipeline</i> desde GitHub?.....	144
9.2.1	Creando un repositorio con Jenkinsfile.....	145

9.2.2	<i>Plugins</i> necesarios	145
9.2.3	Configurar Credenciales de acceso a GitHub	146
9.2.4	Creación de <i>job</i> de tipo <i>pipeline</i>	148
9.3	Tu primer <i>pipeline</i> paso a paso.....	151
9.3.1	Preparación del entorno de trabajo	152
9.3.2	Definición del <i>pipeline</i>	152
9.3.3	Creación del Jenkinsfile	154
9.3.4	Creación del <i>job</i>	157
9.3.5	Ejecución manual del <i>pipeline</i>	161
9.3.6	Automatización de la ejecución del <i>pipeline</i>	165
10	Construcción y empaquetado de software	171
10.1	Construcción VS Empaquetado de software.....	172
10.2	Actividades de construcción y empaquetado de software.....	175
10.3	Maven como herramienta de construcción y empaquetado	179
10.4	Comandos comunes en Maven.....	182
10.5	Integración de Maven con Jenkins	185
11	Espacio de trabajo de los pipelines	189
11.1	¿Qué es el espacio de trabajo en el contexto de los <i>pipeline</i> en Jenkins?	190
11.2	Comportamiento del espacio de trabajo entre ejecuciones de un <i>pipeline</i> 192	
11.3	¿Cómo acceder al espacio de trabajo de un pipeline?	195
11.4	GitHub y el espacio de trabajo.....	200
12	Instalación de herramientas en Jenkins	206
12.1	¿Cómo instalar herramientas en un servidor de Jenkins?	207
12.2	Acceso a herramientas desde <i>pipelines</i> declarativos	212
12.2.1	Herramientas instaladas desde la interfaz gráfica.....	213
12.2.2	Herramientas instaladas con el gestor de paquetes	214
12.2.3	2.3 Herramientas instaladas con binarios	214
13	Integración de Maven en <i>pipelines</i> declarativos	218
13.1	Integración de Maven en <i>pipelines</i> declarativos paso a paso	219

13.1.1	Preparativos para la práctica	219
13.1.2	Herramientas para ejecutar el proyecto	220
13.1.3	Configuración de Git en Jenkins	221
13.1.4	Trabajando con el proyecto en GitHub	222
13.1.5	<i>Snippet Generator</i>	224
13.1.6	Configurando Maven en Jenkins	229
13.1.7	Utilizando Maven en el <i>pipeline</i>	230
14	Pruebas y pipelines	236
14.1	Pruebas de software y <i>pipelines</i> de Jenkins	237
14.2	Herramientas de prueba de software	244
14.3	Integración de JUnit con Jenkins	248
14.3.1	Uso de JUnit como herramienta de prueba	248
14.3.2	<i>Plugins</i> de Jenkins para integrar JUnit	253
14.3.3	Comandos de Junit para recordar	256
14.3.4	Acceso a Junit desde <i>pipelines</i> declarativos en Jenkins	259
14.4	Integración de Postman y Newman con Jenkins	263
14.4.1	Uso de Postman y Newman como herramientas de prueba	263
14.4.2	Newman en los <i>pipelines</i> declarativos de Jenkins	268
14.5	Integración de herramientas de prueba en <i>pipelines</i> declarativos paso a paso 274	
14.5.1	Prerrequisitos	275
14.5.2	Herramientas para ejecutar el proyecto	276
14.5.3	Agregando las pruebas unitarias al proyecto	276
14.5.4	Agregando las pruebas de integración al proyecto	281
14.5.5	Configuración de <i>plugins</i> necesarios en Jenkins	282
14.5.6	Modificación de la declaración del <i>pipeline</i>	284
14.5.7	<i>Pipeline</i> desde Jenkins	286
15	Docker y <i>pipelines</i>	292
15.1	Uso de Docker en <i>pipelines</i> de despliegue	293
15.2	Componentes clave de un Dockerfile	296

15.3	<i>Plugins</i> en Jenkins para la integración con Docker.....	300
15.4	Comandos de Docker para recordar	302
15.4.1	Comandos para gestionar imágenes de Docker.....	303
15.4.2	Comandos para gestionar contenedores de Docker	303
15.4.3	Comandos para utilizar DockerHub.....	304
15.5	¿Cómo configurar Docker en Jenkins?	307
16	Docker Compose y <i>pipelines</i>	314
16.1	Uso de Docker Compose en <i>pipelines</i> de despliegue.....	315
16.2	Componentes clave de un archivo YAML	317
16.3	Comandos de Docker Compose para recordar	322
16.4	Uso de Docker Compose en <i>pipelines</i> de despliegue.....	326
17	Integración de Docker y Docker Compose en <i>pipelines</i> declarativos paso a paso	330
17.1	Integración de Docker y Docker Compose en <i>pipelines</i> declarativos paso a paso	331
17.1.1	Prerrequisitos de la práctica	332
17.1.2	Preparación del entorno de trabajo	334
17.1.3	Creación del Dockerfile del proyecto.....	335
17.1.4	Creación y publicación de la imagen del proyecto	336
17.1.5	Usando Docker Compose para levantar un contendor.....	338
17.1.6	Conexión SSH de Jenkins hacia la máquina anfitriona.....	342
18	Respuestas de cuestionarios	349
18.1	Respuestas para el capítulo 2	350
18.2	Respuestas para el capítulo 3	350
18.3	Respuestas para el capítulo 4	350
18.4	Respuestas para el capítulo 5	351
18.5	Respuestas para el capítulo 6	351
18.6	Respuestas para el capítulo 7	351
18.7	Respuestas para el capítulo 8	351
18.8	Respuestas para el capítulo 9	352

18.9	Respuestas para el capítulo 10	352
18.10	Respuestas para el capítulo 11	353
18.11	Respuestas para el capítulo 12	353
18.12	Respuestas para el capítulo 13	353
18.13	Respuestas para el capítulo 14	353
18.14	Respuestas para el capítulo 15	354
18.15	Respuestas para el capítulo 16	355
18.16	Respuestas para el capítulo 17	356
19	Bibliografía y recursos	357
19.1	Fuentes actualizadas	358



Capítulo 1

1 Introducción

Contenido del capítulo

- 1.1. Objetivo
- 1.2. Alcance y público objetivo
- 1.3. Contexto de uso

1.1 Objetivo

La presente guía tiene como objetivo explicar la configuración y uso de Jenkins en el despliegue automatizado de software, con el fin de implementar *pipelines* de integración y despliegue continuo (CI/CD), con prácticas reales y actuales.

1.2 Alcance y público objetivo

El contenido que se presenta a continuación está dirigido a cualquier persona interesada en el área de la ingeniería de software, específicamente en el ámbito de la automatización del despliegue de software con Jenkins.

Para que el lector tenga una mejor comprensión de los temas desarrollados en la guía, se espera que cuente con conocimientos y bases sólidas en:

1. El ciclo de vida de desarrollo de software,
2. Uso de herramienta de gestión de repositorios (GitHub),
3. Manejo de contenedores (Docker),
4. Experiencia en la creación de máquinas virtuales, y
5. Uso de línea de comandos para la gestión y manejo de servidores basados en Linux.

1.3 Contexto de uso

El orden recomendado para la lectura de esta guía es la secuencia en la que fue escrita, pues conforme se avanza en los capítulos, se asume que el lector ya se encuentra familiarizado con los conceptos y temas abordados en los capítulos previos.

Al final de los capítulos explicativos, encontrará un capítulo dedicado a dar respuesta para las preguntas de los cuestionarios ([Respuestas de cuestionarios](#)) dentro de la guía, así como un capítulo con las fuentes utilizadas para realizar el contenido de la guía y recursos que pueden resultar de gran interés para el lector ([Bibliografía y recursos](#)).

Si el lector ya tiene conocimientos previos sobre:

1. Qué es Jenkins: puede omitir la lectura del capítulo 2 - [Jenkins](#).
2. El concepto de *Job* en Jenkins, así como los tipos: puede omitir la lectura del capítulo 3 - [Jobs](#).
3. El concepto de *pipeline*, *pipeline* de despliegue y *pipelines* en Jenkins: puede omitir la lectura del capítulo 4 - [Introducción a los pipelines](#). Si no tiene conocimiento de *pipelines* en Jenkins, pero sí de los otros dos conceptos, omita la lectura del capítulo 4 parcialmente, profundizando solamente en la sección 4.7 - [Pipelines en Jenkins](#).
4. Construcción y empaquetado de software con Maven: omita la lectura del capítulo 10, exceptuando la sección 10.5 - [Integración de Maven con Jenkins](#).
5. Conceptos generales sobre pruebas de software y herramientas de prueba: omita la lectura de las secciones 14.1 - 14.4 del capítulo 14 - [Pruebas y pipelines](#).

En cada uno de los capítulos, se utilizan iconos para facilitar la identificación de los contenidos. A continuación se muestra cada uno de ellos, con su respectivo significado:

**Lectura**

Espacios que tienen objetivos de aprendizaje específicos. Se acompañan de una breve descripción y de contenido teórico.

**Cuestionario**

Conjunto de preguntas que sirven para autoevaluar los aprendizajes teóricos adquiridos durante las lecturas.

**Práctica**

Secciones estructuradas en las que se presentan secuencias de pasos para seguir, con el fin de reforzar aprendizajes en temas específicos.

**Rúbrica**

Acompañan a las prácticas y sirven para autoevaluar el desempeño durante las prácticas.

**Nota**

Secciones informativas breves.



Capítulo 2

2 Jenkins

Contenido del capítulo

- 2.1. Herramientas de Automatización de CI/CD
- 2.2. ¿Qué es Jenkins?
- 2.3. Jenkins como Herramienta de Automatización de CI/CD
- 2.4. Cuestionario sobre Jenkins

2.1 Herramientas de Automatización de CI/CD



Lectura 1. Herramientas de Automatización de CI/CD

Nivel básico

Objetivo: el estudiante memoriza el concepto y características principales de las herramientas de automatización en el contexto de la Integración y Despliegue Continuo.

Descripción

Esta lectura introduce el concepto de herramienta de automatización de CI/CD, su propósito en el ciclo de vida de desarrollo de software y sus características generales.

Actualmente, las herramientas de automatización de CI/CD (Integración y Despliegue Continuo) son piezas indispensables en cualquier organización que pretende integrar las prácticas de CI/CD en sus procesos de desarrollo de software. Sin embargo, en los primeros años en que surgió CI/CD, esta se implementaba de manera manual, mediante una intervención constante por parte de los desarrolladores para realizar la integración, prueba y despliegue del software, requiriendo un esfuerzo constante, siendo propenso a errores y difícil de administrar con el tiempo (Sistla, 2021).

Esta situación dio paso a la creación de los primeros sistemas de compilación automatizados como Jenkins o Travis CI, los cuales sentaron las bases para los sistemas modernos de automatización de CI/CD. En términos general, una herramienta de automatización CI/CD es un software que permite a los equipos de desarrollo automatizar procesos de construcción, prueba y despliegue de software de manera coordinada y continua, con el objetivo de reducir errores manuales y acortar los tiempo que toma liberar una aplicación en producción (Sistla, 2021).

En la actualidad existen diversas herramientas de automatización de CI/CD: Jenkins, Circle CI, GitHub Actions, AWS CodePipeline, Azure DevOps, Travis CI, TeamCity, CodeFresh, entre muchas otra. Si bien cada una presenta particularidades que las diferencian entre sí, Beach et al. (2023) coinciden en que estas comparten varias característica comunes:

- Decrementan la intervención humana mediante la utilización de *pipelines* automatizados.
- Permiten la ejecución automatizada de actividades relacionadas a la integración, prueba y despliegue de software (aunque no se limitan únicamente a estas etapas).
- Están estrechamente relacionadas con los sistemas controles de versiones, ya que estos funcionan como el punto de partida para empezar la ejecución de los procesos.
- Permiten la integración de herramientas externas para ejecutar una o más actividades dentro del flujo de trabajo.
- Permiten la definición y administración de *pipelines* de Integración y Despliegue Continuo.

2.2 ¿Qué es Jenkins?



Lectura 2. ¿Qué es Jenkins?

Nivel básico

Objetivo: el estudiante memoriza el concepto de Jenkins.

Descripción

Esta lectura introduce la definición de la herramienta Jenkins y su origen en el campo de la ingeniería de software.

La definición más común de Jenkins según la CD Foundation (2025e) es:

Jenkins es un servidor de automatización autónomo y de código abierto, que se puede utilizar para automatizar todo tipo de tareas relacionadas con la construcción, prueba y entrega o implementación de software.

En otras palabras, Jenkins es una herramienta para la automatización de procesos, independiente de otros servidores o plataformas para funcionar, la cuál es gratuita y de código abierto; es desarrollada y mantenida por la comunidad, cuyo propósito principal es ayudar a automatizar parte del proceso de desarrollo de software. Esta herramienta está enfocada particularmente en aquellas actividades relacionadas a la

Integración y el Despliegue Continuo, como la compilación, empaquetado, ejecución de pruebas, despliegue de software, entre otras.

Jenkins encuentra sus orígenes a finales del 2004 en la empresa *Sun Microsystems*, donde el desarrollador Kohsuke Kawaguchi, cansado de lo tedioso que resultaba el proceso de actualización del software, creó un servidor de automatización con el fin de simplificar las tareas de compilación y ejecución de pruebas. A este proyecto se le denominó Hudson y con el tiempo se convirtió en lo que hoy se conocemos como Jenkins (Ferguson & Hudson, 2011).

En 2010, tras la compra de *Sun Microsystems* por parte de Oracle, surgieron discrepancias entre la comunidad y la empresa, lo que ocasionó que Jenkins surgiera como una bifurcación (*fork*) del proyecto Hudson, y se convirtiera en un proyecto independiente, de código abierto y respaldado ampliamente por la comunidad (Ferguson & Hudson, 2011).

2.3 Jenkins Como Herramienta de Automatización de CI/CD



Lectura 3. Jenkins como herramienta de automatización de CI/CD.

Nivel Intermedio

Objetivo: el estudiante comprende el rol de la herramienta Jenkins como herramienta de automatización dentro del ciclo de vida de desarrollo de software.

Descripción

Esta lectura ofrece una visión general de Jenkins como herramienta de automatización, se describe su propósito general, sus principales características y su arquitectura. Además, se resalta su utilidad en distintas etapas del ciclo de vida del desarrollo de software, destacando cómo facilita la integración y el despliegue continuo de aplicaciones.

La principal razón por la que se creó Jenkins y por la cual hasta la fecha es la herramienta de CI/CD más utilizada, es porque pretende automatizar en lo más

posible gran parte del proceso de desarrollo de software al compilar, probar y desplegar software continuamente.

Es importante entender que Jenkins no ejecuta directamente estas tareas, sino que actúa como un orquestador que coordina y delega las actividades a otras herramientas especializadas. Por ejemplo, en un flujo de trabajo típico para una aplicación en Java, Jenkins podría:

- Llamar a herramientas como GitHub para recuperar el código fuente.
- Usar a Maven para compilar y empaquetar el código.
- Ejecutar las pruebas unitarias con Junit.
- Utilizar a Docker para generar una imagen.
- Y valerse de una herramienta como Ansible para desplegar la aplicación en un servidor.

En todo este proceso, Jenkins no realiza la compilación, prueba o despliegue por sí mismo, sino que únicamente orquesta y automatiza cada tarea para que ocurra de manera continua y con la menor intervención manual posible.

La arquitectura sobre la que Jenkins trabaja es un modelo conocido como Jefe-Trabajador, en la que existe un servidor principal (denominado Jefe) y múltiples servidores secundarios (denominados trabajadores). El servidor jefe se encarga de administrar las configuraciones, asignar las tareas a los servidores secundarios y de coordinar la ejecución del *pipeline* de CI/CD, mientras que los servidores secundarios se encargan de llevar a cabo las tareas de desarrollo, prueba y despliegue de software encomendadas por el servidor principal (CD Foundation, 2025b).



Nota - También les puedes llamar nodos.

A los servidores principales y secundarios también se les conoce como nodos. Al nodo maestro, o principal, se le denomina controlador o *controller*.

Como toda herramienta de automatización, Jenkins comparte varias características con otras soluciones de CI/CD; sin embargo, también tiene particularidades que lo distinguen. Dileepkumar & Mathew (2023) destacan algunas de sus características principales:

- Orientado a *plugins*: Jenkins destaca por tener un amplio catálogo de *plugins* y un ecosistema muy flexible que permite integrar casi cualquier herramienta, lo que permite adaptarlo a las necesidades específicas de cada equipo de desarrollo.
- *Pipeline* como código: Jenkins incorpora el enfoque de *pipeline-as-code*, que permite definir y gestionar los *pipelines* de CI/CD mediante código escrito en lugar de depender de una configuración gráfica.
- Ejecuciones distribuidas: Jenkins soporta la ejecución de *pipelines* de forma distribuida, lo que permite ejecutar múltiples tareas en paralelo en distintos servidores.
- Código Abierto: Jenkins es una herramienta gratuita y de código abierto, la cual recibe un soporte constante y actualizaciones por parte de la comunidad.
- Servidor local: Jenkins es un servidor que se instala y administra de manera local en tu máquina o infraestructura; a diferencia de otras herramientas de CI/CD que operan en la nube, Jenkins te da control de total sobre el entorno (Aunque existen versiones en la nube, como Jenkins X).

2.4 Cuestionario sobre Jenkins



Cuestionario 1. Introducción a Jenkins

Contesta las siguientes preguntas para evaluar los conocimientos adquiridos en las lecturas del capítulo, con el objetivo de reforzar los conocimientos básicos sobre Jenkins.

1. ¿Qué tipo de herramientas son esenciales para la adopción e implementación directa de prácticas de Integración y Despliegue Continuo?

A) Herramientas de pruebas. B) Herramientas de automatización de CI/CD. C) Herramientas de análisis de código. D) Herramientas de virtualización.
2. ¿Cuáles son algunos inconvenientes de implementar CI/CD de forma manual?

- | | | | |
|--|--|---|---|
| A) Menor cooperación y comunicación dentro del equipo. | B) Desarrollo de productos menos competentes y de menor calidad. | C) Propenso a errores y difícil de administrar a largo plazo. | D) Dificulta la escalabilidad y colaboración entre equipos. |
|--|--|---|---|

3. ¿Cuál de las siguientes definiciones describe mejor el concepto de herramienta de automatización de CI/CD?

- | | | | |
|--|--|---|--|
| A) Herramienta que permite automatizar las pruebas unitarias y de integración. | B) Software que permite automatizar procesos y tareas del ciclo de desarrollo de software. | C) Plataforma que gestiona e integra compilación, prueba y despliegue de manera automática. | D) Plataforma para la virtualización automatizada de entornos de desarrollo. |
|--|--|---|--|

4. De las siguientes opciones, ¿Cuáles **no** son herramientas de automatización de CI/CD? (Seleccione tres).

- | | | | |
|---------------------------------------|-----------------------------------|------------------------------------|------------------------------------|
| ☐ Jenkins | ☐ GitHub | ☐ Circle CI | ☐ Junit |
| ☐ Azure DevOps | ☐ TeamCity | ☐ Terraform | ☐ Travis CI |

5. No es una característica principal de una herramienta de automatización de CI/CD:

- | | | | |
|---------------------------------------|--|-------------------------------------|---|
| A) Disminuyen la intervención manual. | B) Se integran con otras herramientas. | C) Mejoran la calidad del software. | D) Se valen de pipelines y flujos de trabajo para automatizar procesos. |
|---------------------------------------|--|-------------------------------------|---|

6. ¿Cuál de las siguientes opciones describe mejor la definición de Jenkins?

- | | | | |
|--|--|--|---|
| A) Un sistema operativo especializado en | B) Una herramienta exclusiva para pruebas de | C) Un orquestador de <i>pipelines</i> de | D) Un servidor de automatización para la creación y |
|--|--|--|---|

servidores de integración y despliegue continuo.	software automatizadas.	CI/CD automatizado.	administración de <i>pipelines</i> de CI/CD
--	-------------------------	---------------------	---

7. ¿A partir de que necesidad surge Jenkins?

- | | | | |
|---|--|---|---|
| A) Simplificar la ejecución de tareas de compilación y pruebas. | B) Sustituir completamente a los sistemas de control de versiones. | C) Automatizar todo el ciclo de vida de desarrollo de software. | D) Orquestar y organizar diversos <i>pipelines</i> y flujos de trabajo. |
|---|--|---|---|

8. Verdadero o falso: Jenkins es la herramienta encargada de ejecutar y llevar a cabo directamente las actividades automatizadas en un *pipeline* de CI/CD.

- | | |
|--------------|----------|
| A) Verdadero | B) Falso |
|--------------|----------|

9. ¿Cuál de las siguientes opciones describe mejor la arquitectura general de Jenkins?

- | | | | |
|--|--|--|---|
| A) Funciona como una base de datos distribuida para gestionar pipelines. | B) Modelo en el que un servidor central coordina la ejecución y los agentes realizan las tareas. | C) Utiliza un sistema de archivos compartido como núcleo para ejecutar todos los procesos. | D) Es un modelo <i>peer-to-peer</i> donde cada nodo tiene las mismas funciones y responsabilidades. |
|--|--|--|---|

10. Dentro de la arquitectura de Jenkins, es elemento encargado de administrar configuraciones y asignar las tareas.

- | | | | |
|-----------|-----------------------|---------------------|------------------|
| A) Broker | B) nodo de Middleware | C) Nodo Coordinador | D) Nodo maestro. |
|-----------|-----------------------|---------------------|------------------|

11. De las siguientes opciones, ¿Cuál **no** es una característica de Jenkins?

- | | | | |
|---|--|---|--|
| A) Permite integrar
variedad de
herramientas
mediante <i>plugins</i> . | B) Puede instalarse
en diferentes
entornos,
incluyendo local y
en la nube. | C) Consume pocos
recursos de
hardware en
cualquier
escenario. | D) Es un software
de código abierto y
sin costo. |
|---|--|---|--|



Capítulo 3

3 Jobs

Contenido del capítulo

- 3.1. ¿Qué es un Job en Jenkins?
- 3.2. Tipos de *jobs* comunes
- 3.3. Cuestionario sobre *jobs* en Jenkins

3.1 ¿Qué es un Job en Jenkins?



Lectura 4. ¿Qué es un Job en Jenkins?

Nivel básico

Objetivo: el estudiante memoriza el concepto de *Job* en Jenkins.

Descripción

Esta lectura introduce el concepto de Job en Jenkins, su propósito en el contexto de Jenkins y sus características generales con el objetivo de introducir este concepto.

Un **Job** en Jenkins, también llamado proyecto, es una unidad de trabajo definida y ejecutada por usuarios dentro de Jenkins para realizar tareas automatizadas. Jenkins utiliza estos proyectos (*jobs*) para llevar a cabo su función principal a través de diversos tipos de *jobs*: *Pipeline*, *Multibranch Pipeline*, *Organization folder*, *Freestyle*, *Multi-configuration (matrix)*, *Maven* y *External Job* (CD Foundation, 2025f).

La función principal de un Job en Jenkins es actuar como la unidad de trabajo que Jenkins ejecuta para realizar tareas automatizadas; por ejemplo, construir, probar o desplegar software. Jenkins organiza y estructura estas acciones dentro de los Jobs, que pueden representarse mediante diversos tipos, como se mencionó anteriormente (CD Foundation, 2025f).

Puede imaginar a los *jobs* como un archivo con instrucciones (aunque no lo son); instrucciones relacionadas que tienen un objetivo común y que se disparan de forma automática ante eventos, como la llegada de una hora específica del día, un evento desde otra aplicación o la propia acción humana.



Nota - Evita el uso de “Maven projects”

Los *Maven projects* son un tipo de Jobs en Jenkins. Aunque fueron muy utilizados por mucho tiempo, los expertos recomiendan evitar la creación de *jobs* de este tipo.

3.2 Tipos de *jobs* comunes



Lectura 5. Tipos de *jobs* comunes

Nivel básico

Objetivo: el estudiante reconoce los tipos de *jobs* comunes que se pueden crear en Jenkins: *freestyle*, *pipeline* y *multi-branch pipelines*.

Descripción

La lectura explica las definiciones, propósito, características y casos de uso de los tres tipos más comunes de *jobs* en Jenkins: *freestyle*, *pipeline* y *multi-branch pipelines*.

Tres tipos de *jobs* comunes en Jenkins y que vale la pena destacar, son: *freestyle*, *pipeline* y *multi-branch pipeline*. Aunque no son los únicos, destacan entre los demás tipos debido a su uso constante por parte de los usuarios de Jenkins.

Freestyle job es el tipo de *job* más versátil. Es el tipo más básico y sencillo de implementar, pues permite automatizar tareas sin necesidad de *scripting* o habilidades de codificación; la configuración se hace a través de la interfaz gráfica de Jenkins. Dada esta flexibilidad, no tienen un propósito acotado más allá de automatizar tareas, como envío de notificaciones, guardado de información, generación de artefactos de software, entre otros. (GeeksforGeeks, 2025b).

Pipeline es posiblemente el tipo de *job* más popular en la actualidad. Este es un tipo de *job* más estructurado que el *freestyle job* y que proporciona un conjunto de herramientas para modelar [pipelines de despliegue](#) (explicados con detalle en el capítulo 3.2) como código. Piense en estos *jobs* como un conjunto de instrucciones que tienen como propósito principal entregar software (además de construirlo, asegurar su calidad, empaquetarlo y demás) (CD Foundation, 2025c).

Los *multi-branch pipeline jobs* llevan la idea anterior más lejos. De los tres tipos, estos son los más robustos. Su propósito es detectar automáticamente las ramas que contiene un repositorio de código y ejecutar un *pipeline job* por cada una, sin necesidad de crear *pipeline jobs* manualmente por cada una de las ramas. Esta es la característica principal que los diferencia de los dos anteriores, pues su objetivo, al igual que los *pipeline jobs*, es automatizar tareas para la entrega de software (CD Foundation, 2025a).

3.3 Cuestionario sobre *jobs* en Jenkins



Cuestionario 2. *Jobs* en Jenkins

Contesta las siguientes preguntas para evaluar los conocimientos adquiridos en las lecturas de este capítulo sobre *jobs* en Jenkins y refuerza tus conocimientos en el tema.

1. ¿Qué es un *job* en Jenkins?

- | | | | |
|---|--|--|--|
| A) Es archivo de configuración para automatizar tareas. | B) Una tarea manual que los usuarios ejecutan. | C) Es una unidad de trabajo definida por los usuarios para automatizar tareas. | D) Es una unidad de trabajo que tiene como fin desplegar software. |
|---|--|--|--|

2. Verdadero o falso: un Job en Jenkins es un archivo ejecutable que representa un contenedor de ejecución separado del servidor Jenkins

- A) Verdadero B) Falso.

3. ¿Cuál es la función principal de un Job dentro de Jenkins?

- | | | | |
|--|---|--|--|
| A) Actuar como la unidad de trabajo que Jenkins ejecuta para automatizar tareas. | B) Automatizar las actividades de despliegue. | C) Estructurar las instrucciones que definen las automatizaciones. | D) Almacenar a través de un archivo las instrucciones para automatizar tareas. |
|--|---|--|--|

4. Tres tipos de Jobs comunes que se pueden crear en Jenkins son:

- | | | | |
|---|---------------------------------------|--|--|
| A) <i>Deployment, Pipeline y Freestyle.</i> | B) <i>Pipeline, Trigger y Matrix.</i> | C) <i>Build, External y Freestyle.</i> | D) <i>Freestyle, Pipeline y Multi-branch pipeline.</i> |
|---|---------------------------------------|--|--|

5. Relaciona cada tipo de *job* con su característica principal:

A) <i>Pipeline</i>	Es el tipo de <i>job</i> más básico y sencillo de implementar.
B) <i>Freestyle job</i>	
C) <i>Multi-branch pipeline</i>	Su propósito es detectar automáticamente las ramas que contienen un repositorio de código y automatizar tareas para cada una.
	Proporciona un conjunto de herramientas para modelar pipelines de despliegue como código.

6. Verdadero o falso: un *Freestyle job* permite crear *jobs* sin necesidad de *scripting* o habilidades de codificación.

A) Verdadero. B) Falso.

7. Verdadero o falso: un *Pipeline* permite definir todo el flujo de trabajo como código.

A) Verdadero. B) Falso.

8. Verdadero o falso: un *Multi-branch Pipeline* requiere que se creen *jobs* manualmente para cada rama del repositorio.

A) Verdadero B) Falso.



Capítulo 4

4 Introducción a los *pipelines*

Contenido del capítulo

- 4.1. Concepto general de *pipeline*
- 4.2. *Pipelines* de despliegue
- 4.3. Etapas comunes de los *pipelines* de despliegue
- 4.4. *Pipelines* en Jenkins
 - 4.4.1. Concepto de *pipeline* en Jenkins
 - 4.4.2. Estilos para definir *pipelines* en Jenkins
 - 4.4.3. Estructura general de los *pipelines* declarativos
 - 4.4.4. Directivas esenciales de los *pipelines* declarativos

4.1 Concepto general de *pipeline*



Lectura 6. ¿Qué es un *pipeline*?

Nivel básico

Objetivo: el estudiante identifica el concepto de *pipeline*.

Descripción

La lectura explica la definición de *pipeline* como concepto general, sin acotarlo a algún contexto como CI/CD o Jenkins, resaltando sus características clave.

El concepto de *pipeline* es amplio y no se acota a Jenkins. Según el estándar 2675 IEEE (2021), un *pipeline* es una técnica de diseño de hardware o software en la que la salida de un proceso sirve como la entrada de uno segundo, la salida de ese segundo proceso sirve como la entrada de uno tercero y así sucesivamente.

Tal como una tubería (*pipeline* en inglés), en la que se conectan diversas secciones para generar un sistema mayor que permite transportar líquidos (comúnmente), el concepto de *pipeline* toma esta idea de interconectar diversos pasos para mover o procesar “cosas” hacia un destino específico; ya sea que hablemos de un flujo de información, una serie de procesos de software o una secuencia de etapas en la manufactura de un producto, la idea se mantiene.

Así, podemos resumir que un *pipeline* se caracteriza por:

1. Ser una técnica de diseño,
 2. En la que múltiples procesos se interconectan,
 3. Para lograr algún objetivo.
-



Cuestionario 3. ¿Pipelines?

Contesta las siguientes preguntas para evaluar los conocimientos adquiridos en la lectura introductoria a los *pipelines* como concepto general.

1. Un *pipeline* es:

- | | | | |
|---|---|----------------------------|---|
| A) Un tipo de software específico de Jenkins. | B) Una técnica de diseño en la que procesos se interconectan. | C) Un estándar de la IEEE. | D) Un concepto que solo existe en el contexto de Jenkins. |
|---|---|----------------------------|---|

2. Verdadero o falso: un *pipeline* se caracteriza por interconectar múltiples procesos.

- A) Verdadero. B) Falso.

3. Verdadero o falso: el objetivo de un *pipeline* es solo automatizar tareas en Jenkins.

- A) Verdadero. B) Falso.

4. Selecciona el ejemplo que mejor represente la idea de *pipeline*:

- | | | | |
|---|--|---|--|
| A) Un script de automatización que contiene instrucciones secuenciales. | B) Un flujo de tareas que se ejecutan de forma paralela dentro de un proyecto. | C) Un conjunto de comandos que definen cómo compilar y probar una aplicación. | D) Una serie de etapas de manufactura donde cada salida alimenta la siguiente. |
|---|--|---|--|

5. Selecciona el ejemplo que mejor represente la idea de *pipeline*:

- | | | | |
|------------------------|--------------|------------------------|---------------|
| A) ISO/IEC/IEEE 12207. | B) IEEE 830. | C) ISO/IEC/IEEE 29119. | D) IEEE 2675. |
|------------------------|--------------|------------------------|---------------|

4.2 Pipelines de despliegue



Lectura 7. ¿Qué es un *pipeline* de despliegue?

Nivel básico

Objetivo: el estudiante identifica el concepto de *pipeline* de despliegue.

Descripción

La lectura explica la definición de *pipeline* de despliegue y el objetivo general de estos *pipelines*, resaltando sus características clave. Además, se destaca la diferencia entre un *pipeline* de despliegue y un *pipeline* (como concepto general).

El concepto de *pipeline*, ya entrando en el contexto del despliegue de software, es muy similar al abordado en la sección previa. Un ***pipeline* de despliegue** es a una secuencia de procesos que son disparados por un cambio en el código fuente de un software; este cambio en el código fuente normalmente se realiza en un sistema de control de versiones, que inicializa el *pipeline* de despliegue, dando paso a los procesos de compilación, prueba, configuración y despliegue del software y de los artefactos que se hayan generado durante la ejecución del *pipeline* (Humble & Farley, 2010).

De forma resumida, un *pipeline* de despliegue es una manifestación automatizada del proceso de desarrollo de software, que cubre desde tener el software en un sistema de control de versiones, hasta la entrega de este al usuario final, contemplando todas las tareas intermedias.

Retomando la analogía de la tubería del capítulo anterior, en este caso la “llave” que abre el flujo, es el cambio en el código fuente dentro del sistema de control de versiones. A partir de ese momento, el agua (nuestro código) comienza a recorrer las distintas secciones de la tubería: compilación, pruebas, configuración; hasta llegar finalmente a los usuarios que esperan consumirla.

Así pues, las características se mantienen: una técnica de diseño en la que se conectan múltiples procesos para lograr un objetivo; solamente que ahora acotando la idea a la entrega de software.

Imagínalo con un ejemplo sencillo: estás implementando nuevas funcionalidades en un sitio web dentro de un equipo de trabajo. Realizas un cambio en Git y haces un *push* a GitHub. De manera automática, un proceso clona tu rama en un espacio de trabajo de prueba, donde se ejecuta un análisis estático de código y se corren pruebas unitarias, de integración y de sistema. Luego, el código se empaqueta en un contenedor Docker, se despliega en un servidor y se aplican pruebas de aceptación y de carga. Al finalizar, recibes un correo con el informe de resultados y, si todo salió bien, el sitio web visible para los usuarios se actualiza. Así, un *pipeline* de despliegue lleva sus cambios directamente hasta los clientes finales.



Cuestionario 4. Pipelines de despliegue

Contesta las siguientes preguntas para evaluar los conocimientos adquiridos en la lectura sobre *pipelines* de despliegue y valida tus conocimientos en el tema.

1. Un *pipeline* de despliegue es:

- | | | | |
|--|--|---|--|
| A) Un sistema manual de entrega de software. | B) Una secuencia automatizada de procesos disparada por cambios en el código fuente. | C) Un sistema automatizado de despliegue de software. | D) Un tipo de <i>job</i> en Jenkins para automatizar la entrega de software. |
|--|--|---|--|

2. ¿Cuál es el objetivo principal de un *pipeline* de despliegue?

- | | | | |
|--|--|--|---|
| A) Ejecutar pruebas unitarias, de integración y sistema. | B) Disparar de forma automática el proceso de entrega de software. | C) Recuperar el código fuente del sistema de control de versiones. | D) Llevar automáticamente los cambios en el código desde el control de versiones hasta el usuario final |
|--|--|--|---|

3. Verdadero o falso: un *pipeline* de despliegue puede automatizar los procesos de compilación, prueba, configuración y despliegue.

- A) Verdadero. B) Falso.

4. Verdadero o falso: el inicio del *pipeline* depende de un cambio en el código fuente en el sistema de control de versiones.

- A) Verdadero. B) Falso.

5. ¿Cuál es la diferencia principal entre un *pipeline* y un *pipeline* de despliegue?

- | | | | |
|---|--------------------------------------|--|---|
| A) Un <i>pipeline</i> de despliegue acota la idea de <i>pipeline</i> al proceso de entrega de software. | B) Ninguna, son conceptos idénticos. | C) Un <i>pipeline</i> solo funciona para hardware. | D) Un <i>pipeline</i> de despliegue requiere intervención manual en cada etapa. |
|---|--------------------------------------|--|---|

4.3 Etapas comunes de los *pipelines* de despliegue



Lectura 8. ¿Cuáles son las etapas típicas en un *pipeline* de despliegue?

Nivel básico

Objetivo: el estudiante enlista las etapas típicas de un *pipeline* de despliegue.

Descripción

La lectura describe de manera secuencial las principales etapas que se presentan en un *pipeline* de despliegue. Para cada etapa se explica su objetivo y funcionamiento dentro del *pipeline*, destacando las principales actividades que se suelen automatizar.

Recordemos que un *pipeline* de despliegue modela un proceso: el proceso de entrega de software; partiendo desde su construcción y pasando por los procesos para probarlo y entregarlo a los usuarios finales. Como lo mencionan Humble & Farley (2010), este proceso forma parte de uno mayor: el de llevar características en la mente de los clientes hasta sus “manos”.

Dicho esto, Humble & Farley (2010) destacan cuatro etapas comunes en los *pipelines* de despliegue:

1. **Etapas de *commit*:** esta etapa evalúa que el sistema funcione a nivel técnico. El código se compila, se corren pruebas “rápidas” (principalmente unitarias), se realiza análisis estático de código y se preparan artefactos para su uso en las etapas posteriores.

El objetivo de esta primera etapa es eliminar *builds* que no cumplen lo necesario para llegar a producción, además de advertir al equipo que la aplicación está rota lo más rápido posible.

Idealmente, esta etapa no debería tomar más de cinco minutos; es por esto por lo que se habla de pruebas “rápidas”. Estas pruebas básicamente hacen referencia a un conjunto de pruebas optimizadas para ejecutarse rápidamente, pero seleccionadas cuidadosamente con el fin de obtener el mayor nivel de confianza de que la aplicación realmente funciona como debería.

2. **Etapas de aceptación:** aquí se evalúa que el sistema trabaje correctamente a nivel funcional y no funcional con la ayuda de pruebas de aceptación automatizadas. Se asegura que el comportamiento cumpla las necesidades de los usuarios y las especificaciones del cliente.
3. **Etapas de pruebas manuales:** en esta etapa se busca asegurar que el sistema sea usable y que cumpla los requisitos, detectando defectos no encontrados por la etapa de pruebas previa. Esta etapa mueve el producto desde el dominio del equipo de desarrollo hacia el dominio de los usuarios.
4. **Etapas de liberación:** en esta etapa final se entrega el sistema a los usuarios, ya sea en forma de un paquete de software, o desplegándolo a un ambiente de producción.

Esta propuesta de Humble & Farley (2010) se puede trazar hacia una más simplificada con solo tres etapas: integración (etapa 1), prueba (etapa 2 y 3) y despliegue (etapa 4). En el siguiente diagrama se muestra cómo estas dos propuestas para las etapas se alinean; además, en la parte inferior del diagrama, se destacan algunas actividades comúnmente automatizadas en Jenkins en *pipelines* de despliegue.

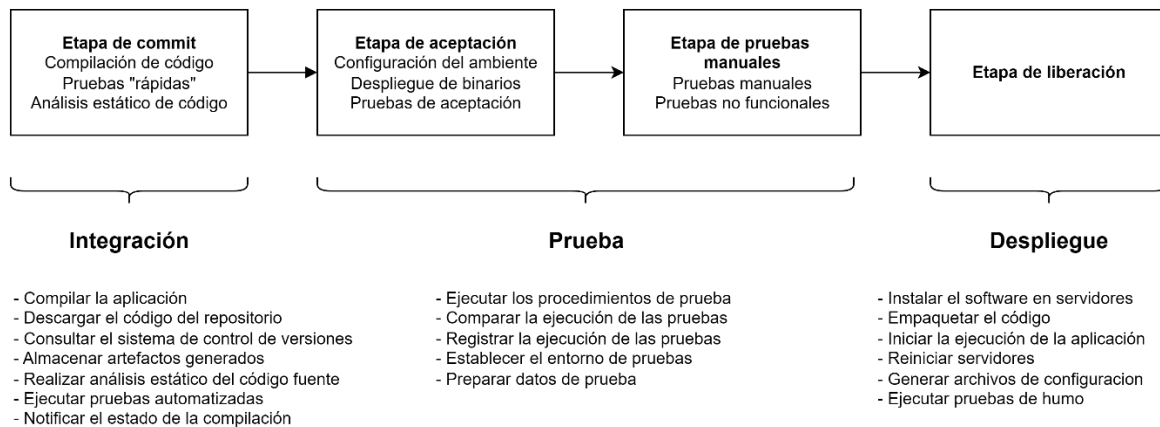


Figura 1. Representación esquemática de las etapas típicas de un pipeline

Tal como se representa en el diagrama, estas etapas son secuenciales y la idea es que no se continúe a una etapa posterior si una de las etapas falla, pues cada etapa evalúa el software construido desde una perspectiva diferente, con el fin de entregar un producto de calidad a los usuarios.

Con lo anterior, no hay que perder de vista que otros tipos de pruebas o etapas en el proceso de entrega de software pueden ser necesarias y consideradas en los *pipelines* de despliegue. Sin embargo, las mencionadas previamente son las más comunes para la mayoría de los proyectos según Humble & Farley (2010).



Cuestionario 5. Etapas comunes de los *pipelines* de despliegue.

Contesta las siguientes preguntas para evaluar los conocimientos adquiridos en la lectura previa y valida tus conocimientos sobre las principales etapas en *pipelines* de despliegue.

1. ¿Cuál de las siguientes no es una etapa típica de un *pipeline* de despliegue según la propuesta de Humble y Farley?

- A) Etapa de *commit*. B) Etapa de aceptación. C) Etapa de pruebas manuales. D) Etapa de despliegue.

2. ¿Cuál de las siguientes es una etapa típica de un *pipeline* de despliegue según la propuesta simplificada?

- | | | | |
|--------------------------|----------------------|-------------------------|--------------------------|
| A) Etapa de integración. | B) Etapa de pruebas. | C) Etapa de despliegue. | D) Todas las anteriores. |
|--------------------------|----------------------|-------------------------|--------------------------|

3. Es la primera etapa de un pipeline de despliegue según la propuesta de Humble y Farley.

- | | | | |
|-----------------------------|--------------------------|-------------------------------|----------------------|
| A) Etapa de <i>commit</i> . | B) Etapa de integración. | C) Etapa de pruebas manuales. | D) Etapa aceptación. |
|-----------------------------|--------------------------|-------------------------------|----------------------|

4. Se refiere a la etapa en la que el código fuente se convierte en archivos ejecutables o paquetes, e involucra actividades como compilación y ejecución de pruebas “rápidas”.

- | | | | |
|-----------------------------|-------------------------|-------------------------------|-------------------------|
| A) Etapa de <i>commit</i> . | B) Etapa de aceptación. | C) Etapa de pruebas manuales. | D) Etapa de liberación. |
|-----------------------------|-------------------------|-------------------------------|-------------------------|

5. “Instalar el software en servidores”, “Empaquetar el código” y “Generar archivos de configuración” son actividades comúnmente automatizadas en *pipelines* de despliegue de Jenkins que pertenecen a la etapa de:

- | | | | |
|--------------------------|---------------------|-------------------------|-------------------------------|
| A) Etapa de integración. | B) Etapa de prueba. | C) Etapa de despliegue. | D) Ninguna de las anteriores. |
|--------------------------|---------------------|-------------------------|-------------------------------|

6. Verdadero o falso: si ocurre un fallo en alguna de las etapas del *pipeline* de despliegue, este no se detiene ya que puede seguir con la ejecución hasta llegar a la etapa final.

- | | |
|---------------|-----------|
| A) Verdadero. | B) Falso. |
|---------------|-----------|

7. Es el orden común de las etapas típicas de un pipeline de despliegue:

- | | | | |
|--------------------------------|--|--------------------------------------|--------------------------------|
| A) <i>Commit</i> , aceptación, | B) <i>Commit</i> , aceptación, pruebas | C) <i>Commit</i> , pruebas manuales, | D) <i>Commit</i> , liberación, |
|--------------------------------|--|--------------------------------------|--------------------------------|

liberación y pruebas manuales.	manuales y liberación.	aceptación y liberación.	aceptación y pruebas manuales.
-----------------------------------	---------------------------	-----------------------------	-----------------------------------

8. La etapa de aceptación tiene como objetivo:

- | | | | |
|--|---|---|--------------------------|
| A) Evaluar que el sistema trabaje correctamente a nivel funcional. | B) Evaluar que el sistema trabaje correctamente a nivel no funcional. | C) Asegurar que el comportamiento del sistema cumpla las necesidades de los usuarios. | D) Todas las anteriores. |
|--|---|---|--------------------------|

9. Verdadero o falso: en la etapa de pruebas manuales el producto se mueve desde el dominio del equipo de desarrollo hacia el dominio de los usuarios.

- | | |
|--------------|-----------|
| A) Verdadero | B) Falso. |
|--------------|-----------|

10. El objetivo principal de la etapa de liberación es:

- | | | | |
|---|--|--|-------------------------------|
| A) Desplegar el sistema en un servidor. | B) Empaquetar el producto de software. | C) Entregar el sistema a los usuarios. | D) Ninguna de las anteriores. |
|---|--|--|-------------------------------|

11. Verdadero o falso: actividades como “Compilar la aplicación”, “Ejecutar pruebas automatizadas” y “Almacenar artefactos generados” son comúnmente automatizadas en *pipelines* de despliegue en Jenkins.

- | | |
|--------------|-----------|
| A) Verdadero | B) Falso. |
|--------------|-----------|

12. “Preparar datos de prueba”, “Establecer el entorno de pruebas” y “Ejecutar los procedimientos de prueba” son actividades comúnmente automatizadas en *pipelines* de despliegue de Jenkins que pertenecen a la etapa de:

- | | | | |
|--------------------------|---------------------|-------------------------|-------------------------------|
| A) Etapa de integración. | B) Etapa de prueba. | C) Etapa de despliegue. | D) Ninguna de las anteriores. |
|--------------------------|---------------------|-------------------------|-------------------------------|

4.4 Pipelines en Jenkins

4.4.1 Concepto de *pipeline* en Jenkins



Lectura 9. ¿Qué es un *pipeline* en Jenkins?

Nivel básico

Objetivo: el estudiante identifica el concepto de *pipeline* en Jenkins.

Descripción

La lectura explica la definición de *pipeline* en Jenkins y el objetivo general de estos, resaltando sus características clave. Además, se destaca la diferencia entre un *pipeline* de despliegue y un *pipeline* en Jenkins.

En el contexto de Jenkins, el concepto de *pipeline* mantiene el mismo sentido que un *pipeline* de despliegue, pero la CD Foundation (2025c) agrega un elemento clave, los *plugins* como parte fundamental del propio *pipeline*:

Un ***pipeline* de Jenkins** es un conjunto de *plugins* que permiten la implementación e integración de *pipelines* de despliegue en Jenkins.

En otras palabras, un ***pipeline* de Jenkins** no es más que la orquestación de varios *plugins* (normalmente herramientas de software de todo tipo) que, trabajando todos juntos, de manera coordinada dentro de Jenkins, construyen un flujo de trabajo automatizado; como una cadena de pasos en la que cada *plugin* cumple con una función específica (tales como compilar, probar, desplegar, entre otras actividades) y Jenkins los coordina para que ocurran de manera continua sin intervención manual.

En términos simples, un *pipeline* de despliegue es el concepto general que describe una secuencia de pasos o etapas automatizadas, mientras que un *pipeline* de Jenkins es un tipo de *Job* que, a su vez, es la implementación práctica de un *pipeline* de despliegue dentro de Jenkins, constituida a partir de *plugins*.

Los *pipelines* en Jenkins se pueden crear de dos maneras:

1. Mediante interfaz gráfica de usuario (GUI): es la forma más sencilla, únicamente se configuran las herramientas y etapas del *pipeline* de manera visual.

2. Mediante texto plano (también conocido como *pipeline-as-code*): es un enfoque más complejo, pero permite mayor personalización. Las etapas y tareas se escriben usando una sintaxis de lenguaje específico de dominio (DSL) para *pipelines* de Jenkins, toda instrucción se escribe y guarda en un archivo denominado **Jenkinsfile**, el cual Jenkins interpreta para crear y ejecutar el *pipeline* de manera automatizada (CD Foundation, 2025c).

**Nota - Utiliza Jenkinsfiles.**

En general, es mejor práctica crear tus *pipelines* mediante Jenkinsfiles y subirlos a un sistema de control de versiones, para tener un mejor control de versiones y organización de tus *pipelines*.

Tal como se explica en el capítulo dedicado a los [jobs](#), Jenkins permite utilizar varios tipos de patrones de automatización. Sin embargo, la CD Foundation (2025c) hace énfasis en utilizar *pipelines* sobre otros tipos de *jobs*, ya que agrega un conjunto poderoso de herramientas de automatización a Jenkins, permitiendo crear desde *pipelines* de integración, hasta *pipelines* de despliegue más complejos. Así, los usuarios pueden crear proyectos que se ajusten a sus necesidades, aprovechándose de las características de este tipo de *jobs*:

- Durabilidad: los *pipelines*, a diferencia de otros *jobs*, resisten los reinicios planeados o inesperados del servidor.
- Interrumpible: los *pipelines* pueden ser detenidos opcionalmente durante su ejecución, para permitir la entrada de un parámetro por parte del usuario.
- Versátiles: los *pipelines* permiten implementar requisitos complejos de CI/CD, como la posibilidad de hacer *forks*, *join*, *loops* o trabajos paralelos.
- Extensibilidad: además de la amplia variedad de *plugins* que soportan los *pipelines*, también permiten integrar extensiones personalizadas.

En la siguiente figura se muestra un flujo de trabajo común de un *pipeline* de Jenkins.

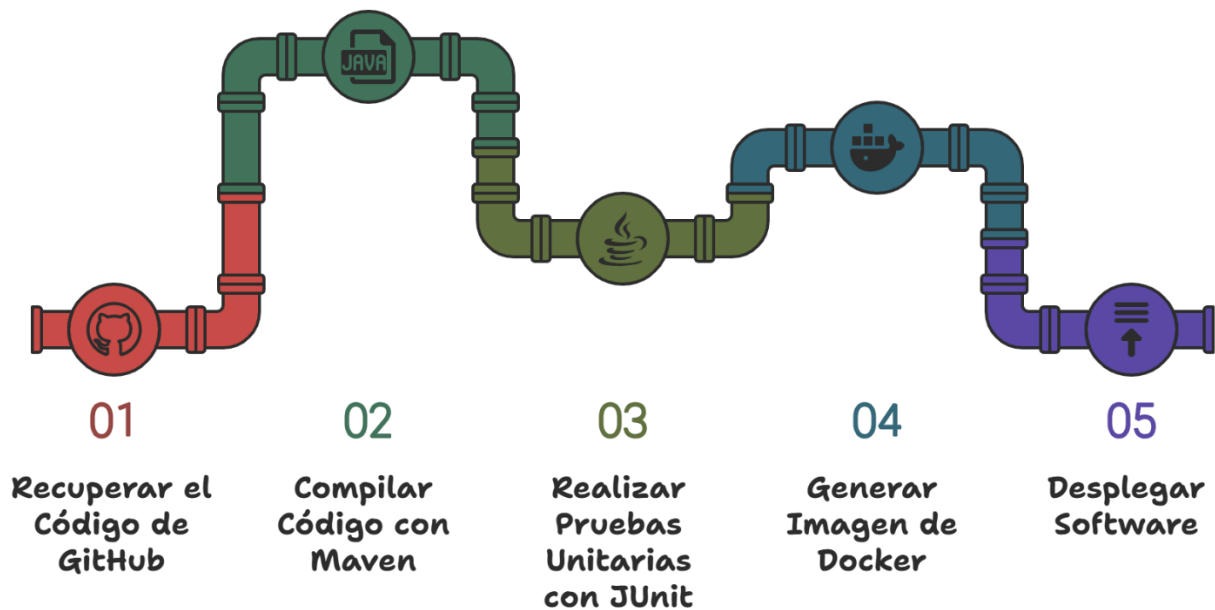


Figura 2. Representación de un flujo de trabajo de un pipeline de Jenkins para un proyecto de Java

En este ejemplo, se modelan cinco etapas generales y se utilizan *plugins* para integrar herramientas como GitHub, Maven, Junit y Docker para realizar una o más actividades relacionadas al ciclo de vida de desarrollo del software. Todas estas herramientas y tareas son coordinadas por Jenkins para ejecutarse de acuerdo con el flujo de trabajo que se busca automatizar.



Cuestionario 6. Pipelines en Jenkins

Contesta las siguientes preguntas para evaluar los conocimientos adquiridos en la lectura previa y así validar sus conocimientos en *pipelines* de Jenkins.

1. ¿Cuál es el elemento clave que permite crear los *pipelines* en el contexto de Jenkins?

A) Entornos Virtuales.

B) Repositorio de Código.

C) *Plugins*.

D) Servidor de automatización.

2. Un *pipeline* de Jenkins es:

- | | | | |
|--|--|--|---|
| A) Un archivo de configuración de sistema operativo. | B) Un conjunto de <i>plugins</i> coordinados por Jenkins para ejecutar un flujo de trabajo automatizado. | C) Un lenguaje de programación de propósito general para creación de <i>pipelines</i> de despliegue. | D) Un mecanismo de automatización de tareas y actividades independientes. |
|--|--|--|---|

3. Verdadero o falso: un *pipeline* de Jenkins es exactamente lo mismo que un *pipeline* de despliegue.

- A) Verdadero. B) Falso.

4. Verdadero o falso: utilizar la interfaz gráfica de Jenkins es la forma más sencilla y recomendable de crear *pipelines*.

- A) Verdadero. B) Falso.

5. Es una práctica recomendada para la creación de *pipelines* en Jenkins.

- | | | | |
|-----------------------|---|---|---|
| A) Usar Jenkinsfiles. | B) Configurar <i>pipelines</i> manualmente. | C) Crear los <i>pipelines</i> únicamente desde la interfaz gráfica. | D) Evitar el uso de control de versiones. |
|-----------------------|---|---|---|

6. ¿Cuáles son las dos formas principales de crear un *pipeline* en Jenkins?

R = _____

7. ¿Cuál es una ventaja principal de utilizar *pipelines* en Jenkins frente a otros tipos de *jobs*?

- | | | | |
|---|---|--|---|
| A) Requieren menos memoria para ejecutarse. | B) Permiten únicamente procesos lineales y simples. | C) Ofrecen características como durabilidad, | D) Solo se pueden usar en entornos locales. |
|---|---|--|---|

versatilidad y
extensibilidad.

8. Verdadero o falso: los *pipelines* en Jenkins solo pueden ejecutarse en un flujo lineal y no permiten trabajos paralelos.
- A) Verdadero. B) Falso.
9. Verdadero o falso: los *pipelines* en Jenkins son un flujo de trabajo secuencial y automatizado, por lo que no se pueden interrumpir durante su ejecución
- A) Verdadero. B) Falso.
10. Verdadero o falso: los *pipelines* en Jenkins resisten reinicios planeados o inesperados del servidor sin perder su estado.
- A) Verdadero. B) Falso.

4.4.2 Estilos para definir *pipelines* en Jenkins



Lectura 10. Estilos de *pipelines* en Jenkins

Nivel básico

Objetivo: el estudiante reconoce los estilos para definir *pipelines* en Jenkins: declarativo y *scripted*.

Descripción

La lectura presenta los dos estilos disponibles para definir *pipelines* en Jenkins, destacando las características de ambos, sus casos de uso y diferencias.

En la lectura previa se mencionan dos formas de crear *pipelines* en Jenkins: mediante la interfaz gráfica de usuario, o la manera recomendada, escribiendo código en un archivo denominado Jenkinsfile. Estos archivos pueden ser escritos utilizando dos tipos de sintaxis: **declarativa** (*declarative*) o **scripted**. (CD Foundation, 2025c)

Los ***pipelines de tipo scripted*** son la sintaxis original de Jenkins y fue la primera forma de crear *pipelines*. Estos están basados fuertemente en el lenguaje de programación Groovy, y eran los que predominaban antes de la versión 2.0 de Jenkins (Laster, 2018, p. 19).

Los *pipelines* que utilizan este tipo de sintaxis también son llamados *pipelines imperativos*, pues funcionan como un “programa” donde el usuario debe escribir la lógica y el flujo de cada etapa del *pipeline*. Es decir, uno mismo indica paso a paso qué hacer y en qué orden (Laster, 2018, p. 38-39).

```
// Scripted Pipeline
node('worker_node1') {
    stage('Source') { // Get code
        // get code from our Git repository
        git 'git@diyvb2:/home/git/repositories/workshop.git'
    }
    stage('Compile') { // Compile and do unit testing
        // run Gradle to execute compile and unit testing
        sh "gradle clean compileJava test"
    }
}
```

Figura 3. Ejemplo de un Scripted Pipeline

Normalmente las instrucciones de un *pipeline* de tipo *scripted* se escriben dentro de un bloque “*node*”, que refiere al nodo en donde se ejecutará el *pipeline*. Dentro de este bloque se encuentran los *stages*, que describen las etapas principales del *pipeline*. Dentro de cada *stage*, se describen las acciones específicas a realizar. En el ejemplo de la figura anterior, se clona un repositorio en el primer *stage* denominado “*Source*”, y en el segundo *stage* (*Compile*), se compila el código y se ejecutan pruebas unitarias.

A pesar de que este tipo de *pipelines* soportan una amplia variedad de funcionalidades para ejecutar tareas, tienen un inconveniente clave: los *scripted pipelines* no incluían algunas funcionalidades clave de Jenkins, como la gestión automática después de una construcción (*post-build*), el envío de notificaciones o la validación de la estructura del *pipeline*. Aunque muchas de estas funcionalidades se podían emular con Groovy, esto requería conocimientos muy avanzados que no todos los usuarios tenían (Laster, 2018, p. 19).

Esto dio origen a que en 2016 se introdujera la sintaxis de los *pipelines* declarativos, agregando una estructura más clara y predecible a los *pipelines*, así como elementos y constructos mejorados del lenguaje específico de dominio, haciéndolo más

amigable y similar a construir el *pipeline* mediante la interfaz gráfica de usuario. Esta nueva sintaxis permitió reducir la utilización de código Groovy para emular comportamientos y funcionalidades inherentes a Jenkins (Laster, 2018, p.19).

A diferencia de los *pipelines* de tipo *scripted*, los declarativos se organizan en secciones claras que describen (o “declaran”) los estados y resultados que se desean obtener en las partes principales del *pipeline*, en lugar de concentrarse en la lógica para lograr lo mismo (Laster, 2018). Se enfocan en el qué y no en el cómo.

```
// Declarative Pipeline
pipeline {
  agent {label 'worker_node1'}
  stages {
    stage('Source') { // Get code
      steps {
        // get code from our Git repository
        git 'git@diyvb2:/home/git/repositories/workshop.git'
      }
    }
    stage('Compile') { // Compile and do unit testing
      steps {
        // run Gradle to execute compile and unit testing
        sh "gradle clean compileJava test"
      }
    }
  }
}
```

Figura 4. Ejemplo de un Pipeline Declarativo

En los *pipelines* declarativos, todas las instrucciones deben estar contenidas dentro del bloque principal denominado “*pipeline*”. Dentro de este bloque se definen etiquetas similares a las usadas en los *scripted pipelines*, como *stages*, pero además se incorporan nuevas instrucciones específicas, como *agent* y *steps*.

Aunque los *pipelines scripted* y los declarativos se basan en las mismas reglas de sintaxis de Groovy, la CD Foundation (2025d) recalca que los *pipelines* declarativos utilizan una sintaxis más simplificada y presentan algunas excepciones importantes:

- No es posible usar llaves { } como separadores de declaraciones; cada instrucción debe escribirse en una línea independiente.
- Cada bloque puede contener únicamente secciones, directivas, pasos o declaraciones de asignación.
- El bloque principal y más grande siempre debe ser *pipeline* { }.

De manera general, se puede pensar en la sintaxis declarativa como el uso de un formulario en la interfaz web: basta con llenar información clave en secciones preestablecidas, donde cada una tiene un propósito específico y un comportamiento esperado. Esto los hace más fácil de aprender y de mantener para los usuarios nuevos. Sin embargo, provee de una menor flexibilidad para implementar acciones que no estén contempladas en la estructura predefinida.

Mientras que la sintaxis *scripted* se asemeja a crear un *script* o programa escrito de forma imperativa, donde se define directamente la lógica y flujo de ejecución. Este enfoque ofrece mayor flexibilidad y control, dando más libertad al usuario para implementar *pipelines* más complejos y personalizables sin las restricciones de una estructura fija.



Nota - ¿Pipelines declarativos o scripted?

Los expertos recomiendan el uso de la sintaxis declarativa sobre *scripted* para definir los *pipelines*, dada su facilidad de lectura y que suele resultar más amigables para los principiantes.

En la siguiente tabla se muestran los aspectos más relevantes de cada tipo de sintaxis, así como sus ventajas, desventajas y los casos de uso más favorables para cada tipo de *pipeline* (CD Foundation, 2025d; Laster, 2018, pp. 39-41).

Tabla 1. Cuadro comparativo de pipelines scripted contra pipelines declarativos

Característica	Tipo de Sintaxis	
	<i>Scripted Pipeline</i>	<i>Pipeline Declarativo (Declarative)</i>
<i>Enfoque</i>	Basado en programación imperativa (definir paso a paso la lógica y flujo).	Basado en estructura declarativa (secciones predefinidas con un propósito claro).
<i>Complejidad</i>	Puede ser complejo de aprender, requiere conocimientos de Groovy.	Más fácil de aprender, incluso para usuarios nuevos.
<i>Flexibilidad</i>	Muy flexible: permite lógica avanzada (loops, condiciones, programación personalizada).	Menos flexible: se limita a lo soportado por la sintaxis declarativa.

<i>Mantenimiento</i>	Puede ser más difícil de mantener en proyectos grandes debido a su libertad.	Más fácil de mantener por su estructura clara y validación de sintaxis.
<i>Sintaxis</i>	Lenguaje de <i>scripting</i> con Groovy. Más libre y flexible.	Sintaxis más simple y estructurada. Menos flexible pero más ordenada.
<i>Bloque de sintaxis principal</i>	No tiene un bloque obligatorio, pero normalmente empieza con <code>node { ... }</code>	Siempre debe estar dentro de <code>pipeline { ... }</code> .
<i>Agente de ejecución</i>	Se define con <code>node('nombre_agente')</code> .	Se define dentro de la sección <code>agent { ... }</code> .
<i>Definir Etapas</i>	<code>stage('Nombre') { ... }</code> .	<code>stages { stage('Nombre') { steps { ... } } }</code>
<i>Acciones dentro de las etapas</i>	Se escriben directamente dentro del stage. Ejemplo: <code>sh "mvn clean install"</code> .	Se colocan dentro del bloque <code>steps { ... }</code> . Ejemplo: <code>steps { sh "mvn clean install" }</code> .
<i>Condicionales y ciclos</i>	Se usa la sintaxis de Groovy (<code>if</code> , <code>for</code> , <code>while</code> , etc.).	Algunas directivas (<code>when { ... }</code>), pero no hay bucles complejos.
<i>Post-build (acciones finales)</i>	Se deben programar manualmente en Groovy.	Existe la sección <code>post { success { ... } failure { ... } }</code> predefinida.
<i>Flexibilidad de Groovy</i>	Puedes usar cualquier expresión o función de Groovy.	Sintaxis simplificada: no se permiten llaves como separadores ni expresiones arbitrarias de Groovy.
<i>Legibilidad</i>	Menos estructurado, se parece más a un script de programación.	Más ordenado y predecible, parecido a llenar un formulario con secciones.
<i>Ventajas</i>	• Generalmente requieren menos secciones y menos especificaciones. • Es similar a crear un programa. • Es más flexible para realizar operaciones personalizadas y flujos de trabajo más complejos.	• Puede generarse mediante la interfaz gráfica Blue Ocean. • Contiene secciones que se relacionan con conceptos familiares de Jenkins. • Mejor verificación de sintaxis e identificación de errores. • Mayor consistencia entre diferentes <i>pipelines</i>.

Desventajas	• En general requiere que el usuario escriba más código en Groovy. • La verificación de la sintaxis se limita al lenguaje y entorno Groovy. • Se aleja del modelo tradicional de Jenkins. • Puede resultar más complejo que implementar el <i>pipeline</i> de manera declarativa.	• Estructura más rígida (más difícil de manejar código de <i>pipeline</i> personalizado). • Actualmente no está bien adaptado para <i>pipelines</i> o flujos de trabajo más complejos.
Casos de uso ideales	<i>Pipelines</i> complejos o muy personalizados.	<i>Pipelines</i> estándar, fáciles de entender y de mantener.

Cuestionario 7. Estilos para definir *pipelines*



Contesta las siguientes preguntas para evaluar los conocimientos adquiridos en la lectura previa y validar tus conocimientos sobre los estilos de *pipelines* en Jenkins.

1. ¿Cuáles son los dos estilos principales para definir *pipelines* en Jenkins?

R = _____

2. Verdadero o falso: un *pipeline* declarativo en Jenkins siempre empieza por la palabra clave "*pipeline*".

A) Verdadero. B) Falso.

3. Los expertos recomiendan el uso de la sintaxis *scripted* para la creación de *pipelines* de Jenkins.

A) Verdadero. B) Falso.

4. ¿Qué tipo de *pipelines* son más simples y estructurados?

- A) Declarativos. B) *Scripted*. C) Jenkinsfiles. D) Todas las anteriores.

5. ¿Qué tipo de *pipelines* ofrecen mayor flexibilidad al usar Groovy?

- A) Declarativos. B) *Scripted*. C) Jenkinsfiles. D) Todas las anteriores.

6. Relaciona cada una de las siguientes características con el tipo de *pipeline* según corresponda, anotando la letra del tipo de *pipeline* en la casilla que corresponde a una de sus características.

A) Declarativo

B) *Scripted*

(1) Se debe definir paso a paso la lógica y flujo manualmente.

(2) Requiere amplios conocimientos de Groovy.

(3) Se limita a lo soportado por la sintaxis declarativa.

(4) Tiene secciones predefinidas con un propósito claro.

(5) Más fácil de mantener

(6) No tiene un bloque obligatorio, pero normalmente empieza con *node { ... }*

(7) Utiliza bloques *steps { ... }*

(8) No permite usar bucles muy complejos

(9) No contiene una funcionalidad de *post-build* nativa.

(10) Mejor verificación de sintaxis e identificación de errores.

(11) Tiene una estructura bastante rígida

(12) Se aleja del modelo tradicional de Jenkins.

7. Los *pipelines* declarativos se recomiendan para casos avanzados que requieren de una lógica compleja.

A) Verdadero. B) Falso.

8. ¿Cuál estilo de *pipeline* debes elegir para un equipo que recién comienza a utilizar Jenkins y por qué?

R = _____, porque _____

9. ¿A qué estilo de sintaxis corresponde el siguiente fragmento de código de Jenkins?

```
pipeline {
  agent any
  stages {
    stage('Login') {
      steps {
        echo "Active user is now ${params.USERID}"
      }
    }
  }
}
```

A) *Scripted*. B) Declarativo.

10. Un equipo de desarrollo pretende construir un *pipeline* de Jenkins el cual debe contar con bucles y condiciones complejas, ¿Qué estilo de *pipeline* es el más adecuado?

- A) Declarativos. B) *Scripted*. C) Jenkinsfiles. D) Iterativos.

4.4.3 Estructura general de los *pipelines* declarativos



Lectura 11. Estructura de pipelines declarativos

Nivel intermedio

Objetivo: el estudiante explica cómo están compuestos los *pipelines* declarativos: bloques, secciones, directivas y condicionales.

Descripción

La lectura explica la estructura general de los *pipelines* declarativos, incluyendo los diferentes tipos de elementos que lo pueden componer y el propósito de cada uno de ellos.

Según Laster (2018), la estructura común de un *pipeline* declarativo consiste en un bloque padre denominado "*pipeline { }*", que contiene dos tipos de elementos: **directivas** y **secciones**. A su vez cada sección puede contener más secciones, directivas, pasos (*steps*) y condicionales. Aunque en la práctica los bloques, secciones y directivas se ocupan indiscriminadamente y a veces como sinónimos, sí existe una diferencia en cuanto a su propósito y funcionalidad:

- **Bloque:** un bloque se refiere a un conjunto de código que tiene un inicio y un fin. En otras palabras, es el código que se encuentra dentro de una llave que abre "{" y una que cierra "}". Aunque este término aplica a varias partes del *pipeline*, normalmente se utiliza para referirse únicamente al bloque principal que contiene todo el código asociado al *pipeline*.

```
pipeline {  
  // code in declarative syntax  
}
```

Figura 5. Ejemplo de directiva de bloque *pipeline{}*

- **Sección (Section):** las secciones son formas de agrupar elementos que necesitan ser ejecutados en puntos particulares del *pipeline*. Actualmente existen tres tipos de secciones.
 - **Stages:** es el contenido principal del *pipeline* y aquí se agrupan todas las definiciones individuales de directivas *stage* que forman la lógica del *pipeline*.

```
stages {  
  stage('Build') {  
    //Steps and actions  
  }  
  stage('Test') {  
    //Steps and actions  
  }  
}
```

Figura 6. Ejemplo de sección Stage{}

- **Steps:** se usan dentro de cada declaración *stage*, y agrupan los comandos que realmente se ejecutan. De manera general sirve para separar acciones concretas como “sh”, “git”, “echo”, entre otras.

```
stage('Deploy') {  
  steps {  
    sh "docker build -t myapp ."  
    sh "docker run myapp"  
  }  
}
```

Figura 7. Ejemplo sección steps{}

- **Post:** agrupa *steps* y condiciones que se deben ejecutar o verificar al final de la ejecución del *pipeline* o de un *stage*. Normalmente se usa para manejar acciones según los resultados obtenidos durante la ejecución.

```
post {  
    success {  
        echo "Pipeline completado con éxito 🎉"  
    }  
    failure {  
        echo "Pipeline falló ❌"  
    }  
    always {  
        echo "Esto se ejecuta siempre"  
    }  
}
```

Figura 8. Ejemplo sección post{}

- **Directivas:** son instrucciones que configuran, definen o controlan el comportamiento del *pipeline* o de una etapa. Sirven esencialmente para tres cosas:
 - **Definir valores:** para asignar un valor a alguna variable de entorno del *pipeline*, por ejemplo la directiva `agent { }`, para asignar al nodo trabajador que se va a usar en la ejecución del *pipeline*.

```
agent { label 'worker' }
```

Figura 9. Ejemplo directiva agent{}

- **Configurar comportamientos:** para establecer configuraciones de comportamiento del *pipeline*. Un ejemplo de esto es la directiva `triggers { }`, que permite establecer cuándo se debe ejecutar el *pipeline* automáticamente.

```
triggers { cron('0 7 * * 1-5') }
```

Figura 10. Ejemplo de la directiva triggers{}

- **Especificar acciones que se deben realizar:** para definir etapas o acciones que se deben ejecutar y que no pueden o no pertenecer a una sección. Como la directiva `stage { }`, que define una etapa y dentro de la cual habrá pasos.

```
stage('Build') {  
    steps {  
        sh "mvn clean install"  
    }  
}
```

Figura 11. Ejemplo directiva stage

- **Condicionales:** establecen criterios bajo los cuales cierta acción debe ocurrir. Normalmente se pueden encontrar dos tipos de condicionales en los *pipelines* declarativos:
 - **when:** aunque técnicamente es una directiva, se considera un tipo de condicional que define un criterio para que un *stage* sea ejecutado o no.

```
stage ('build') {  
    when {  
        branch 'foo'  
    }  
    <steps>  
}
```

Figura 12. Ejemplo condicional `when{}`

- **post:** es un bloque dentro de la sección *post* que define criterios para realizar acciones posteriores a la ejecución del *pipeline*. Las condiciones y criterios en este bloque normalmente hacen referencia a el estatus final del *pipeline*, como *success*, *failure* u *always*.

```
post {  
    success {  
        echo "Pipeline completado con éxito 🎉"  
    }  
    failure {  
        echo "Pipeline falló ❌"  
    }  
    always {  
        echo "Esto se ejecuta siempre"  
    }  
}
```

Figura 13. Ejemplo condicional `post{}`

Como resumen, los *pipelines* declarativos se componen de cuatro partes fundamentales: bloques, secciones, directivas y condicionales.

- Los bloques y secciones definen la estructura general del *pipeline*,
- Las directivas establecen su comportamiento y configuración, y
- Los condicionales aportan flexibilidad, permitiendo que ciertas acciones se ejecuten solo si se cumplen condiciones lógicas.



Cuestionario 8. Estructura de *pipelines* declarativos

Contesta las siguientes preguntas para evaluar los conocimientos adquiridos en esta lectura previa y así validar tus conocimientos sobre la estructura de *pipelines* declarativos.

1. ¿Cuál es el bloque principal contiene todo el código asociado al *pipeline*?

R= _____

2. ¿Cuál es la sección que se ejecuta al final de un *pipeline* o *stage* para manejar acciones según los resultados obtenidos?

R= _____

3. ¿Qué elemento define la estructura general del pipeline?

- A) Directivas. B) Bloques y secciones. C) Condicionales. D) *steps*.

4. ¿Qué sección agrupa las definiciones individuales de etapas (*stage*)?

- A) *steps*. B) *post*. C) *stages*. D) *triggers*.

5. ¿Cuál es el propósito principal de la sección *steps*?

- | | | | |
|----------------------------------|--|--|--|
| A) Definir agentes de ejecución. | B) Configurar cuándo se dispara un <i>pipeline</i> . | C) Ejecutar comandos y acciones concretas. | D) Controlar la ejecución posterior al <i>pipeline</i> . |
|----------------------------------|--|--|--|

6. ¿Qué tipo de directiva permite establecer condiciones automáticas de ejecución del *pipeline*?

A) *stage*.

B) *triggers*.

C) *post*.

D) *when*.

7. Verdadero o falso: las secciones en un *pipeline* declarativo solo pueden contener *steps*.

A) Verdadero.

B) Falso.

8. Verdadero o falso: los condicionales permiten que ciertas acciones se ejecuten solo si se cumplen criterios lógicos.

A) Verdadero.

B) Falso.

9. La sección *post* se utiliza para ejecutar *steps* únicamente al inicio del *pipeline*.

A) Verdadero.

B) Falso.

10. De manera resumida, los pipelines declarativos se componen de cuatro partes fundamentales: _____, _____, _____ y _____.

4.4.4 Directivas esenciales de los pipelines declarativos



Lectura 12. Directivas esenciales de los *pipelines* declarativos.

Nivel intermedio

Objetivo: el estudiante explica las directiva que normalmente se utilizan para la creación los *pipelines* declarativos: *pipeline*, *enviroment*, *tool*, *options*, *trigger*, *parameters*, *libraries*, *stages*, *agent*, *stage*, *steps* y *post*.

Descripción

La lectura explica las principales directivas que se suelen utilizar en los *pipelines* declarativos de Jenkins, para cada directiva se detalla su propósito y ubicación recurrente dentro de los Jenkinsfiles.

En la lectura previa se explica que las directivas son las instrucciones que permiten configurar y controlar el comportamiento del *pipeline*, así como ejecutar las actividades automatizadas. Estas se definen a lo largo de todo el Jenkinsfile, normalmente dentro del bloque principal y pueden contenerse dentro de otras secciones de manera anidada. Las directivas generalmente se componen de una palabra reservada (la directiva en sí) y uno o varios argumentos o bloques de configuración.

En la siguiente figura se muestran las principales directivas que se suelen usar para los *pipelines* declarativos. Cada “caja” representa una sección específica con su correspondiente directiva; aquellas con líneas punteadas son directivas opcionales, mientras que las que tienen líneas continuas son obligatorias en todo *pipeline* declarativo (CD Foundation, 2025d; Laster, 2018, pp 237 - 259).

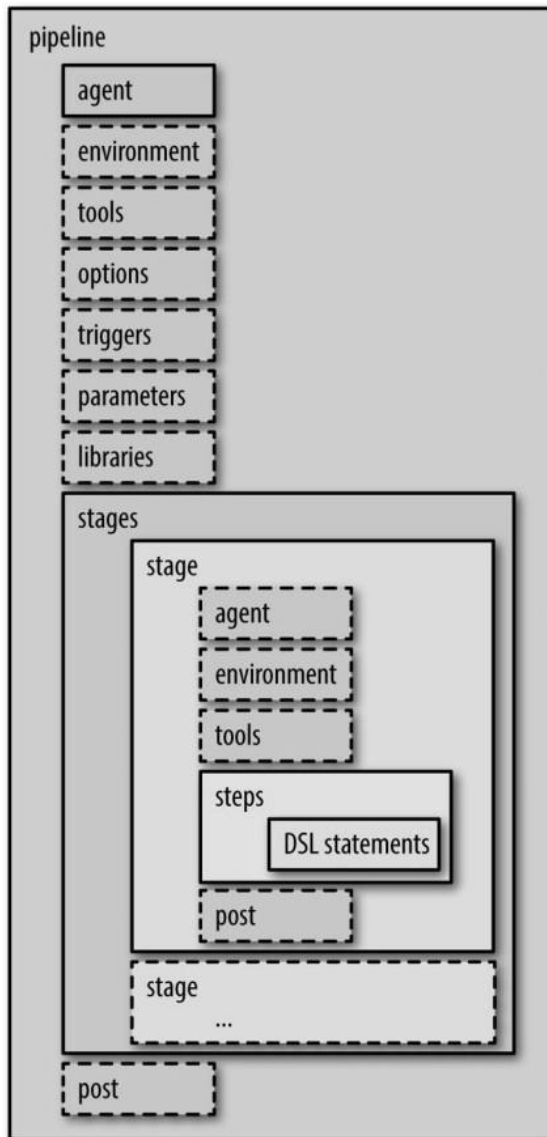


Figura 14. Estructura general de un pipeline declarativo. Adaptado de Jenkins 2: Up and Running (p. 237), por O. Laster, 2018, O'Reilly Media. Copyright 2018 por O'Reilly Media.

- **pipeline:** es la primera directiva de todo *pipeline* declarativo y define el bloque general del *pipeline* que contendrá en su interior las demás secciones y directivas.
- **agent:** es la directiva que especifica el agente o nodo que ejecutará el *pipeline* completo o alguno de sus etapas (*stage*). Esta directiva es obligatoria a nivel de *pipeline*, pero opcional a nivel de etapa (*stage*). Admite tres posibles argumentos:
 - **agent {any}:** indica que el *pipeline* o *stage* puede ejecutarse en cualquier nodo libre.

- **agent {none}**: señala que el *pipeline* general no se ejecutará en un nodo específico, lo que obliga a definir un agente en cada *stage*.
- **agent {label "<etiqueta del nodo>"}**: indica que el *pipeline* o *stage* puede ejecutarse en cualquier nodo que tenga la etiqueta indicada entre comillas.

```
pipeline {  
  agent {label 'linux'}
```

Figura 15. Ejemplo de uso de la directiva agent {}

- **environment**: esta directiva permite declarar una serie de variables de entorno bajo el formato clave-valor. El alcance de las variables depende de la ubicación de la directiva, pudiendo ser a nivel global o acotarse a un *stage*.

```
environment {  
  AN_ACCESS_KEY = credentials('my-predefined-secret-text')  
}
```

Figura 16. Ejemplo de uso de la directiva environment {}

- **tools**: esta directiva permite indicar qué herramientas se quieren utilizar en el *pipeline*, haciendo que Jenkins las instale automáticamente si no están presentes en el agente. Además, hace que las herramientas se encuentren accesibles desde la ruta del agente seleccionado, para que cualquier instrucción del *pipeline* pueda usarla sin indicar la ruta absoluta.

```
tools {  
  maven 'apache-maven-3.0.1'  
}
```

Figura 17. Ejemplo de uso de la directiva tools {}



Nota - ¿Qué requiere la directiva Tools?

Para poder usar la directiva *tools* se deben de configurar las herramientas previamente en la sección "Configuración Global de Herramientas". La directiva requiere que se indique un agente de ejecución mediante la directiva *agent*, o no tendrá efecto alguno en el *pipeline*.

- **options:** esta directiva permite realizar configuraciones globales del *pipeline*, como asignar marcas de tiempos en los logs (*timestamps*), habilitar o inhabilitar ejecuciones paralelas (*disableConcurrentBuilding*), establecer límites de tiempo de ejecución (*timeout*), entre otras opciones más. Todo esto sin la necesidad de realizar las configuraciones manualmente desde la interfaz gráfica de usuario, lo que permite tener un *pipeline* replicable.

```
options {  
    buildDiscarder(logRotator(numToKeepStr: '5')) // Mantener solo 5 builds  
    timeout(time: 30, unit: 'MINUTES')           // Cancelar si dura más de 30 min  
    disableConcurrentBuilds()                     // Evitar ejecuciones simultáneas  
    timestamps()                                  // Mostrar marcas de tiempo en los logs  
}
```

Figura 18. Ejemplo de uso de la directiva `options {}`

- **triggers:** esta directiva permite establecer algún criterio para que el *pipeline* se inicialice de manera automática. Actualmente existen tres formas de disparar la inicialización del *pipeline*:
 - **cron:** utiliza una [expresión Cron](#) para establecer un intervalo de tiempo para ejecutar el *pipeline*.
 - **pollSCM:** también utiliza una expresión Cron para establecer un intervalo de tiempo, pero en lugar de ejecutar el *pipeline*, Jenkins primero verifica que haya cambios en el repositorio; si existen, entonces ejecuta el *pipeline*.
 - **upstream:** permite ingresar una referencia a un *job* externo y, con base al estado de ese *job*, se puede o no ejecutar el *pipeline*.

```
triggers {  
    cron('H 2 * * *') // Todos los días a las 2 am  
    pollSCM('H/30 * * * *') // Revisar cambios cada 30 min  
    upstream(upstreamProjects: 'CompileJob,TestJob', threshold: hudson.model.Result.SUCCESS)  
    githubPush()  
}
```

Figura 19. Ejemplo de uso de la directiva `triggers {}`



Nota - ¿Y si utilizo Git?

Existe un *trigger* especial (`githubPush()`), que permite que un *pipeline* se ejecute automáticamente al recibir un *webhook* desde GitHub (o

Bitbucket). Para usarlo, debe configurarse previamente el *webhook* en el repositorio; así, cada vez que haya un *commit*, GitHub enviará una notificación a Jenkins, que realizará un *SCM polling* y ejecutará el *pipeline* si detecta cambios.

- **parameters:** esta directiva permite establecer valores de entrada para el *pipeline*. El valor de estos parámetros puede ser ingresado por el usuario al ejecutar el *pipeline* o mediante una llamada a alguna API. Sirve para que las ejecuciones de un mismo *pipeline* puedan variar según opciones, textos, o archivos que el usuario indique.

```
parameters {  
    string(name: 'APP_VERSION', defaultValue: '1.0.0', description: 'Versión de la aplicación')  
    booleanParam(name: 'RUN_TESTS', defaultValue: true, description: '¿Ejecutar pruebas?')  
    choice(name: 'DEPLOY_ENV', choices: ['dev', 'qa', 'prod'], description: 'Seleccionar entorno de despliegue')  
}
```

Figura 20. Ejemplo de uso de la directiva *parameters {}*

- **libraries:** esta es una directiva nueva y más compleja, pero de manera general, permite importar *shared libraries* en el *pipeline* para utilizar su código. Las *shared libraries* no son más que archivos con colecciones de código Groovy que contienen funciones reutilizables para diversos *pipelines*. Este concepto es más avanzado dentro de Jenkins y en esta guía no se profundiza.

```
libraries {  
    lib('mi-shared-lib')  
}  
stages {  
    stage('Ejemplo') {  
        steps {  
            echo "Usando librería compartida"  
            miFuncionCompartida()  
        }  
    }  
}
```

Figura 21. Ejemplo de uso de la directiva *libraries*

- **stages:** esta declarativa obligatoria agrupa una colección de uno o más directivas *stage*.

```
stages {  
    stage('Example 1') {}  
    stage('Example 2') {}  
    stage('Example 3') {}  
}
```

Figura 22. Ejemplo de uso de la directiva `stages {}`

- **stage:** dentro de la sección `stages` debe haber al menos una directiva `stage`. Cada `stage` tiene al menos un nombre y uno o más `steps`; además, opcionalmente cada `stage` puede contener directivas locales de `environment`, `tools` y `agent`.

```
stages {  
    stage('Compilación') {  
        agent { label 'linux' }  
        environment {  
            APP_ENV = 'dev'  
        }  
        tools {  
            maven 'Maven3'  
        }  
        steps {  
            echo "Compilando aplicación en entorno: ${APP_ENV}"  
            sh 'mvn clean install'  
        }  
    }  
}
```

Figura 23. Ejemplo de uso de la directiva `stage {}`

- **steps:** esta directiva es obligatoria e indica las tareas específicas que ocurren dentro de un `stage`. Cada `step` corresponde normalmente a una instrucción de Jenkins, y a este nivel de sintaxis es prácticamente equivalente a la de los *pipelines scripted*, con la diferencia de que en los declarativos no se permite incluir código Groovy arbitrario, sino únicamente pasos y constructores compatibles.
- **Statement del lenguaje específico de dominio (DSL):** aunque los enunciados DSL no son directiva propiamente, estas son las instrucciones o comandos predefinidos por Jenkins que representan acciones que el *pipeline* puede ejecutar, y normalmente se escriben dentro de las directivas `steps`.


```
pipeline {
  agent any
  stages {
    stage('Ejemplos de Steps') {
      steps {
        // 👉 1. Mensajes en consola
        echo "Iniciando el pipeline..."
        // 👉 2. Obtener código de un repositorio Git
        git url: 'https://github.com/ejemplo/mi-repo.git', branch: 'main'
        // 👉 3. Ejecutar comandos en la terminal (Linux)
        sh 'echo "Compilando aplicación..."'
        sh 'mvn clean install'
        // 👉 4. Archivar artefactos generados
        archiveArtifacts artifacts: 'target/*.jar', fingerprint: true
      }
    }
  }
}
```

Figura 24. Ejemplo de uso de las directivas steps y DLS



Nota - ¡Ayuda, soy nuevo!

Desde la versión 2.0 de Jenkins, se agregó una herramienta llamada [Snippet Generator](#). Esta ayuda a crear código para *steps* individuales; los *steps* que se pueden generar dependen directamente de los *plugins* que se tengan instalados.

- **post**: finalmente, esta directiva opcional siempre va al final del *pipeline* o al final de un *stage*, y contine directivas *steps* que se ejecutan si se cumple alguna condición definida dentro de la misma sección *post*.

```
post {
  success {
    echo "Pipeline completado con éxito 🎉"
  }
  failure {
    echo "Pipeline falló ❌"
  }
  always {
    echo "Esto se ejecuta siempre"
  }
}
```

Figura 25. Ejemplo de uso de la directiva post {}

Como pudimos observar a lo largo de este capítulo, los *pipelines* declarativos tienen una estructura y organización preestablecida y cuentan con secciones obligatorias que todo *pipeline* declarativo debe tener (*pipeline*, *agent*, *stages*, *stage* y *step*). Sin embargo, también permiten omitir o agregar otras directivas con el fin de modificar y

personalizar su comportamiento. Aunque hay que tener en cuenta que todas estas directivas e instrucciones DSL ya están predefinidas, y no es posible utilizar código Groovy directamente, lo que limita la flexibilidad del *pipeline* en comparación con los de tipo *scripted*.



Cuestionario 9. Directivas esenciales de los *pipelines* declarativos

Contesta las siguientes preguntas para evaluar los conocimientos adquiridos en la lectura previa y validar tus conocimientos sobre las directivas utilizadas en los *pipelines* declarativos.

1. Verdadero o falso: la directiva *agent* es opcional a nivel de *pipeline*, pero obligatoria a nivel de *stage*.
A) Verdadero. B) Falso.
2. Verdadero o falso: la directiva *enviroment* solo puede declararse a nivel global del *pipeline* y no dentro de *stages*.
A) Verdadero B) Falso
3. Verdadero o falso: la directiva *steps* puede contener código Groovy arbitrario dentro de *pipelines* declarativos.
A) Verdadero B) Falso
4. Verdadero o falso: la directiva *post* es opcional y siempre se coloca al final de un *pipeline* o *stage*.
A) Verdadero B) Falso
5. ¿Qué directiva define el bloque principal de un *pipeline* declarativo?

- A) *stages*. B) *pipeline*. C) *agent*. D) *tools*.

6. ¿Cuál de estas directivas permite ejecutar el *pipeline* automáticamente al recibir un *webhook* de GitHub?

- A) *triggers*. B) *pollSCM*. C) *githubPush()*. D) *cron*.

7. ¿Qué se necesita configurar previamente para poder usar *tools* correctamente en un *pipeline*?

- | | | | |
|---|-------------------------|---|-------------------------|
| A) Instalar los <i>plugins</i> de las herramientas. | B) Agente de ejecución. | C) Herramientas en la Configuración Global de Jenkins | D) Todas las anteriores |
|---|-------------------------|---|-------------------------|

8. ¿Qué permite importar la directiva *libraries* en el *pipeline* con el fin de reutilizar su código?

- A) Variables de entorno. B) Shared libraries. C) *triggers*. D) *stages*.

9. Cada *stage* dentro de la sección *stages* debe contener al menos un _____.

- A) *agent*. B) *step*. C) *tool*. D) *trigger*.

10. Un equipo de desarrollo quiere que su *pipeline* se ejecute automáticamente todos los días a las 2:00 a.m. ¿Qué directiva deberían utilizar para configurar este comportamiento?

R = _____.

11. Un equipo de desarrollo necesita que un *stage* realice algunas acciones solo si el *pipeline* falla o tiene éxito. ¿Qué directiva les ayudaría a lograr esto?

R = _____.

12. Un desarrollador novato debe configurar un *pipeline* de tal manera que pueda elegir el entorno de despliegue (*dev*, *qa*, *prod*) de su aplicación web cada vez que ejecuta el *pipeline*. ¿Qué directiva le permitirá esto?

R = _____.

13. ¿Cuáles son las cuatro directivas obligatorias para todo *pipelines* declarativo?

R = _____, _____, _____ y _____.

14. ¿Cuáles son las tres directivas opcionales que pueden estar a nivel de *pipeline* o de *stage*?

R = _____, _____ y _____.



Capítulo 5

5 Jenkins en entornos Linux

Contenido del capítulo

- 5.1. Comandos en Linux esenciales para el uso de Jenkins
- 5.2. Requisitos técnicos de un servidor de Jenkins
- 5.3. Instalación paso a paso de un servidor Jenkins en Ubuntu Server
 - 5.3.1. Preparación del entorno
 - 5.3.2. Configuración de la máquina virtual
 - 5.3.3. Instalación de Java en Ubuntu Server
 - 5.3.4. Instalación de Jenkins en Ubuntu Server
 - 5.3.5. Personalización inicial del entorno de Jenkins
 - 5.3.6. Ingreso a Jenkins por primera vez

5.1 Comandos en Linux esenciales para el uso de Jenkins



Lectura 13. Esenciales de Linux para Jenkins

Nivel básico

Objetivo: el estudiante recuerda el uso de los comandos `wget`, `systemctl` y `apt-get` en Linux.

Descripción

La lectura presenta los comandos básicos de distribuciones Linux basadas en Debian esenciales para la instalación de Jenkins, que se utilizarán en capítulos posteriores.

Posiblemente te estés preguntando cuál es la relación entre Linux y Jenkins. Aunque pareciera que no están relacionados, un servidor en Jenkins preparado para producción deberá alojarse y exponerse en la red, accediendo a él remotamente para su gestión. Es ahí donde aparece Linux.

Para no hablar de forma tan general, centraremos el discurso en Ubuntu Server, una variante del sistema operativo Ubuntu basado en el kernel de Linux, que proporciona una gestión remota eficiente a través de una interfaz de línea de comandos, optimizado para alojar sitios web, ejecutar aplicaciones y gestionar servicios de red (Carter, 2024).

Introducimos esta distribución particular de Linux, pues las secciones posteriores partirán de ello. Sin embargo, esta sección tiene como objetivo recordar tres comandos esenciales en Linux que serán de gran utilidad durante la instalación de Jenkins: **`wget`**, **`systemctl`** y **`apt-get`**.

El comando **`wget`** se utiliza para descargar archivos desde un servidor y soporta HTTP, HTTPS y FTP. Es ideal para conexiones lentas o inestables, pues si la descarga falla, seguirá intentando hasta completarla (GeeksforGeeks, 2024). Su sintaxis básica es la siguiente:

```
wget [opciones] [url]
```

Figura 26. Sintaxis general para el comando `wget`

En esta sintaxis previamente mostrada, [url] indica la dirección desde la cual se desea descargar el archivo y [opciones] hace referencia a alguno de los parámetros básicos, como `-b` para iniciar el proceso en segundo plano, `-i <archivo>` para leer la URL desde un archivo o `-t <número>` para definir el número máximo de intentos de descarga (GeeksforGeeks, 2024). Aquí, vale la pena resaltar que estos no son todos los parámetros que el comando puede tomar.

Ejemplos de su uso se muestran en la imagen siguiente, con la línea 1 presentando el uso más simple del comando, donde solo se hace la descarga del archivo especificado. La línea 2 muestra el uso del parámetro `-b` para una descarga en segundo plano. Y finalmente, la línea 3 muestra el uso del comando especificando 10 intentos de descarga con ayuda del parámetro `-t`.

```
1 wget http://example.com/sample.php
2 wget -b http://www.example.com/samplepage.php
3 wget -t 10 http://example.com/samplefile.tar.gz
```

Figura 27. Ejemplos de uso del comando `wget`

Siguiendo con **apt-get**, esta es una herramienta de línea de comandos utilizada en sistemas Linux basados en Debian (como Ubuntu server), que permite recuperar información y paquetes desde fuentes autorizadas, para instalar, actualizar o eliminar software (GeeksforGeeks, 2025a). La sintaxis general es la siguiente:

```
sudo apt-get [opciones] [comando] [paquete(s)]
```

Figura 28. Sintaxis general de la herramienta `apt-get`

En esta sintaxis previa, `sudo` es un comando que inicia la ejecución con privilegios de administrador, necesarios en este contexto pues la herramienta `apt-get` realiza modificaciones del sistema; [opciones] hace referencia a modificadores como `-y` (responder “sí” automáticamente) o `-s` (simular sin efectuar cambios); [comando] refiere a la acción que se quiere realizar con la herramienta, como `update` (actualiza la lista de paquetes desde los repositorios), `upgrade` (instala las versiones más recientes de los paquetes), `install` (instala o actualiza paquetes), `remove` (elimina paquetes), etc.; finalmente, [paquete(s)] hace referencia al nombre del paquete (o paquetes) objetivo, en caso de que [comando] así lo requiera (GeeksforGeeks, 2025a).

```
1 sudo apt-get update
2 sudo apt-get upgrade
3 sudo apt-get install vim
```

Figura 29. Ejemplos de uso de la herramienta *apt-get*

Las líneas 1, 2 y 3 del ejemplo anterior, presentan el uso de esta herramienta con los comandos *update*, *upgrade* e *install* previamente explicados. Como puede ver, el comando *install* sí requiere especificar el paquete (o paquetes en otros casos) que se desean instalar, mientras que los otros dos comandos, no lo requieren.

Aquí vale la pena mencionar que *apt-get* también puede ser mencionado y utilizado solamente como *apt*. Esta última es una versión más reciente de la herramienta *apt-get*, sin embargo, para fines prácticos de esta guía, puede que vea el uso de una u otra versión.

Por último, **systemctl** es una herramienta de línea de comandos utilizada para controlar el sistema y el gestor de servicios *systemd*. Permite gestionar servicios realizando acciones como iniciar, detener, reiniciar, habilitar o deshabilitar (GeeksforGeeks, 2025b). La sintaxis general de este comando es:

```
systemctl [comando] [unidad]
```

Figura 30. Sintaxis general de la herramienta *systemctl*

En esta sintaxis, *[comando]* indica la acción a realizar: *start* (iniciar servicios), *stop* (detener servicios), *restart* (reiniciar servicios), *status* (muestra el estado actual de un servicio), *enable* (habilitar el servicio para iniciarlo durante el arranque), *disable* (revertir la acción de *enable*) y *reboot* (reiniciar el sistema completamente). Mientras que *[unidad]* hace referencia al nombre del servicio o unidad sobre el que se aplica el comando (GeeksforGeeks, 2025c).

```
1 systemctl start jenkins
2 systemctl stop jenkins
3 systemctl status jenkins
```

Figura 31. Ejemplo de uso de la herramienta *systemctl*

Los ejemplos previos muestran el uso del comando para iniciar (línea 1), parar (línea 2) y ver el estado (línea 3) del servicio nombrado *jenkins*.

A forma de resumen para cerrar el capítulo, vale la destacar que:

1. `wget` es un comando que sirve para descargar archivos desde internet,
2. `apt-get` es una herramienta que permite gestionar software en el sistema y
3. `systemctl` es una herramienta para gestionar servicios del sistema.



Cuestionario 10. Esenciales de Linux para Jenkins

Contesta las siguientes preguntas para evaluar los conocimientos adquiridos en la lectura previa y valida tus conocimientos sobre comandos en Linux.

1. ¿Qué comando de Linux se utiliza para descargar archivos desde internet?
A) `wget`. B) `apt-get`. C) `systemctl`. D) Todos los anteriores.
2. Verdadero o falso: `systemctl` es una herramienta que sirve para gestionar servicios del sistema.
A) Verdadero. B) Falso.
3. ¿Qué comando se usa para gestionar software en distribuciones basadas en Debian?
A) `wget`. B) `apt-get`. C) `systemctl`. D) Todos los anteriores.
4. Selecciona la opción con la afirmación correcta:
A) `wget` administra servicios del sistema. B) `systemctl` instala paquetes en Linux. C) `apt-get` descarga archivos de internet. D) Ninguna de las anteriores.
5. ¿Qué comando utilizarías para descargar un archivo llamado `jenkins.war` desde <https://get.jenkins.io>?

- A) *wget*. B) *apt-get*. C) *systemctl*. D) Ninguno de los anteriores.

6. ¿Qué comando utilizarías para iniciar el servicio de Jenkins en Linux?

- A) *systemctl run jenkins*. B) *systemctl enable jenkins*. C) *systemctl start jenkins*. D) *systemctl status jenkins*.

7. ¿Qué comando utilizarías para instalar el paquete git usando *apt-get*?

- A) *apt-get install git*. B) *sudo apt-get install git*. C) *sudo apt-get update git*. D) *sudo-apt get upgrade*.

5.2 Requisitos Técnicos de un Servidor de Jenkins



Lectura 14. Requisitos técnicos de un servidor de Jenkins

Nivel básico

Objetivo: el estudiante enlista los requisitos técnicos recomendados para configurar un servidor de Jenkins en un sistema operativo Linux (Ubuntu Server).

Descripción

La lectura presenta los requisitos técnicos de hardware y software necesarios para instalar un servidor de Jenkins en un equipo con sistema operativo Linux (distribución Ubuntu). Se detallan las especificaciones relacionadas con el procesador, la memoria RAM, el almacenamiento, así como los programas y dependencias necesarias.

La gestión de la infraestructura para alojar un servidor de Jenkins es un tema muy amplio. Sin embargo, esta sección tiene como objetivo introducir al estudiante a este tema, presentando de forma general los requisitos de software y hardware necesarios para instalar Jenkins en un equipo y hacer que este funcione adecuadamente como Servidor de Jenkins.

Antes de profundizar en el tema, recordemos que Jenkins trabaja bajo un modelo conocido como maestro-trabajador. Esto es esencial, pues la información en este capítulo se basa en describir el entorno mínimo recomendado para alojar al nodo maestro de Jenkins: el encargado de orquestar los nodos trabajadores.



Nota - ¡Échame una mano!

Si bien aquí solamente explicamos los requisitos para configurar el servidor de Jenkins, los expertos recomiendan aprovechar los sistemas de ejecución distribuidos y la arquitectura propia de Jenkins para orquestar los *jobs* y optimizar el uso de recursos. Idealmente, el nodo maestro o servidor principal, debería estar destinado a gestionar los nodos trabajadores, repartir el trabajo entre estos y nada más.

Ahora bien, toda la información proporcionada en este capítulo es cierta para la versión 2.516.2 de Jenkins, que es la versión LTS (*long-term support*) de la herramienta en agosto de 2025. Esto no quiere decir que las características descritas dejen de ser ciertas para versiones anteriores o posteriores. Sin embargo, hay que tenerlo presente al leer esta guía, pues algunos de los requisitos podrían cambiar con otras versiones; principalmente aquellos relacionados con el software.

Comenzando con los requisitos de software necesarios para ejecutar Jenkins en un equipo, la documentación oficial de CD Foundation (2025b) menciona como único requisito de software tener una versión de Java 17 o 21 instalada.

Pasando a los requisitos de hardware, el mismo documento de CD Foundation (2025b), destaca como requisito mínimo tener 256MB de memoria RAM, aunque más de 2GB es lo que recomiendan; hablando de espacio en disco, un mínimo de 10GB libres en lo recomendable. Otro documento de CD Foundation (2025d), menciona que las exigencias de RAM pueden llegar hasta más de 70GB para servidores masivos de Jenkins y resaltan que el cálculo del hardware dependerá mucho de la demanda, como en cualquier otro servidor.



Nota - ¿Qué pasa cuando la carga de trabajo aumenta?

Esta es una pregunta muy válida y que, aunque no se responde aquí dado el alcance de esta sección, te recomendamos que consultes la

página en la documentación oficial titulada [architecting for scale](#), para conocer más al respecto.

A pesar de las diferencias previas, KodeKloud (s/f) presenta requisitos similares a los mencionados por CD Foundation (2025b), pues destacan como requisitos mínimos dos núcleos en CPU, 256MB de memoria RAM y 1GB en espacio de disco; sin embargo, estos requisitos suben a cuatro núcleos en CPU, 4GB de memoria RAM y al menos 50GB en espacio de disco como requisitos recomendados para servidores pequeños. Esto es curioso, pues estos requisitos coinciden con los de otra página en la documentación oficial de CD Foundation (2025e), donde mencionan estos requisitos de hardware particulares para la instalación de Jenkins en sistemas operativos basados en Linux.

Para este punto, es muy probable que los requisitos técnicos de hardware y software estén un poco difusos. Es importante destacar que estos requisitos dependerán mucho de la carga de trabajo a la que se someterá el servidor de Jenkins, así como el tipo de instalación, pues Jenkins puede instalarse en distintos ambientes, como Windows, Docker, Kubernetes, macOS, entre otros.

Aún con lo anterior, podemos resumir los requisitos de hardware y software para una instalación de Jenkins en sistemas operativos Linux:

Requisitos mínimos:

1. (software) Java 17 o 21
2. (hardware) 2 núcleos de CPU
3. (hardware) 256MB de memoria RAM
4. (hardware) 1GB de espacio en disco

Requisitos recomendados para un servidor pequeño:

1. (software) Java 17 o 21
2. (hardware) 4 núcleos de CPU
3. (hardware) al menos 4GB de memoria RAM
4. (hardware) al menos 50GB de espacio en disco

Fuera de estos requisitos de hardware y software, es recomendable tener acceso a un navegador web dentro del servidor, o tener dicho acceso de forma remota hacia el

servidor, pues Jenkins ofrece una interfaz web (expuesta por defecto en localhost en el puerto 8080) para gestionarlo. Si desea conocer más sobre esta interfaz web que proporciona Jenkins, consulte el capítulo “[Conociendo la interfaz gráfica de Jenkins](#)” de esta guía.



Cuestionario 11. Requisitos técnicos de un servidor de Jenkins.

Escribe en el siguiente cuadro los requisitos de hardware, software y de red recomendados para instalar Jenkins en un equipo con sistema operativo Linux y valida los conocimientos adquiridos.

1. ¿Cuántos núcleos de CPU son el requisito mínimo para realizar una instalación de Jenkins en sistemas operativos Linux?
A) 1 núcleo de CPU. B) 2 núcleos de CPU. C) 4 núcleos de CPU. D) 16 núcleos de CPU.
2. ¿Qué versión mínima de Java requiere Jenkins?
A) Java 8. B) Java 11. C) Java 17. D) Java 21.
3. ¿En qué puerto se expone Jenkins por defecto?
A) Puerto 3000. B) Puerto 80. C) Puerto 443. D) Puerto 8080.
4. Verdadero o falso: para ejecutar Jenkins, solamente se necesitan considerar los requisitos de hardware.
A) Verdadero. B) Falso.
5. ¿Cuánta memoria RAM es recomendable para realizar una instalación de Jenkins en sistemas operativos Linux?
A) 256MB. B) 256GB. C) 4GB. D) 70GB.

6. Verdadero o falso: los requisitos de hardware reales dependerán mucho de la carga de trabajo a la que se someterá el servidor de Jenkins.

A) Verdadero. B) Falso.

7. Verdadero o falso: al menos 256GB de espacio en disco son recomendables para una instalación de Jenkins en un servidor pequeño en sistemas operativos Linux.

A) Verdadero. B) Falso.

5.3 Instalación Paso a Paso de un Servidor Jenkins en Ubuntu Server



Práctica 1. ¿Cómo instalar Jenkins en Ubuntu Server?

Nivel avanzado

Objetivo: el estudiante demuestra la capacidad de instalar y configurar un servidor de Jenkins plenamente funcional en un entorno virtualizado con Linux (Ubuntu Server).

Prerrequisitos: navegador web (Chrome, Mozilla, etc.), software de virtualización (VirtualBox preferentemente) con un sistema operativo Linux funcional (Ubuntu Server) y un cliente SSH (BitViseSSH de preferencia).

Descripción

En esta práctica se explican detalladamente los pasos para instalar un servidor de Jenkins en un entorno Linux (Ubuntu Server). Así como las configuraciones iniciales recomendadas y las acciones básicas para poder acceder al servidor por primera vez.

5.3.1 Preparación del entorno

Como se mencionó en secciones previas, Jenkins es una aplicación autónoma (*standalone app*), lo que significa que su instalación y funcionamiento no requiere de servicios o servidores externos. Además, al ser de código abierto, cuenta con compatibilidad con casi cualquier sistema operativo o plataforma que soporte Java (Jenkins está escrito en Java). En la generalidad, lo único que se requiere para utilizar Jenkins es un sistema operativo que soporte alguna versión de Java 17 o 21 y un navegador web.

Para esta práctica se utilizarán las siguientes herramientas:

- VirtualBox versión 7.2
- BitViseSSH Client versión 9.33
- Sistema Operativo Linux-Distribución Ubuntu Server versión 24.04.03 LTS.
- Java 21 distribución Eclipse Temurin.
- Navegador Web Chrome de 64 bits.

Antes de comenzar a instalar Jenkins, es necesario contar con el Navegador web Chrome (preferentemente) instalado y funcionando en nuestra máquina anfitriona, en caso de necesitar ayuda con esto, puede consultar el siguiente recurso: [Tutorial: Instalar Chrome](#).

Además, es necesario contar con VirtualBox correctamente instalado y funcionando, en caso de necesitar ayuda con esto, puedes consultar los siguientes recursos: [Video-Tutorial: Instalar VirtualBox 7.0](#) o [Tutorial: Instalar VirtualBox](#).

Por otra parte, también es necesario contar con un Servidor Ubuntu correctamente instalado y funcionando en una máquina virtual, en caso de necesitar ayuda con esto, puedes consultar los siguientes recursos: [Video-Tutorial: Instalar Ubuntu Server en VirtualBox](#) o [Tutorial: Instalar Ubuntu Server en VirtualBox](#).

Debido a que Ubuntu Server no tiene interfaz gráfica, el acceso al servidor de Jenkins se realizará desde la máquina anfitriona. Aunque es posible instalar Jenkins directamente en el host o utilizar una distribución de Linux con Interfaz gráfica, en esta práctica se decidió trabajar de esta manera para simular un escenario más cercano a una implementación real, en un equipo de desarrollo, donde Jenkins se

despliega en un servidor dedicado y los miembros del equipo se conectan remotamente para gestionarlo.

5.3.2 Configuración de la máquina virtual

Omita estos pasos si no está utilizando una máquina virtual para alojar el servidor de Jenkins.

Al utilizar Jenkins dentro de una máquina virtual, es necesario habilitar el modo de conexión de red “*bridge*”, para acceder posteriormente a la máquina virtual desde la maquina anfitriona, utilizando la IP de la máquina virtual.

Para esto, sigue los pasos a continuación:

1. Abre VirtualBox y busca tu máquina virtual de Ubuntu Server. Da clic en configuración.

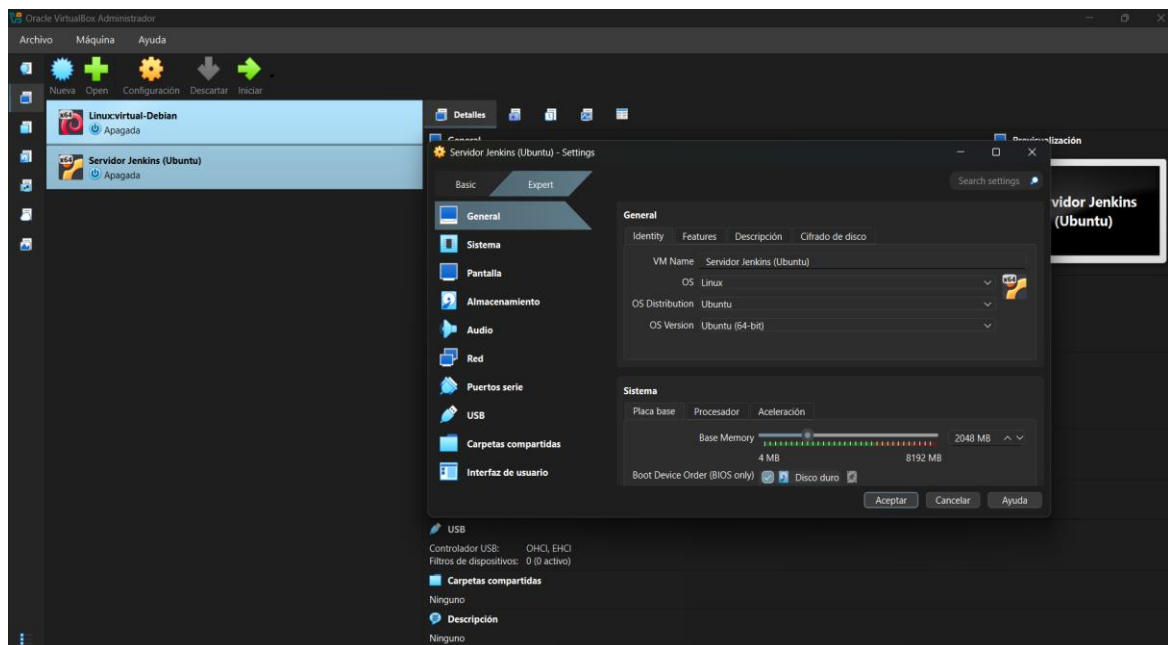


Figura 32. Pantalla de configuración de VirtualBox

2. Ve a la sección de Red, marca la casilla para habilitar el “Adaptador de red 1”, en el campo “conectado a” selecciona “adaptador puente” y en el campo “nombre” selecciona tu tarjeta de red de tu equipo *host* con la que tienes acceso a Internet. Finalmente guarda los cambios.

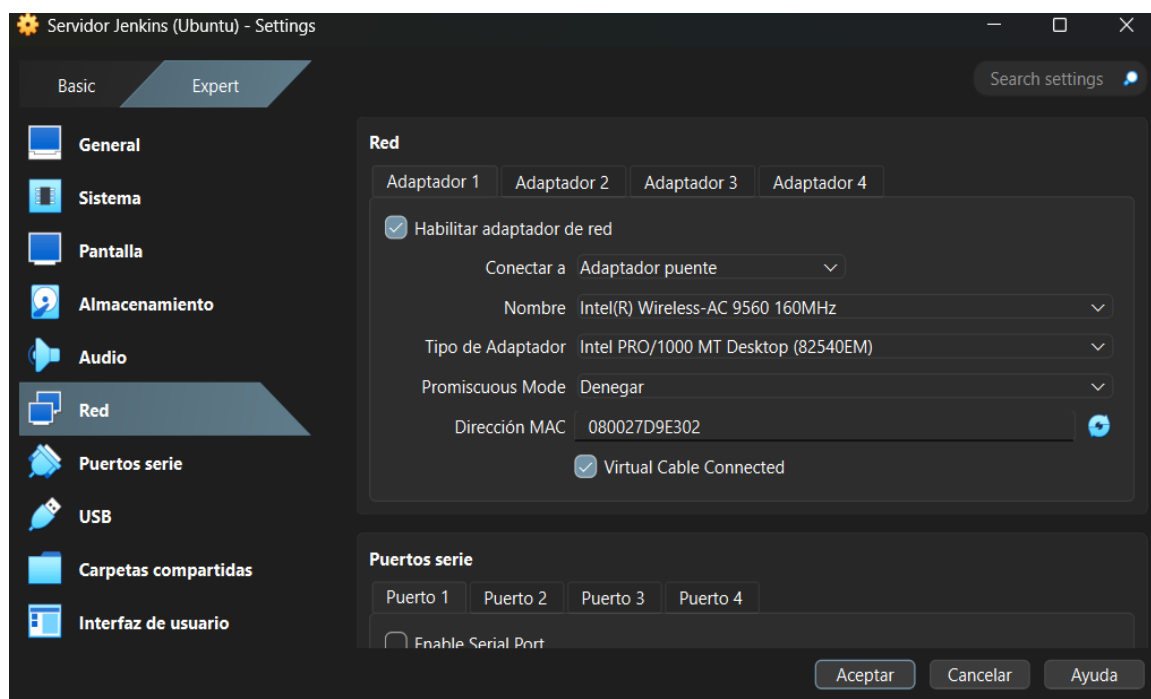


Figura 33. Pantalla de configuración de Red de VirtualBox

- Después, inicia la máquina virtual. Inicia sesión en el servidor Ubuntu, e ingresa el comando `ip a`, para consultar la IP de la máquina virtual, que normalmente tiene el formato `192.168.X.X`. Anota la dirección en otro lugar para utilizarla posteriormente.

```
jenkins@servidorjenkins:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:d9:e3:02 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.78/24 metric 100 brd 192.168.1.255 scope global dynamic enp0s3
        valid_lft 86387sec preferred_lft 86387sec
    inet6 2806:10a6:c:e7ed:a00:27ff:fed9:e302/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 2591987sec preferred_lft 2591987sec
    inet6 fe80::a00:27ff:fed9:e302/64 scope link
        valid_lft forever preferred_lft forever
jenkins@servidorjenkins:~$ _
```

Figura 34. Resultado de la ejecución del comando `ip a`

Por defecto, VirtualBox no permite copiar y pegar directamente en la terminal de Ubuntu Server. Aunque existen formas para habilitar esto (habilitar el portapapeles compartido, o instalar los complementos de invitado), estas soluciones no siempre funcionan. Bajo este contexto, la solución más sencilla y efectiva para trabajar de forma más cómoda en este entorno, es conectarse mediante SSH al servidor desde nuestro equipo anfitrión. Para ello:

1. Primero hay que instalar el servicio de SSH en la máquina virtual con el comando: **`sudo apt install openssh-server -y`**.

```
jenkins@servidorjenkins:~$ sudo apt install openssh-server -y
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
openssh-server ya está en su versión más reciente (1:9.6p1-3ubuntu13.13).
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 0 no actualizados.
jenkins@servidorjenkins:~$
```

Figura 35. Instalar Openssh-Server

2. Posteriormente hay que instalar, en nuestra máquina anfitriona, el software BitViseSSH Client para conectarnos al servidor Ubuntu. Si necesitas ayuda para descargar este programa, puedes consultar los siguientes recursos: [Tutorial Instalar: BitViseSSH Client](#) o [Video-Tutorial: Instalar BitViseSSH Client](#)
3. Y finalmente, simplemente hay que conectarse a mediante BitViseSSH Client a la IP de nuestra máquina virtual, e ingresar el usuario y contraseña de esta. Así podremos copiar y pegar en la línea de comandos del servidor, lo que será muy útil para la instalación de JAVA.

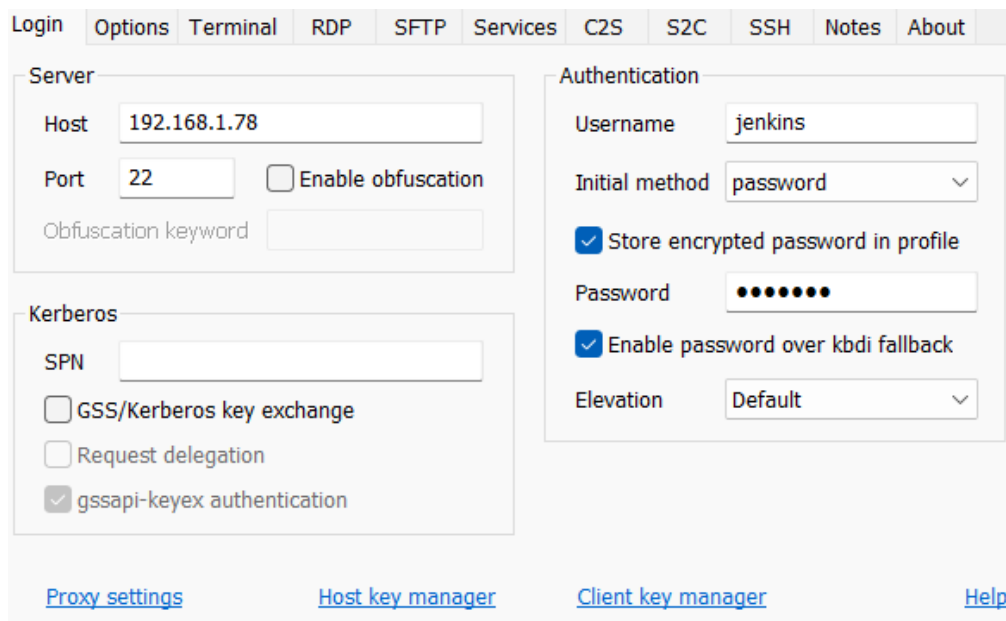


Figura 36. Pantalla de inicio de BitViseSSH

5.3.3 Instalación de JAVA (Eclipse Temurin) en Ubuntu Server.

Ahora que ya está configurada la conexión de red de la máquina virtual, procederemos a instalar la versión 21 de Java (recuerda verificar las versiones compatibles de Java con la versión de Jenkins en tu equipo). Para esta práctica, se utilizará la distribución

Eclipse Temurin debido a que es una versión gratuita y de código abierto como Jenkins. Pero antes, por buenas prácticas es recomendable actualizar el servidor.

Como recomendación sugerimos copiar y pegar los comandos requeridos para instalar Temurin, debido a que algunos son bastantes largos. Comencemos:

1. Ejecuta el comando **`sudo apt update`**, para actualizar toda la información sobre los paquetes disponibles configurados en el sistema y asegurarse que la máquina conoce la versión más reciente disponible de Java y de Jenkins.

```
jenkins@servidorjenkins:~$ sudo apt update
[sudo] password for jenkins:
Obj:1 http://mx.archive.ubuntu.com/ubuntu noble InRelease
Des:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Des:3 http://mx.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Obj:4 http://mx.archive.ubuntu.com/ubuntu noble-backports InRelease
Des:5 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [8.708 B]
Des:6 http://mx.archive.ubuntu.com/ubuntu noble/main Translation-es [325 kB]
Des:7 http://mx.archive.ubuntu.com/ubuntu noble/restricted Translation-es [816 B]
Des:8 http://mx.archive.ubuntu.com/ubuntu noble/universe Translation-es [1.371 kB]
Des:9 http://mx.archive.ubuntu.com/ubuntu noble/multiverse Translation-es [63,1 kB]
Descargados 2.020 kB en 24s (84,0 kB/s)
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se pueden actualizar 5 paquetes. Ejecute «apt list --upgradable» para verlos.
jenkins@servidorjenkins:~$ _
```

Figura 37. Actualizar directorio de paquetes

2. Posteriormente, hay que asegurarse de que los paquetes necesarios están disponibles mediante el comando **`sudo apt install -y wget apt-transport-https gpg`**. Este comando instala los paquetes `wget`, para descargar archivos de internet; `apt-transport-https`, para instalar paquetes de repositorios https y no solo http; y `gpg`, para verificar las firmas de los paquetes.

```
jenkins@servidorjenkins:~$ sudo apt install -y wget apt-transport-https gpg
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
wget ya está en su versión más reciente (1.21.4-1ubuntu4.1).
fijado wget como instalado manualmente.
gpg ya está en su versión más reciente (2.4.4-2ubuntu17.3).
fijado gpg como instalado manualmente.
Se instalarán los siguientes paquetes NUEVOS:
  apt-transport-https
0 actualizados, 1 nuevos se instalarán, 0 para eliminar y 5 no actualizados.
Se necesita descargar 3.970 B de archivos.
Se utilizarán 36,9 kB de espacio de disco adicional después de esta operación.
Des:1 http://mx.archive.ubuntu.com/ubuntu noble-updates/universe amd64 apt-transport-https all 2.8.3 [3.970 B]
Descargados 3.970 B en 1s (7.868 B/s)
Seleccionando el paquete apt-transport-https previamente no seleccionado.
(Leyendo la base de datos ... 84075 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../apt-transport-https_2.8.3_all.deb ...
Desempaquetando apt-transport-https (2.8.3) ...
Configurando apt-transport-https (2.8.3) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
jenkins@servidorjenkins:~$
```

Figura 38. Instalar paquetes más nuevos

- Posteriormente, hay que usar el comando **`sudo wget -qO - https://packages.adoptium.net/artifactory/api/gpg/key/public | gpg --dearmor | sudo tee /etc/apt/trusted.gpg.d/adoptium.gpg > /dev/null`**, para descargar la llave pública GPG de Adoptium, convertirla en formato binario (.gpg), y guardarla en el sistema como una clave confiable para montar APT. Esto es necesario para poder agregar el repositorio de Adoptium y poder instalar Java de manera segura.

```
tristan@tristan-MS-7A40:~$ sudo wget -qO - https://packages.adoptium.net/artifactory/
api/gpg/key/public | gpg --dearmor | sudo tee /etc/apt/trusted.gpg.d/adoptium.gpg > /
dev/null
tristan@tristan-MS-7A40:~$
```

Figura 39. Descargar llave pública de Adoptium

- Luego se debe utilizar el comando **`sudo echo "deb https://packages.adoptium.net/artifactory/deb $(awk -F= '/^VERSION_CODENAME/{print$2}' /etc/os-release) main" | sudo tee /etc/apt/sources.list.d/adoptium.list`**, para añadir el repositorio oficial de Adoptium a la lista de fuentes de APT de la máquina virtual.

```
tristan@tristan-MS-7A40:~$ sudo echo "deb https://packages.adoptium.net/artifactory/d
eb $(awk -F= '/^VERSION_CODENAME/{print$2}' /etc/os-release) main" | sudo tee /etc/ap
t/sources.list.d/adoptium.list
deb https://packages.adoptium.net/artifactory/deb jammy main
tristan@tristan-MS-7A40:~$
```

Figura 40. Agregar repositorios de Adoptium

5. Finalmente, volvemos actualizar la lista de paquete con el comando **`sudo apt update`**, para evitar posibles errores durante la instalación de JAVA; mediante el comando **`sudo apt install temurin-21-jdk`**, se instala JAVA Temurin. Es importante mencionar que puedes cambiar el número "21" a otra versión de JAVA, pero recuerda revisar las compatibilidad con Jenkins primero.

```
jenkins@servidorjenkins:~$ sudo apt install temurin-21-jdk
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  adoptium-ca-certificates fonts-dejavu-extra java-common liboss4-salsa-asound2
  liboss4-salsa2 libxi6 libxrender1 libxtst6 p11-kit p11-kit-modules x11-common
Paquetes sugeridos:
  default-jre
Se instalarán los siguientes paquetes NUEVOS:
  adoptium-ca-certificates fonts-dejavu-extra java-common liboss4-salsa-asound2
  liboss4-salsa2 libxi6 libxrender1 libxtst6 p11-kit p11-kit-modules temurin-21-jdk
  x11-common
0 actualizados, 12 nuevos se instalarán, 0 para eliminar y 4 no actualizados.
Se necesita descargar 179 MB de archivos.
Se utilizarán 371 MB de espacio de disco adicional después de esta operación.
¿Desea continuar? [S/n] s
Des:1 http://mx.archive.ubuntu.com/ubuntu noble-updates/main amd64 p11-kit-modules am
d64 0.25.3-4ubuntu2.1 [328 kB]
```

Figura 41. Instalar Java Temurin

6. Podemos verificar que la instalación fue correcta mediante el comando **`java -version`**.

```
jenkins@servidorjenkins:~$ java -version
openjdk version "21.0.8" 2025-07-15 LTS
OpenJDK Runtime Environment Temurin-21.0.8+9 (build 21.0.8+9-LTS)
OpenJDK 64-Bit Server VM Temurin-21.0.8+9 (build 21.0.8+9-LTS, mixed mode, sharing)
jenkins@servidorjenkins:~$
```

Figura 42. verificar la versión instalada de Java

5.3.4 Instalación de Jenkins en Ubuntu Server

Una vez que se tiene instalado JAVA en la máquina virtual, ya es posible instalar Jenkins. Actualmente existen dos distribuciones de Jenkins, la distribución con soporte a largo plazo, y la distribución con actualizaciones semanales. Para servidores de producción es recomendable seleccionar la versión con soporte a largo plazo, ya que es la versión más estable del momento.

1. Al igual que con Java, es necesario descargar la clave GPG de Jenkins y guardarla en el sistema mediante el comando **`sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key`**, para luego utilizarla para verificar la autenticidad de los paquetes de Jenkins que se instalen mediante **`apt`**.

```
jenkins@servidorjenkins:~$ sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc https://
/pkg.jenkins.io/debian-stable/jenkins.io-2023.key
--2025-09-09 23:11:17-- https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
Resolving pkg.jenkins.io (pkg.jenkins.io)... 2a04:4e42:2e::645, 151.101.198.133
Connecting to pkg.jenkins.io (pkg.jenkins.io)|2a04:4e42:2e::645|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3175 (3,1K) [application/pgp-keys]
Saving to: '/etc/apt/keyrings/jenkins-keyring.asc'

/etc/apt/keyrings/jen 100%[=====] 3,10K --.-KB/s in 0s

2025-09-09 23:11:17 (62,6 MB/s) - '/etc/apt/keyrings/jenkins-keyring.asc' saved [3175
/3175]

jenkins@servidorjenkins:~$
```

Figura 43. Descargar clave GPG de Jenkins

- Posteriormente hay que agregar el repositorio oficial de Jenkins al sistema de la máquina virtual mediante el comando **echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]" **
**https://pkg.jenkins.io/debian-stable binary/ | sudo tee **
/etc/apt/sources.list.d/jenkins.list > /dev/null, utilizando la clave GPG descargada, lo que permitirá instalar Jenkins de manera segura con APT.

```
jenkins@servidorjenkins:~$ echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc
]" \
> https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
> /etc/apt/sources.list.d/jenkins.list > /dev/null
jenkins@servidorjenkins:~$
```

Figura 44. Agregar repositorio de Jenkins

- Finalmente, hay que actualizar una vez más la lista de paquetes con el comando **sudo apt update**, para verificar que el repositorio de Jenkins aparece en los resultados. Con ello, instalamos Jenkins con el comando **sudo apt install jenkins**.

```
jenkins@servidorjenkins:~$ sudo apt-get update
Obj:1 http://mx.archive.ubuntu.com/ubuntu noble InRelease
Obj:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Obj:3 http://mx.archive.ubuntu.com/ubuntu noble-updates InRelease
Ign:4 https://pkg.jenkins.io/debian-stable binary/ InRelease
Obj:5 http://mx.archive.ubuntu.com/ubuntu noble-backports InRelease
Des:6 https://pkg.jenkins.io/debian-stable binary/ Release [2.044 B]
Des:7 https://pkg.jenkins.io/debian-stable binary/ Release.gpg [833 B]
Des:8 https://pkg.jenkins.io/debian-stable binary/ Packages [29,7 kB]
Des:9 https://packages.adoptium.net/artifactory/deb noble InRelease [7.501 B]
Descargados 40,0 kB en 1s (28,4 kB/s)
Leyendo lista de paquetes... Hecho
jenkins@servidorjenkins:~$
```

Figura 45. Actualizar paquetes del servidor

```
jenkins@servidorjenkins:~$ sudo apt-get install jenkins
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  net-tools
Se instalarán los siguientes paquetes NUEVOS:
  jenkins net-tools
0 actualizados, 2 nuevos se instalarán, 0 para eliminar y 4 no actualizados.
Se necesita descargar 87,3 MB de archivos.
Se utilizarán 88,4 MB de espacio de disco adicional después de esta operación.
¿Desea continuar? [S/n] s
Des:2 http://mx.archive.ubuntu.com/ubuntu noble-updates/main amd64 net-tools amd64 2.
10-0.1ubuntu4.4 [204 kB]
Des:1 https://pkg.jenkins.io/debian-stable binary/ jenkins 2.516.2 [87,1 MB]
Ign:1 https://pkg.jenkins.io/debian-stable binary/ jenkins 2.516.2
Des:1 https://pkg.jenkins.io/debian-stable binary/ jenkins 2.516.2 [87,1 MB]
14% [1 jenkins 4.184 kB/87,1 MB 5%] 15,9 kB/s 1h 26min 36s
```

Figura 46. Instalar Jenkins

- Una vez finalizada la instalación, podemos verificar el funcionamiento de Jenkins con el comando **systemctl --full status jenkins**, que debe mostrar el estatus activo:

```
jenkins@servidorjenkins:~$ systemctl --full status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabl>
   Active: active (running) since Wed 2025-09-10 00:55:37 UTC; 1min 33s ago
     Main PID: 4618 (java)
        Tasks: 43 (limit: 2266)
      Memory: 524.5M (peak: 526.0M)
         CPU: 23.947s
       CGroup: /system.slice/jenkins.service
              └─4618 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenk>
```

Figura 47. Comprobación de la instalación de Jenkins



Nota - ¿Y si Jenkins no enciende?

De manera predeterminada, Jenkins utiliza el puerto 8080, el cual generalmente se encuentra libre. Sin embargo, si el servicio no logra iniciarse justo después de la instalación, es recomendable verificar que el puerto 8080 no esté ocupado ejecutando el comando **sudo ss -tulnp | grep 8080**.

5.3.5 Personalización inicial del entorno de Jenkins (opcional)

Hasta este punto Jenkins ya se encuentra instalado y en ejecución, pero antes de continuar, es recomendable realizar algunas configuraciones en el entorno de ejecución, con el fin de mejorar la organización y administración de los *plugins*, así como la propia ejecución de Jenkins.

1. Antes de editar cualquier archivo de configuración, es recomendable detener el servicio de Jenkins con el comando **`sudo systemctl stop jenkins`**, y verificarlo con el comando **`systemctl --full status Jenkins`**.

```
jenkins@servidorjenkins:~$ sudo systemctl stop jenkins
[sudo] password for jenkins:
jenkins@servidorjenkins:~$ systemctl --full status jenkins
o jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabl
   Active: inactive (dead) since Wed 2025-09-10 00:59:50 UTC; 5s ago
   Duration: 4min 13.489s
   Process: 4618 ExecStart=/usr/bin/jenkins (code=exited, status=143)
   Main PID: 4618 (code=exited, status=143)
   Status: "Jenkins stopped"
   CPU: 24.847s
```

Figura 48. Detener el servicio de Jenkins

2. Con el servicio de Jenkins detenido, ya es seguro editar el archivo de configuración de arranque con el comando **`sudo systemctl edit jenkins`**, que abrirá un ventana con el archivo de configuración "override.conf". Este sirve para sobrescribir de manera segura las configuraciones predeterminadas de Jenkins.

```
/etc/systemd/system/jenkins.service.d/.#override.conf8e521fe4e8383
### Editing /etc/systemd/system/jenkins.service.d/override.conf
### Anything between here and the comment below will become the contents of the drop

### Edits below this comment will be discarded

### /usr/lib/systemd/system/jenkins.service
# #
# # This file is managed by systemd(1). Do NOT edit this file manually!
# # To override these settings, run:
# #
# #     systemctl edit jenkins
# #
# # For more information about drop-in files, see:
# #
# #     https://www.freedesktop.org/software/systemd/man/systemd.unit.html
# #
#
```

Figura 49. Archivo de configuración de Jenkins

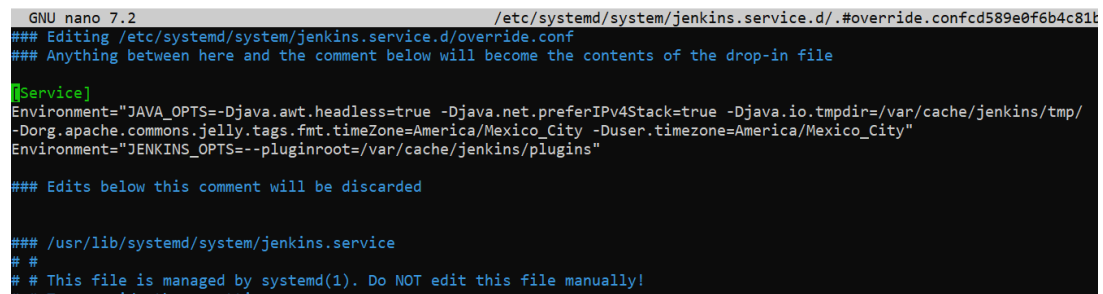
3. Dentro del archivo, la única sección que se toma en cuenta es la que está entre los comentarios 2 y tres. Todo lo que este fuera de ese espacio, es descartado por el servicio de Jenkins (esto se menciona en los comentarios mencionados). Dentro de este archivo, vamos a ingresar la siguiente configuración, sin saltos de línea, ya que cada declaración ***Enviroment*** debe ir en la misma línea:

[Service]

```

Environment="JAVA_OPTS=-Djava.awt.headless=true -
Djava.net.preferIPv4Stack=true -Djava.io.tmpdir=/var/cache/jenkins/tmp/ -
Dorg.apache.commons.jelly.tags.fmt.timeZone=America/Mexico_City -
Duser.timezone=America/Mexico_City"
Environment="JENKINS_OPTS=--pluginroot=/var/cache/jenkins/plugins"

```



```

GNU nano 7.2 /etc/systemd/system/jenkins.service.d/override.conf
## Editing /etc/systemd/system/jenkins.service.d/override.conf
## Anything between here and the comment below will become the contents of the drop-in file

[Service]
Environment="JAVA_OPTS=-Djava.awt.headless=true -Djava.net.preferIPv4Stack=true -Djava.io.tmpdir=/var/cache/jenkins/tmp/
-Dorg.apache.commons.jelly.tags.fmt.timeZone=America/Mexico_City -Duser.timezone=America/Mexico_City"
Environment="JENKINS_OPTS=--pluginroot=/var/cache/jenkins/plugins"

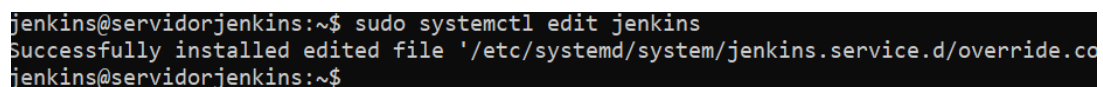
## Edits below this comment will be discarded

## /usr/lib/systemd/system/jenkins.service
# #
# # This file is managed by systemd(1). Do NOT edit this file manually!

```

Figura 50. Archivo de configuración modificado

Esta configuración asegura que Jenkins se pueda ejecutar en servidores sin interfaz gráfica como Ubuntu Server, que Jenkins utilice IPv4 en lugar de IPv6 y define la zona horaria de Ciudad de México. Además, crea un directorio para guardar los archivos temporales de Jenkins y que no se mezclen con los del sistema; finalmente cambia la ruta en donde se almacenan los *plugins*, para no guardarlos en la carpeta raíz de Jenkins. Guarda la configuración con la combinación de teclas **CTRL+O** y **Enter**.



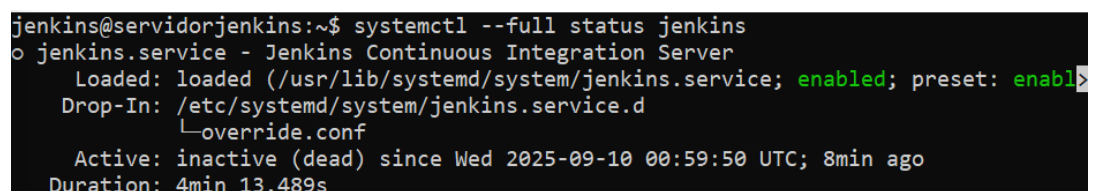
```

jenkins@servidorjenkins:~$ sudo systemctl edit jenkins
Successfully installed edited file '/etc/systemd/system/jenkins.service.d/override.conf'
jenkins@servidorjenkins:~$

```

Figura 51. Guardar archivos de configuraciones

- Una vez modificado el archivo de configuración, podemos verificar los cambios guardados con el comando **systemctl --full status jenkins**, donde se puede apreciar que en el campo *Drop-in* ahora aparece el archivo que recién se modificó (*override.conf*).



```

jenkins@servidorjenkins:~$ systemctl --full status jenkins
o jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Drop-In: /etc/systemd/system/jenkins.service.d
            └─override.conf
   Active: inactive (dead) since Wed 2025-09-10 00:59:50 UTC; 8min ago
   Duration: 4min 13.489s

```

Figura 52. Verificar configuraciones aplicadas a Jenkins

5. Antes de reiniciar el servicio, es necesario crear la carpeta para archivos temporales que se indicó en la configuración, esto con el comando **`sudo mkdir -p /var/cache/jenkins/tmp`**. Posteriormente es necesario darle los permisos adecuados con el comando **`sudo chown -R jenkins:jenkins /var/cache/jenkins/tmp`**.

```
jenkins@servidorjenkins:~$ sudo mkdir -p /var/cache/jenkins/tmp
jenkins@servidorjenkins:~$ sudo chown -R jenkins:jenkins /var/cache/jenkins/tmp
jenkins@servidorjenkins:~$
```

Figura 53. Crear carpeta de archivos temporales

6. Finalmente, hay que verificar que las configuraciones son adecuadas con el comando **`systemd-analyze verify jenkins.service`**. Este hace que el sistema analice el servicio Jenkins para asegurarse que es funcional y que no habría problema en iniciarlo. Este comando no devuelve nada a menos que exista un problema.

```
jenkins@servidorjenkins:~$ systemd-analyze verify jenkins.service
jenkins@servidorjenkins:~$
```

Figura 54. Verificar integridad del servicio de Jenkins

7. Con todo configurado y verificado, podemos iniciar el servicio con el comando **`systemctl start jenkins`**; Este puede tardar unos minutos en activar el servicio. Se puede verificar el *status* con el comando **`systemctl --full status jenkins`**. Aquí podremos ver reflejadas las configuraciones realizadas previamente.

```
jenkins@servidorjenkins:~$ sudo systemctl start jenkins
jenkins@servidorjenkins:~$ sudo systemctl --full status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabl>
   Drop-In: /etc/systemd/system/jenkins.service.d
            └─override.conf
   Active: active (running) since Wed 2025-09-10 01:26:18 UTC; 28s ago
     Main PID: 5733 (java)
       Tasks: 48 (limit: 2266)
      Memory: 358.5M (peak: 366.7M)
         CPU: 19.708s
    CGroup: /system.slice/jenkins.service
            └─5733 /usr/bin/java -Djava.awt.headless=true -Djava.net.preferIPv4Stac>

sep 10 01:25:29 servidorjenkins jenkins[5733]: Jenkins initial setup is required. An>
sep 10 01:25:29 servidorjenkins jenkins[5733]: Please use the following password to >
sep 10 01:25:29 servidorjenkins jenkins[5733]: 823821b8514640e6bd4c910a3b13508c
sep 10 01:25:29 servidorjenkins jenkins[5733]: This may also be found at: /var/lib/j>
sep 10 01:25:29 servidorjenkins jenkins[5733]: <
sep 10 01:25:29 servidorjenkins jenkins[5733]: <
sep 10 01:25:29 servidorjenkins jenkins[5733]: <
sep 10 01:25:29 servidorjenkins jenkins[5733]: <
sep 10 01:26:18 servidorjenkins jenkins[5733]: 2025-09-10 01:26:18.769+0000 [id=39] >
sep 10 01:26:18 servidorjenkins jenkins[5733]: 2025-09-10 01:26:18.801+0000 [id=30] >
sep 10 01:26:18 servidorjenkins systemd[1]: Started jenkins.service - Jenkins Contin>

jenkins@servidorjenkins:~$
```

Figura 55. Consultar estado del servicio Jenkins



Nota - ¿Qué parámetros utiliza Jenkins para funcionar?

Como datos extra, es importante conocer los parámetros que utiliza *systemd* para controlar el servicio de Jenkins, esto se puede consultar mediante el comando: ***systemctl show jenkins***.

5.3.6 Ingreso a Jenkins por primera vez

Una vez que se ha instalado Jenkins correctamente, se realizaron las configuraciones de personalización iniciales (opcionales) y el servicio se está ejecutando correctamente, es momento de ingresar a la instancia de Jenkins mediante el navegador web.

1. Cuando se instala Jenkins por primera vez, este comienza bloqueado, por lo que hay que utilizar una contraseña para desbloquearlo. Esta contraseña se encuentra en el archivo `/var/lib/jenkins/secrets/initialAdminPassword`, y se puede recuperar con el comando ***sudo cat /var/lib/jenkins/secrets/initialAdminPassword***; o también se puede consultar con el comando ***systemctl --full status jenkins***. En el bloque inferior nos indica que se requiere una instalación inicial y que la contraseña es la siguiente:

823821b8514640e6bd4c910a3b13508c (distinta para cada servidor de Jenkins)

```
jenkins.service - Jenkins Continuous Integration Server
  Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabl>
  Drop-In: /etc/systemd/system/jenkins.service.d
           └─override.conf
  Active: active (running) since Wed 2025-09-10 01:26:18 UTC; 28s ago
  Main PID: 5733 (java)
  Tasks: 48 (limit: 2266)
  Memory: 358.5M (peak: 366.7M)
  CPU: 19.708s
  CGroup: /system.slice/jenkins.service
           └─5733 /usr/bin/java -Djava.awt.headless=true -Djava.net.preferIPv4Stac>

sep 10 01:25:29 servidorjenkins jenkins[5733]: Jenkins initial setup is required. An>
sep 10 01:25:29 servidorjenkins jenkins[5733]: Please use the following password to >
sep 10 01:25:29 servidorjenkins jenkins[5733]: 823821b8514640e6bd4c910a3b13508c
```

Figura 56. Consultar contraseña de Jenkins

2. Copiamos la contraseña y, en nuestra máquina anfitriona, utilizando el navegador web Chrome, ingresamos a la dirección IP de nuestra máquina virtual (la misma dirección usada para conectarse por SSH), en el puerto 8080 (Jenkins se ejecuta por default en este puerto, pero se puede modificar en los archivos de configuración antes editados).

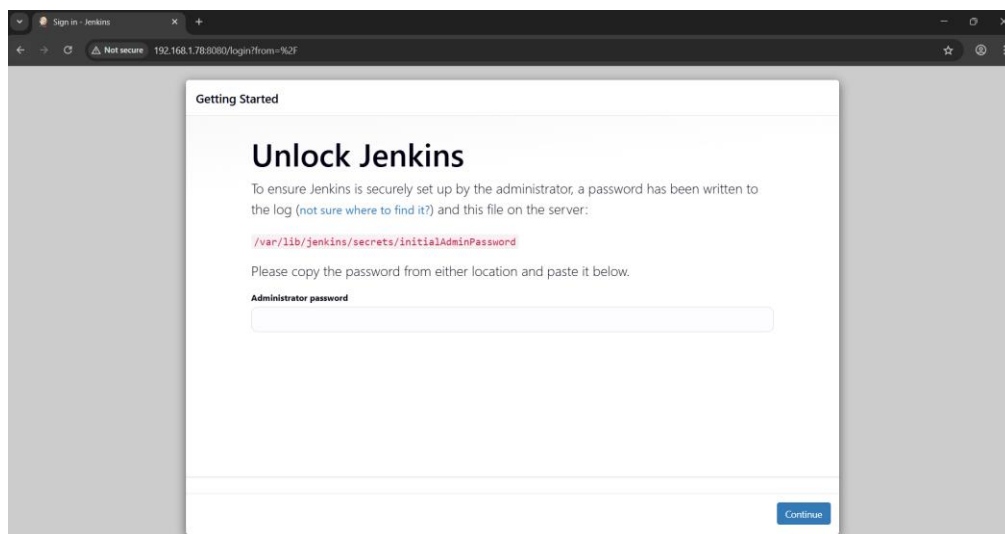


Figura 57. Pantalla para desbloquear Jenkins

3. Para empezar el desbloqueo, ingresamos la contraseña copiada y hacemos clic en el botón *Continue*.

Administrator password

Figura 58. Contraseña de desbloqueo de Jenkins

4. Posteriormente nos lleva la ventana para instalar los *plugins* de Jenkins. No te preocupes por ahora, pues hay un capítulo dedicado a explicar con detalle cómo se instalan y en qué consisten los *plugins*.

La primera opción es instalar los *plugins* recomendados por el propio Jenkins y la comunidad; la segunda opción es seleccionarlos manualmente. Debido a que esto es un tutorial para principiantes, seleccionaremos la opción uno, que iniciará la descarga automática de los *plugins* (esto puede tomar unos minutos).

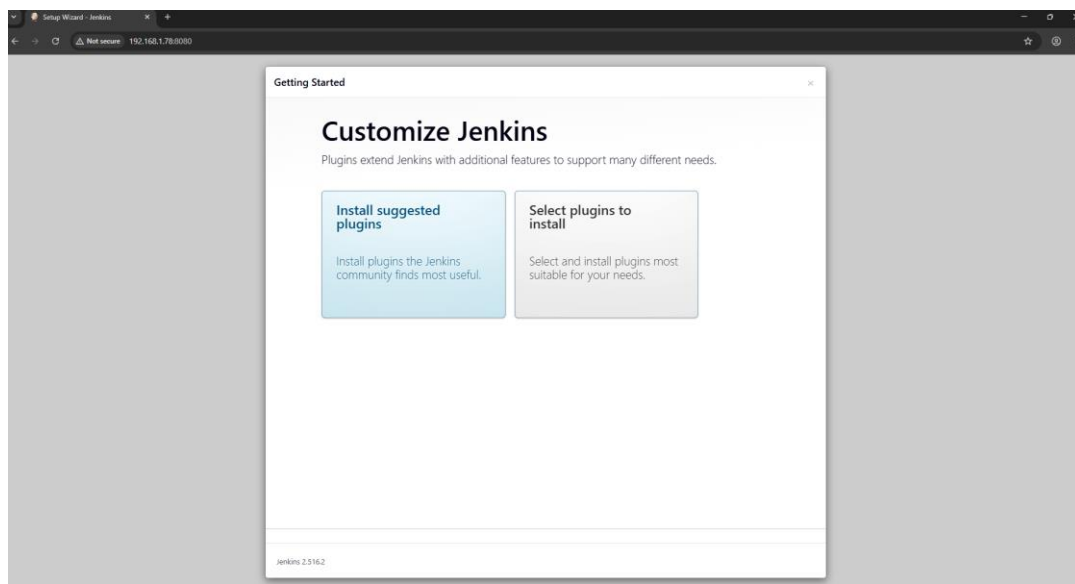


Figura 59. Pantalla de selección de *plugins* iniciales

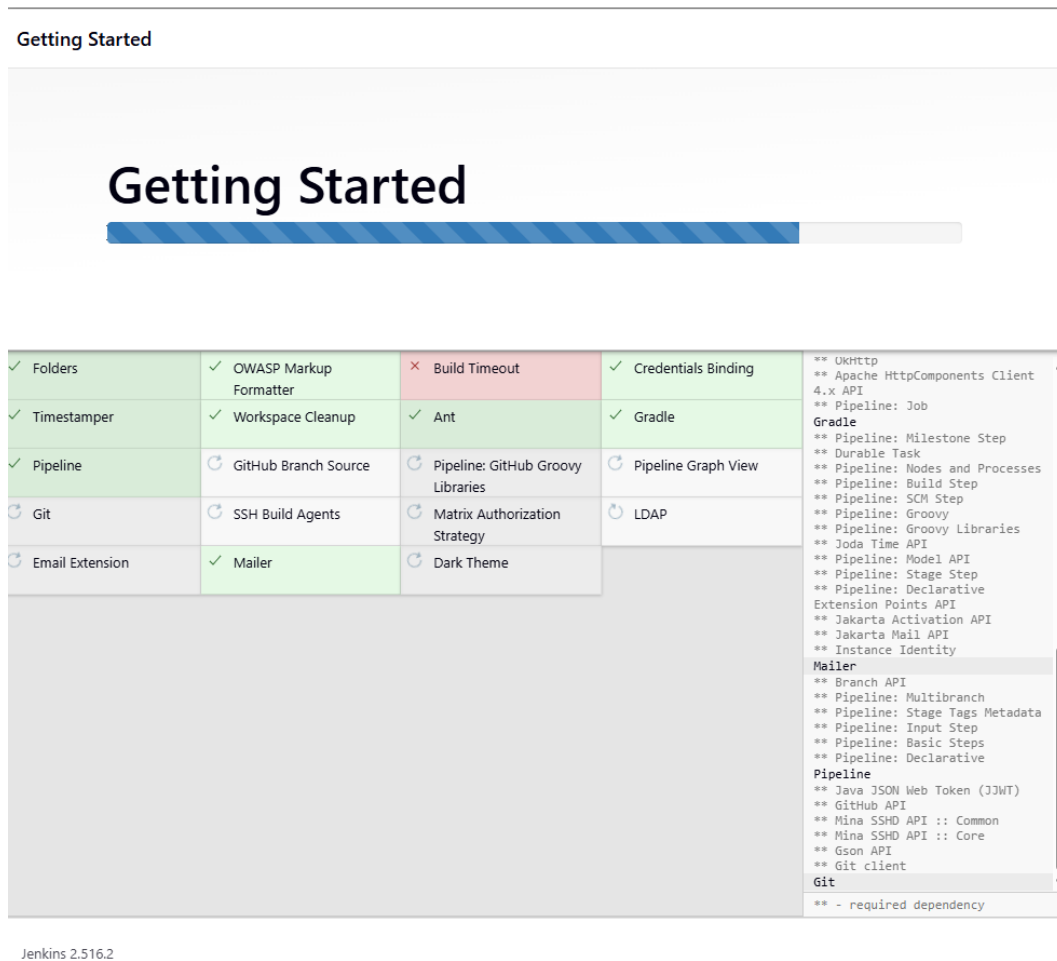


Figura 60. Pantalla de instalación de plugins recomendados

Si algún plugin no se descarga correctamente, no se preocupe, pues en las siguientes lecciones aprenderás a instalarlos desde cero.

- Una vez se terminan de instalar los *plugins*, avanzaremos a la siguiente ventana para crear el primer usuario con permisos de administrador, ingresando un nombre de usuario, contraseña, nombre completo y correo electrónico. Damos clic en el botón *Save and Continue*.

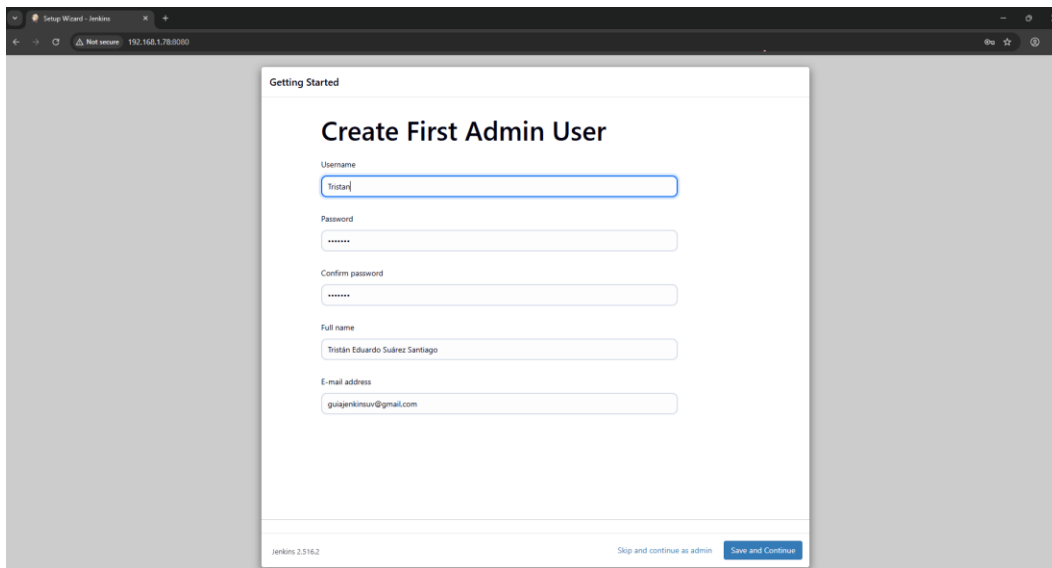


Figura 61. Pantalla para crear usuario administrador

6. Posteriormente hay que confirmar nuestra URL para acceder a la instancia de Jenkins. En este caso confirmamos la URL 192.168.1.78:8080 (tú confirma que sea la de tu máquina virtual), y hacemos clic en el botón *Save and Finish*. Posteriormente damos clic en el botón *Start using Jenkins*.

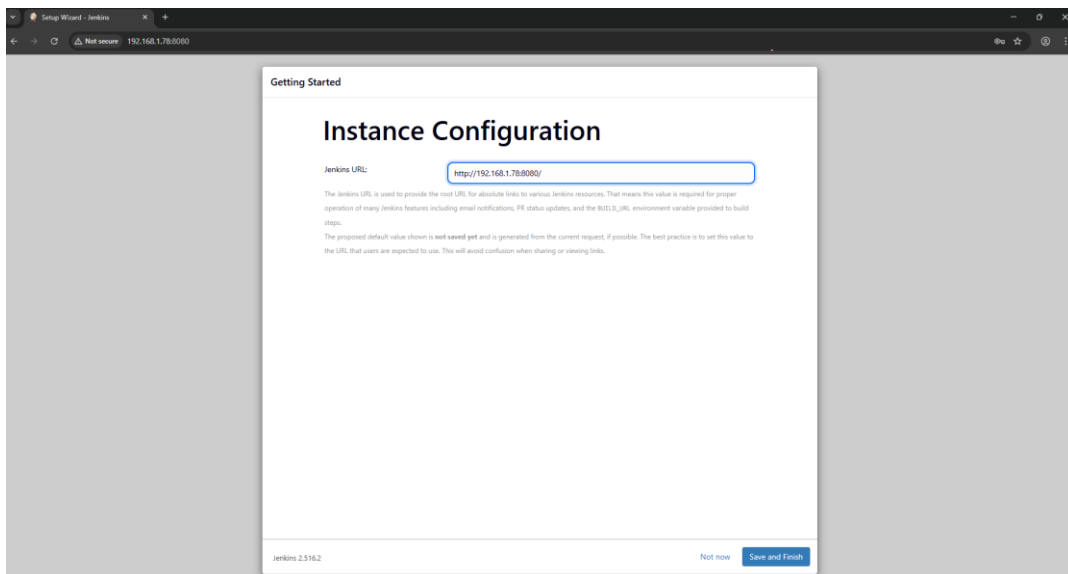


Figura 62. Pantalla de confirmación de la URL

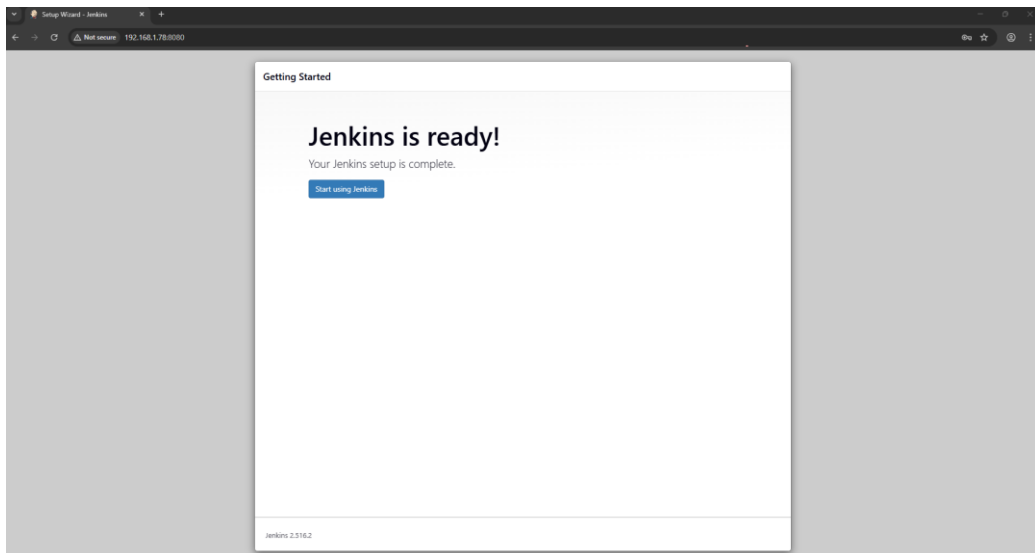


Figura 63. Pantalla final de configuración de Jenkins

Con esto, estaría finalizada la instalación de una instancia de Jenkins en una máquina virtual con sistema operativo Linux, distribución Ubuntu Server. En el futuro, para acceder a la instancia de Jenkins, debemos encender la máquina virtual y acceder a la URL de la instancia de Jenkins mediante nuestro navegador web, en nuestra máquina *host*.

En los siguientes capítulos estaremos explorando la interfaz gráfica de Jenkins, aprenderemos a configurar credenciales, descargar e instalar *plugins*, integrar herramientas como GitHub, Maven y Junit, administrar diversos aspectos de Jenkins y empezaremos a crear los primeros *jobs* de tipo *pipeline*, utilizando sintaxis declarativa.



Rúbrica 1. ¿Cómo instalar Jenkins en Ubuntu Server?

En la siguiente rúbrica, marca “Sí” o “No” en cada aspecto de evaluación, para determinar si tu implementación cumple con los criterios establecidos para la instalación de un servidor Jenkins en un entorno Linux (Ubuntu Server).

Rubrica de Evaluación

Aspecto por evaluar	Registro de Cumplimiento	
	Si	No
Consulté la versión de JAVA mediante el comando: <i>java -version.</i>		
Consulté el estado del servicio <i>jenkins</i> mediante el comando <i>systemctl –full status jenkins.</i>		
Verifiqué la correcta ejecución de Jenkins mediante el comando <i>system-analyze verify jenkins.service.</i>		
Recuperé la contraseña de desbloqueo mediante el comando <i>sudo cat /var/lib/jenkins/secrets/initialAdminPassword,</i> o: <i>systemctl –full status jenkins.</i>		
Pude ingresar a la interfaz web de Jenkins mediante el navegador web, ingresado la IP de mi máquina virtual, en el puerto 8080.		
Instalé los <i>plugins</i> recomendados.		
Pude ingresar a la instancia de Jenkins con las credenciales que registré al crear el primer usuario administrador en el servidor.		



Capítulo 6

6 Conociendo la Interfaz Gráfica de Jenkins

Contenido del capítulo

- 6.1. Gestión de Jenkins desde la Interfaz
- 6.2. Creación de Jobs desde la Interfaz

6.1 Gestión de Jenkins desde la Interfaz



Lectura 15. Familiarizándose con la interfaz gráfica de Jenkins

Nivel intermedio

Objetivo: el estudiante comprende los componentes principales de la interfaz gráfica de Jenkins relacionados a la gestión del servidor.

Descripción

La lectura explica en qué consiste el menú inicial de Jenkins, así como las secciones y menús secundarios de la interfaz gráfica de la herramienta, haciendo énfasis en la sección de “Gestión de Jenkins”.

Para el momento en el que se redacta este escrito, la versión LTS de Jenkins es la 2.516.3. Considerando esto, para este capítulo y los posteriores, se toma como referencia la interfaz gráfica de esta versión. Esto es importante mencionarlo, pues con el pasar de las versiones en Jenkins, la interfaz gráfica ha sido una de las características que se ha visto más afectada.

Dicho lo anterior, la primer pantalla con la que nos encontraremos tras iniciar sesión en Jenkins, se presenta a continuación.

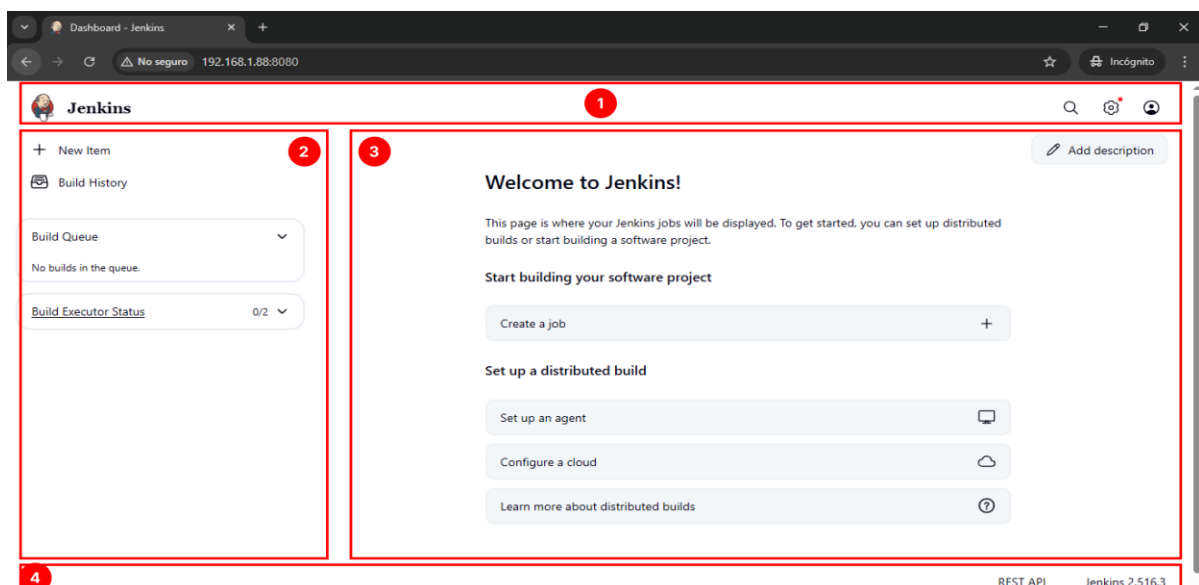


Figura 64. Pantalla principal de Jenkins

Esta primera pantalla principal se divide en cuatro secciones: la cabecera (1), una barra lateral (2), la sección central o principal (3) y el pie de página (4). La cabecera y el pie de página se mantienen inmutables sea cual sea la sección que estemos visitando dentro de Jenkins; la cabecera fijada a la parte superior sin importar cuándo bajemos en la página y el pie de página en la parte inferior de todas las páginas.

En la cabecera (1) encontramos cuatro elementos: el logo de Jenkins y tres iconos: de búsqueda, configuración y perfil (de izquierda a derecha). El logo de Jenkins sirve como enlace para regresar a esta pantalla de inicio; el icono de búsqueda abre una sección conocida como *command palette*, que permite buscar y navegar directamente hacia configuraciones, secciones, opciones y demás dentro de Jenkins; el icono de configuración redirige a la sección de gestión (*manage Jenkins*), en la que se profundizará más adelante; finalmente, el icono de perfil redirige al perfil del usuario que inició sesión, desde donde se pueden cambiar configuraciones de la cuenta, apariencia de Jenkins, preferencias, contraseña, entre otras.

La barra lateral (2) brinda opciones estrechamente relacionadas con el monitoreo de los *jobs*. La opción *new item* redirige a la sección para crear nuevos *jobs* y que se explica detalladamente en el siguiente capítulo; *build history* muestra el historial completo de ejecuciones de los *jobs* dentro del servidor; *build queue* y *build executor status* trabajan en conjunto para resumir las ejecuciones de *jobs* que están siendo procesadas: la primera muestra la lista de *jobs* que están siendo ejecutados, mientras que la segunda muestra los resultados de dichas ejecuciones.

La sección principal (3) proporciona enlaces interesantes. El primero de ellos, *create a job* redirige a la sección para crear nuevos *jobs*, la misma de la opción *new item* de la barra lateral; *set up an agent* muestra la pantalla para registrar un nuevo nodo (trabajador): recuerde la arquitectura de Jenkins; *configure a cloud* abre la página para configurar una nube (como Kubernetes o AWS) y habilitar el aprovisionamiento dinámico de nodos según la demanda; la última opción, *learn more about distributed builds*, abrirá la documentación de Jenkins donde se explica más sobre esta arquitectura distribuida de Jenkins y cómo sacarle el mayor provecho.

El botón *add description* en la parte superior derecha de la sección principal, aparece en muchas de las páginas. Al presionar este botón, se habilitará un campo de texto y algunos botones para guardar o descartar cambios. El botón justamente tiene esta función, agregar un texto de descripción que consideremos relevante para que persista en la parte superior de la página.

Finalmente, en esta pantalla principal se encuentra el pie de página (4), donde se muestran dos cosas: *REST API* que es un enlace hacia la API del servidor de Jenkins y la versión de Jenkins. Esta API previamente mencionada es una interfaz en el servidor que permite gestionarlo a través de una API REST.

De todas estas opciones que presenta la pantalla principal de Jenkins, posiblemente la más importante después de la que inicia la creación de un *job*, es la que abre la sección de gestión (*manage Jenkins*) y que se muestra a continuación.

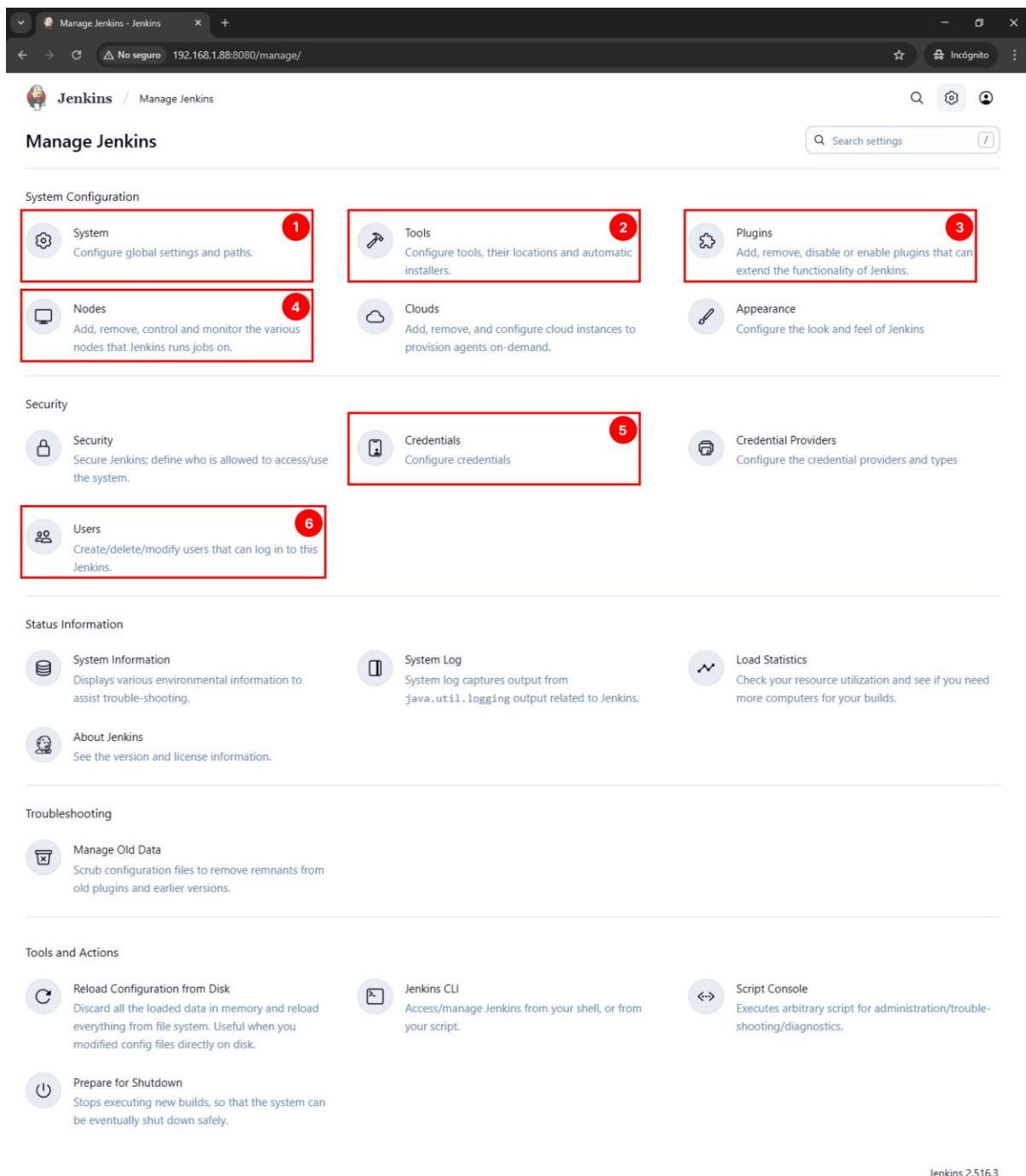


Figura 65. Pantalla de gestión de Jenkins

Como podemos ver a través de la ilustración, esta sección es muy compleja y cuenta con muchas opciones. Sin embargo, revisaremos con detalle solo seis de estas opciones, pues son las que comúnmente se estarán referenciando a lo largo de esta guía. Además, no buscamos abrumar al lector explicando con detalle todas las opciones, pero lo animamos a explorar estas secciones y profundizar más en algunas de esas características más avanzadas de Jenkins, como *Jenkins CLI*, *Security*, *Clouds*, entre otras.

6.1.1 System

La página de *system* engloba las configuraciones directamente relacionadas con Jenkins. De primera mano, se muestra el nombre del directorio base donde Jenkins está instalado, así como configuraciones relacionadas con el comportamiento del nodo ante la ejecución de un *job*: si deberá ejecutar *jobs*, cuántos *jobs* paralelos pueden ejecutarse en ese nodo, el tiempo de espera entre el evento que dispara un *job* y su ejecución real, entre otras configuraciones.

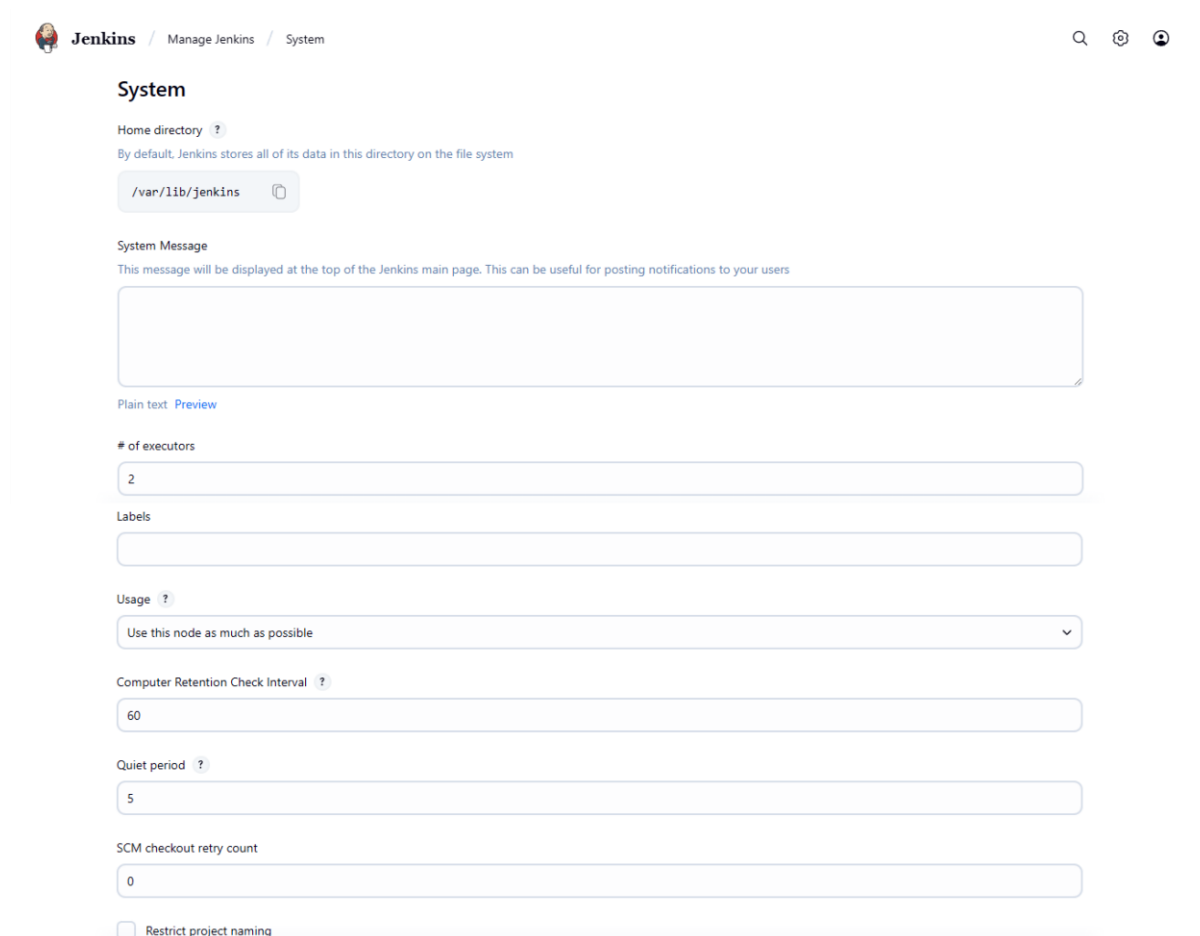
The screenshot shows the Jenkins 'System' configuration page. At the top, there's a breadcrumb trail: 'Jenkins / Manage Jenkins / System'. The main title is 'System'. Below it, the 'Home directory' is set to '/var/lib/jenkins'. A 'System Message' section has a text area for notifications. The '# of executors' is set to 2. The 'Labels' field is empty. The 'Usage' dropdown is set to 'Use this node as much as possible'. The 'Computer Retention Check Interval' is 60. The 'Quiet period' is 5. The 'SCM checkout retry count' is 0. At the bottom, there's a checkbox for 'Restrict project naming' which is unchecked.

Figura 66. Pantalla de gestión del sistema dentro de Jenkins

Además de estas configuraciones, aquí encontraremos opciones para gestionar la ubicación de Jenkins (URL), propiedades globales como variables de entorno o

ubicaciones de herramientas, el uso de métrica y estadísticas, configuración de un proxy en caso de requerirlo, tiempo de espera límite para la ejecución de *Jobs*, gestión del uso de la API de Jenkins, gestión de servidores de GitHub, gestión global del *plugin* de Git, notificaciones, entre algunas otras configuraciones globales.



Nota - Nodos maestros/controladores

Una de las configuraciones que podemos encontrar en esta página de sistema, es la de *# of executors (number of executors)*. Esta configuración permite configurar el número máximo de *jobs* paralelos que se pueden ejecutar en el nodo. Si nosotros colocamos 0 como valor, estaremos indicando que ese nodo no ejecutará ningún *job*, lo cual es recomendable cuando configuramos el nodo maestro o controlador.

Lo importante para recordar de esta página, es que nos permite configurar Jenkins a nivel global. En caso de requerir algún cambio de la configuración que aplique a todo el nodo de Jenkins o todos los *jobs* dentro de este, es muy probable que en esta sección se encuentre esta configuración.

6.1.2 Tools

La página de *tools* permite configurar las herramientas a las que Jenkins tiene acceso, como Git, Java o Maven. En esta ventana es desde donde se crean, eliminan y modifican las referencias (rutas) hacia cada una de las herramientas a las que Jenkins tiene acceso, para que estas sean accedidas y utilizadas desde las implementaciones de los *jobs*.

The screenshot displays the Jenkins 'Tools' configuration page. At the top, the navigation bar includes the Jenkins logo, 'Manage Jenkins', and 'Tools'. The main content area is titled 'Tools' and contains several sections:

- Maven Configuration:** Includes dropdowns for 'Default settings provider' (set to 'Use default maven settings') and 'Default global settings provider' (set to 'Use default maven global settings').
- JDK installations:** Features an 'Add JDK' button.
- Git installations:** This section is expanded, showing a modal form for adding a new Git installation. The form includes:
 - A title bar with a hamburger menu icon, the text 'Git', and a close button (X).
 - A 'Name' field with the value 'Default'.
 - A 'Path to Git executable' field with the value 'git'.
 - An 'Install automatically' checkbox, which is currently unchecked.
 - An 'Add Git' button with a dropdown arrow.
- Gradle installations:** Features an 'Add Gradle' button.
- Ant installations:** Features an 'Add Ant' button.
- Maven installations:** Features an 'Add Maven' button.

At the bottom of the page, there are two buttons: 'Save' (in blue) and 'Apply' (in light gray).

Figura 67. Pantalla de configuración de herramientas en Jenkins

6.1.3 Plugins

Esta página permite gestionar los *plugins* instalados en la instancia de Jenkins: agregar nuevos, desinstalarlos, deshabilitarlos o habilitarlos, actualizarlos, entre otras acciones.

En un capítulo posterior se profundiza en los *plugins*, sobre qué son, su propósito y su importancia en Jenkins. Por ahora solo debe saber que dentro de la interfaz gráfica de Jenkins, esta página es desde donde se gestionan; además de tener claro que los *plugins* son paquetes de software que extienden la funcionalidad de Jenkins.

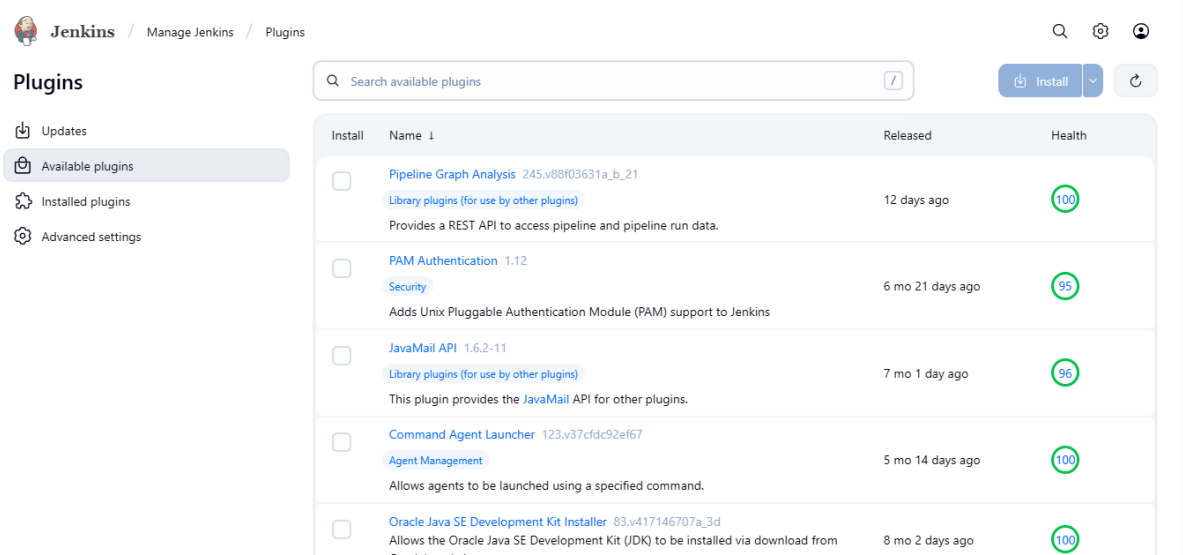


Figura 68. Pantalla de gestión de plugins en Jenkins

La imagen previa muestra la página de gestión de *plugins*. A la izquierda podemos observar una barra lateral de navegación, donde se tiene seleccionada la opción *available plugins*. A la derecha, la lista de *plugins* disponibles para instalar. Estos *plugins* son creados por la comunidad de Jenkins y distribuidos para compartir las extensiones en funcionalidad dentro de Jenkins. Tan solo el uso de herramientas como Git o la creación de *pipelines*, requieren instalar *plugins* que permitan integrar estas funciones dentro de la herramienta.

6.1.4 Nodes

En la página de *nodes* es desde donde se gestionan los nodos que soportan la infraestructura de Jenkins. Recordemos la arquitectura bajo la que funciona Jenkins (jefe/trabajador); en caso de que tuviéramos múltiples nodos trabajando en conjunto para ejecutar los *jobs*, esta pantalla nos permitiría registrar nuevos nodos, ver la lista de nodos que tenemos registrados y acceder a cada uno de ellos para cambiar su configuración.

En la siguiente imagen solamente se muestra un nodo: el nodo actual al que accedimos; pues no se han registrado más nodos y no se ha establecido un flujo jefe-trabajador.

Nodes

+ New Node Configure Monitors ↻

S	Name ↓	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	Built-In Node	Linux (amd64)	In sync	5.06 GiB	2.23 GiB	5.06 GiB	0ms
	Data obtained	83 ms	83 ms	80 ms	79 ms	75 ms	66 ms

Icon: S M L Legend

REST API Jenkins 2.516.3

Figura 69. Pantalla de gestión de nodos en Jenkins

Aun cuando en esta guía no profundizaremos en el uso de múltiples nodos y en la gestión de estos para comenzar a escalar Jenkins (pues este no es el enfoque), tenga muy presente que esta es una parte importante de Jenkins cuando el número de *jobs* que se manejan comienza a crecer, y que esta pantalla es de suma importancia para la gestión de los nodos.

6.1.5 Credentials

Como contexto general, las credenciales dentro de Jenkins son un mecanismo para guardar de forma segura datos sensibles, como las contraseñas o *API keys*. Este tema se explora en profundidad en el siguiente capítulo, sin embargo, debe saber que es desde esta pantalla desde donde se pueden gestionar.

Credentials

ⓘ

T	P	Store ↓	Domain	ID	Name

Stores scoped to Jenkins

P	Store ↓	Domains
	System	(global)

Icon: S M L

REST API Jenkins 2.516.3

Figura 70. Pantalla de gestión de credenciales en Jenkins

6.1.6 Users

Esta última página permite consultar la lista de usuarios registrados en la instancia de Jenkins a la que accedimos, así como registrar nuevos y gestionar aquellos que ya están registrados. Acceder a la configuración de cada usuario nos mostrará una página muy similar a la del perfil de un usuario, pudiendo configurar aspectos de la cuenta, apariencia de Jenkins, preferencias, contraseña, entre otras.

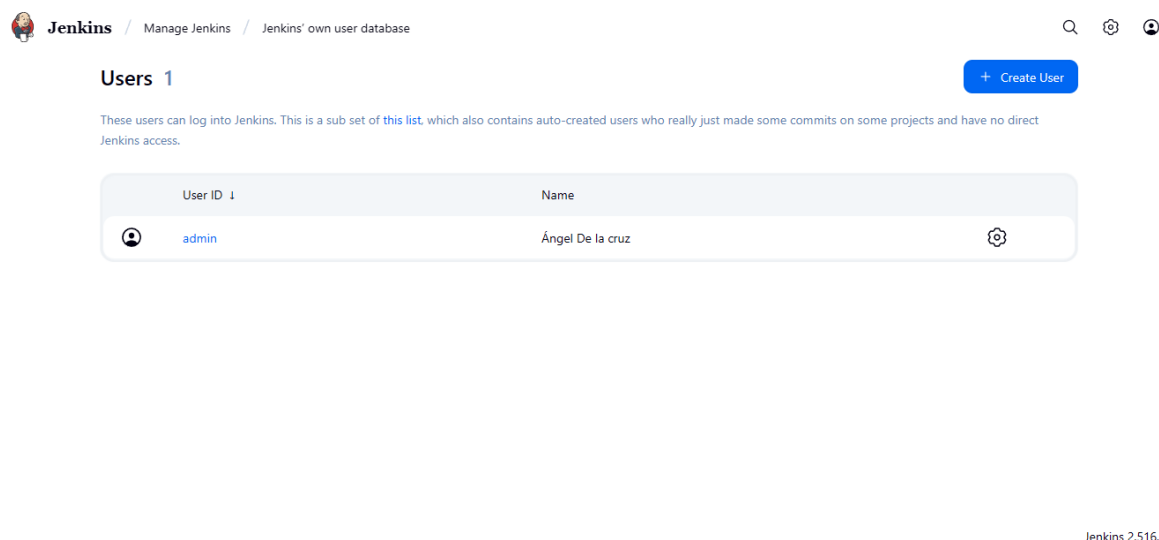


Figura 71. Pantalla de gestión de usuarios en Jenkins

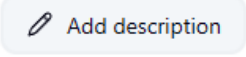
Como hemos visto hasta ahora, estamos accediendo a Jenkins a través de una URL en un navegador, ingresamos nuestro usuario y contraseña y podemos comenzar a configurar todo dentro de la herramienta. En un equipo de trabajo que utiliza Jenkins, el flujo podría verse muy similar: cada desarrollador tiene su propio usuario para acceder al servidor de Jenkins, cuenta con ciertos permisos para realizar ciertas acciones y puede acceder en cualquier momento a los nodos de Jenkins en caso de ser necesario.



Cuestionario 12. Familiarizándose con la interfaz gráfica de Jenkins.

Contesta las siguientes preguntas para evaluar tu comprensión en la lectura previa y validar tus conocimientos respecto a la interfaz gráfica de Jenkins.

1. ¿Qué opción dentro de la pantalla principal permite crear un nuevo proyecto en Jenkins?

A)	B)	C) Icono de configuración.	D) Ninguna de las anteriores.
			

2. Verdadero o falso: dentro de la pantalla principal de Jenkins solamente existe una opción para iniciar la creación de un *job*.

A) Verdadero. B) Falso.

3. Verdadero o falso: desde la pantalla principal de Jenkins, se resumen las ejecuciones de los *jobs*, pero solamente de aquellos que están siendo procesados.

A) Verdadero. B) Falso.

4. Verdadero o falso: en la sección de gestión (*manage Jenkins*) se encuentra la configuración del sistema, usuarios y credenciales.

A) Verdadero. B) Falso.

5. Verdadero o falso: desde la cabecera, que se mantiene presente en la mayoría de las páginas de Jenkins, podemos acceder a la configuración del perfil, la búsqueda rápida (*command palette*) y la sección de gestión.

A) Verdadero. B) Falso.

6. Imagina que necesitas almacenar tu contraseña de acceso a un repositorio privado de *GitHub*. ¿En qué sección de Jenkins deberías configurarla?

A) Manage Jenkins / System	B) Manage Jenkins / Users	C) Manage Jenkins / Nodes	D) Manage Jenkins / Credentials
----------------------------	---------------------------	---------------------------	---------------------------------

7. Imagina que estás trabajando con *Kubernetes* en tu proyecto de software. Has investigado y encontraste una extensión que te permite integrar esta herramienta con tus *pipelines* en Jenkins. ¿Desde dónde instalarías esta extensión?

- A) Manage Jenkins / System B) Manage Jenkins / Plugins C) Manage Jenkins / Tools D) Manage Jenkins / Clouds

8. Uno de los miembros de tu equipo ha olvidado su contraseña de acceso. Tú como administrador, puedes gestionar las cuentas y te han solicitado realizar un cambio de contraseña para la de este miembro del equipo. ¿Desde dónde aplicarías este cambio?

- A) Manage Jenkins / System B) Manage Jenkins / Credentials C) Manage Jenkins / Users D) Manage Jenkins / Security

9. Uno de los administradores de Jenkins dentro de tu equipo de trabajo te ha asignado la tarea de registrar una nueva instancia de Jenkins para ejecutar los *jobs* que ejecutan pruebas en un entorno de pre-producción del proyecto en el que trabajan. ¿Desde dónde registrarías esta nueva instancia?

- A) Manage Jenkins / System B) Manage Jenkins / Tools C) Manage Jenkins / Nodes D) Manage Jenkins / Credentials

10. Tu equipo de trabajo ha comenzado a utilizar una versión de Java más reciente y necesitan probar su uso dentro de los *jobs* en Jenkins. Ya has instalado esta nueva versión de Java en el servidor donde se encuentra corriendo Jenkins. ¿Desde dónde podrías registrar esta nueva versión de Java para que pueda ser accedida desde los *jobs* que corren en tu instancia de Jenkins?

- A) Manage Jenkins / Tools B) Manage Jenkins / System C) Manage Jenkins / Nodes D) Manage Jenkins / Credentials

6.2 Creación de Jobs desde la interfaz



Lectura 16. Creación de *jobs* desde la interfaz gráfica

Nivel intermedio

Objetivo: el estudiante comprende los componentes principales de la interfaz gráfica de Jenkins relacionados a la creación de *jobs*.

Descripción

La lectura explica cómo acceder a la sección de Jenkins destinada a la creación de *jobs*, con énfasis en los de tipo *pipeline*. Se explica en qué consisten las configuraciones iniciales de este tipo de *job*.

Ahora que conoce cómo navegar dentro de la interfaz gráfica de usuario de Jenkins y dónde encontrar las opciones para gestionar Jenkins de forma general, pasaremos a explicar uno de los componentes principales de esta herramienta: la creación de *jobs*.

Como primer paso, ya sabemos que desde la pantalla principal de Jenkins tenemos dos opciones para iniciar la creación de un *job*: *new item* y *create a job*. Ambas opciones mostrarán la siguiente página en caso de seleccionarlás.

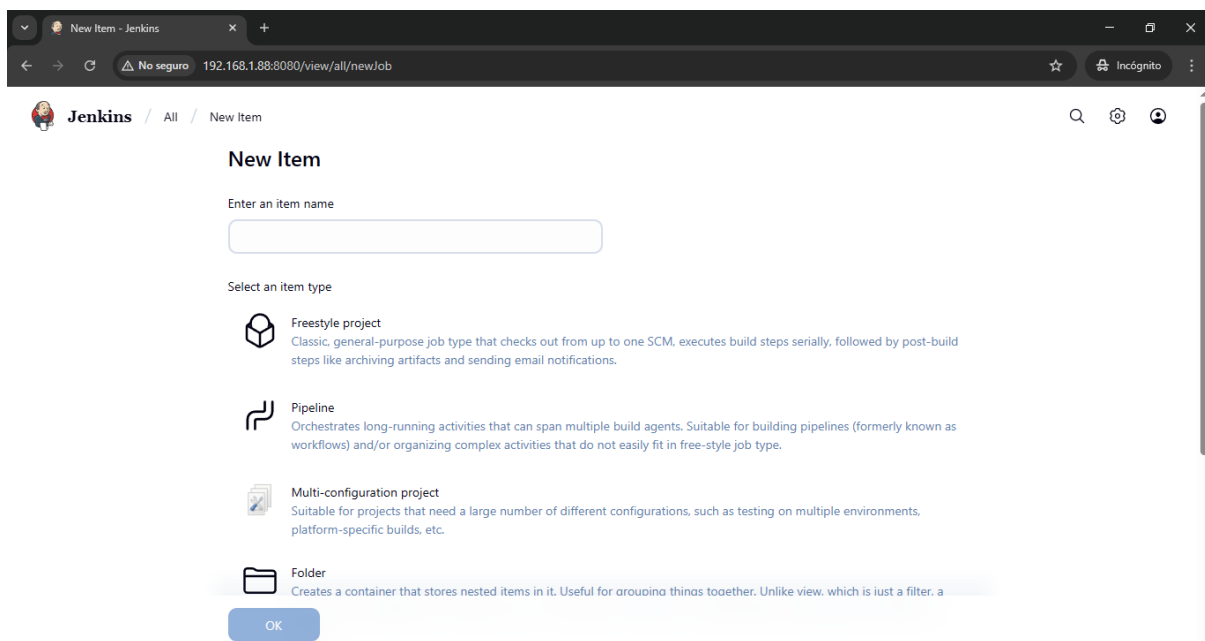


Figura 72. Pantalla de creación de jobs

Como lo muestra la imagen previa, esta primera pantalla requiere especificar el nombre del nuevo *job* y el tipo de *job*. Una vez que seleccionamos el tipo de *job* y le asignamos un nombre, el botón *ok* se habilita para continuar a la siguiente ventana. Sin embargo, hay que tener en cuenta que esto creará el *job* y la página posterior solamente proporcionará las opciones necesarias para configurarlo.



Nota - Nombrado de *jobs*

Los expertos recomiendan limitar el nombrado de los proyectos (*jobs*) a caracteres alfanuméricos. Toma esto en cuenta a la hora de darle un nombre a tus nuevos *jobs* y si quieres separar palabras en el nombrado, utiliza guiones (-) o guiones bajos (_).

La página de configuración de los *jobs* será muy similar sin importar su tipo. Pueden que cambien las secciones, pero la estructura y sus opciones serán muy similares. A continuación, se muestra la página de configuración de un *job* de tipo *pipeline*.

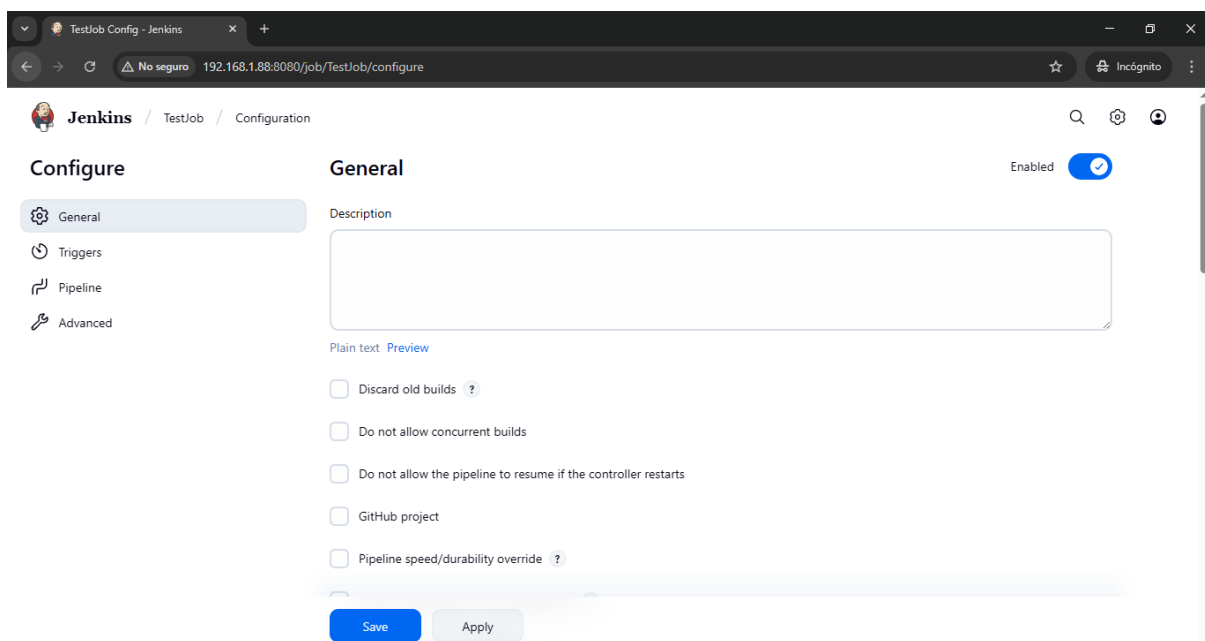


Figura 73. Pantalla de configuración general del pipeline

Como se puede observar en la imagen previa, esta página de configuración cuenta con cuatro secciones visibles en la barra lateral a la izquierda: *general*, *triggers*, *pipeline* y *advanced*. De estas secciones, la única compartida entre los diversos tipos de *jobs* es la primera. Sin embargo y como ya se mencionó antes, la estructura de la página es similar sin importar el tipo de *job*.

La primera sección (*general*) permite configurar el comportamiento del *job* en lo general, como: eliminar las compilaciones viejas, restringir las ejecuciones en paralelo, agregar parámetros, entre otras opciones. La mayoría de las opciones en esta sección están acompañadas de un signo de interrogación, que al presionarlo proporcionan una explicación completa de la función que tiene cada opción dentro de la configuración. Le invitamos a que en su propia instalación lea más sobre estas opciones para conocer un poco más sobre cómo se configura el comportamiento de los *jobs*.

La segunda sección (*triggers*), responde a la pregunta ¿Cuándo quiero que se ejecute mi *pipeline*? Esta sección permite configurar el cuándo de un *pipeline*: si su ejecución se disparará por algún evento en el repositorio del proyecto, por la finalización de otro *job*, si se ejecutará periódicamente, entre otras opciones.

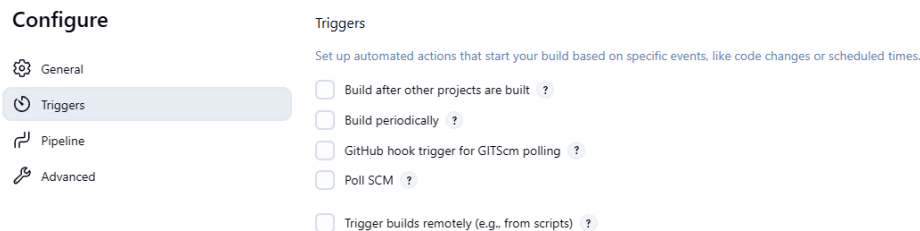
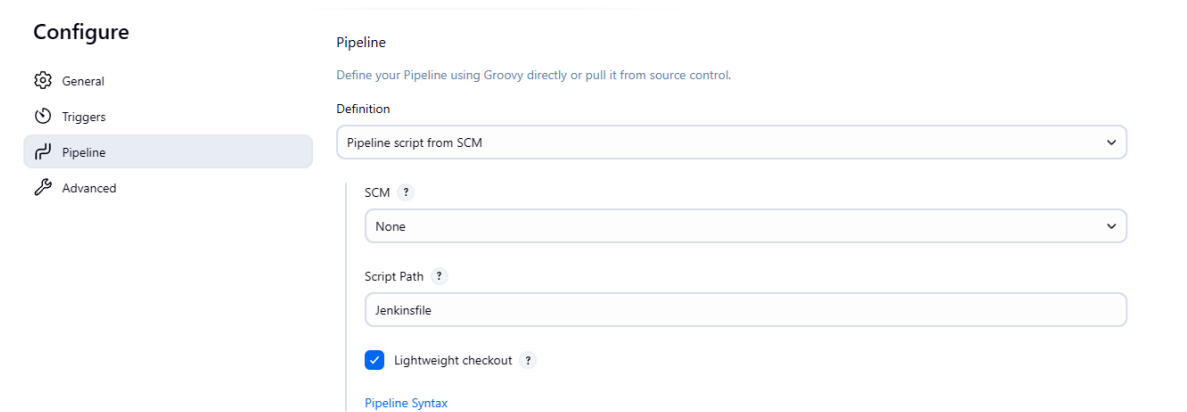


Figura 74. Pantalla de configuración de triggers del pipeline

La tercera sección (*pipeline*) ya nos habla de la estructura del *pipeline*. Aquí ya se nos permite configurar su definición, ya sea a través de un script (el código directamente) o a través de un sistema de control de versiones, como Git con algún repositorio de código como GitHub.

Esta sección de la configuración es donde especificamos la ubicación de nuestro archivo *Jenkinsfile* o directamente escribimos el código del *pipeline* de Jenkins que automatizará nuestro *pipeline* de despliegue. Independientemente del tipo de definición seleccionada, se puede usar la sintaxis *scripted* o declarativa para crear el pipeline.



Configure

- General
- Triggers
- Pipeline**
- Advanced

Pipeline
Define your Pipeline using Groovy directly or pull it from source control.

Definition
Pipeline script from SCM

SCM ?
None

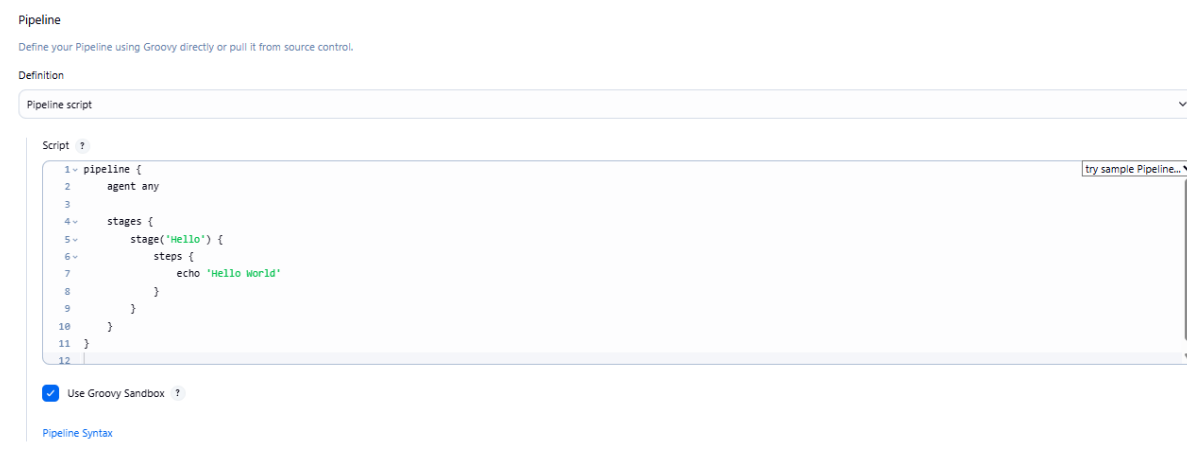
Script Path ?
Jenkinsfile

☒ Lightweight checkout ?

[Pipeline Syntax](#)

Figura 75. Sección de configuración "pipeline"

Si se escoge la primera opción de definición, se debe escribir el código del pipeline directamente en Jenkins. Existen algunas plantilla predeterminadas que se pueden usar como base para escribir los pipelines declarativos o *scripted*.



Pipeline
Define your Pipeline using Groovy directly or pull it from source control.

Definition
Pipeline script

Script ?

```

1 pipeline {
2   agent any
3
4   stages {
5     stage('Hello') {
6       steps {
7         echo 'Hello World'
8       }
9     }
10  }
11 }
12

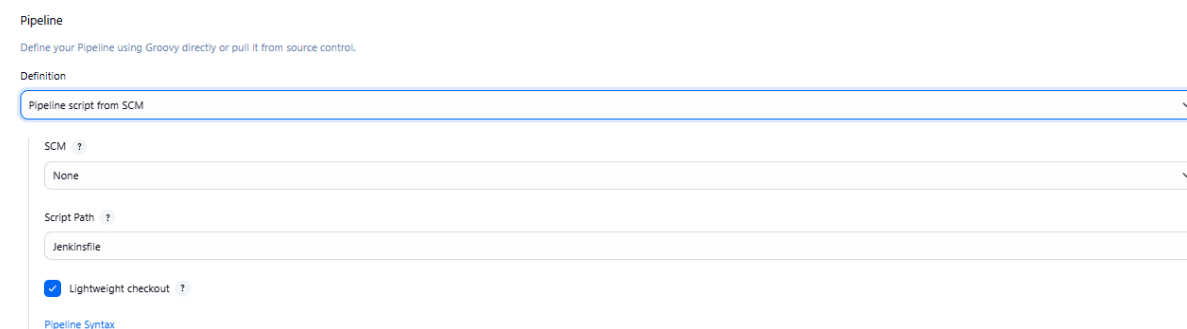
```

☒ Use Groovy Sandbox ?

[Pipeline Syntax](#)

Figura 76. Ejemplo de declaración de pipeline scripted

Y en el caso de seleccionar la segunda opción de definición, se requiere ingresar información extra, respecto al repositorio en el que se encuentra ubicado el Jenkinsfile.



Pipeline
Define your Pipeline using Groovy directly or pull it from source control.

Definition
Pipeline script from SCM

SCM ?
None

Script Path ?
Jenkinsfile

☒ Lightweight checkout ?

[Pipeline Syntax](#)

Figura 77. Formulario de información de pipeline declarativo

La última sección (*advanced*), nos permite hacer dos cosas: la primera es establecer un *quiet period*, que básicamente sirve para especificar el tiempo en segundos que Jenkins esperará entre recibir la notificación de comenzar la ejecución del *job* y realmente comenzarla; y segundo, nos permite especificar un *display name* que es un nombre para referenciar a este *job* a través de la interfaz gráfica de Jenkins (distinto al que asignamos al crear el *job*).

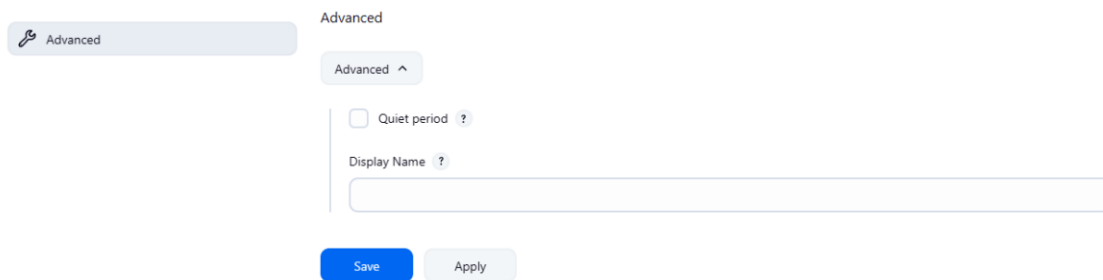
The image shows the 'Advanced' configuration section of a Jenkins pipeline. On the left, there is a tab labeled 'Advanced' with a wrench icon. The main area has a title 'Advanced' with a dropdown arrow. Below this, there is a checkbox labeled 'Quiet period' with a help icon. Underneath is a text input field labeled 'Display Name' with a help icon. At the bottom, there are two buttons: 'Save' (in blue) and 'Apply' (in light blue).

Figura 78. Pantalla de configuración avanzada del pipeline

Para finalizar, se encuentran los botones *save* y *apply* dentro de esta ventana de configuración del *job*. Ambos guardan los cambios en la configuración del *job* sin ejecutarlo. Sin embargo, el primero mencionado, redirecciona a la página principal del *job*, mientras que el segundo solamente aplica los cambios y nos deja en la misma pantalla.



Cuestionario 13. Creación de Jobs desde la interfaz gráfica.

Contesta las siguientes preguntas relacionadas a la lectura previa y valida tus conocimientos respecto a la creación de *jobs* desde la interfaz gráfica de Jenkins.

1. Verdadero o falso: para crear un *job* solamente es necesario asignarle un nombre y seleccionar su tipo.

A) Verdadero.

B) Falso.

2. Para el nombrado de *jobs* es recomendable utilizar:

- | | | | |
|---|--------------------------------|--|--|
| A) Caracteres alfanuméricos y espacios. | B) Cualquier tipo de carácter. | C) Solamente caracteres alfanuméricos. | D) Caracteres alfanuméricos y guiones. |
|---|--------------------------------|--|--|

3. Verdadero o falso: la pantalla de configuración de un *job* no cambia sin importar el tipo de *job* que estemos configurando.

- | | |
|---------------|-----------|
| A) Verdadero. | B) Falso. |
|---------------|-----------|

El administrador de Jenkins dentro de tu equipo de trabajo te ha asignado la tarea de configurar un *job* dentro de Jenkins de tipo *pipeline*. Responde las siguientes preguntas tomando esto en cuenta.

4. ¿Desde la interfaz gráfica qué sección de la configuración del *job* deberías revisar si necesitas que la definición del *pipeline* se descargue desde un repositorio de GitHub?

- | | | | |
|---------------------|----------------------|----------------------|----------------------|
| A) <i>General</i> . | B) <i>Pipeline</i> . | C) <i>Triggers</i> . | D) <i>Advanced</i> . |
|---------------------|----------------------|----------------------|----------------------|

5. Verdadero o falso: si quieres configurar la definición del pipeline directamente en Jenkins, la sección que visitarías sería la misma que en la pregunta previa.

- | | |
|---------------|-----------|
| A) Verdadero. | B) Falso. |
|---------------|-----------|

6. ¿Desde la interfaz gráfica qué sección de la configuración del *job* deberías revisar si necesitas que tu *pipeline* se ejecute cada vez que hay un nuevo *commit* en el repositorio del proyecto?

- | | | | |
|---------------------|----------------------|----------------------|----------------------|
| A) <i>General</i> . | B) <i>Pipeline</i> . | C) <i>Triggers</i> . | D) <i>Advanced</i> . |
|---------------------|----------------------|----------------------|----------------------|

7. ¿Desde la interfaz gráfica qué sección de la configuración del *job* deberías revisar si necesitas que tu *pipeline* no permita ejecuciones concurrentes?

- | | | | |
|---------------------|----------------------|----------------------|----------------------|
| A) <i>General</i> . | B) <i>Pipeline</i> . | C) <i>Triggers</i> . | D) <i>Advanced</i> . |
|---------------------|----------------------|----------------------|----------------------|

8. ¿Desde la interfaz gráfica qué sección de la configuración del *job* deberías revisar si necesitas que tu *pipeline* no se ejecute inmediatamente cada que se realiza un *commit*, sino que se esperen 5 minutos entre el *commit* y la ejecución del *pipeline*?

A) *General*. B) *Pipeline*. C) *Triggers*. D) *Advanced*.

9. Verdadero o falso: la sección *general* de la configuración de un *job* permite gestionar el comportamiento general de un *job*, como la concurrencia de ejecuciones y la especificación de *triggers*.

A) Verdadero. B) Falso.

10. Verdadero o falso: ambos botones *save* y *apply* guardan los cambios en la configuración de un *job*, pero el primero de ellos redirecciona hacia la pantalla principal del *job*.

A) Verdadero. B) Falso.



Capítulo 7

7 Uso de credenciales en Jenkins

Contenido del capítulo

- 7.1. ¿Qué es una credencial en Jenkins?
- 7.2. ¿Cómo se usan las credenciales en Jenkins?

7.1 ¿Qué es una credencial en Jenkins?



Lectura 17. Credenciales en Jenkins

Nivel básico

Objetivo: el estudiante describe el concepto de credencial en Jenkins.

Descripción: la lectura explica el concepto de credencial en Jenkins, las ventajas de adoptar su uso en Jenkins y algunos ejemplos comunes de uso.

Según la documentación oficial de CD Foundation (2025b), una credencial en Jenkins es un mecanismo seguro para almacenar información sensible, como contraseñas, claves SSH, tokens de acceso o certificados, para que puedan ser utilizados en *jobs* sin exponerlos directamente en el código ni en la configuración visible para los usuarios.

El beneficio principal del uso de credenciales en Jenkins es la seguridad, pues Jenkins guarda las credenciales cifradas en el controlador y las maneja mediante identificadores, reduciendo el riesgo de exposición accidental. Esto permite que los *pipelines* (y *jobs* en general) interactúen con aplicaciones externas (repositorios, servicios en la nube, sistemas de despliegue, etc.) de manera segura y controlada (CD Foundation, 2025b).

Y no todas las credenciales son iguales. Si bien todas se guardan de forma segura, según la documentación de CD Foundation (2025p), Jenkins maneja diversos tipos de credenciales:

- *Secret text*: por ejemplo un token de una API.
- *Username and password*: los cuales pueden ser manejados por separado o como una cadena separada por dos puntos, en el formato *username:password*.
- *Secret file*: básicamente un archivo que contiene información sensible.
- *SSH username with private key*: un par de claves SSH pública/privada.
- *Certificate*: un archivo de certificado PKCS #12 (una sintaxis especial para la transferencia de información personal).

- *Docker Host Certificate Authentication*: certificados para autenticar conexiones con Docker.

Para todo esto, vale la pena mencionar que las credenciales almacenadas en Jenkins pueden estar definidas a nivel global, para un *job* o *pipeline* específico o para un solo usuario (CD Foundation, 2025p).



Cuestionario 14. Credenciales en Jenkins

Responde las siguientes preguntas para evaluar la lectura anterior y demostrar tu comprensión sobre el concepto de credenciales en Jenkins.

1. ¿Qué es una credencial en Jenkins?

- | | | | |
|---|--|---|---|
| A) Un registro donde Jenkins guarda el historial de ejecuciones y logs. | B) Una configuración que permite definir variables de entorno. | C) Un mecanismo seguro para almacenar información sensible. | D) Un recurso que gestiona la instalación de <i>plugins</i> en el servidor. |
|---|--|---|---|

2. Verdadero o falso: las credenciales en Jenkins permiten almacenar información sensible de forma segura.

- A) Verdadero. B) Falso.

3. Ejemplos de tipos de credenciales que puedes guardar en Jenkins son:

- | | | | |
|---|---|--|--|
| A) <i>Secret text, Username and password</i> y <i>Secret file</i> . | B) <i>Groovy password, Username and password</i> y <i>Secret file</i> . | C) <i>Secret text, Password</i> y <i>Secret file</i> . | D) <i>Secret text, Username and password</i> y <i>Encrypted file</i> . |
|---|---|--|--|

4. ¿Cuál es la ventaja de usar credenciales en Jenkins en lugar de poner contraseñas directamente en el pipeline?

- | | | | |
|--|--|--|--|
| A) Reducir el tiempo de ejecución de los <i>jobs</i> . | B) Facilitar la portabilidad de contraseñas entre <i>pipelines</i> . | C) Facilitar el acceso a las contraseñas desde los <i>jobs</i> . | D) Aportar seguridad a la hora de manejar datos sensibles. |
|--|--|--|--|

5. Verdadero o falso: las credenciales solamente pueden ser definidas a nivel de *jobs*.

- A) Verdadero. B) Falso.

7.2 ¿Cómo se usan las credenciales en Jenkins?



Lectura 18. Uso de credenciales en Jenkins

Nivel intermedio

Objetivo: el estudiante explica el uso de credenciales en Jenkins.

Descripción

La lectura expone cómo utilizar credenciales en Jenkins, dónde gestionarlas y cómo acceder a estas credenciales desde *pipelines* declarativos.

Ahora que conoces el concepto de credencial dentro de Jenkins, es momento de entender cómo utilizarlas. Con este fin, debemos saber que en Jenkins, las credenciales primero se configuran en el sistema de credenciales de Jenkins (a través de la interfaz gráfica). Luego, desde un *pipeline* (por ejemplo), se pueden referenciar para que puedan ser usadas sin exponer directamente los valores.

Sin embargo, antes de profundizar en el uso de credenciales, debemos entender dos cosas: ¿Por qué es posible gestionar credenciales dentro de Jenkins? Y ¿Quiénes pueden gestionar estas credenciales?

Respondiendo a la primera de estas preguntas, en Jenkins podemos gestionar credenciales gracias al *plugin* [credentials](#). En este contexto, surgen dos conceptos importantes: *credential stores* y *credentials domains*.

CD Foundation (2024a) mencionan dentro del manual de usuario del *plugin* *credentials*, que las *credential stores* son componentes que agrupan credenciales y

están asociados a un contexto determinado dentro de Jenkins (por ejemplo: el contexto global del sistema, el contexto de un usuario, el contexto de una carpeta, etc.). También mencionan que los *credentials domains* son una forma de clasificar o particionar credenciales dentro de una *credentials store*, donde se definen criterios sobre dónde o cómo pueden usarse ciertas credenciales.

Pasando a dar respuesta a la segunda de las preguntas, debemos saber que esta gestión de credenciales solo puede ser realizada por usuarios con permisos para crear credenciales (CD Foundation, 2025p). Por defecto, cada vez que creamos un usuario en Jenkins, este cuenta con todos los permisos para gestionar todos los aspectos de la instancia de Jenkins. Sin embargo, Jenkins de forma interna maneja niveles de permisos, que pueden ser configurados a través de *plugins*. Así, conocer esto sobre las credenciales cobra sentido si en algún momento el lector se encuentra trabajando en un equipo de desarrollo y descubre que no tiene acceso a la gestión de credenciales con la cuenta de Jenkins que le fue asignada.

Pasando al manejo de las credenciales, durante la exploración de la interfaz gráfica de Jenkins en un capítulo previo, aprendimos cómo es que podemos llegar al panel de gestión de credenciales en Jenkins; ahora profundizaremos en sus elementos.

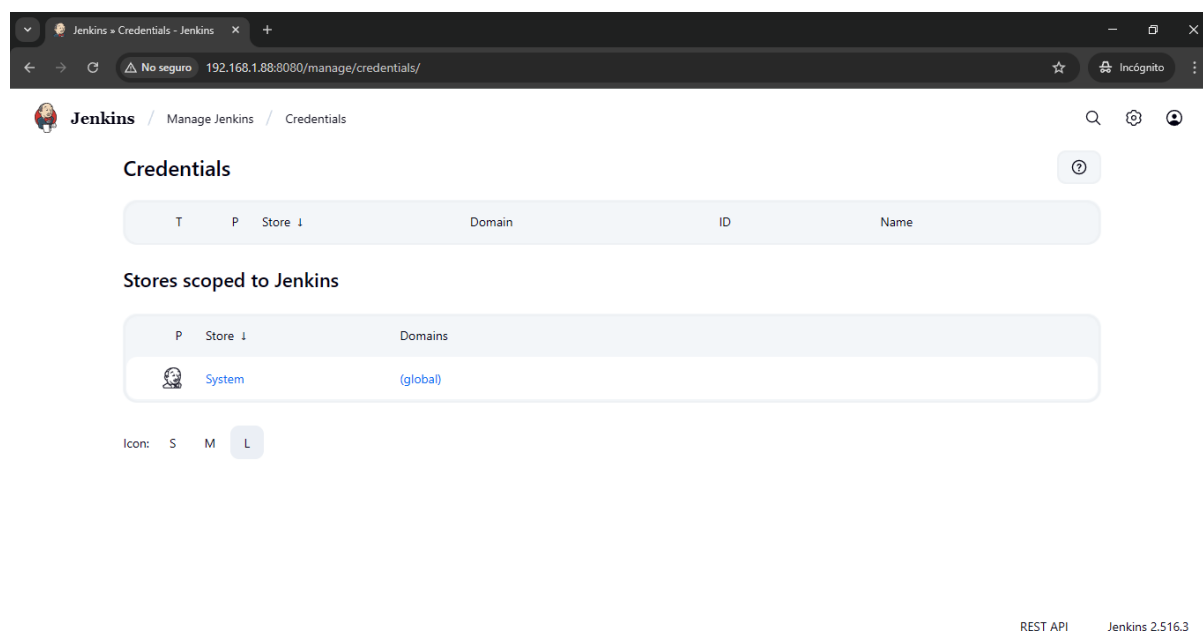


Figura 79. Panel de gestión de credenciales en Jenkins

En esta página de gestión de credenciales, dentro de la lista *stores scoped to Jenkins*, podemos ver los campos *stores* y *domains*. La *store System* mostrada por defecto, hace referencia al contexto global del sistema (la instancia de Jenkins). El *domain*

global en esa *store* define los criterios para asegurar las credenciales ahí registradas realmente sean accesibles desde cualquier parte, incluyendo los *pipelines*.

Así pues, ya sea que seleccionemos *system* y después el dominio *global* o directamente seleccionemos este dominio, al final llegaremos al panel de gestión de credenciales globales. Este panel es el que se muestra en la siguiente imagen.

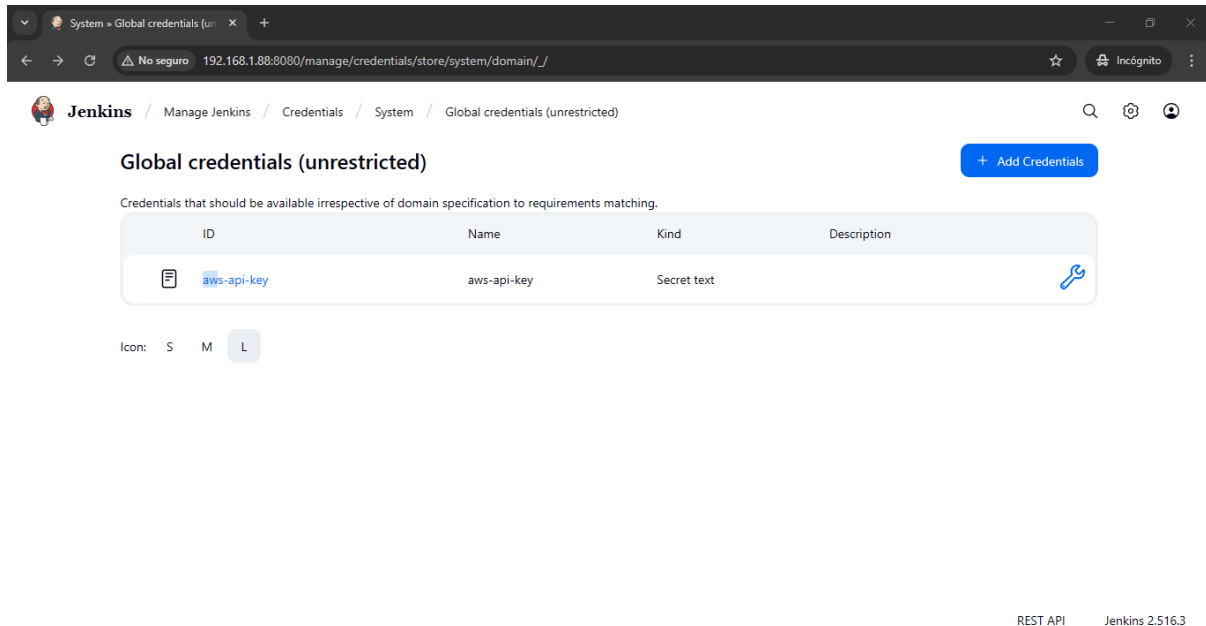


Figura 80. Panel de gestión de credenciales globales

En este panel se nos presenta la lista de credenciales registradas, en este caso, dentro del dominio *global* (pues fue este al que accedimos). Podemos gestionar estas credenciales de forma individual, seleccionando su nombre o el icono de herramienta que acompaña la credencial. Además, podemos agregar nuevas credenciales con el botón *add credentials*. Al seleccionar este botón, se nos muestra la página de creación de credenciales.

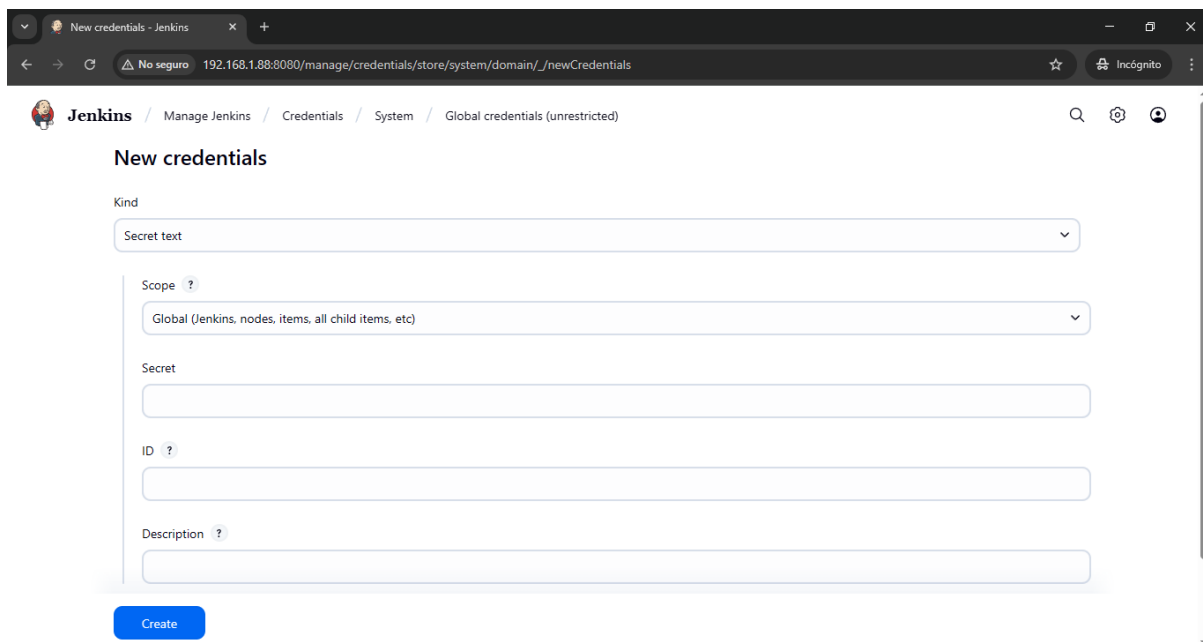


Figura 81. Formulario de registro de credenciales

En esta pantalla de registro de credenciales, podemos seleccionar el tipo de credencial (campo *kind*) de alguno de los que fueron mencionados en el capítulo anterior. Esto cambia los campos que se muestran en la pantalla; sin embargo, campos como *scope* y *ID* se suelen mantener inmutables sin importar el tipo de credencial que seleccionemos. El primero de ellos proporciona las opciones *system* y *global*: *system* registra las credenciales para su uso dentro de la instancia (configuraciones del sistema), mientras que *global* expone las credenciales a nivel global; *ID* simplemente es el nombre de la credencial.



Nota - Nombrado de credenciales

Los expertos recomiendan establecer una estrategia consistente para el nombrado de los identificadores de credenciales. Toma esto en cuenta a la hora de gestionar tus credenciales.

Ahora bien, con esta primer parte ya hemos explorado la configuración de credenciales dentro de Jenkins. Solamente queda responder, ¿Cómo accedemos a estas credenciales?

Para los tipos más comunes de credenciales (*secret text*, *username and password* y *secret files*), Jenkins provee el método ***credentials(...)*** que puede ser usado dentro de un bloque *environment* en pipelines declarativos. Para los otros tipos de credenciales, se puede usar ***withCredentials(...)*** { ... } dentro de bloques de *steps*, proporcionando

un acceso temporal a la credencial. Entonces, el acceso a las credenciales en *pipelines* ocurre mediante estas dos funciones: ***credentials(...)*** y ***withCredentials(...)*** (CD Foundation, 2025o).

Hablando particularmente de *credentials()* y como se mencionó previamente, este enfoque es útil cuando trabajamos con los tipos de credenciales *secret text*, *username and password* o *secret file*. En este caso, la credencial está disponible como variable de entorno en los *steps* del *pipeline* y se declaran como se muestra a continuación.

```
1 pipeline {
2   agent any
3   environment {
4     API_TOKEN = credentials('api-token')
5   }
6   stages {
7     stage('Ejemplo de stage 1') {
8       steps {
9         // aquí y en otros stages y steps se puede utilizar $API_TOKEN
10        sh '''
11          curl -H "Authorization: Bearer $API_TOKEN"
12            -X POST https://api.ejemplo.com/database/seed
13          '''
14        }
15      }
16    }
17  }
```

Figura 82. Ejemplo de uso de la función *credentials()*

En este ejemplo, la credencial se inserta en un *header* de una petición HTTP, como un *bearer token*. Es importante mencionar que *api-token* (el argumento pasado a la función *credentials*) es el ID de la credencial tal cual lo configuramos durante su creación. También note la sintaxis para el acceso a la variable de entorno que almacena la credencial: *\$NOMBRE*.

Para credenciales de tipo *username and password*, hay un comportamiento especial. Si hacemos:

```
1 environment {
2   BITBUCKET_COMMON_CREDS = credentials('bitbucket-common-creds')
3 }
```

Figura 83. Ejemplo de comportamiento de la función *credentials()* para credenciales *user:password*

Y *bitbucket-common-creds* es una credencial de tipo *username and password*, Jenkins creará tres variables de entorno (CD Foundation, 2025o):

- `BITBUCKET_COMMON_CREDS` que contiene *username:password*
- `BITBUCKET_COMMON_CREDS_USR` que contiene solo el usuario
- `BITBUCKET_COMMON_CREDS_PSW` que contiene solo la contraseña

Para credenciales de tipo *secret file*, Jenkins almacena el secreto como un archivo temporal y la variable de entorno apunta al *path* temporal del archivo (CD Foundation, 2025o).

Dos puntos importantes para considerar mencionados por CD Foundation (2025o):

1. Cuando se imprimen o exponen estas variables que guardan credenciales a través de logs, Jenkins automáticamente reemplaza el valor real con `****` para evitar exposición.
2. No usar interpolación de Groovy ("`${NAME}`") en lugar de ("`$NAME`") con variables sensibles, pues se estaría exponiendo su valor.

Pasando a *withCredentials()*, este es un bloque que permite asociar temporalmente una o más credenciales a variables de entorno. Pero tome en cuenta que saliendo de ese bloque, esas variables ya no están disponibles (CD Foundation, 2025o).

```
1 pipeline {
2   agent any
3   stages {
4     stage('Ejemplo de stage 1') {
5       steps {
6         withCredentials([
7           string(credentialsId: 'api-token', variable: 'API_TOKEN')
8         ]) {
9           // solamente dentro de este bloque se puede usar $API_TOKEN
10          sh '''
11            curl -H "Authorization: Bearer $API_TOKEN" \
12              -X POST https://api.ejemplo.com/database/seed
13            '''
14          }
15        }
16      }
17    }
18  }
```

Figura 84. Ejemplo de uso de la función `withCredential()`

Este ejemplo anterior expone la misma funcionalidad que el ejemplo con *credentials*, solo que esta vez hace uso de *withCredentials* y pasa como argumento un arreglo de las credenciales. En este caso se usa la función *string* para recuperar una credencial de tipo *secret text*, pero tome en cuenta que la forma de acceder las credenciales cambia con este método de acceso.

Sin embargo, para la generación del código para otros tipos de credenciales utilizando *withCredentials*, la propia documentación de (CD Foundation, 2025o) recomienda hacer uso del *snippet generator*.



Nota - ¡Una ayudita!

El *snippet generator*, *pipeline syntax* o *declarative snippet generator* son distintas formas de referirse a lo mismo: una herramienta en Jenkins para la generación de bloques de código de *pipelines* declarativos. Esta herramienta es altamente recomendada por los expertos y la puedes acceder desde *tu-pipeline* / *Pipeline Syntax* (en la barra lateral del menú).

Considere que la opción *withCredentials()* está disponible gracias al *plugin* [Credentials Binding Plugin](#). Sin este *plugin* instalado, no es posible utilizar esta funcionalidad.

Finalmente, es importante saber que en otros tipos de *jobs* que no son *pipelines* también se pueden usar las credenciales, sin embargo su consumo es distinto, estrechamente relacionado con su configuración. Aunque no se pretende profundizar en este aspecto en esta guía, es importante saber que esta función también está disponible en otros tipos de *jobs*.



Cuestionario 15. Uso de credenciales en Jenkins

Responde las siguientes preguntas para evaluar la lectura previa y verificar tu comprensión en el uso de credenciales en Jenkins.

1. ¿En qué parte de Jenkins se gestionan las credenciales?

- A) *Manage Jenkins / System* B) *Manage Jenkins / Users* C) *Manage Jenkins / Nodes* D) *Manage Jenkins / Credentials*

2. Verdadero o falso: las credenciales pueden usarse tanto en *freestyle jobs* como en *pipelines*.

- A) Verdadero. B) Falso.

3. Seleccione la opción correcta:

- A) *withCredentials()* permite inyectar globalmente las credenciales en forma de variables de entorno dentro de los *pipelines*. B) Las credenciales deben escribirse en el código fuente para que Jenkins pueda leerlas. C) *credentialsId()* solo se usa en *freestyle jobs*. D) Ninguna de las anteriores.

4. ¿Cuál representa mejor el uso de credenciales en un *pipeline* de Jenkins?

- A) Usarlas para pasar información sensible entre *stages* del *pipeline*. B) Usarlas cuando el secreto ya está publicado en el repositorio y quieres C) Usarlas para autenticarse en un servicio externo y no exponer el secreto en el código. D) Usarlas para ejecutar un comando local sin requerir

referenciarlo desde
allí.

autenticación
externa.

5. ¿Cuál de las siguientes opciones representa el camino para registrar una nueva credencial global en Jenkins?

- | | | | |
|---|---|---|--|
| A) Crear un pipeline nuevo y declarar <code>environment {}</code> . | B) Navegar a <i>Manage Jenkins</i> -> <i>Credentials</i> -> <i>Global</i> -> <i>Add Credentials</i> . | C) Subir el token a un repositorio y recuperarlo con SSH desde el <i>pipeline</i> . | D) Guardar la credencial en un archivo dentro del espacio de trabajo y leerla desde un <i>sh</i> . |
|---|---|---|--|

6. Verdadero o falso: suponiendo que tienes una credencial de tipo *certificate*, utilizar *credentials()* para accederla desde un *pipeline* es la mejor opción para hacerlo.

A) Verdadero.

B) Falso.

Con base en el siguiente código, responde las siguientes preguntas.

```
1 pipeline {
2   agent any
3   environment {
4     GITHUB_TOKEN = credentials('github-api-token')
5   }
6   stages {
7     stage('Fetch repository info') {
8       steps {
9         sh 'curl -H "Authorization: token $GITHUB_TOKEN" https://api.github.com/repos/mi-org/mi-repo'
10        sh 'echo "Token de GitHub: ${GITHUB_TOKEN}"'
11      }
12    }
13  }
14 }
```

7. ¿Cuál es el ID de la credencial que se está accediendo en este ejemplo?

- A) GITHUB_TOKEN. B) mi-org-token. C) token-github. D) github-api-token.

8. ¿Cómo se está utilizando la credencial de forma segura en el *pipeline*?

- | | | | |
|--|--|--|-------------------------------|
| A) Se utiliza de forma segura en el <i>header</i> de autorización. | B) Se utiliza de forma segura con el comando <i>echo</i> . | C) Se convierte en parámetro del pipeline para usarlo en todos los <i>jobs</i> . | D) Ninguna de las anteriores. |
|--|--|--|-------------------------------|

9. ¿Hay algún error de exposición de credenciales en el *pipeline*?

- | | | | |
|---|--|---|---|
| A) En la asignación de la variable en la línea 4. | B) En la llamada como <i>header</i> en la línea 9. | C) En la llamada en consola en la línea 10. | D) No hay un error de exposición de credenciales. |
|---|--|---|---|

10. ¿Cuál es el tipo de credencial que se está accediendo con *credentials()* en este *pipeline*?

- | | | | |
|-------------------------|-------------------------|-------------------------------|-------------------------|
| A) <i>Secret file</i> . | B) <i>Secret text</i> . | C) <i>User and password</i> . | D) <i>Certificate</i> . |
|-------------------------|-------------------------|-------------------------------|-------------------------|



Capítulo 8

8 Plugins

Contenido del capítulo

- 8.1. ¿Qué es un *plugin*?
- 8.2. ¿Cómo integrar *plugins* en Jenkins?
 - 8.2.1. ¿Cómo acceder a los *plugins*?
 - 8.2.2. ¿Cómo instalar un *plugin*?
 - 8.2.3. ¿Cómo actualizar un *plugin*?
 - 8.2.4. Gestión de los plugins
- 8.3. Jenkins y GitHub

8.1 ¿Qué es un *plugin*?



Lectura 19. ¿Qué es un *plugin* en Jenkins?

Nivel básico

Objetivo: el estudiante describe el concepto de *plugin* en Jenkins.

Descripción

La lectura explica qué son los *plugins* en Jenkins, presentando su definición, su importancia en Jenkins y cómo es que se accede a todos ellos.

Una de las características más útiles de Jenkins es su fácil integración con distintas herramientas a través de *plugins*. La CD Foundation (2025c) define a un ***plugin*** como una extensión de las funcionalidades de Jenkins suministrada independientemente del núcleo de Jenkins. En otras palabras, los *plugins* son como piezas extras que se pueden añadir a Jenkins para otorgarle nuevas o mejores capacidades, más allá de las que tiene de por defecto.

En Jenkins los *plugins* son el principal medio para que las organizaciones y usuarios puedan crear entornos y proyectos que se adapten a sus necesidades reales. Estos son desarrollados y mantenidos por la amplia comunidad de Jenkins, lo que ha propiciado la creación de miles de *plugins*. Es tal la flexibilidad de Jenkins que cualquier persona puede crear su propio *plugin*; sin embargo, siempre es mejor revisar si el *plugin* requerido ya existe.

Actualmente existen más de dos mil *plugins* activos con diversos propósitos: ya sean para automatizar alguna actividad de dentro del pipeline, mejorar la usabilidad de Jenkins, tener una mejor administración de los *jobs*, o atender una necesidad específica de los usuarios (CD Foundation, s/f-b).

Tal como se realizó en la práctica 1 ([Instalación Paso a Paso de un Servidor Jenkins en Ubuntu Server](#)), al instalar Jenkins existe la opción de descargar e instalar los *plugins* recomendados por la comunidad. Esto son los *plugins* más usados de manera general y que ofrecen funcionalidades que casi cualquier proyecto en Jenkins utilizará en algún momento. Algunos de estos *plugins* son: Git, Gradle, Ant, Pipeline, Credentials Binding, Folders, Email Extension, Mailer, OWASP, SSH Slaves, Blue Ocean, entre otros.

Todos estos *plugins* y más, se encuentran indexados y organizados en un [repositorio de distribución centralizado](#) en el que se pueden buscar, categorizar y filtrar, así como revisar a detalle su funcionalidad y configuración.



Nota - No te fíes de todos

Jenkins se caracteriza por su flexibilidad y apertura. Cualquiera puede crear un *plugin* y publicarlo. Sin embargo, esta ventaja también implica ciertos riesgos al momento de instalarlos. Por ello, es recomendable instalar únicamente *plugins* provenientes de fuentes confiables, como el catálogo oficial de Jenkins y, en caso de ser necesario, verificar el código antes de agregarlos a tu proyecto.



Cuestionario 16. *Plugins* en Jenkins

Responde las siguientes preguntas para evaluar la lectura introductoria previa y verificar tu comprensión sobre el concepto de *plugin* en Jenkins.

1. ¿Qué es un *plugin* en Jenkins?

- | | | | |
|--|--|--|--|
| A) Un programa independiente que reemplaza al núcleo de Jenkins. | B) Un archivo de configuración de Jenkins. | C) Una extensión de las funcionalidades de Jenkins independiente de su núcleo. | D) Una librería exclusiva para integrarse con Git. |
|--|--|--|--|

2. ¿Dónde se pueden encontrar todos los *plugins* de Jenkins?

- | | | | |
|---------------|-------------------------------------|---------------------------------------|-------------------|
| A) En GitHub. | B) En la CD Foundation Marketplace. | C) En el <i>Plugin Index</i> oficial. | D) En Docker Hub. |
|---------------|-------------------------------------|---------------------------------------|-------------------|

3. ¿Qué ventaja ofrecen los *plugins* frente a una instalación de Jenkins sin extensiones?

A) Mayor flexibilidad y adaptabilidad a los equipos.	B) Añade funcionalidades a Jenkins que no se tenían.	C) Mejora la capacidad de automatización, usabilidad y organización.	D) Todas las anteriores.
--	--	--	--------------------------

4. ¿Cuál de las siguientes opciones describe mejor la flexibilidad del sistema de *plugins* en Jenkins?

A) Que únicamente soporta <i>plugins</i> oficiales.	B) Que cualquier persona puede desarrollar su propio plugin.	C) Que los plugins solo funcionan en sistemas Linux.	D) Que los plugins se instalan sin necesidad de Internet.
---	--	--	---

5. Verdadero o falso: los *plugins* son opcionales en Jenkins, pero sin ellos la mayoría de las integraciones no serían posibles.

A) Verdadero.	B) Falso.
---------------	-----------

6. Verdadero o falso: los *plugins* de Jenkins son desarrollados exclusivamente por el equipo oficial que mantiene Jenkins.

A) Verdadero.	B) Falso.
---------------	-----------

7. Verdadero o falso: solo los administradores del proyecto Jenkins pueden crear nuevos *plugins*.

A) Verdadero.	B) Falso.
---------------	-----------

8. Verdadero o falso: *plugins* como Git, Pipeline y Blue Ocean se consideran parte de los recomendados al instalar Jenkins.

A) Verdadero.	B) Falso.
---------------	-----------

8.2 ¿Cómo integrar *plugins* en Jenkins?



Lectura 20. ¿Cómo integrar *plugins* en Jenkins?

Nivel Intermedio

Objetivo: el estudiante explica los pasos necesarios para integrar un plugin en Jenkins.

Descripción

A través de la lectura, se presenta de forma general la interfaz gráfica que permite gestionar los *plugins* en Jenkins, destacando principalmente su instalación. Además, se destacan prácticas altamente recomendadas para gestionar los *plugins*.

8.2.1 ¿Cómo acceder a los *plugins*?

En Jenkins toda la gestión (descarga, instalación, actualización y eliminación) de *plugins* se realiza mediante el *update center*. Este es un mecanismo interno del núcleo de Jenkins que permite al controlador principal de Jenkins descargar todos los metadatos necesarios e instalar los *plugins*. Cuando se instala un *plugin*, este se guarda como un archivo *.hpi*, el cual contiene todo el código, imágenes y demás recursos necesarios para que el *plugin* funcione correctamente (CD Foundation, 2025g).

Actualmente, existen dos formas de instalar un *plugin*:

- Usando el gestor de *plugins* de la interfaz gráfica de Jenkins, o
- Mediante la línea de comando de Jenkins.

Aunque ambos enfoque son correctos, es mucho más sencillo utilizar la interfaz gráfica de Jenkins, ya que se tiene una sección específicamente dedicada a gestionar los *plugins*. En la sección “Manage Jenkins”, se encuentra la opción *plugins*.

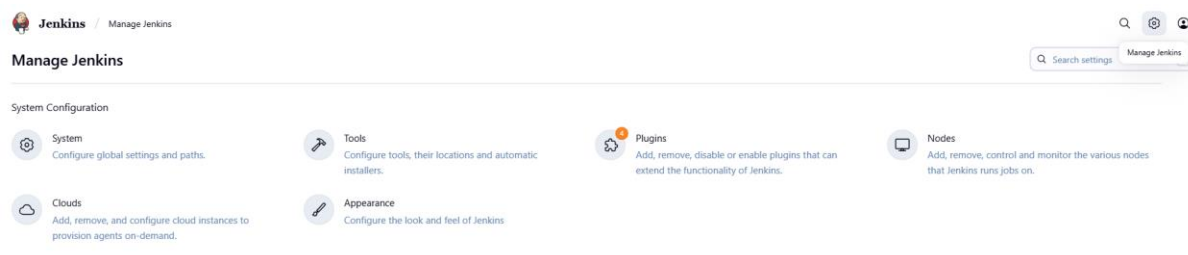


Figura 85. Panel de administración de Jenkins

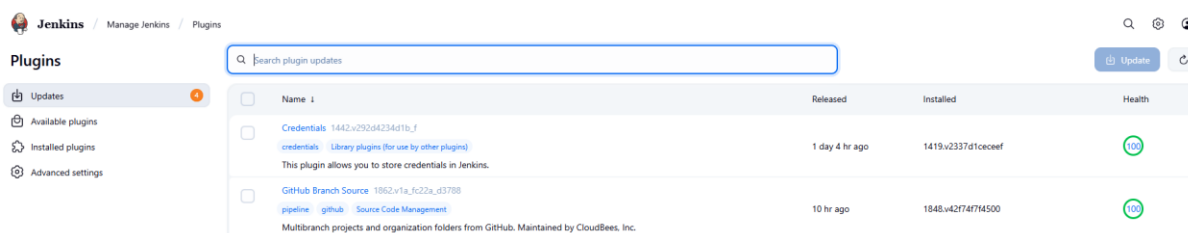


Figura 86. Panel de gestión de plugins

Dentro de este panel de gestión de *plugins* hay cuatro apartados principales:

- **Updates:** esta ventana muestra los *plugins* instalados que tienen actualizaciones disponibles. Desde esta ventana se pueden actualizar los *plugins* seleccionados con tan solo unos cuantos clics.

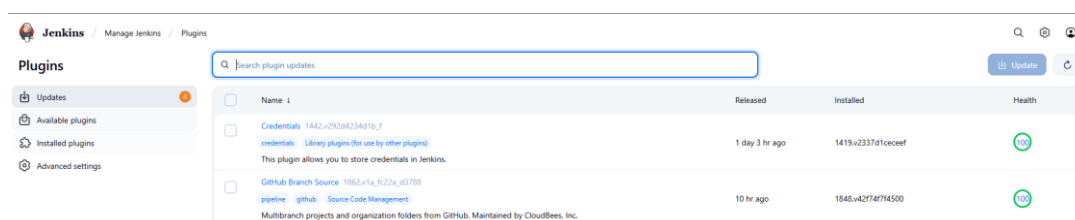


Figura 87. Panel para la actualización de plugins

- **Available plugins:** esta ventana muestra todos los *plugins* disponibles que no están instalados en el servidor. Desde esta venta se pueden buscar e instalar los *plugins* que se deseen.

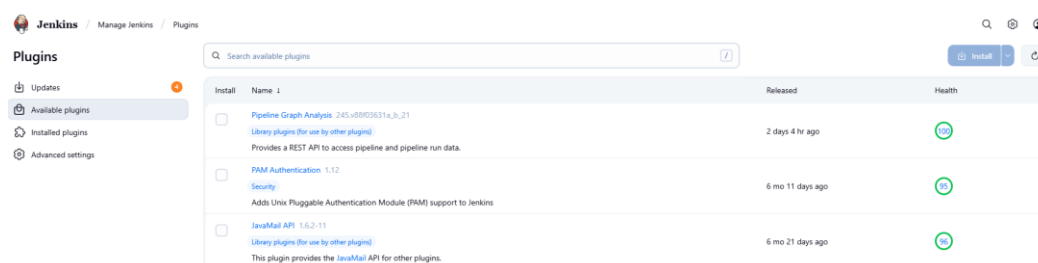


Figura 88. Panel para la descarga de plugins

- **Installed plugins:** como su nombre indica, esta ventana muestra todos los *plugins* instalados en el servidor. Desde aquí se puede consultar información detallada de cada *plugin*, desactivarlos o eliminarlos de ser necesario.

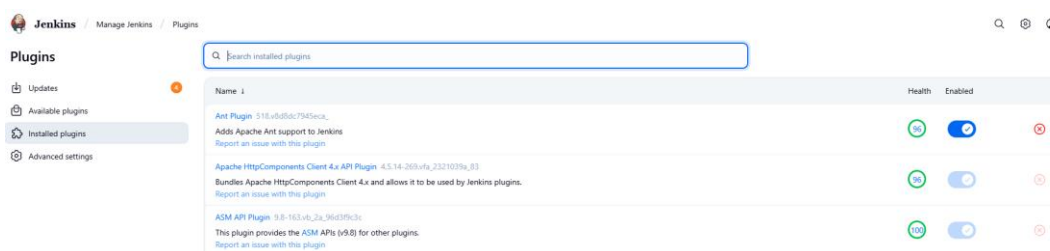


Figura 89. Panel para la gestión de plugins instalados

- **Advance Settings:** la sección de configuraciones avanzadas permite instalar *plugins* de manera manual, y modificar la ruta del *update center*.

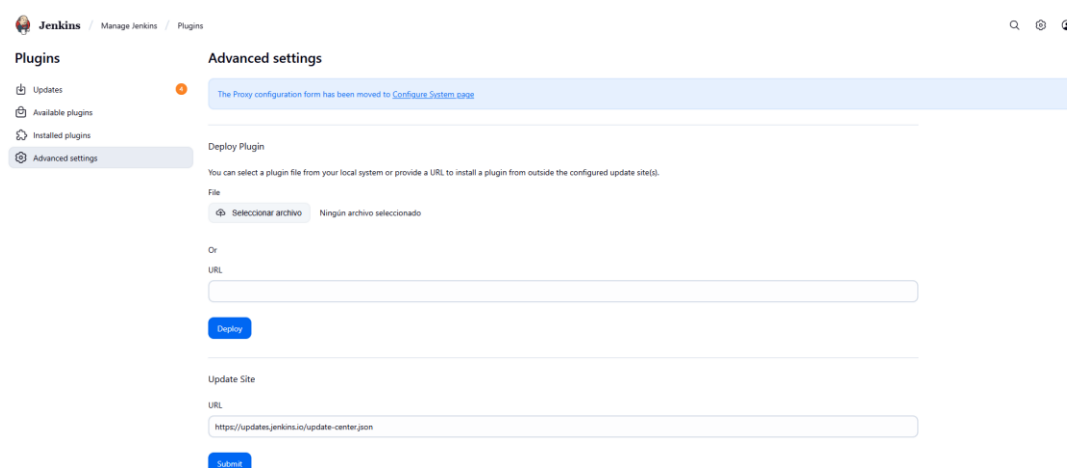


Figura 90. Panel de opciones avanzadas en la gestión de plugins

8.2.2 ¿Cómo instalar un *plugin*?

Como se mencionó previamente, hay dos formas de instalar *plugins*: mediante línea de comandos y mediante la interfaz web. A través de la interfaz web también existen dos formas de instalar un *plugin*: a través de la sección *available plugins* o en la sección *advance settings*, aunque esta última se usa comúnmente para instalar *plugins* propios o que no se encuentran en el catálogo oficial (CD Foundation, 2025I).

1. Para instalar un *plugin* desde la interfaz web simplemente hay que ir a la sección *available plugins*. En esta sección se muestra información general de los *plugins* disponibles: nombre, categoría, hace cuánto tiempo se liberó y una columna *health* que es un indicador general de confiabilidad y mantenibilidad del *plugin*. Así mismo, en esta ventana se pueden buscar *plugins* específicos por nombre o categoría (ej. *UI Theme* o *Docker*) (CD Foundation, 2025I).



Figura 91. Pantalla de plugins disponibles

- Una vez encontramos el *plugin* requerido, hay que seleccionarlo dando clic en el recuadro izquierdo y hacer clic en el botón *install*. Adicionalmente está la opción de que el *plugin* se instale solo después de reiniciar el servidor de Jenkins.

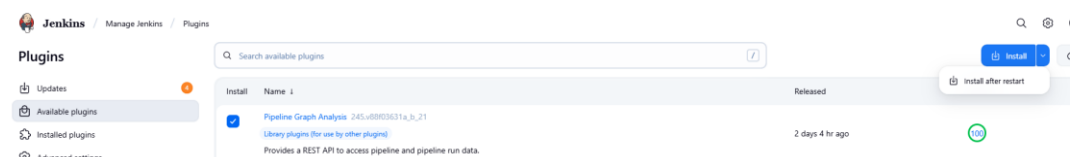


Figura 92. Pantalla de plugins disponibles con uno de ellos listo para ser instalado

- Una vez se empieza a instalar, se habilita la sección *download progress*, en la que se puede monitorear el progreso de la descarga, así como reiniciar el servidor una vez instalado el *plugin*.

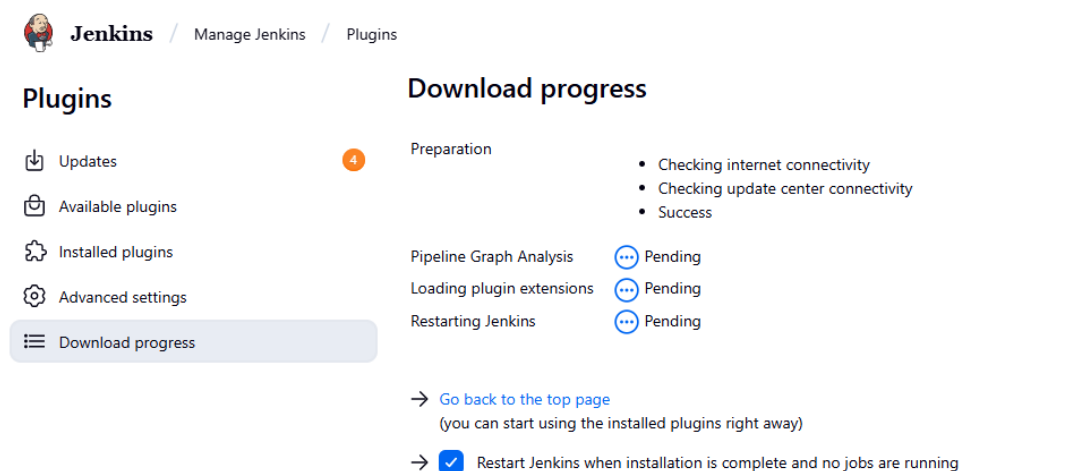


Figura 93. Pantalla para monitorear la instalación de plugins



Nota - ¡Dale un reinicio!

Después de instalar un *plugin*, se recomienda reiniciar el servidor de Jenkins, aunque no es obligatorio. De forma general, los expertos recomiendan reiniciar Jenkins tras realizar cambios de configuración de cualquier tipo.

- Una vez se termina de instalar el *plugin*, si se marcó la casilla para reiniciar el servidor, al tratar de navegar a otra sección, el servidor te redireccionará al inicio de sesión para poder volver a ingresar.



Nota - Para amantes de Linux y CLI

Aunque menos usado pero igual de efectivo, se puede usar la línea de comandos de Jenkins para instalar un *plugin* mediante el comando: ***install-plugin*** e indicando la fuente de donde se descarga.

La otra forma de instalar *plugins*, desde la interfaz web, es subiendo el archivo *.hpi* de nuestro *plugin* personal o ingresando la URL de descarga de un *plugin* que no se encuentre en el catálogo oficial (CD Foundation, 2025I). El proceso es como se muestra a continuación:

- Ir la sección *advanced settings*.

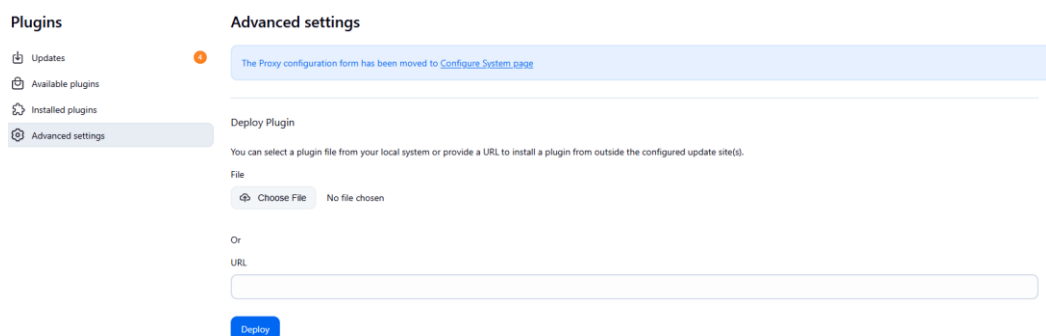


Figura 94. Pantalla para instalar plugins manualmente

- Hacer clic en el botón *choose file* y seleccionar el archivo *.hpi* que concentra el contenido del *plugin*.

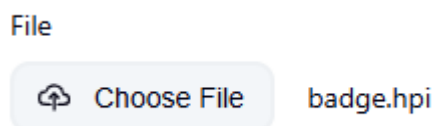


Figura 95. Botón para cargar un plugin

- U opcionalmente ingresar la URL de descarga del *plugin*.



Figura 96. Campo para ingresar la URL de descarga del plugin

4. Y finalmente hacer clic en el botón *deploy*, y de la misma forma se instalará el *plugin* automáticamente.



Nota - Menos es más

Aunque puedes instalar tantos *plugins* como quieras en tu servidor de Jenkins, es recomendable reducir el número de *plugins* utilizados tanto como sea posible.

Normalmente después de instalar un *plugin* y reiniciar el servidor, el *plugin* debería estar funcionando. Esto se puede verificar si aparece instalado y habilitado en la sección *installed plugins*. Además, algunos *plugins* requieren configuraciones posteriores como ingresar credenciales o parámetros de uso, pero esto varía para cada *plugin*. En general, estas configuraciones se hacen el apartado de *manage Jenkins* (CD Foundation, 2025I).

8.2.3 ¿Cómo actualizar un *plugin*?

Debido a que la mayoría de los *plugins* son desarrollados por la comunidad, suelen recibir actualizaciones y parches constantemente. Por lo que es importante mantenerse al día y monitorear revisar constantemente el apartado de actualizaciones.

1. Para actualizar un *plugin* hay que ir a la sección *updates*, donde se listan los *plugins* con actualizaciones disponibles. En esta sección se muestra: nombre, fecha del lanzamiento de la última actualización, el identificador del último *build*, y nuevamente el indicador *health* (CD Foundation, 2025I).

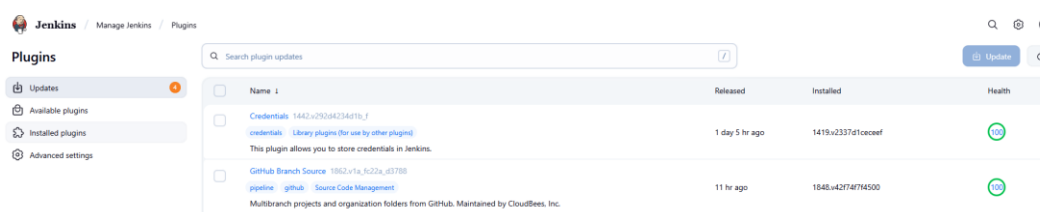


Figura 97. Pantalla en Jenkins para la actualización de *plugins*

2. Por defecto el controlador de Jenkins verifica por actualizaciones cada 24hrs, pero se puede realizar un chequeo al momento haciendo clic el botón *Check now* con un icono de flecha, ubicado en la parte superior derecha de la pantalla (CD Foundation, 2025I).

3. Igual que en la ventana para instalar *plugins*, se puede buscar por nombre o categoría entre los *plugins* instalados. Y para actualizarlos simplemente hay que seleccionarlos y hacer clic en el botón *update*.

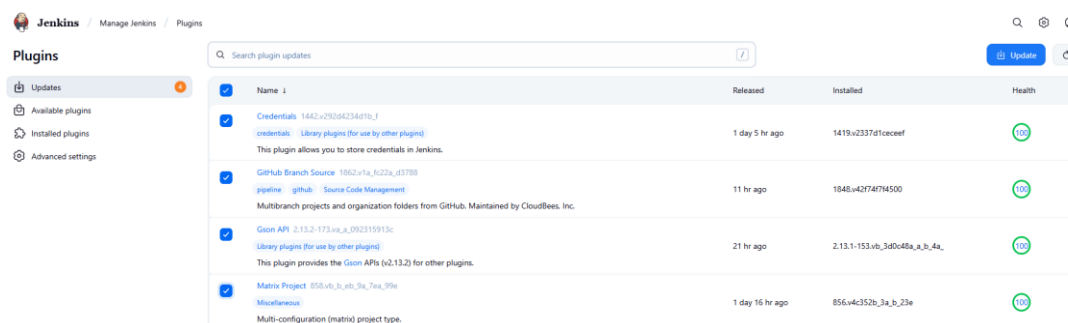


Figura 98. Pantalla de actualización de *plugins* con algunos seleccionados listos para actualizar

4. De la misma forma, esto habilitará una nueva ventana para monitorear la actualización de los *plugins*. Es recomendable reiniciar el servidor después de actualizar todos los *plugins*.



Nota - Mantente al día

Toma en cuenta que los expertos recomiendan actualizar periódicamente Jenkins y sus *plugins* por cuestiones de seguridad, ya que estas actualizaciones pueden ser parches de seguridad o de bugs.

8.2.4 Gestión de los *plugins*

Una vez que has instalado y actualizado todos los *plugins* que se van a necesitar, los puedes consultar y gestionar en la sección *installed plugins*. En esta sección los podrás deshabilitar, habilitar o eliminar permanentemente.

En la sección *installed plugins* también se pueden buscar por nombre o categoría entre los *plugins* instalados.

Para desactivar un *plugin* basta con hacer clic en el botón de tipo “switch”. Para reactivarlo es de la misma forma.

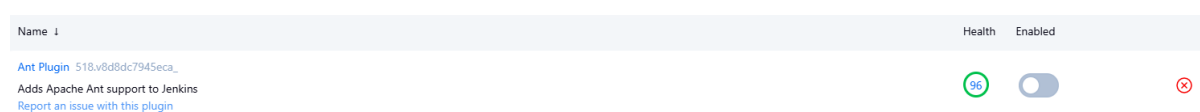


Figura 99. Pantalla de gestión de *plugins* donde se muestra el activador del *plugin* asociado (botón “toggle”), con el *plugin* desactivado

En el pasado, muchas funcionalidades de Jenkins existían directamente en el núcleo de la propia herramienta, pero con el tiempo estas se fueron abstrayendo en *plugins*. Esto generó que otros *plugins* que ocupaban esas funcionalidades ahora dependan de *plugins* externos y ya no del núcleo de Jenkins. A esto se le conoce como dependencia implícita (CD Foundation, 2025l).

Se le dice implícita porque la dependencia de un *plugin* sobre otro normalmente no está declarada de manera explícita en los metadatos de los *plugins*. Como el núcleo de Jenkins no puede saber si un *plugin* todavía necesita esas funcionalidades que antes eran parte del núcleo, asume que todos los *plugins* que antes requerían de una funcionalidad que se extrajo del núcleo a un *plugin*, todavía lo requieren por defecto (de ahí que sea implícita) y lo marca como una dependencia necesaria, aunque ya no lo necesiten (CD Foundation, 2025l).



Nota - Gestiona tus *plugins* con ¿*Plugins*?

Aunque suene extraño, en Jenkins existen *plugins* que permiten gestionar los *plugins* instalados en tu servidor. Los expertos recomiendan utilizar estas herramientas de administración de *plugins* para optimizar la gestión de estos dentro de Jenkins.

Para eliminar permanentemente un *plugin*, hay que hacer clic en el botón con icono de "X". Sin embargo, si el *plugin* tiene dependencias implícitas, no será posible eliminar el *plugin* hasta que se eliminen primero todas los *plugins* dependientes.

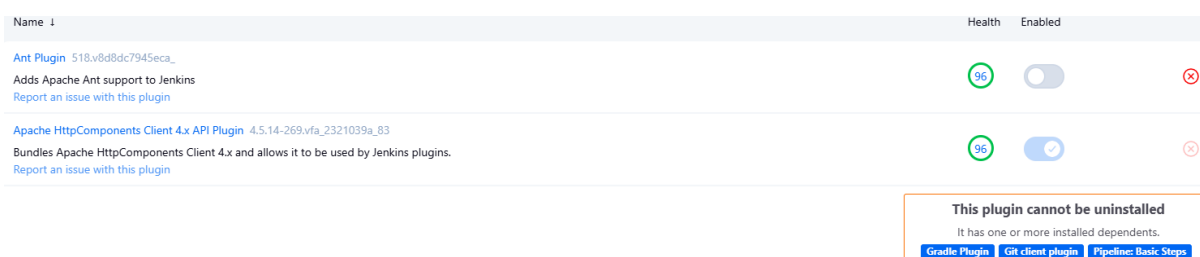


Figura 100. Pantalla de gestión de *plugins* donde se muestra el intento de eliminación de un *plugin*, bloqueado por dependencias implícitas no eliminadas previamente



Cuestionario 17. Integración de *plugins* en Jenkins

Responde las siguientes preguntas para evaluar la lectura anterior y verificar tu comprensión en la integración de *plugins* con Jenkins.

1. ¿Qué mecanismo interno utiliza Jenkins para la descarga e instalación de *plugins*?
A) *Plugin Manager* externo. B) *Update Center*. C) *Package Installer*. D) Jenkins Core.
2. ¿En qué formato se almacena un plugin una vez instalado en Jenkins?
A) .jar B) .zip C) .hpi D) .war
3. ¿Cuál de las siguientes no es una sección del gestor de *plugins*?
A) *Updates*. B) *Available plugins*. C) *Installed plugins*. D) *System logs*.
4. ¿Qué opción se utiliza para instalar un *plugin* que no está en el catálogo oficial?
A) *Updates*. B) *Advanced settings*. C) *Available plugins*. D) *Installed plugins*.
5. ¿Qué acción se recomienda después de instalar o actualizar un *plugin*?
A) Eliminar el caché de Jenkins. B) Reiniciar el servidor de Jenkins. C) Reiniciar el *plugin* instalado. D) Reiniciar el núcleo de Jenkins.
6. Verdadero o falso: los *plugins* en Jenkins pueden instalarse únicamente desde la interfaz gráfica.
A) Verdadero. B) Falso.

7. Verdadero o falso: la sección *installed plugins* permite consultar, desactivar o eliminar *plugins*.

A) Verdadero. B) Falso.

8. Verdadero o falso: el *update center* es responsable de proveer los metadatos necesarios para la gestión de *plugins*.

A) Verdadero. B) Falso.

9. Menciona las dos formas principales de instalar un *plugin* en Jenkins:

R = _____ y _____

10. Explica qué es una dependencia implícita y por qué puede afectar la eliminación de un *plugin*.

R= _____
_____.

8.3 Jenkins y GitHub



Lectura 21. Jenkins y GitHub

Nivel básico

Objetivo: el estudiante identifica el *plugin* necesario para recuperar la definición de un *pipeline* de Jenkins desde GitHub.

Descripción

Con la lectura, se presentan dos *plugins* relevantes: *Git plugin* y *Pipeline plugin*. Se destacan las capacidades principales que agregan a Jenkins, sus objetivos y cómo es que aportan para conectar Jenkins con repositorios de Git, específicamente, aquellos alojados en GitHub.

El control de versiones es un aspecto importante en el contexto del desarrollo de software y en la práctica de CI/CD. Hoy en día, utilizar un repositorio de código que centralice nuestros proyectos y que permita darles seguimiento a los cambios, es

indispensable. Siguiendo esta misma idea, mantener un control y seguimiento de nuestras configuraciones, es igual de importante.

Por esto anterior, este capítulo explica cómo es que esto se logra en Jenkins. No para mantener un control de versiones del servidor completo de Jenkins, pero sí para gestionar las versiones de los *pipelines* declarativos, específicamente de su definición a través de archivos *Jenkinsfile*.

Como abordamos en el capítulo [Creación de Jobs desde la interfaz](#), podemos crear *jobs* desde la interfaz gráfica de Jenkins. En el ejemplo que presentamos sobre la creación de un *job*, este fue de tipo *pipeline*. Esta creación de *pipelines* dentro de Jenkins se debe gracias al *plugin* que lleva el mismo nombre: [Pipeline](#). Y su objetivo es justamente ese: permitir la creación de *jobs* de tipo *pipeline*.

Si recordamos, al crear un *pipeline* llegaremos a una sección de configuración llamada *pipeline*. En esta sección ingresamos la definición del *pipeline* (su código); y una de las opciones para establecer la definición, es traerla desde un sistema de control de versiones o *Pipeline script from SCM* desde la interfaz gráfica.

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM

SCM ?

None

Script Path ?

Jenkinsfile

☒ Lightweight checkout ?

[Pipeline Syntax](#)

Figura 101. Sección "pipeline" durante la configuración de un job de tipo pipeline

Esta sección específica es propia del *plugin pipeline* durante la creación del *job*. Ahora bien, a través de la opción SCM podemos seleccionar el sistema de control de versiones. Si nosotros seleccionamos la opción desplegable, podemos ver algo como lo siguiente:

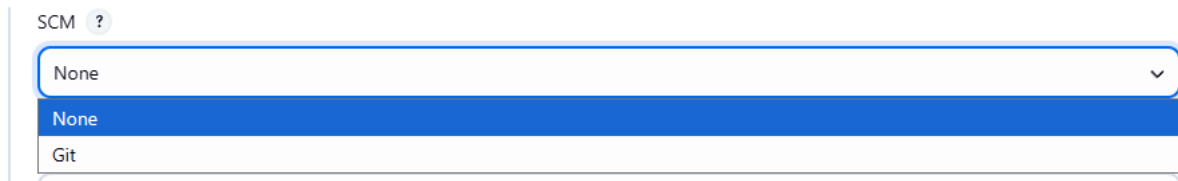


Figura 102. Opciones por defecto durante la selección de sistema de control de versiones en la configuración de un pipeline

El hecho de que la opción *Git* aparezca en esta opción de SCM no es coincidencia y no es nativo del *plugin pipeline*. Esto, es gracias a otro *plugin*, llamado [Git](#).

El *plugin* Git proporciona las operaciones fundamentales de Git necesarias para que Jenkins interactúe con repositorios de este tipo. Su objetivo principal es que dentro de Jenkins se pueda operar sobre repositorios Git de forma automatizada, permitiendo la obtención y actualización del código y facilitando la ejecución de procesos que integren el uso de Git (CD Foundation, 2025d).

The screenshot displays the Jenkins 'Configure' page for a pipeline. The 'Definition' is set to 'Pipeline script from SCM'. Under 'SCM', 'Git' is selected. The 'Repositories' section is active, showing a 'Repository URL' field with a red error message 'Please enter Git repository.' and a 'Credentials' dropdown set to '- none -'. The 'Branches to build' section is also active, showing a 'Branch Specifier (blank for \'any\')' field with the value '*/master'. The 'Repository browser' is set to '(Auto)'. The 'Additional Behaviours' section has an 'Add' button. The 'Script Path' is set to 'Jenkinsfile'. The 'Lightweight checkout' checkbox is checked. The page includes a sidebar with 'Configure' options: General, Triggers, Pipeline (selected), and Advanced. The top navigation bar shows the Jenkins logo and the path 'example-pipeline / Configuration'.

Figura 103. Panel de configuración del pipeline con detalles sobre el sistema de control de versiones

Al seleccionar la opción *Git*, se despliegan múltiples opciones que permiten configurar la conexión con el repositorio de Git desde el que queremos traer la definición del *pipeline*. Así, recuperar un *Jenkinsfile* desde GitHub es tan simple como recorrer estas opciones y llenar los campos con la información adecuada. Sin embargo, en este capítulo no se entra en detalles sobre cuáles son todas estas opciones y su significado.

Ahora bien, hay otro *plugin* relevante en la interacción entre Jenkins y GitHub, llamado *GitHub Plugin*. CD Foundation (2025e) menciona que este *plugin* aporta tres

funcionalidades principales: crear hipervínculos entre Jenkins y GitHub (*commits/issues*), disparar *jobs* cuando hay *pushes* mediante webhooks y reportar el estado de las compilaciones de vuelta a GitHub.

El *plugin* GitHub no participa directamente en el recuperado de los *Jenkinsfiles*: su función principal en este contexto es orquestar *triggers*. En la práctica, el plugin GitHub puede provocar que Jenkins inicie la acción (por ejemplo, ejecutar el *pipeline* cuando llega un *push*), pero la obtención real de las definiciones de los *pipelines* es responsabilidad del plugin de Git (CD Foundation, 2025e).

Para cerrar este capítulo, recapitulemos:

1. Para consumir definiciones de *pipelines* (*Jenkinsfiles*) desde GitHub (o cualquier otro repositorio de Git), es necesario tener instalados los *plugins* Pipeline y Git.
2. La función principal del *plugin* Pipeline es crear *jobs* de tipo pipeline.
3. La función principal del *plugin* Git es habilitar la conexión con los repositorios Git y recuperar los *Jenkinsfiles* desde ahí, durante la configuración del *pipeline*.
4. Aunque el *plugin* GitHub no participa directamente en la obtención de definiciones de *pipelines*, aporta con la orquestación de *triggers* para monitorear eventos desde GitHub y disparar la ejecución de *pipelines*.



Cuestionario 18. Jenkins y GitHub

Responde las siguientes preguntas para evaluar la lectura anterior y verifica tu comprensión sobre los *plugins* que permiten conectar a Jenkins con GitHub.

-
1. ¿Qué *plugin*(s) permite(n) que en Jenkins se puedan recuperar definiciones de *pipelines* (*Jenkinsfiles*) directamente desde GitHub?

A) El <i>plugin</i> Pipeline.	B) El <i>plugin</i> Git.	C) Los <i>plugins</i> Pipeline y Git.	D) El <i>plugin</i> GitHub.
-------------------------------	--------------------------	---------------------------------------	-----------------------------
 2. Verdadero o falso: basta con solo el Git Plugin para obtener definiciones de *pipelines* desde GitHub.

A) Verdadero. B) Falso.

3. Verdadero o falso: el Git Plugin permite recuperar definiciones de *pipelines* desde repositorios, pero solamente de GitHub.

A) Verdadero. B) Falso.

4. Verdadero o falso: el *plugin* GitHub no es necesario para recuperar definiciones de *pipelines* desde GitHub.

A) Verdadero. B) Falso.

5. ¿Qué *plugin* permite específicamente la definición de *jobs* como *pipelines*?

A) El *plugin* Pipeline. B) El *plugin* Git. C) El *plugin* Job as Pipeline. D) El *plugin* GitHub.

6. ¿Cuál es la función principal del *plugin* GitHub en el contexto de las definiciones de los *pipelines*?

A) Obtener la definición de los *pipelines*. B) Permitir la creación de *jobs* de tipo *pipeline*. C) Habilitar específicamente la conexión con GitHub. D) Orquestrar *triggers* para iniciar la ejecución de *pipelines*.



Capítulo 9

9 Creación de pipelines

Contenido del capítulo

- 9.1. ¿Cómo ejecutar *pipelines*?
- 9.2. ¿Cómo recuperar un *pipeline* desde GitHub?
- 9.3. Tu primer *pipeline* paso a paso
 - 9.3.1. Preparación del entorno de trabajo
 - 9.3.2. Definición del *pipeline*
 - 9.3.3. Creación del Jenkinsfile
 - 9.3.4. Creación del *job*
 - 9.3.5. Ejecución manual del *pipeline*
 - 9.3.6. Automatización de la ejecución del *pipeline*

9.1 ¿Cómo ejecutar pipelines?



Lectura 22. Ejecución de pipelines

Nivel básico

Objetivo: el estudiante nombra los pasos para ejecutar un *pipeline* de Jenkins.

Descripción

La lectura aborda las diversas opciones que existen para ejecutar un *pipeline*, así como los pasos previos necesarios que permiten realizar esto, enfatizando los pasos a seguir para realizar estas ejecuciones de forma manual.

En el capítulo [Tipos de Jobs comunes](#) se menciona que existen diversos tipos de proyectos en Jenkins. Uno de los más recientes y usados en la actualidad son los *pipelines*; para crear este tipo de proyectos revisa el capítulo [Creación de Jobs desde la Interfaz](#). De manera general, para ejecutar un *pipeline* en Jenkins, hay que crear un *job* de tipo *pipeline*, seleccionar el tipo de definición (*script* o script desde un SCM) y guardarlo.

Una vez creado un *job* en Jenkins, este siempre aparecerá visible en la pantalla de inicio (*Dashboard*) de Jenkins.

The screenshot shows the Jenkins Dashboard. On the left, there's a sidebar with navigation links: 'New Item', 'Build History', 'Project Relationship', 'Check File Fingerprint', 'Build Queue' (showing 'No builds in the queue'), and 'Build Executor Status' (showing '0 of 2 executors busy'). The main area displays a table of jobs. At the top right, there are search, settings, and user icons, along with an 'Add description' button. Below the table, there are filters for 'S' (Success), 'W' (Warning), and 'L' (Error), and a 'More' button (three dots).

S	W	Name ↓	Last Success	Last Failure	Last Duration	
...	☀	Pipeline_Beta	N/A	N/A	N/A	▶
✓	☀	Pipeline_Jenkins	34 min #3	N/A	8.9 sec	▶

Figura 104. Pantalla principal de Jenkins mostrando dos jobs

Para ejecutar el *pipeline* manualmente, se puede hacer clic en el botón con el icono de "play", ubicado a la derecha de la pantalla, acompañando el *pipeline* que nos interesa ejecutar.

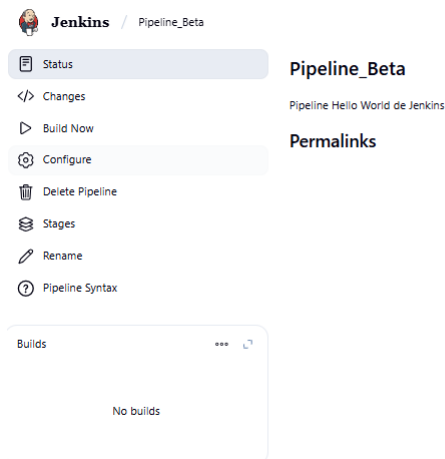


Figura 105. Menú de opciones de un pipeline en la interfaz gráfica de Jenkins

La segunda opción para ejecutar un *pipeline* manualmente es ingresando al espacio de trabajo del *pipeline*, haciendo clic en el nombre del proyecto, lo cual abrirá una nueva ventana. En esta ventana se puede monitorear el estado y cambios que hayan ocurrido en el *pipeline* con cada ejecución, realizar configuraciones nuevas, revisar los *stages* declarados, renombrar el *pipeline* e incluso eliminarlo. Para ejecutarlo, es tan simple como ubicar la opción *Build Now* y dar clic en ella.

Al ejecutar el *pipeline*, este genera un registro en la sección *Builds*, donde se muestra el estado de la ejecución. Haciendo clic en el nombre de algun *build* se pueden consultar los detalles de la ejecución.

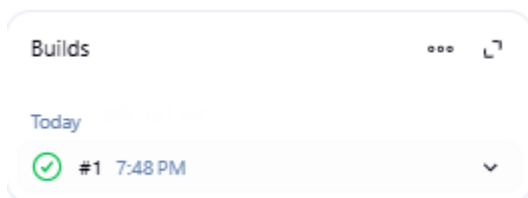


Figura 106. Panel lateral de historial de builds desde la pantalla de configuración de pipeline en Jenkins

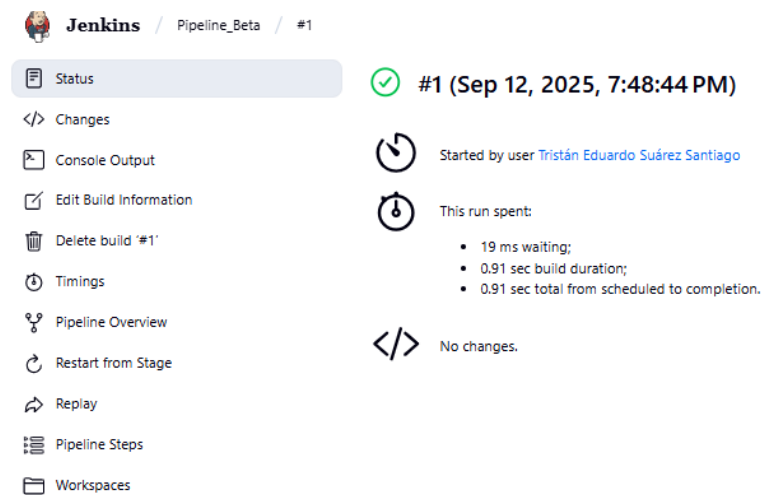


Figura 107. Pantalla de gestión de un build para un pipeline desde la interfaz de Jenkins

En la imagen previa se muestra más información sobre la ejecución (*build*) específica de un *pipeline*. Se pueden revisar los datos generados durante la ejecución, editar información del *build*, eliminarlo o volver a ejecutarlo.

En el apartado de *Console output* o *Pipeline overview*, se pueden consultar los resultados y el estado de toda la ejecución del *pipeline* con más detalle.

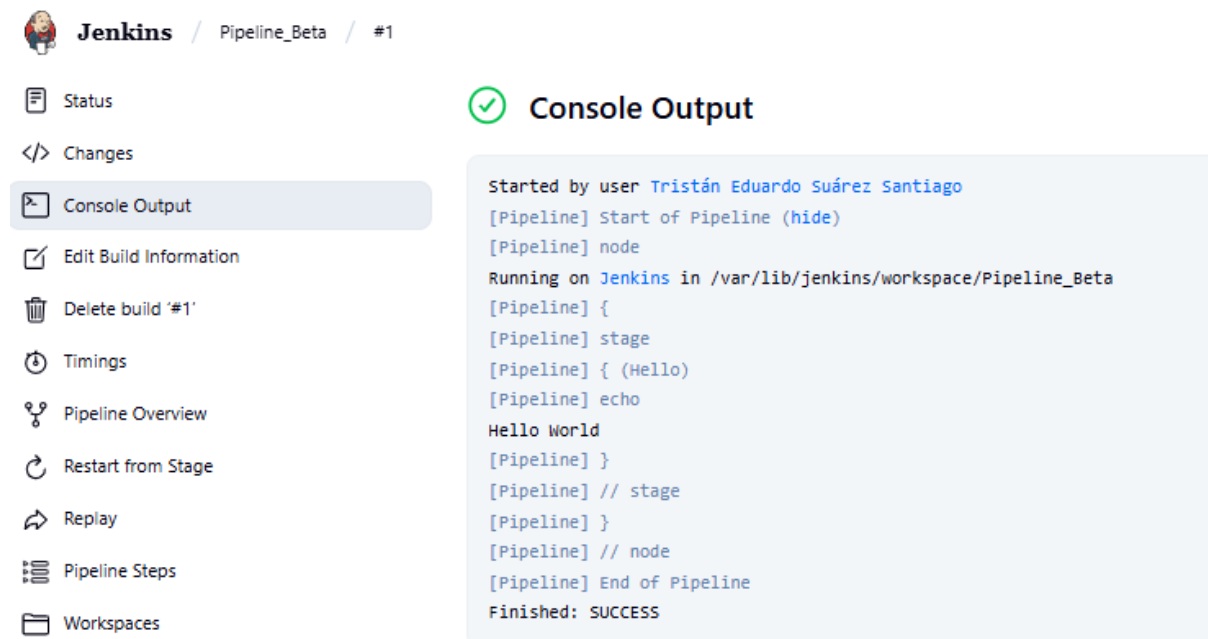


Figura 108. Panel de salida de la consola de un build asociado a un pipeline desde la interfaz de Jenkins

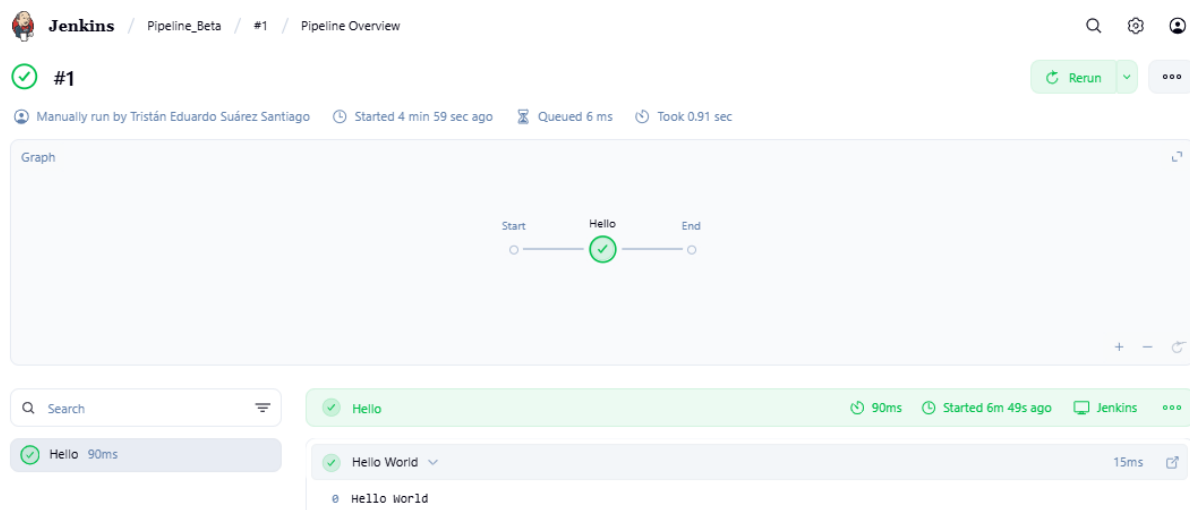


Figura 109. Pantalla de pipeline overview desde la interfaz gráfica de Jenkins

Como se puede observar, el apartado *Console output* y *Pipeline overview* son muy similares, ambos tienen el propósito de resumir el estado y salidas de la ejecución del *pipeline*. Sin embargo, *Pipeline overview* es más gráfico y fácil de entender, mientras que *Console Output* es más útil para resolver problemas al ejecutar los *pipelines*.

Como se explicó a lo largo de este capítulo, los *pipelines* se ejecutan manualmente haciendo clic en algún botón. Sin embargo, Jenkins permite la ejecución automática de los *pipelines* mediante la configuración de eventos que funcionan como disparadores (*triggers*) del *pipeline*.

Dentro de la sección de configuración del *pipeline* se encuentra el apartado de *triggers*, donde se puede establecer un evento para ejecutar el *pipeline* automáticamente.

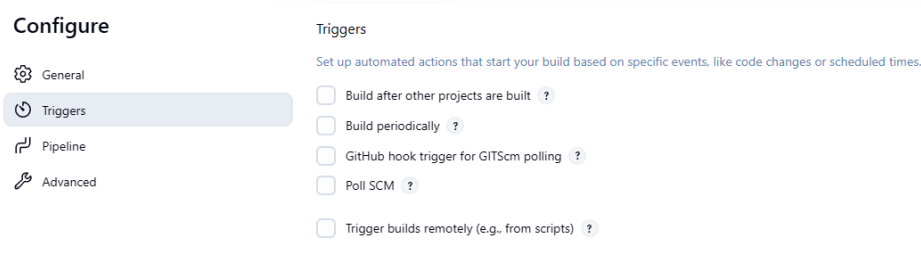


Figura 110. Sección de configuración de triggers desde la configuración de un pipeline



Nota - Primero lo primero

Una práctica recomendada por los expertos es validar la ejecución de los *pipelines* de forma manual antes de automatizarla.

Actualmente hay cinco formas de inicializar un *pipeline* de manera automática basándose en eventos específicos:

- *Build after other Project are built*: esta opción inicializa el *pipeline* una vez que termina de ejecutarse otro proyecto.
- *Build periodically*: esta opción utiliza la notación Cron para establecer un horario específico para ejecutar el *pipeline*.
- *GitHub hook trigger*: esta opción utiliza una funcionalidad de GitHub (WeebHook) para enviar una señal a Jenkins cada vez que se hace un *commit* en el repositorio, ejecutando automáticamente el *pipeline*.
- *Poll SCM*: esta opción también utiliza la notación Cron para establecer un horario específico, pero en lugar de ejecutar el *pipeline*, Jenkins realiza un escaneo al sistema control de versiones (como Git) en busca de cambios en el código; si existe alguno, entonces ejecuta el *pipeline* automáticamente.
- *Trigger builds remotly*: esta opción utiliza un token para realizar una llamada HTTP a una URL específica que inicializa el *pipeline*.



Cuestionario 19. Ejecución de *pipelines*

Responde las siguientes preguntas para evaluar la lectura anterior y verificar tu comprensión en la ejecución de *pipelines*.

1. Verdadero o falso: los *jobs* que se crean en Jenkins se encuentran en la sección “Jobs” en el menú “Manage Jenkins”.

- A) Verdadero. B) Falso.

2. Verdadero o falso: para ejecutar un *pipeline* es necesario ingresar el comando ***jenkins <nombre de tu pipeline> run***.

- A) Verdadero B) Falso

3. En este apartado del *dashboard* del *pipeline* se muestran los resultados de todas las ejecuciones previas y la más reciente.

- A) *triggers* B) *Console Output* C) *Pipeline Overview* D) *Builds*

4. En este apartado del *dashboard* del *pipeline* se pueden ver los detalles de la ejecución del *pipeline*, así como las salidas obtenidas y los estados de cada *stage*.

- A) *Pipeline details* B) *Console input* C) *Pipeline Overview* D) *Builds*

5. En este apartado se puede configurar un evento que permita ejecutar el *pipeline* automáticamente.

- A) *execution* B) *configurations* C) *Pipeline Overview* D) *triggers*

6. Verdadero o falso: una práctica recomendada es automatizar el *pipeline* sin necesidad de probarlo manualmente primero.

- A) Verdadero. B) Falso.

7. Esta opción de automatización de la ejecución del *pipeline* utiliza notación Cron para establecer un horario fijo de ejecución.

- A) *GitHub hook trigger* B) *Poll SCM* C) *Build Periodically* d) *Build after other Project are built*

8. Esta opción de automatización de la ejecución del *pipeline* permite ejecutar el *pipeline* cada vez que se realiza un *commit* en GitHub.

- A) *GitHub hook trigger* B) *Poll SCM* C) *Build Periodically* D) *Build after other Project are built*

9.2 ¿Cómo recuperar un *pipeline* desde GitHub?



Lectura 23. Recuperando la definición de un *pipeline* desde GitHub.

Nivel Intermedio

Objetivo: el estudiante explica los pasos requeridos para consumir la definición de un *pipeline* de Jenkins desde GitHub.

Descripción

La lectura describe los pasos necesarios para recuperar un *Jenkinsfile* alojado en un sistema de control de versiones (GitHub), el cual contiene la definición de un *pipeline* funcional en Jenkins.

Un *Jenkinsfile* es un archivo que contiene la definición y flujo de trabajo de un *pipeline*. Una de las practicas más recomendadas al crear *pipelines* en Jenkins es utilizar *Jenkinsfiles* y subirlos en un repositorio de un sistema control de versiones, para consumirlo desde de ahí, y así mantener una mayor organización y manejo de versiones (CD Foundation, 2025k).

El sistema de control de versiones que más se usa con Jenkins en la actualidad es GitHub con mucha diferencia, por lo que en este capítulo y en prácticas posteriores se utiliza esta herramienta para almacenar el *Jenkinsfile*.

Por esta razón, lo primero que se debe de hacer es instalar el servicio de Git en el servidor donde se aloja Jenkins, para que los *plugins* puedan utilizar esta herramienta para comunicarse con GitHub. En capítulos posteriores se explica cómo instalar y configurar herramientas dentro del entorno de Jenkins. Aunque por ahora no se configure una instalación de Git dentro de Jenkins, este utilizará la instalación global de Git que este en el servidor, en caso de no existir marcará un error al ejecutar el *pipeline* (CloudBees, s/f-c).

Puedes instalar la herramienta Git mediante el comando **`sudo apt install git -y`**. Esta es la única herramienta que siempre se debe instalar manualmente cuando se usan *Jenkinsfiles*, ya que no se puede delegar a Jenkins su instalación cuando la definición del *pipeline* se recupera de un repositorio en GitHub; si no existe Git, entonces no se puede recuperar un repositorio de GitHub.

9.2.1 Creando un repositorio con Jenkinsfile

Para traer un *pipeline* desde GitHub a Jenkins, primero se debe crear el archivo Jenkinsfile correspondiente que contenga la definición del *pipeline* y subirlo a un repositorio en GitHub. En el siguiente capítulo se detalla cómo crear un Jenkinsfile.

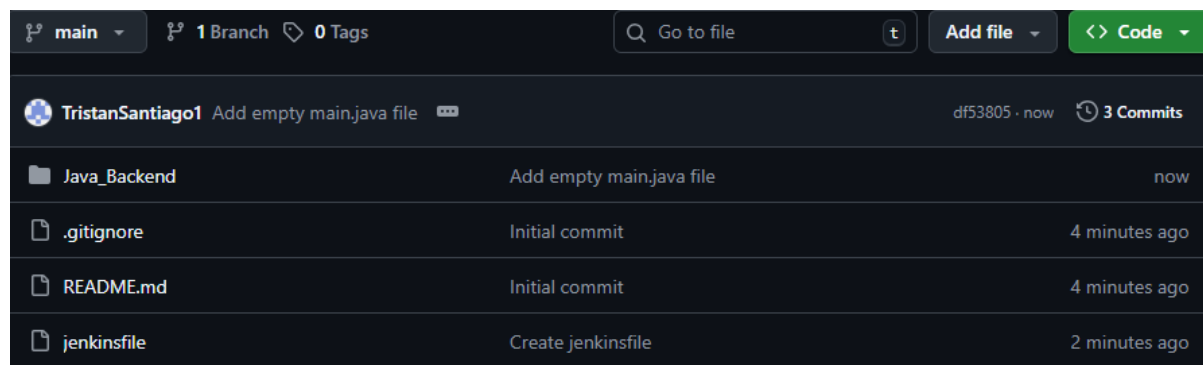


Figura 111. Repositorio de GitHub con Jenkinsfile incluido

Esto permitirá que cualquiera con acceso a la URL del repositorio pueda recuperar el Jenkinsfile y crear su propio *pipeline* a partir de lo definido en el archivo.

9.2.2 Plugins necesarios

Como se explicó en el capítulo [Jenkins y GitHub](#), para poder utilizar GitHub con Jenkins es necesario instalar algunos *plugins*: **Git Plugin**, **GitHub Plugin** y **Pipeline Plugin**. Estos son necesarios para poder comunicar Jenkins con el servicio de Git y recuperar un repositorio de GitHub. Puedes verificar la disponibilidad de estos *plugins* en tu servidor en la sección: *Manage Jenkins* -> *plugins*, tal como se explica en el capítulo [¿Cómo integrar plugins en Jenkins?](#).

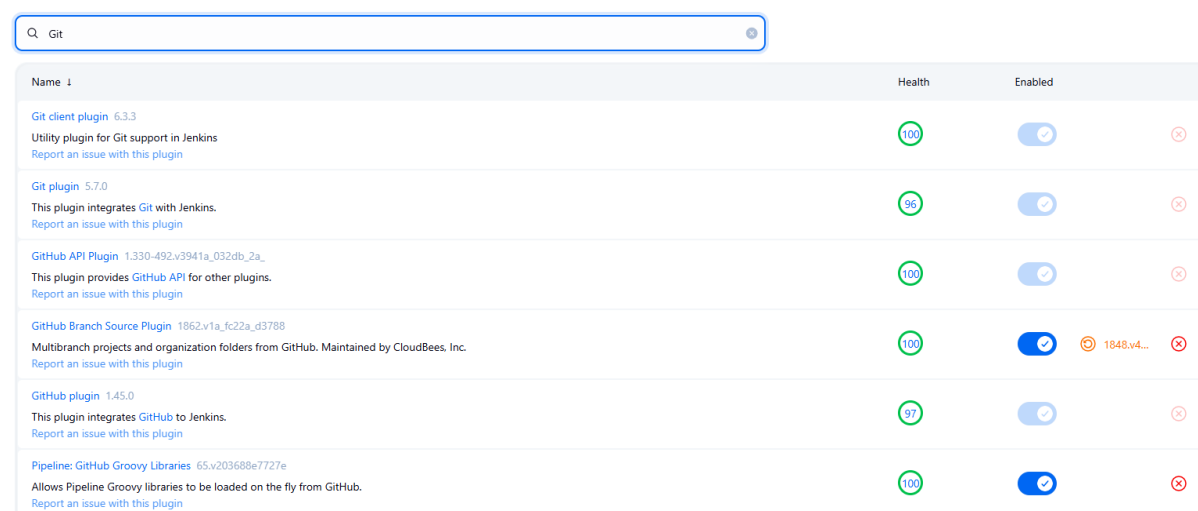


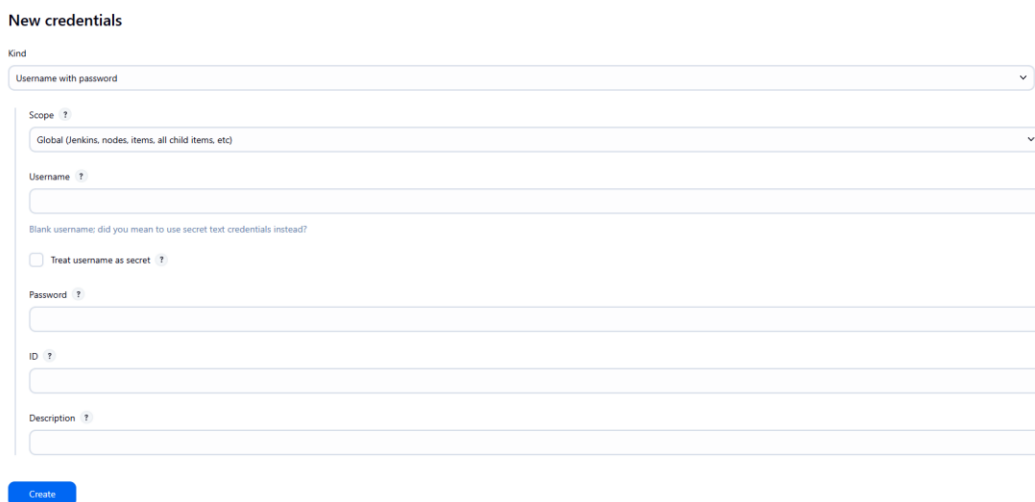
Figura 112. Captura de pantalla del panel de gestión de plugins instalados, mostrando aquellos que coinciden con la búsqueda "Git"

Es importante mencionar que los *plugins* no instalan el servicio de Git en Jenkins o en tu máquina anfitriona, por lo que es necesario instalar Git en la máquina o configurar una instalación directamente en la instancia de Jenkins. Estas configuraciones se detallan en capítulos posteriores.

9.2.3 Configurar Credenciales de acceso a GitHub

Posteriormente, en caso de ser un repositorio privado, es necesario configurar las credenciales de GitHub que permitan el acceso de Jenkins. Tal como se explicó en el capítulo [¿Cómo se usan las credenciales en Jenkins?](#), esto se realiza desde la sección: *Manage Jenkins -> Credentials*.

1. Se debe seleccionar el almacén *System* para consultar los dominios de credenciales, seguido del dominio *Global credentials*, para acceder a las credenciales que pueden ser utilizadas en cualquier *pipeline* o *job*, y hacer clic en el botón *Add Credential*, para agregar una nueva.



New credentials

Kind
Username with password

Scope
Global (Jenkins, nodes, items, all child items, etc)

Username
Blank username: did you mean to use secret text credentials instead?

☐ Treat username as secret

Password

ID

Description

Create

Figura 113. Formulario de registro de credenciales con el tipo "user with password" seleccionado

2. Posteriormente, se debe llenar el formulario correspondiente al tipo de credenciales que se quiere registrar para GitHub. Los expertos recomiendan utilizar la opción *username with password*.
3. Sin embargo, en lugar de ingresar nuestra contraseña real de GitHub, es recomendable utilizar un token de autenticación en su lugar, para esto en GitHub hay que ir a: Perfil -> Configuraciones -> Configuración del Desarrollador -> Token de Acceso Personal -> Tokens de grano fino (o clásicos, ambos funcionan) -> hacer clic en el botón "Generar nuevo Token".

Si necesitas ayuda para generar un token de autenticación en GitHub, puedes revisar los siguientes recursos: [Tutorial: Generar Token de Autenticación GitHub](#) o [Video-Tutorial: Generar Token de Autenticación GitHub](#).

En el formulario hay que ingresar el nombre del token, una descripción opcional, marcar el acceso a los repositorios y asignarles los permisos requeridos: permiso *Administration*, para dar permiso de lectura y escritura, y *Web hooks*, para habilitar los webhooks en prácticas posteriores. Y finalmente dar clic en el botón “Generar Token”.



Figura 114. Captura de pantalla de GitHub donde se muestra un token de grano fino previamente generado

- Una vez se tiene el token, este ya se puede usar en el lugar de la contraseña de usuario, colocándolo en el campo *password* en el formulario de Jenkins.

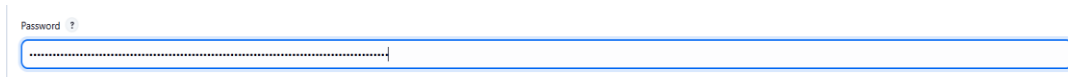


Figura 115. Apartado *password* del registro de credenciales habiendo ingresado un token en lugar de una contraseña

- también se debe asignar un ID único que identifique las credenciales. Es importante que este ID sea claro y conciso, ya que se utilizará posteriormente como referencia en el *Jenkinsfile*.

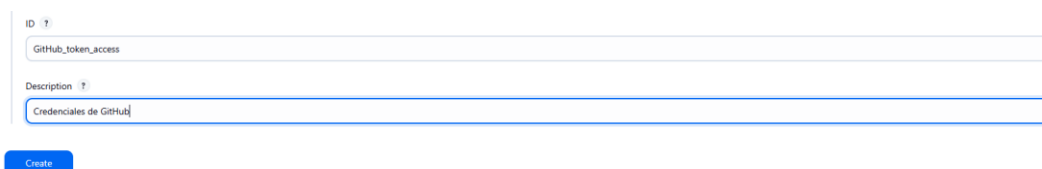


Figura 116. Apartado *ID* del formulario de registro de credenciales



Nota - ¿No quieres vulnerar tu usuario y contraseña?

Si prefieres no usar tu usuario y contraseña como credenciales de autenticación, los expertos también recomiendan usar credenciales de tipo *secret file* en su lugar.

9.2.4 Creación de *job* de tipo *pipeline*

Una vez que se tiene el *Jenkinsfile* en un repositorio de GitHub, se han instalado los *plugins* necesarios, y se han registrado las credenciales necesarias, es posible comunicar Jenkins con GitHub. Pero antes, hay que crear un *Job* de tipo *pipeline*, siguiendo las instrucciones explicadas en la práctica [Creación de Jobs desde la Interfaz](#).

Y justo en el apartado *Definition* de la configuración del *pipeline*, se debe seleccionar la opción ***pipeline script from SCM***, lo que indica que el *pipeline* no se va a escribir directamente en Jenkins, en su lugar se va a recuperar de un sistema de control de versiones como GitHub (Git).

Configure

- General
- Triggers
- Pipeline
- Advanced

Repositories ?

Repository URL ?

Please enter Git repository.

Credentials ?

- none -

+ Add

Advanced ▾

Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/master

Add Branch

Repository browser ?

(Auto)

Additional Behaviours

Add ▾

Script Path ?

Jenkinsfile

☒ Lightweight checkout ?

If selected, try to obtain the Pipeline script contents directly from the SCM without performing a full checkout. The advantage of this mode is its efficiency; however, you will not get any changelogs or polling based on the SCM. (If you use checkout: scm during the build, this will populate the changelog and initialize polling.) Also build parameters will not be substituted into SCM configuration in this mode. Only selected SCM plugins support this mode.

(from Pipeline: Groovy)

Pipeline Syntax

Figura 117. Apartado para configurar el acceso al repositorio para recuperar el *Jenkinsfile* desde Jenkins

Al seleccionar esta opción, se debe registrar información sobre la ubicación del repositorio, así que es necesario que cuentes con:

1. La URL del repositorio en la que se encuentra el Jenkinsfile.
2. El ID de las credenciales registradas previamente, asociadas a la cuenta de GitHub donde está alojado el repositorio con el Jenkinsfile.
3. El nombre de la rama (*branch*) del repositorio en la que se encuentra el Jenkinsfile.
4. Y el nombre específico del archivo Jenkinsfile, tal y como está escrito en el repositorio, porque si no coincide, Jenkins no lo podrá recuperar.

En este apartado también se encuentra la opción ***Ligthweight checkout*** marcada por defecto. Esta opción permite recuperar únicamente la definición del *pipeline* sin clonar el repositorio completo, es recomendable marcar esta opción por cuestiones de eficiencia, para verificar que el *pipeline* se crea correctamente antes de clonar el resto del contenido del repositorio (CloudBees, s/f).

Con estas configuraciones Jenkins ya puede recuperar la definición del *pipeline* automáticamente del repositorio de GitHub, y ejecutar las etapas que estén declaradas en el mismo. En el siguiente capítulo pondrás todo esto en práctica y crearas tu primer *pipeline* con sintaxis declarativa desde cero.



Cuestionario 20. Recuperando la definición de un *pipeline* desde GitHub.

Responde las siguientes preguntas para evaluar la lectura previa y demostrar que has comprendido el proceso de recuperación de *pipelines* de Jenkins desde GitHub.

-
1. Es un elemento indispensable para recuperar *pipelines* desde GitHub.

A) Archivos binarios.	B) Jenkinsfile.	C) Archivo .hpi.	D) <i>Plugin Blue Ocean</i> .
-----------------------	-----------------	------------------	-------------------------------
 2. ¿Qué *plugin* es necesario para que Jenkins pueda trabajar con repositorios de GitHub?

- A) Git Plugin. B) GitHub Plugin. C) Pipeline Plugin. D) Todos los anteriores.
3. ¿En qué sección de Jenkins se configuran las credenciales para acceder a GitHub?
- A) Manage Jenkins → Plugins B) Manage Jenkins → System C) Manage Jenkins → Credentials D) New Item → Pipeline
4. Al crear una credencial en Jenkins para GitHub, ¿Qué es recomendable ingresar en el campo *password*?
- A) La contraseña de la cuenta de GitHub. B) El token de autenticación generado en GitHub. C) La clave SSH pública. D) Ninguno, se deja vacío.
5. Qué opción se debe seleccionar en la definición del *pipeline* para que Jenkins use un Jenkinsfile desde GitHub?
- A) *Pipeline script*. B) *Pipeline script from SCM*. C) *Freestyle Project*. D) *Declarative pipeline*.
6. ¿Qué sucede si el *Script Path* en Jenkins no coincide con el nombre del Jenkinsfile en el repositorio?
- A) Jenkins intenta crear uno por defecto. B) El *pipeline* falla al ejecutarse. C) Jenkins lo busca en la rama *master*. D) No pasa nada, se ignora el campo.
7. ¿Qué permisos mínimos deben asignarse al token de GitHub para que Jenkins pueda acceder y gestionar repositorios?
- A) Solo lectura. B) *Administration y Webhooks*. C) *Push y Pull*. D) Ninguno, el token no requiere permisos.

8. Verdadero o falso: es recomendable guardar el Jenkinsfile en un repositorio de GitHub, porque facilita la organización y control de versiones.
- A) Verdadero. B) Falso.
9. Verdadero o falso: es más seguro usar usuario y contraseña de GitHub en Jenkins que un token de autenticación.
- B) Verdadero. C) Falso.
10. Verdadero o falso: en Jenkins, al crear credenciales para GitHub, el ID asignado puede ser cualquier texto porque no tiene relevancia en el Jenkinsfile.
- B) Verdadero. C) Falso.

9.3 Tu primer *pipeline* paso a paso



Práctica 2. Creando tu primer *pipeline* paso a paso

Avanzado

Objetivo: el estudiante crea *pipelines* declarativos en Jenkins a partir de archivos Jenkinsfiles y lo integra con un sistema control de versiones (GitHub).

Prerrequisitos: servidor de Jenkins instalado y funcionando correctamente en una máquina virtual de Linux con Ubuntu Server, cuenta activa en GitHub y editor de texto (preferentemente Visual Studio Code).

Descripción

La siguiente práctica guía paso a paso en la creación de un *pipeline* declarativo de Jenkins, mostrando cómo integrarlo con GitHub para recupera la definición del *pipeline* a través del Jenkinsfile. Se detallan los pasos necesarios para preparar el entorno de Jenkins, configurar

un *job* de tipo *pipeline*, elaborar y definir un Jenkinsfile con su estructura y directivas esenciales, ejecutar el *pipeline* y, por último, configurar su automatización para que se ejecute de manera automática ante cambios en el repositorio.

9.3.1 Preparación del entorno de trabajo

Antes de comenzar esta práctica, debes saber que la definición del *pipeline* y todos los archivos y elementos que se utilizarán en prácticas posteriores, se encuentran ubicados en el [repositorio público de esta guía](#). En este caso, el código base del *pipeline*, que es el que se desarrollará a lo largo de esta práctica, se encuentra en la rama *pipeline-base* como referencia.

Ahora bien, antes de comenzar las configuraciones, asegúrate de tener instalada la herramienta Git en el servidor donde alojas Jenkins. Por otra parte, debes asegurarte de tener instalados los *plugins* Pipeline, Git y GitHub dentro de tu instancia de Jenkins. Para conocer más sobre los *plugins* en general, puedes consultar el capítulo [Plugins](#) de esta guía; para conocer más sobre estos tres *plugins* específicos, puedes consultar específicamente la sección [Jenkins y GitHub](#).

Además, deberás configurar un repositorio privado en GitHub y crear un token de acceso, que deberás registrar como una credencial de Jenkins siguiendo los pasos mencionados en la sección Es importante mencionar que los *plugins* no instalan el servicio de Git en Jenkins o en tu máquina anfitriona, por lo que es necesario instalar Git en la máquina o configurar una instalación directamente en la instancia de Jenkins. Estas configuraciones se detallan en capítulos posteriores.

Configurar Credenciales de acceso a GitHub de esta guía. Esto, para que más adelante subas la definición del *pipeline* hacia ese repositorio y puedas seguir la práctica tal y como está especificada.

Es cierto que puedes consumir la definición del *pipeline* desde el Jenkinsfile en el repositorio público mencionado al inicio, sin la necesidad de configurar un repositorio personal privado. Sin embargo, el hecho de realizar esta configuración por tu cuenta será de gran ayuda, pues seguramente los repositorios desde donde tengas que acceder a las definiciones de los *pipelines* en futuros proyectos del ámbito profesional, serán privados. Por esta razón recomendamos realizar un *fork* del repositorio público de esta guía.

9.3.2 Definición del *pipeline*

En este nivel, la definición de nuestro *pipeline* será muy simple, pues el objetivo de esta sección es aprender a conectar Jenkins con GitHub y traer la definición del *pipeline* desde ahí. Capítulos posteriores demostrarán cómo construir los proyectos, probarlos y desplegarlos, trabajando sobre este “esqueleto” inicial del *pipeline*.

Considerando esto, a continuación se muestra la definición del pipeline:

```
1 pipeline {
2   agent any
3
4   //tools {}
5
6   //environment {}
7
8   stages {
9     stage('Build') {
10      steps {
11        echo 'Compilando la aplicación...'
12      }
13    }
14    stage('Test') {
15      steps {
16        echo 'Ejecutando pruebas...'
17      }
18    }
19    stage('Deploy') {
20      steps {
21        echo 'Desplegando la aplicación...'
22      }
23    }
24  }
25
26  post {
27    always {
28      echo 'Finalizando ejecución del pipeline...'
29    }
30  }
31 }
```

Figura 118. Definición inicial del *pipeline*

Como se mencionó previamente, esta definición del *pipeline* es sumamente básica. Por esto mismo y con base en lo aprendido en el capítulo [Introducción a los pipelines](#), sabemos que el lector entiende muy bien qué significa cada una de las partes que lo componen. Aun con esto, recapitemos la estructura presentada y su significado:

1. **agent any** (línea 2): al estar en el nivel más alto, está respondiendo la pregunta ¿Dónde se ejecutará el *pipeline*? Al colocar *any*, le respondemos “donde se pueda”. Como en este caso nuestra instancia de Jenkins no está configurada para conectarse con otros nodos, la única opción es que se ejecute dentro del propio nodo de la instancia.
2. **tools {}** (línea 4): esta línea está comentada y solamente sirve como referencia futura, para considerar que será necesaria. Realmente no estamos especificando ninguna herramienta y si no estuviera comentada, causaría problemas por este hecho.
3. **environment {}** (línea 6): pasa algo similar a lo que pasó con la directiva *tools*. La colocamos como referencia futura pensando en que ahí podrán vivir las credenciales que necesitemos. Sin embargo, la comentamos dado que colocarla vacía causaría problemas.
4. **stages {...}** (líneas 8-24): aquí se están definiendo tres *stages*, una para cada etapa del *pipeline* de despliegue (*build*, *test* y *deploy*). En cada una de las etapas, solamente se está imprimiendo un mensaje representativo de cada etapa.
5. **post {...}** (líneas 26-30): aquí simplemente se imprime un mensaje final siempre que termina la ejecución de los *stages*, sin importar si el resultado fue exitoso o fallido (por eso la directiva *always*).

En el ejemplo que se muestra en las imágenes siguientes, se trabajó sobre la rama *pipeline-base*, sin embargo, usted puede trabajar sobre la rama *main* o cualquiera que considere adecuada. Esto afectará una configuración en el *job*, pero se hará énfasis cuando lleguemos a ella.

9.3.3 Creación del Jenkinsfile

Ahora bien, aclarado esto anterior y con el entendido del *pipeline* sobre el que estaremos trabajando, tenemos dos alternativas para proceder: podemos realizar el *fork* del repositorio o podemos implementar la definición en nuestro entorno local desde cero. Se explican ambas opciones en este apartado, sin embargo se recomienda seguir la segunda para ganar mayor experiencia práctica en la creación de *pipelines*.

Para realizar simplemente el *fork* con el Jenkinsfile ya explicado:

1. Inicia sesión en tu cuenta de GitHub e ingresa al [repositorio público de esta guía](#).
2. En la esquina superior derecha haz clic en el botón "Fork", esto abrirá una ventana.
3. Deselecciona la opción *Copy the main Branch Only*, para así copiar todas las ramas del repositorio.
4. Adicionalmente, puedes agregar una descripción y cambiar el nombre del repositorio si así lo quieres.
5. Finalmente, haz clic en el botón *Create Fork*. Ahora ya cuentas con un copia privada del repositorio, donde podrás modificar los archivos, el proyecto y el propio Jenkinsfile.

Si tienes dudas respecto a los *fork* de GitHub o sobre el proceso para realizarlos, puedes consultar el siguiente recurso: [Tutorial: Crear Forks en GitHub](#).

La rama *pipeline-base* del repositorio ya cuenta con el código final del Jenkinsfile de esta práctica. Sin embargo, puedes realizar todo desde cero, generando una rama propia a partir de main (o en el propio main) y siguiendo los pasos descritos a continuación:

1. Clona el repositorio privado hacia tu entorno local.
2. Abre la carpeta del proyecto en tu editor de código.
3. Crea un nuevo archivo en la base del proyecto llamado Jenkinsfile y escribe dentro de él la definición del *pipeline* previamente mostrada. Se debería ver algo como lo siguiente:

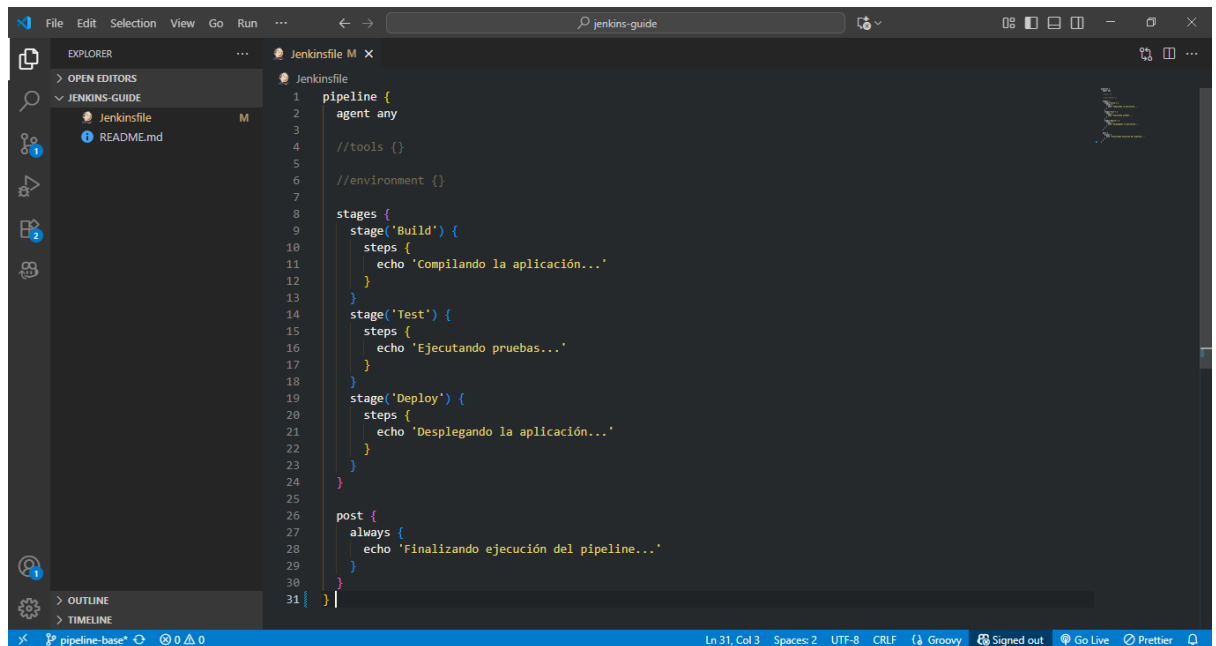
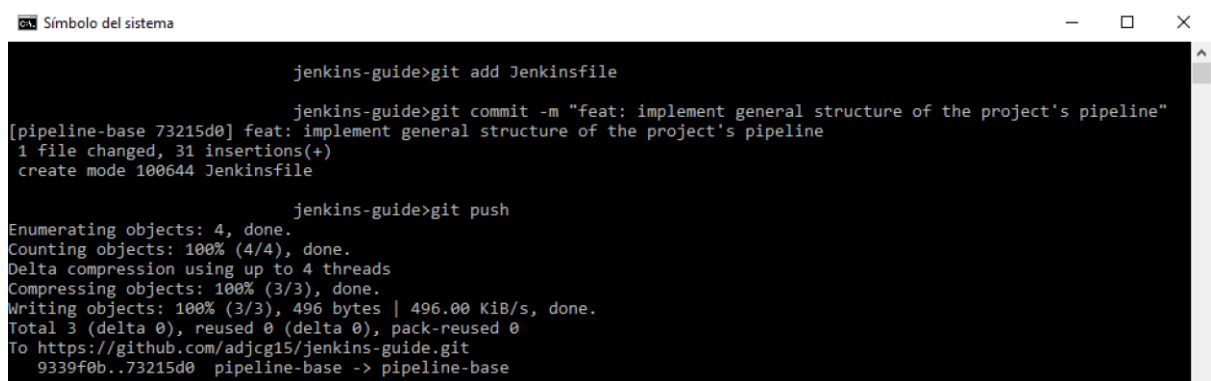


Figura 119. Captura de pantalla del editor de código con el Jenkinsfile abierto

4. Abre una terminal en la ruta del proyecto y realiza un *commit* que incluya el Jenkinsfile previamente creado, para después subirlo al repositorio de GitHub:
 - a. ***git add Jenkinsfile***
 - b. ***git commit -m "Primera versión del pipeline"*** (o el nombre de *commit* de tu agrado)
 - c. ***git push -u origin main*** (o en su defecto el nombre de la rama sobre la que estés trabajando). Si no es tu primer *commit*, bastaría un ***git push***.



Con estos pasos previos, tendremos la definición de nuestro *pipeline* alojada en nuestro repositorio de GitHub, lista para ser consumida desde Jenkins y comenzar a automatizar nuestro *pipeline* de despliegue.

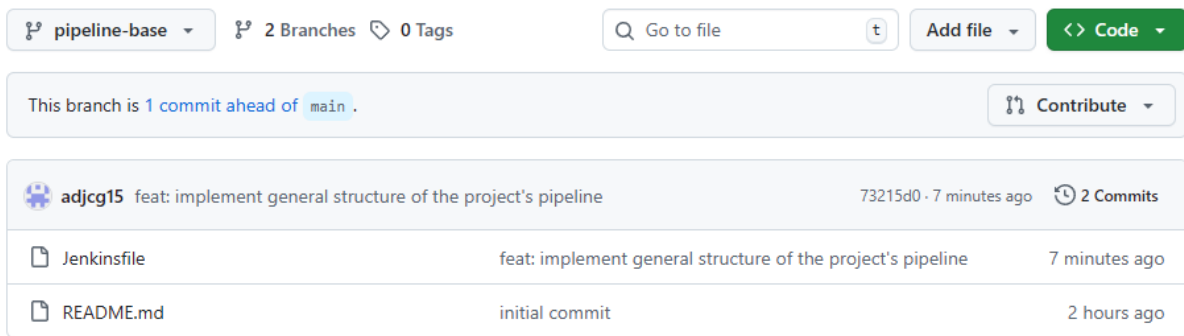


Figura 120. Captura de pantalla de GitHub con el Jenkinsfile cargado

9.3.4 Creación del *job*

Con la definición del *pipeline* lista dentro del repositorio, debemos consumirla desde nuestro *job* de tipo *pipeline* desde Jenkins. Para esto, sigamos los siguientes pasos:

1. Ve a la pantalla de creación de *jobs* y registra uno nuevo. Asigna un nombre que puedas identificar fácilmente y selecciona el tipo de *job pipeline*.

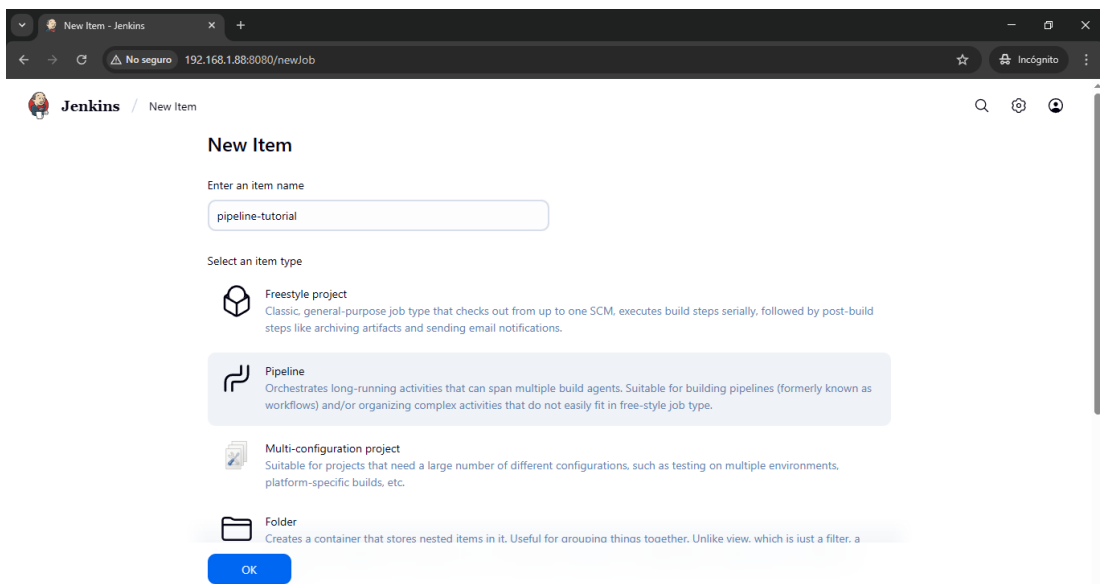
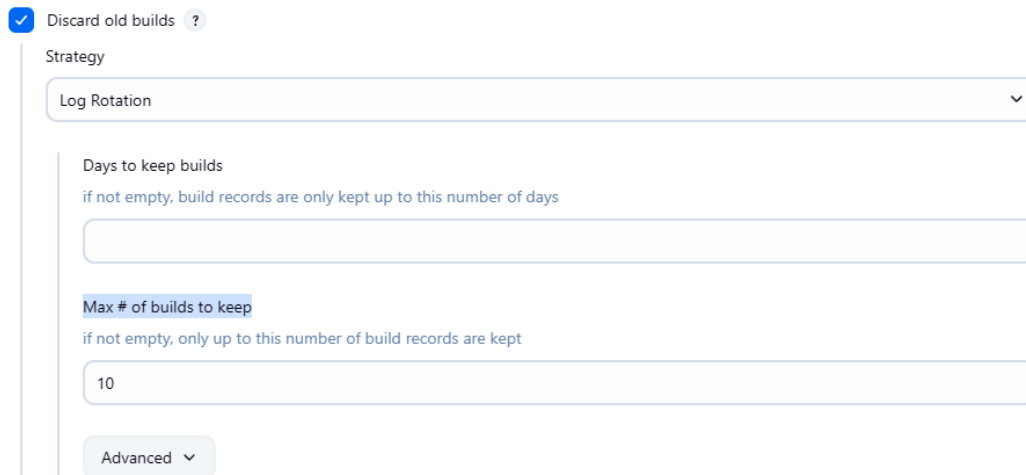


Figura 121. Pantalla de creación de *jobs* en Jenkins

2. Una vez creado el *job*, en la sección general de configuración, selecciona la opción *Discard old builds* y asigna 10 en la opción *Max # of builds to keep* desplegada. Esto hará que Jenkins solamente retenga las últimas 10 ejecuciones del *pipeline*.



The screenshot shows the 'Discard old builds' configuration in Jenkins. At the top, there is a checked checkbox labeled 'Discard old builds' with a help icon. Below it, the 'Strategy' dropdown menu is set to 'Log Rotation'. Under this, there are two input fields: 'Days to keep builds' (with a hint 'if not empty, build records are only kept up to this number of days') and 'Max # of builds to keep' (with a hint 'if not empty, only up to this number of build records are kept'). The 'Max # of builds to keep' field contains the value '10'. At the bottom, there is an 'Advanced' dropdown menu.

Figura 122. Configuración para mantener solamente los últimos 10 builds

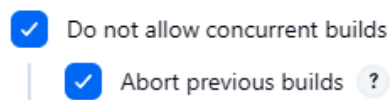


Nota - Limpia tu espacio

Los expertos recomiendan realizar limpieza periódica del historial de *builds* y artefactos dentro de Jenkins. La opción *Discard old builds* en la sección de configuración general de los *pipelines* ayuda con esto, manteniendo solamente los *N builds* más recientes.

- De nuevo dentro de la sección de configuración general del *pipeline*, selecciona la opción *Do not allow concurrent builds* y después la opción *Abort previous builds* desplegada. Esto optimizará la ejecución de nuestro *pipeline*, manteniendo solamente una ejecución a la vez.

Esto nos beneficia cuando, por ejemplo, hay dos eventos que disparan la ejecución del *pipeline* y que son muy cercanos en el tiempo, obligando a Jenkins a mantener solamente el *build* o ejecución más reciente.



The screenshot shows two checked checkboxes in the Jenkins configuration: 'Do not allow concurrent builds' and 'Abort previous builds' (which has a help icon).

Figura 123. Configuración para deshabilitar builds paralelos

- Pasando a la sección de *triggers*, déjala tal cual se encuentra, sin cambiar nada por el momento. Más tarde regresaremos a configurarla, pero por ahora quedará sin cambios.

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- ☐ Build after other projects are built ?
- ☐ Build periodically ?
- ☐ GitHub hook trigger for GITScm polling ?
- ☐ Poll SCM ?
- ☐ Trigger builds remotely (e.g., from scripts) ?

Figura 124. Panel de configuración de los triggers del pipeline

5. Para la sección de *pipeline*, consumiremos la definición del *pipeline* que definimos previamente. Como primer paso, selecciona *pipeline script from SCM* en el campo *definition*.

Definition

Pipeline script from SCM

Figura 125. Selección de "pipeline script from SCM" para la definición del pipeline durante su configuración

6. Al seleccionar esta opción previa, se habilitarán múltiples campos relacionados con la configuración del acceso al repositorio donde se encuentra alojada la definición del *pipeline*. En el campo *SCM*, el primero que aparece, seleccione *Git*, pues en nuestro caso accederemos a un repositorio de GitHub.

SCM ?

Git

Figura 126. Configuración de Git como SCM al configurar el pipeline

7. El siguiente apartado es *repositories*. En este apartado, coloca la URL de tu repositorio privado, donde subiste tu archivo *Jenkinsfile*, dentro del campo *repository URL* (o la del repositorio público de la guía, en su defecto); esta URL debe ser la misma que utilizarías como si fueras a clonar el repositorio, no la URL directa de GitHub. Además, selecciona las credenciales (*user and password*), que configuraste para acceder a este repositorio, en el campo *credentials*.



Nota - ¿Por qué aparece otro nombre de credenciales?

Cuando configuras un *pipeline* y debe seleccionar credenciales, particularmente de tipo *user and password*, puede llegar a pasar que el

nombre de la credencial se muestre como *user/******, y no el ID que le asignaste al crearla. Para evitar esto, al configurar las credenciales, marca la casilla *Treat username as secret*.



Figura 127. Configuración relacionada con el acceso al repositorio durante la creación de un pipeline

8. En el siguiente apartado, *branches to build*, escribe el nombre de la rama donde se encuentra ubicado tu archivo *Jenkinsfile*. En el ejemplo, la rama es */pipeline-base*, sin embargo, tú coloca el nombre de la rama en donde subiste la definición de tu *pipeline*: muy probablemente */master* o */main*.



Figura 128. Selección de la rama del repositorio durante la configuración de un pipeline

9. Las demás opciones déjelas tal como están y pasa directamente al campo *script path*. En él, coloca la ruta del archivo *Jenkinsfile* dentro del repositorio. En este ejemplo, el *Jenkinsfile* se encuentra en la raíz del proyecto y dentro del campo se coloca ese valor. Sin embargo, podría pasar que el archivo *Jenkinsfile* se encuentre dentro de carpetas como */src* o similares; en ese caso, el campo debería especificarse como */src/Jenkinsfile*.

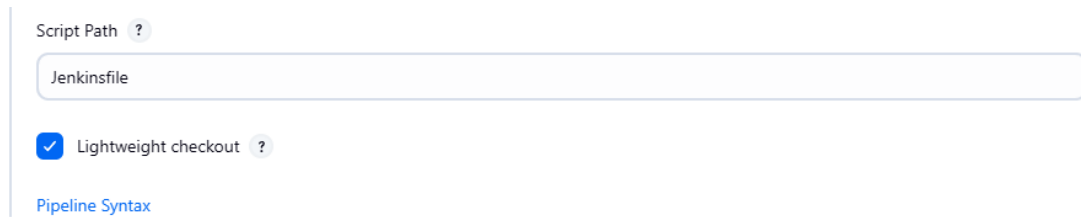


Figura 129. Configuración para indicar la ruta hacia el Jenkinsfile durante la creación de un pipeline

10. La sección *advanced* déjela sin modificar y finaliza dando clic en el botón save.

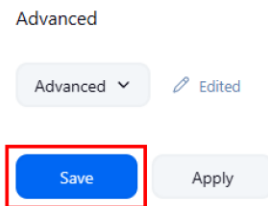


Figura 130. Captura de pantalla que resalta el botón para guardar los cambios al finalizar la configuración de un pipeline

9.3.5 Ejecución manual del pipeline

Con la configuración anterior, queda probar nuestra implementación. Para ello, ejecutaremos manualmente el *job* (nuestro *pipeline*). Con este fin:

1. Abre el panel del *pipeline* y da clic en la sección *build now*, tal como se explicó al inicio de este capítulo, en el apartado [¿Cómo ejecutar pipelines?](#).
2. Una vez que termine la ejecución, en la barra lateral que aparece a la izquierda, aparecerá el resultado de dicha ejecución. Para entrar al detalle, daremos clic en dicho resultado.

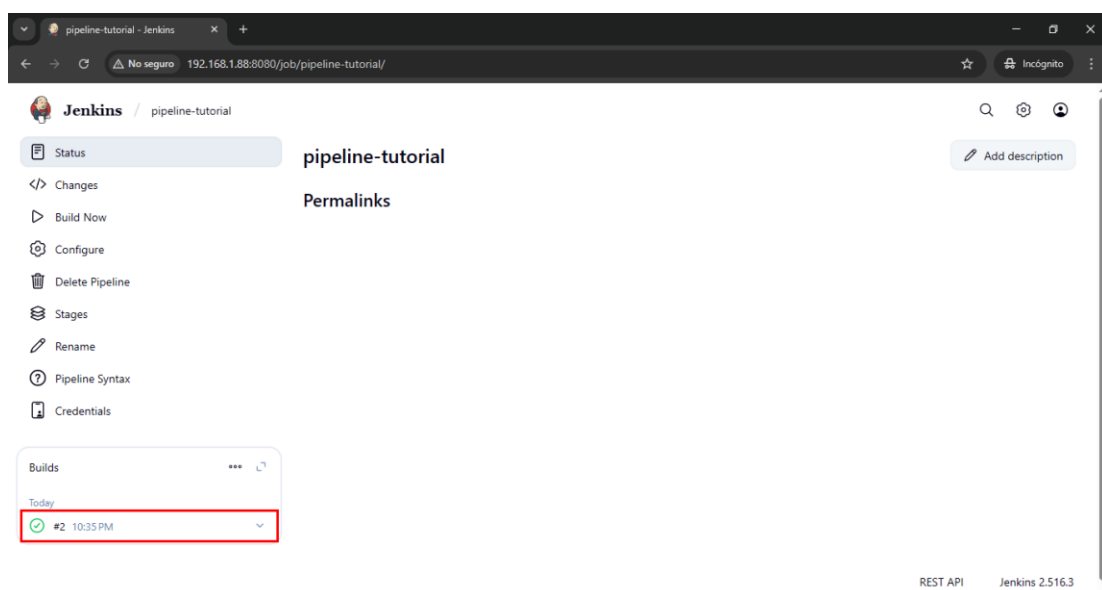


Figura 131. Captura de pantalla que resalta la sección para entrar al detalle de un build

3. Esto abrirá la ventana de detalle de la ejecución (*build*), tal como se muestra a continuación:

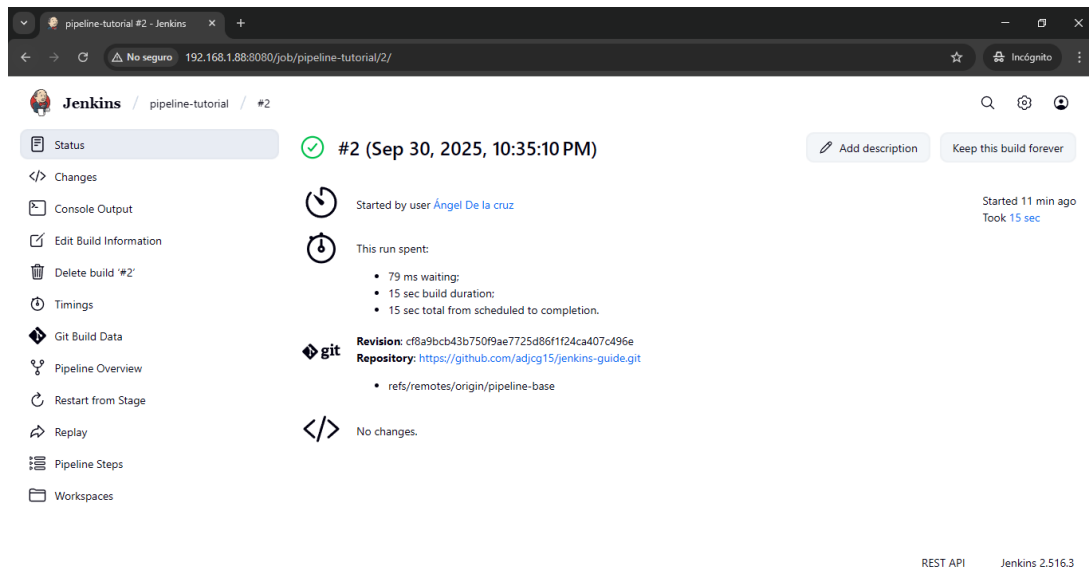


Figura 132. Ventana con el detalle de la ejecución de un pipeline en Jenkins

4. En esta sección podemos consultar detalles generales de la ejecución, como el tiempo que tardó en completarse, el repositorio asociado, el responsable de la ejecución, entre otros datos. Sin embargo, nos enfocaremos en ver los logs de la ejecución. Para ello, haz clic en la sección *console output* de la barra lateral.

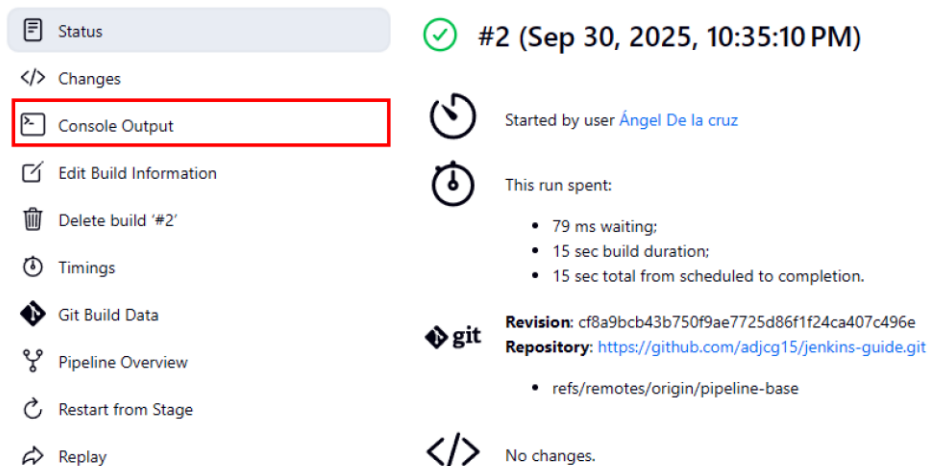


Figura 133. Captura de pantalla que resalta la sección para entrar a los logs de salida de la consola de la ejecución

5. Esto abrirá la salida en consola de la ejecución del *pipeline*, tal como se muestra en la siguiente figura.

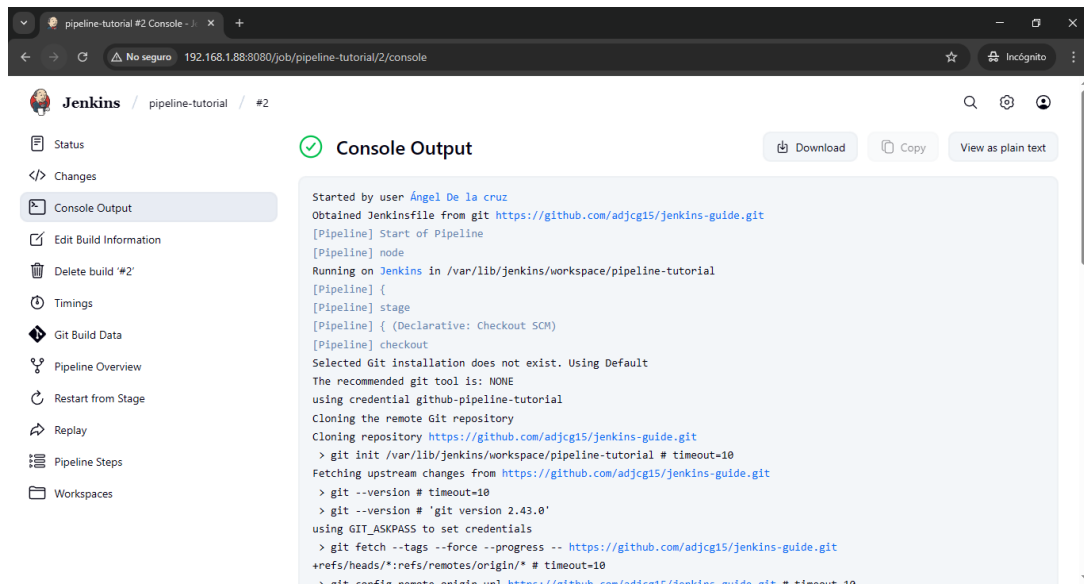


Figura 134. Captura de pantalla de la salida en consola de la ejecución de un pipeline en Jenkins

6. Observa que aquí es donde se muestran los mensajes `echo` que colocamos en la definición del *pipeline*. Considera que estas ejecuciones persisten y pueden ser accedidas dentro de la instancia de Jenkins. Considerando esto, ten mucho cuidado con qué es lo que muestras a través de la consola.

```
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Compilando la aplicación...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Ejecutando pruebas...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Desplegando la aplicación...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Finalizando ejecución del pipeline...
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figura 135. Resultados en consola de la ejecución del pipeline

Con estos pasos, logramos ejecutar manualmente el *pipeline* y verificar su correcta ejecución. Usualmente si ocurre algún error, es a través de la consola previamente abordada donde podremos inspeccionar qué es lo que está pasando.

Sin embargo, antes de cerrar esta sección, valdría la pena destacar dos apartados más dentro de la ventana de detalle de la ejecución: *pipeline overview* y *workspaces*; marcadas como 1 y 2 en la siguiente imagen.

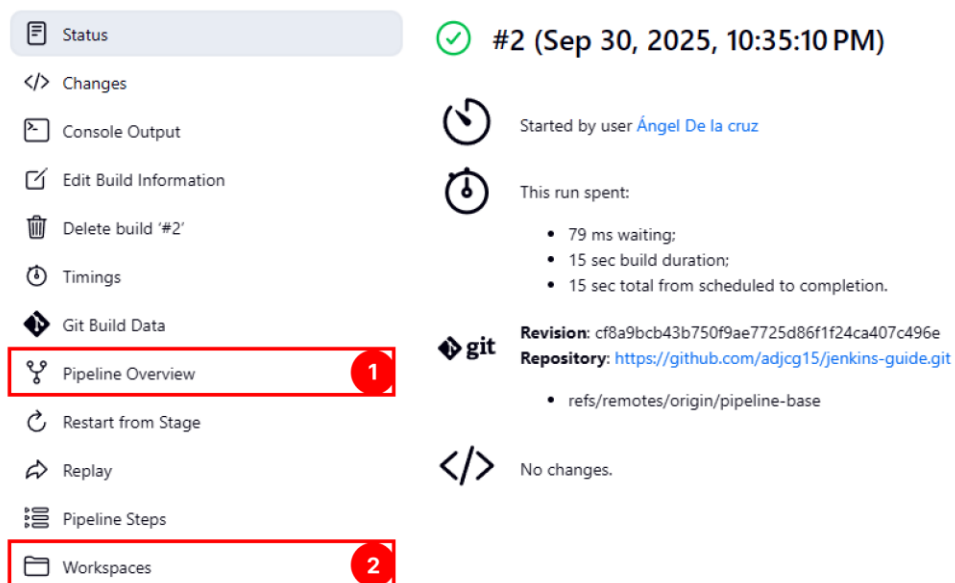


Figura 136. Captura de pantalla con las secciones *Pipeline Overview* (1) y *Workspaces* (2) resaltadas

La primer sección abre una ventana de resumen del *pipeline*. En esta puede consultar un resumen de la ejecución del *pipeline* por etapa. Le animamos a que la explore.

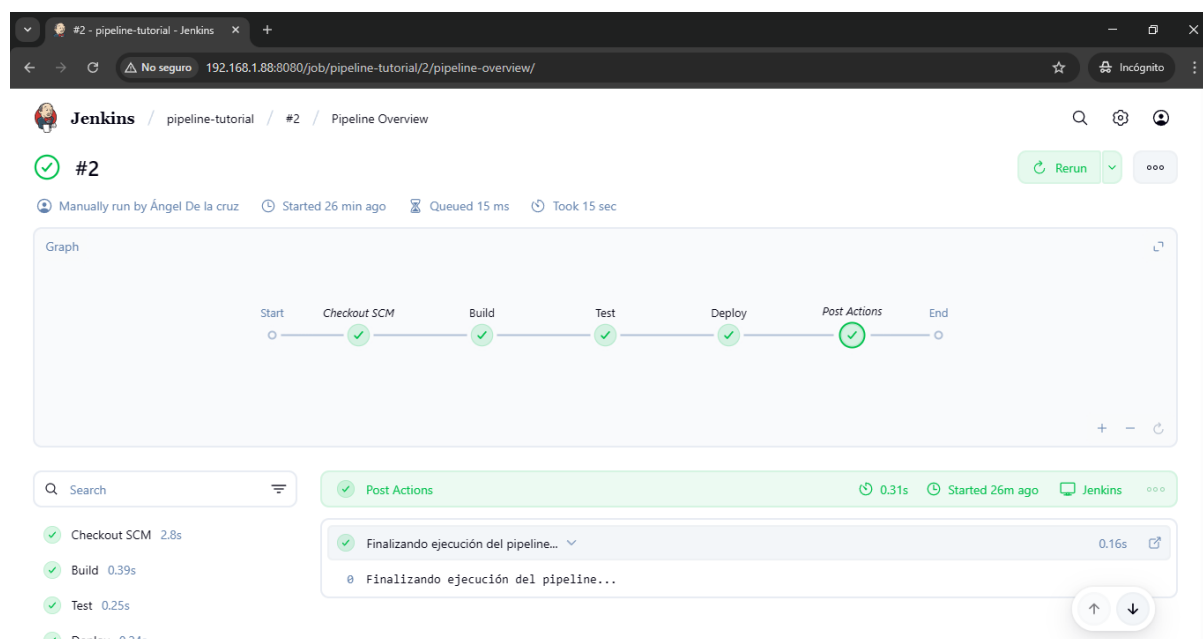


Figura 137. Pantalla de resultados de la ejecución (*Pipeline Overview*)

La segunda sección mencionada abre una ventana con un enlace hacia el espacio de trabajo del *pipeline*. En el cual, si accedemos, podremos ver qué es lo que hay ahí. Te animamos a que lo explores y veas los archivos que se encuentran ahí. ¿Por qué están ahí? Eso es algo que abordaremos más adelante en esta guía.

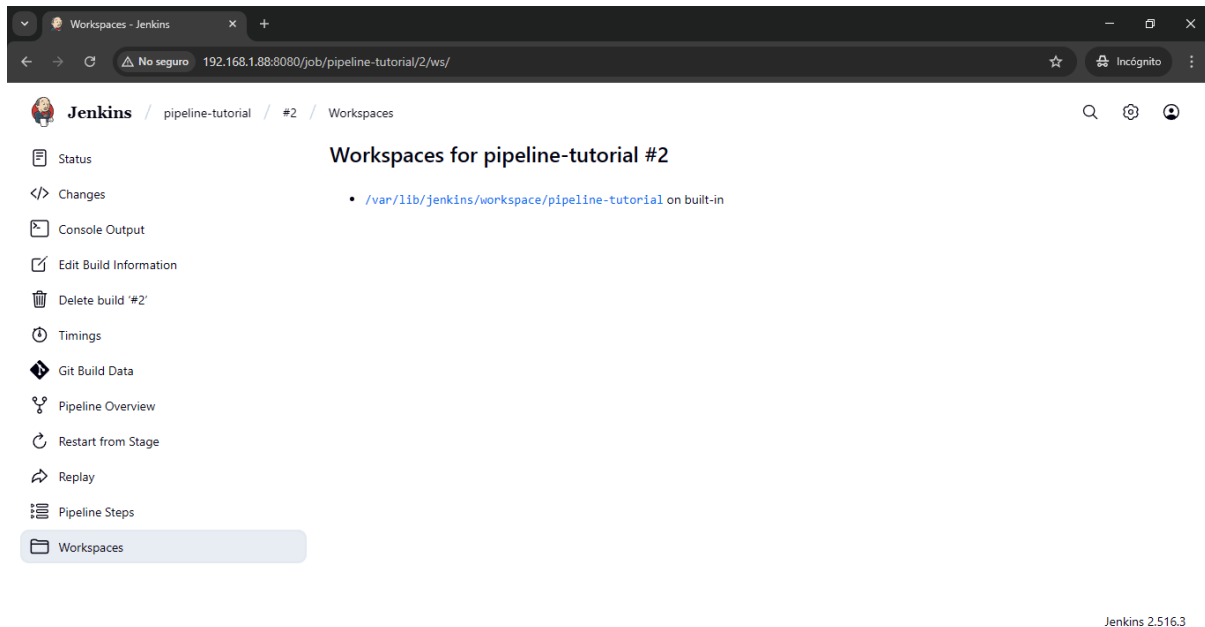


Figura 138. Pantalla del espacio de trabajo del pipeline en Jenkins

9.3.6 Automatización de la ejecución del *pipeline*

Comúnmente buscaremos que nuestro *pipeline* se ejecute tras ciertos eventos. No queremos acceder a Jenkins y seleccionar un botón cada vez que queramos correr nuestro *pipeline*; esto le quitaría un poco el sentido a la automatización.

En el contexto de repositorios Git, hay dos enfoques populares para automatizar la ejecución: *polling* y *webhooks*. *Polling* es el proceso de revisar periódicamente el repositorio para verificar si hay nueva información que procesar; esto se hace desde Jenkins hacia el repositorio. Los *webhooks* básicamente son una suscripción a ciertos eventos; estos eventos se envían desde el repositorio hacia Jenkins (Arun, 2024).

Aquí es importante mencionar que los expertos recomiendan configurar las ejecuciones del *pipeline* de tal forma que solo se activen en eventos importantes, por ejemplo, a través de *webhooks* con eventos como *push* al repositorio. Esto, debido a que estrategias como *polling* suelen consumir muchos recursos dada la revisión periódica que realizan.

Ahora bien, aun considerando esto anterior, en esta demostración utilizaremos *polling* y no *webhooks*. Esta decisión se debe a los requisitos para configurar *webhooks*.

Actualmente estamos trabajando con una instancia de Jenkins local. Los *webhooks* requieren tener el servidor de Jenkins expuesto en Internet, para permitir que el repositorio (como GitHub) envíe los eventos hacia Jenkins, lo cual queda fuera de los alcances de esta guía.



Nota - Webhooks y Jenkins

Aunque la configuración de *webhooks* y GitHub queda fuera del alcance de esta guía, te animamos a que, si cuentas con conocimientos más avanzados en la exposición de servidores a través de internet, explores [tutoriales que aborden la configuración de webhooks](#). ¡Continúa aprendiendo y prueba tus límites!

Con esta breve introducción, pasemos al “paso a paso” de cómo configurar el *polling* de nuestro repositorio GitHub:

1. Como primer paso, abre la configuración del *pipeline* y dirígete a la sección de *triggers*.

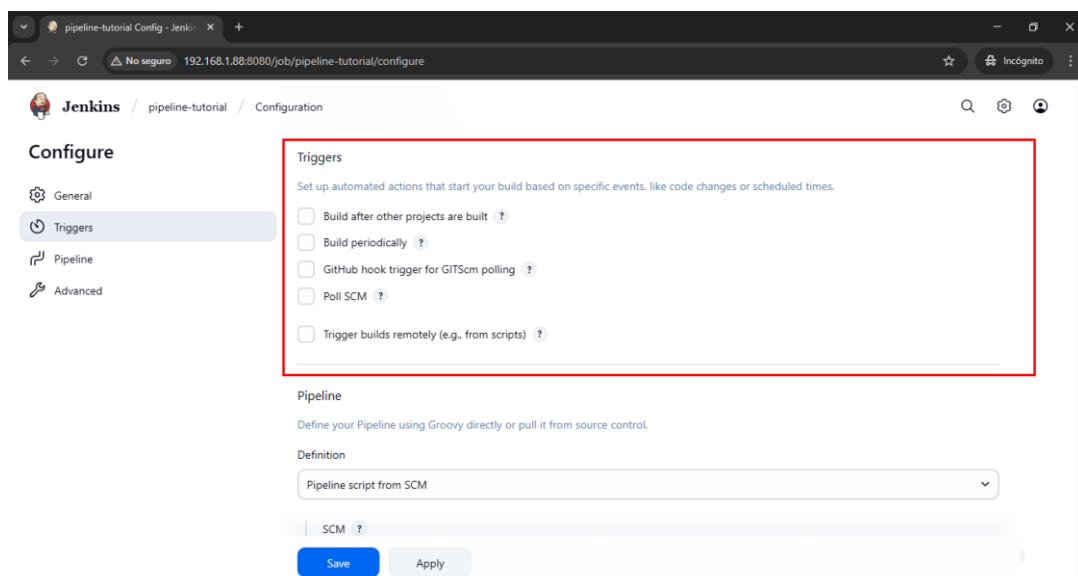


Figura 139. Captura de pantalla que resalta la sección de *triggers* en la configuración de un *pipeline*

2. En esta sección, selecciona la opción *Poll SCM*. Esto mostrará un campo llamado *Schedule*, donde debes ingresar el valor **H/5 * * * ***. Esta es una [expresión cron](#) que significa: “revisa el repositorio cada cinco minutos, comenzando en un minuto calculado automáticamente para distribuir la carga de la instancia de Jenkins; y en caso de que encuentres cambios en el

repositorio, ejecuta el *pipeline*". Esto solo es demostrativo, en un escenario real, revisar por cambios cada cinco minutos muy probablemente no sea la mejor decisión, dada la carga computacional que representaría.

Figura 140. Configuración de la expresión cron como trigger de Poll SCM en pipeline de Jenkins

3. Una vez realizados estos cambios, haz clic en el botón Save para guardar los cambios sobre la configuración de nuestro *pipeline*.
4. Ahora, para probar si este cambio surtió efecto, abre el *Jenkinsfile* en tu editor de texto y cambia la directiva *post* tal y como se muestra a continuación:

```

1 post {
2     always {
3         echo 'Finalizando ejecución del pipeline con Polling!'
4     }
5 }

```

Figura 141. Cambio en la directiva post para probar el Poll SCM

5. Posteriormente, haz *commit* de este cambio y *push* hacia la misma rama que indicaste durante la configuración del *pipeline*.

```

jenkins-guide>git add Jenkinsfile
jenkins-guide>git commit -m "chore: test polling with minimal definition change"
[pipeline-base c6e6adb] chore: test polling with minimal definition change
1 file changed, 1 insertion(+), 1 deletion(-)
jenkins-guide>git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 355 bytes | 355.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/adjcg15/jenkins-guide.git
cf8a9bc..c6e6adb pipeline-base -> pipeline-base

```

Figura 142. Actualización de repositorio para probar Poll SCM

- Posterior a esto, espera en la ventana de gestión del *pipeline* hasta que aparezca el nuevo *build* de forma automática. En el siguiente ejemplo, el *build* marcado con (1) fue disparado por el *polling*.

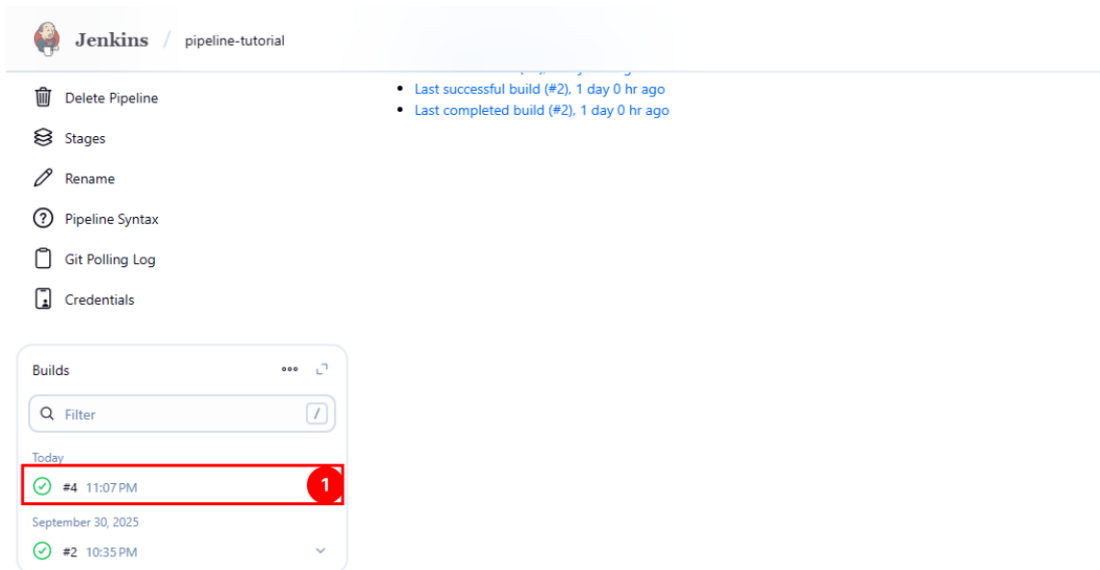


Figura 143. Captura de pantalla donde se resalta el build lanzado automáticamente gracias al Poll SCM

- Entra a la ventana de detalles del *build*. Observa que en el detalle del *build*, indica que este comenzó por un cambio en el sistema de control de versiones, a diferencia del *build* manual, que mostraba el nombre de quien comenzó el *build*.

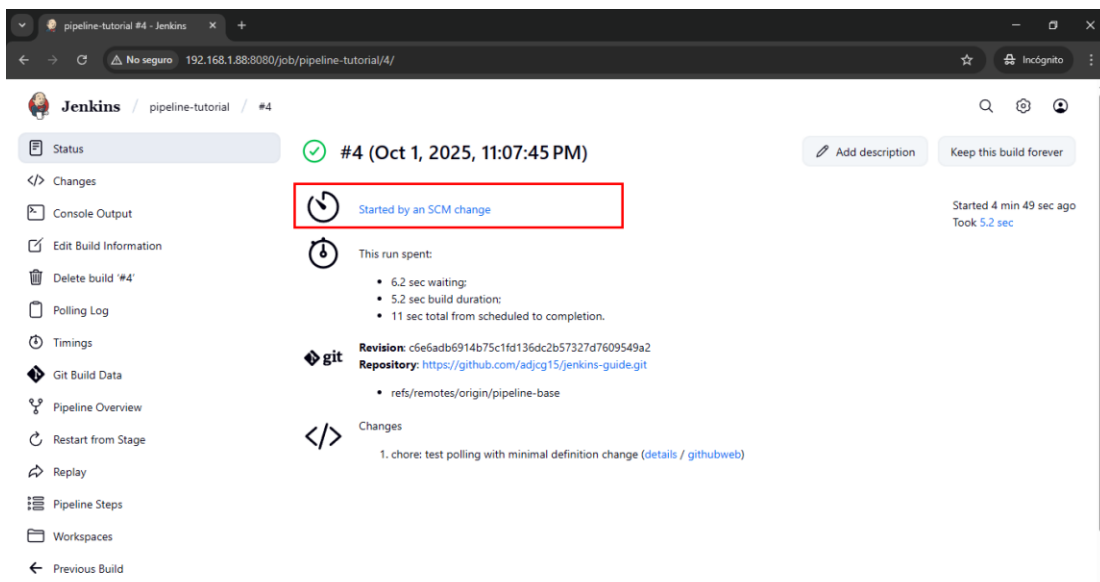


Figura 144. Captura de pantalla donde se resalta la sección que indica que el build fue "disparado" automáticamente después de un cambio en el sistema de control de versiones

- Ahora abre la consola del *build*. Observa que se muestra el mensaje que cambiamos en la directiva *post* y finaliza satisfactoriamente esta práctica.

```
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Finalizando ejecución del pipeline con Polling!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
```

Figura 145. Resultado en consola de la ejecución del pipeline, tras cambiar el mensaje final

Recuerda, con esta configuración de *polling* no estamos estableciendo una ejecución periódica del *pipeline*. Lo que hacemos es establecer una revisión periódica del repositorio (GitHub) para que, en caso de haber cambios comparando con nuestra versión en Jenkins, se ejecute un *build* del *pipeline*.

Probablemente ahora no haga mucho sentido el haber realizado esta última configuración, pues el *pipeline* que tenemos no cambiará y se mantendrá estático. Sin embargo, considere que este *pipeline* lo iremos mejorando a lo largo de la guía y que incluso lo integraremos con un proyecto. Es en esta integración con un proyecto donde cobrará sentido la configuración, pues todo esto no solamente traerá la definición del *pipeline*, sino que clonará el repositorio completo (incluido el proyecto) hacia el espacio de trabajo en Jenkins. Pero esto solo es un vistazo a capítulos posteriores, donde se explica todo esto con mayor detalle.

Antes de cerrar esta práctica, le recomendamos desactivar la configuración del *trigger* previamente realizada para que el servidor de Jenkins no consuma muchos recursos en su equipo.



Rúbrica 2. Evaluación de práctica 2 "Creando tu primer pipeline paso a paso"

En la siguiente rúbrica, marca "Sí" o "No" en cada aspecto de evaluación, para determinar si tu implementación cumple con los criterios requeridos en la implementación de *pipelines* declarativos con Jenkins y GitHub.

Rubrica de Evaluación		
Aspecto por evaluar	Registro de Cumplimiento	
	Si	No

Verifiqué que los plugin necesarios estuvieran instalados antes de comenzar la configuración del <i>pipeline</i> .		
Configuré un repositorio privado de GitHub para seguir la práctica.		
Generé un token de acceso en GitHub y lo utilicé al configurar mis credenciales en Jenkins.		
Creé un Jenkinsfile en mi repositorio local, siguiendo la definición de ejemplo de la práctica.		
Cargué mi Jenkinsfile a mi repositorio privado en GitHub utilizando los comandos de git.		
Configuré un job de tipo pipeline comprendiendo la sección Pipeline script from SCM .		
Ejecuté el pipeline manualmente y todas las etapas se completaron sin errores.		
Revisé los logs de la consola de ejecución del pipeline para confirmar que cada etapa se ejecutó correctamente.		
Exploré las secciones pipeline overview y workspaces dentro de la ventana de detalle de ejecución.		
Automaticé la ejecución del <i>pipeline</i> utilizando polling .		
Comprobé la ejecución automatizada del pipeline realizando un <i>commit</i> hacia mi repositorio privado de GitHub.		



Capítulo 10

10 Construcción y empaquetado de software

Contenido del capítulo

- 10.1. Construcción VS Empaquetado de software.
- 10.2. Actividades de construcción y empaquetado de software.
- 10.3. Maven como herramienta de construcción y empaquetado.
- 10.4. Comandos comunes en Maven.
- 10.5. Integración de Maven con Jenkins.

10.1 Construcción VS Empaquetado de software



Lectura 24. ¿Qué es la construcción y empaquetado de software?

Nivel básico

Objetivo: el estudiante reconoce la definición de construcción de software y de empaquetado de software, y distingue claramente entre ambos conceptos.

Descripción

La lectura aborda el concepto de construcción de software y empaquetado de software como etapas dentro del ciclo de desarrollo de software, resaltando la diferencia entre ambos conceptos.

Entre las primeras actividades que se realizan dentro de un *pipeline* de despliegue, se encuentran la construcción del software (*build*) y el empaquetado (*packaging*) del mismo. Aunque ambas actividades son similares y suelen confundirse, estas presentan diferencias importantes en cuanto a su propósito.

Según el estándar IEEE (2021), la **construcción del software (*build*)** es el proceso de generar un sistema ejecutable y verificable a partir de una versión de origen, mientras que el **empaquetado (*packaging*)** consiste en combinar todos los componentes relacionados en un solo elemento, listo para ser desplegado.

En otras palabras, la **construcción del software o *build*** es el proceso de compilar los archivos de código fuente, convirtiéndolos en código máquina ejecutable. Este proceso suele incluir la ejecución de pruebas unitarias y análisis estáticos de código. Por otra parte, el **empaquete (*packaging*)** se puede entender como un proceso posterior al *build*, en el que el software compilado, junto con todos los recursos necesarios (bibliotecas, imágenes, archivos de configuración, entre otros), se ensamblan en conjunto en un formato listo para su distribución.

Actualmente existen varias herramientas que permiten automatizar estas dos actividades, tales como: Maven, Ant, MSBuild, Rails, Gradle, entre otras que sirven para automatizar la construcción del software y, en la mayoría de los casos, el empaquetado. Estas herramientas se suelen integrar con Jenkins para automatizar

estas etapas dentro de los *pipelines* de despliegue, y suelen ocurrir justo después de recuperar el código fuente del sistema de control de versiones.



Cuestionario 21. ¿Qué es la construcción y empaquetado de software?

Responde las siguientes preguntas para evaluar la lectura previa y verificar tu comprensión de los conceptos de construcción y empaquetado de software.

1. Verdadero o falso: la construcción (*build*) y el empaquetado (*packaging*) hacen referencia al mismo concepto, solo que utilizando un término distinto.

A) Verdadero.

B) Falso.

2. ¿Qué es la construcción de software (*build*)?

A) El proceso de empaquetar todos los componentes relacionados en un solo elemento.

B) El proceso de compilar los archivos de código fuente y generar un sistema ejecutable y verificable.

C) El proceso de desplegar una aplicación en un entorno productivo.

D) El proceso de documentar las pruebas de software.

3. ¿Qué caracteriza al empaquetado de software (*packaging*)?

A) Convierte código fuente en código máquina.

B) Ejecuta pruebas unitarias y análisis estáticos.

C) Ensambla el software compilado con librerías y recursos en un formato listo para distribución.

D) Traduce el software a diferentes lenguajes de programación.

4. Verdadero o falso: el empaquetado generalmente ocurre después del proceso de construcción.

A) Verdadero. B) Falso.

5. Verdadero o falso: herramientas como Maven, MSBuild o Ant pueden integrarse en Jenkins para automatizar la construcción y empaquetado del software.

A) Verdadero. B) Falso.

6. Verdadero o falso: la construcción de software (*build*) genera un solo artefacto final que sirve para el despliegue posterior.

A) Verdadero. B) Falso.

7. Verdadero o falso: el empaquetado reúne diversos elementos de software para crear archivos ejecutables en lenguaje máquina.

A) Verdadero. B) Falso.

8. ¿Cuál de las siguientes herramientas **no** se asocia directamente con construcción y empaquetado de software?

A) Maven. B) Ant. C) MSBuild. D) npm.

10.2 Actividades de construcción y empaquetado de software



Lectura 25. Actividades de construcción y empaquetado de software.

Nivel básico

Objetivo: el estudiante reconoce las actividades específicas que contemplan la construcción y empaquetado de software, y las ubica dentro del flujo del *pipeline* de despliegue.

Descripción

La lectura aborda las actividades concretas que suelen realizarse durante la construcción (*build*) y empaquetado de software, localizando en qué etapa de los *pipelines* de despliegue ocurren.

Tanto la construcción como el empaquetado de software, involucran un conjunto de actividades específicas. Por un lado, la construcción (*build*) suele asociarse únicamente con la compilación del código fuente, sin embargo, esta es solo una de las actividades que forman parte del proceso.

Durante la construcción del software (*build*), Tomas Fernandez (2022) mencionan que suelen ocurrir actividades como:

- Compilar el código fuente: transformar el código fuente en ejecutables binarios.
- Gestionar las dependencias: se descargan y vinculan las bibliotecas y archivos necesarios para crear un aplicación funcional y cohesiva.
- *Linting*: se refiere a un análisis estático de código en busca de errores de estilo y defectos de programación.
- Análisis de calidad: es otro tipo de análisis estático de código enfocado en mejorar la calidad del código, mediante la identificación temprana de vulnerabilidades, riesgos de seguridad y posibles problemas de mantenimiento.

- Pruebas unitarias: en algunos casos se realiza la ejecución de pruebas unitarias durante el proceso de construcción, con el fin de validar la lógica más básica de la aplicación.
- Generación de ejecutables: al finalizar el proceso de construcción (*build*), se generan los archivos ejecutables de la aplicación, listos para usarse. Archivos tales como *.jar* para Java, *.dll* para C#, o *.obj* para C++.

Por otra parte, Horatiu Vladasel (2022) destacan que durante el empaquetado del software, suelen ocurrir actividades como:

- Identificación de recursos: se identifican y recolectan los detalles, archivos, dependencias y configuraciones necesarias para la aplicación.
- Verificación y evaluación: se analiza toda la información recolectada y se determina el formato de empaquetado más adecuado (EXE, MSI, App-V, MSIX, entre otros formatos).
- Empaquetado del software: se integran los elementos recolectados de la aplicación en un paquete, según los requerimientos del proyecto, generando un producto final distribuible.
- Firma y versionado: se aplica una firma digital y un número de versionado al paquete generado, garantizando autenticidad, trazabilidad y mejor organización.
- Pruebas: en algunos casos se realizan pruebas funcionales y de instalación para asegurar que el paquete generado funciona correctamente.
- Preparación para el despliegue: finalmente el paquete es transferido al área correspondiente para su distribución en algún entorno.

Actualmente, es poco común que los equipos de desarrollo realicen todas estas actividades manualmente. Hoy en día existen herramientas como Maven, que permiten automatizar tanto la construcción como el empaquetado del software con la mínima intervención humana y haciendo clic en un solo botón.

Dentro de los *pipelines* de despliegue, la construcción del software y el empaquetado normalmente son secuenciales y ocurren justo después de recuperar el código fuente del sistema del sistema de control de versiones.



Cuestionario 22. Actividades de construcción y empaquetado de software.

Responde las siguientes preguntas para evaluar la lectura 2 y verificar tu comprensión respecto a las actividades de construcción y empaquetado de software.

- ¿Cuál de las siguientes actividades **sí** forma parte del proceso de construcción (*build*) de software?

A) Compilación del código fuente.	B) Identificación de recursos.	C) Firma digital.	D) Preparación para el despliegue.
-----------------------------------	--------------------------------	-------------------	------------------------------------
- Durante la actividad de gestión de dependencias, el objetivo principal es:

A) Detectar vulnerabilidades de seguridad.	B) Descargar y vincular bibliotecas externas necesarias para la aplicación.	C) Aplicar firmas digitales al artefacto final.	D) Empaquetar los recursos en un instalador.
--	---	---	--
- ¿Cuál de las siguientes **no** es una actividad de la construcción de software?

A) Compilación del código fuente.	B) Ejecutar pruebas de aceptación.	C) Gestionar dependencias.	D) Análisis estáticos de código.
-----------------------------------	------------------------------------	----------------------------	----------------------------------
- ¿Qué actividad durante el *build* permite detectar vulnerabilidades y riesgos de seguridad en el código?

A) Análisis de calidad.	B) <i>Linting</i> .	C) Compilación.	D) Pruebas funcionales.
-------------------------	---------------------	-----------------	-------------------------
- En la etapa de empaquetado de software, la verificación y evaluación se refieren a:

- | | | | |
|--------------------------------|---|---|--|
| A) Ejecutar pruebas unitarias. | B) Determinar el formato más adecuado para el paquete generado. | C) Compilar el código fuente en binarios. | D) Almacenar los artefactos en un repositorio. |
|--------------------------------|---|---|--|

6. ¿En qué parte del *pipeline* de despliegue suelen ocurrir el *build* y el empaquetado?

- | | | | |
|--|---|---|---|
| A) Antes de obtener el código del repositorio. | B) Después de la fase de pruebas en producción. | C) Posterior a la obtención del código. | D) Exclusivamente en la etapa de monitoreo. |
|--|---|---|---|

7. Verdadero o falso: el *linting* es un análisis estático de código que busca errores de estilo y defectos de programación.

- | | |
|---------------|-----------|
| A) Verdadero. | B) Falso. |
|---------------|-----------|

8. Verdadero o falso: durante el empaquetado se generan los ejecutables principales como *.jar* o *.dll*.

- | | |
|---------------|-----------|
| A) Verdadero. | B) Falso. |
|---------------|-----------|

9. Verdadero o falso: el formato *.exe* es un ejemplo de artefacto generado en la fase de construcción.

- | | |
|---------------|-----------|
| A) Verdadero. | B) Falso. |
|---------------|-----------|

10. Verdadero o falso: hoy en día es poco común que se realicen las actividades de *build* y *packaging*.

- | | |
|---------------|-----------|
| A) Verdadero. | B) Falso. |
|---------------|-----------|

10.3 Maven como herramienta de construcción y empaquetado



Lectura 26. ¿Para qué sirve Maven?

Nivel básico

Objetivo: el estudiante reconoce el rol de Maven como herramienta de construcción y empaquetado de software.

Descripción

La lectura profundiza en la herramienta Maven, describiendo su funcionalidad y propósito como herramienta de construcción y empaquetado de software, así como su integración en los *pipelines* de despliegue en Jenkins.

Maven es una de las herramientas más populares para trabajar con el lenguaje de programación Java. Esta es una herramienta para la construcción (*build*) de software de proyectos de Java, que permite administrar y automatizar la compilación, prueba y documentación del software (The Apache Software Foundation, 2025).

Maven surgió como un intento de simplificar el proceso de construcción del software, con el objetivo principal de permitir a los desarrolladoras organizar y compilar proyectos de Java en el periodo de tiempo más corto posible, simplificando así el proceso de construcción (*build*), y facilitando la documentación y publicación de la información de los proyectos. Hoy en día, Maven se ha establecido como una herramienta de automatización que ayuda a los desarrolladores a trabajar con proyectos basados en Java en su día a día (The Apache Software Foundation, 2025).

De manera general, Maven funciona mediante un archivo central llamado POM (*Project Object Model*), en formato XML, que contiene toda la información relevante del proyecto y define su estructura, dependencias, y configuraciones necesarias para el proceso de construcción de software (The Apache Software Foundation, 2025b).

Cuando se ejecuta un proyecto, Maven busca el Archivo POM, lo interpreta y obtienen la información y configuraciones requeridas para alcanzar un objetivo específico, como compilar el código, empaquetarlo o ejecutar pruebas.

Entre las actividades de construcción y empaquetado que se pueden automatizar con Maven, se encuentran:

- Verificar dependencia e información del proyecto.
- Ejecutar pruebas unitarias.
- Compilar el código fuente.
- Empaquetar el software en un formato distribuible.
- Realizar verificaciones de los paquetes generado.

Además, Maven permite automatizar la ejecución de pruebas de integración, así como la instalación y despliegue del software en distintos ambientes (The Apache Software Foundation, 2025b).

Maven por sí sola ya es una herramienta muy útil, que reduce el tiempo de construcción del software. Al integrarla con Jenkins como parte de los *pipelines* de despliegue, se puede configurar Jenkins para ejecutar de manera automática los *builds* de Maven sobre demanda, lo que beneficia al proceso de integración continua que promueve Jenkins.

Dentro de los *pipelines* de Jenkins, Maven toma un rol protagónico durante la fase de integración, automatizando la compilación y empaquetado del código que se recupera previamente del repositorio de GitHub.



Cuestionario 23. ¿Para qué sirve Maven?

Responde las siguientes preguntas para evaluar la lectura previa y verificar tu comprensión respecto a Maven.

1. ¿Cuál de las siguientes afirmaciones describe mejor a Maven?

- | | | | |
|---|--|---|---|
| A) Es un sistema de control de versiones. | B) Es una herramienta para gestionar la construcción, prueba y | C) Es un editor de código fuente para Java. | D) Es un entorno de ejecución para aplicaciones Java. |
|---|--|---|---|

documentación de
proyectos en Java.

2. ¿Qué archivo utiliza Maven como base para ejecutar un proyecto?

- A) build.xml B) settings.gradle C) pom.xml D) config.json

3. ¿Cuál de las siguientes **no** es una actividad de construcción o empaquetado que Maven puede automatizar?

- A) Compilación del código fuente. B) Ejecución de pruebas unitarias. C) Gestión de dependencias. D) Edición del código fuente.

4. ¿En qué parte del *pipeline* de Jenkins se integra Maven?

- A) Después de realizar las pruebas unitarias. B) Previo al despliegue en producción. C) Después de recuperar el código de GitHub. D) Durante la ejecución de pruebas automatizadas.

5. Son algunas de las ventajas de usar Maven en lugar de compilar y empaquetar manualmente.

- A) Reduce los tiempos de construcción y calidad del software. B) Reduce los tiempos de construcción del software y facilita la documentación. C) Facilita la documentación y mejora la cobertura de las pruebas. D) Hace el software más seguro y disminuye los tiempos de despliegue.

6. Verdadero o falso: el archivo POM define la estructura, dependencias y configuraciones necesarias para el proceso de construcción de un proyecto en Java.

- A) Verdadero. B) Falso.

7. Verdadero o falso: Maven fue creado con el objetivo de facilitar el proceso de despliegue de contenedores Docker.

A) Verdadero. B) Falso.

8. Verdadero o falso: Maven únicamente permite compilar código, pero no realizar pruebas ni empaquetar software.

A) Verdadero. B) Falso.

10.4 Comandos comunes en Maven



Lectura 27. ¿Cómo utilizo Maven?

Nivel básico

Objetivo: el estudiante reconoce los principales comando utilizados en Maven para construir y empaquetar software.

Descripción

La lectura explica el propósito y funcionamiento de los principales comandos (*clean*, *compile*, *test*, *package*, *install*, entre otros) utilizados en Maven para construir y empaquetar software en los *pipelines* de Jenkins.

The Apache Software Foundation (2025b) define una lista de comandos para ejecutar una o más fases del ciclo de vida del software de manera automática. A estos comandos se les conocen como etapas o fases Maven:

- **validate:** valida que el proyecto es correcto y que todas la información necesaria está disponible.
- **compile:** compila el código fuente del proyecto.
- **test:** realiza pruebas unitaria al código compilado del proyecto. Para ejecutar las pruebas se requiere de una biblioteca o framework de pruebas compatible con Maven, como JUnit.
- **package:** a partir del código compilado, realiza el empaquetado de código en un formato distribuible como JAR o WAR.
- **integration-test:** de ser necesario, despliega el código empaquetado en un ambiente donde se puedan ejecutar las pruebas de integración.

- **verify:** ejecuta análisis al paquete para verificar que es válido y cumple con los criterios de calidad establecidos.
- **install:** instala el software empaquetado en un repositorio local para ser usado como dependencia en otros proyectos locales.
- **deploy:** copia el paquete de software final a un repositorio remoto para ser compartido con otros desarrolladores o proyectos.

Todos estos comandos funcionan de manera jerárquica y secuencial, lo que quiere decir que al ejecutar una fase de Maven, también se ejecutan implícitamente todas las fases previas. Por ejemplo, si se ejecuta el comando *package*, también se ejecuta las fases *validate*, *compile* y *test*.



Nota - ¿Cómo ejecuto Maven?

Para utilizar Maven, siempre es necesario empezar con el comando **mvn**, seguido de los comandos para ejecutar. Además, es posible ejecutar más de un comando a la vez.

Otros comandos de soporte que se suelen usar con Maven, pero que no están relacionados directamente con alguna etapa del ciclo de vida definido por Maven son:

- **clean:** elimina todos los artefactos generados en *builds* previos.
- **site:** genera un sitio de documentación del proyecto basado en las información disponible en el archivo POM.

A pesar de que los *pipelines* declarativos implementan su propia sintaxis, Jenkins utiliza estos mismos comandos para ejecutar Maven. Estos se escriben siempre dentro de las directivas *steps*, y se ejecutan mediante el *step sh*, seguido del comando de Maven completo y entre comillas simples.

```
steps {  
  sh 'mvn clean package'  
}
```

Figura 146. Step en Jenkins que ejecuta Maven con los comandos "clean" y "package"



Cuestionario 24. ¿Cómo utilizo Maven?

Responde las siguientes preguntas para evaluar la lectura anterior y verificar tu comprensión respecto a los comando para utilizar Maven.

1. ¿Qué comando de Maven se utiliza únicamente para compilar el código fuente del proyecto?
A) *validate* B) *site* C) *compile* D) *build*
2. Si quieres eliminar la carpeta `target/` y limpiar artefactos de compilaciones anteriores, ¿Qué comando usarías?
A) *deploy* B) *site* C) *clear* D) *clean*
3. ¿Qué comando empaqueta el proyecto en un archivo `.jar` o `.war` dentro de la carpeta `target/`?
A) *deploy* B) *clean* C) *compile* D) *package*
4. ¿Cuál es el propósito del comando *mvn install*?
A) Subir el paquete al repositorio remoto. B) Instalar el paquete en el repositorio local. C) Generar la documentación del sitio. D) Ejecutar pruebas de integración.
5. ¿Qué comando se usa normalmente en Jenkins para realizar una construcción completa y fresca del proyecto?
A) *mvn install* B) *mvn deploy* C) *mvn clean* D) *mvn clean install*
compile
6. ¿Qué comando de Maven es más adecuado para usarse antes de iniciar cualquier compilación en Jenkins?

- A) *clean* B) *site* C) *verify* D) *deploy*
7. ¿Cuál de las siguientes opciones no es un comando válido de Maven?
- A) *install* B) *integration-test* C) *build* D) *verify*
8. ¿Qué directiva se utiliza para ejecutar los comandos de Maven?
- A) *stages* B) *mvn* C) *steps* D) *run*
9. ¿Qué tipo de *step* se suele utilizar para ejecutar los comandos de Maven?
- A) *echo* B) *sh* C) *mvn* D) *checkout*
10. Verdadero o falso: los comandos que se usan directamente con Maven son distintos a los que se usan en Jenkins para interactuar con dicha herramienta.
- A) Verdadero. B) Falso.

10.5 Integración de Maven con Jenkins



Lectura 28. *Plugins necesarios para integrar Maven con Jenkins*

Nivel básico

Objetivo: el estudiante identifica los *plugins* necesarios para integrar la herramienta Maven con los *pipelines* declarativos de Jenkins.

Descripción

La lectura explica qué *plugins* se requieren para poder utilizar Maven dentro de los *pipelines* declarativos de Jenkins. Así mismo, se profundiza en el propósito de cada *plugin* requerido.

Como regla general, se requiere de uno o más *plugins* para poder integrar herramientas con Jenkins. Maven no es la excepción, ya que requiere de dos *plugins* específicos para poder utilizarse dentro de los *pipelines* de Jenkins.

El primer *plugin* se llama **Maven Integration**. Como su nombre lo describe, este el *plugin* principal que permite integrar Jenkins con proyectos de Maven, permitiendo que Jenkins comprenda los archivos POM y habilitando la ejecución de los principales comandos de Maven dentro de las definiciones de los *jobs* de Jenkins.

Al integrar este *plugin*, es posible crear proyectos de Maven en Jenkins mediante el uso de *jobs* de tipo *free style* con *steps* de *build* de Maven, o utilizando *jobs* de tipo *Maven-style*, que se agregan como opción al crear *jobs* tras descargar el *plugin* ya mencionado. Sin embargo, ambas prácticas se encuentran casi obsoletas y no son recomendadas por la comunidad. Por otro lado, también es posible utilizar Maven dentro de los *jobs* de tipo *pipelines*. Sin embargo, debido a que este *plugin* es antiguo y previo a los *jobs* de tipo *pipelines* en Jenkins, no cuenta con la mejor integración con este tipo de proyectos (CD Foundation, 2025l).

Debido a este problema, se creó un *plugin* enfocado exclusivamente en integrar correctamente Maven con los *pipelines* declarativos o *scripted* de Jenkins. Este *plugin* se llama **Pipeline Maven Integration**; provee de un conjunto avanzado de características que simplifican la utilización de Maven dentro de los *pipelines* de Jenkins y de los Jenkinsfiles, alineándose mejor con las prácticas recomendadas de *pipelines* como código (CD Foundation, 2025o).

Ambos *plugins* se pueden descargar e instalar desde la sección *Manage plugins* en Jenkins, tal como se explicó en el capítulo [¿Cómo integrar plugins en Jenkins?](#).

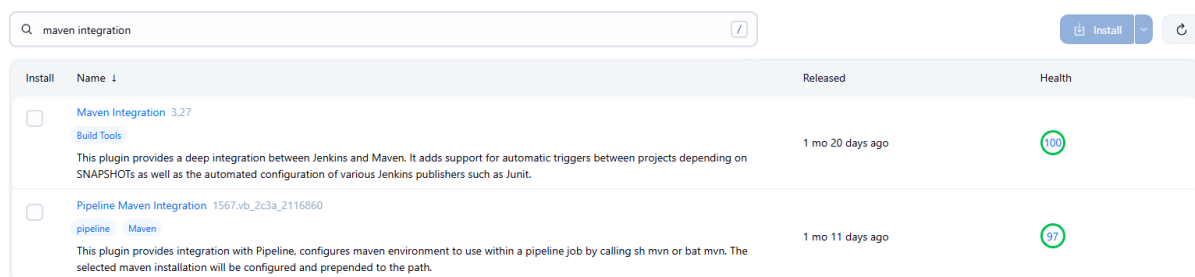


Figura 147. Captura de pantalla en Jenkins de la gestión de plugins, donde se muestran aquellos relacionados con Maven

Es importante mencionar que los *plugins* descargados no son instancias de la propia herramienta de Maven, esto quiere decir que la herramienta todavía no se encuentra

funcional, ni instalada en ningún lado; los *plugins* solo sirven como interfaz que permiten la comunicación entre Maven y Jenkins.

Para instalar Maven y poder utilizar esta herramienta con Jenkins existen dos formas: gestionando manualmente la instalación de la herramienta u orquestarla con ayuda de Jenkins.

La primera es instalar Maven directamente en el servidor donde se aloja Jenkins, para luego configurar la ruta (*path*) de instalación en la configuración de Jenkins y que así el *plugin* utilice esa instalación para ejecutar los comandos. La segunda opción consiste en delegar a Jenkins para realizar la instalación de Maven. Con esta última opción solo se debe indicar la versión de Maven que se quiere instalar y Jenkins gestionará todo el proceso de instalación. En la práctica, ambas alternativas son válidas y ampliamente utilizadas; escoger una u otra es una decisión subjetiva que dependerá de las circunstancias y contextos de cada equipo de desarrollo (Nandita Sahu, 2022).

En esta guía se ha optado por utilizar la segunda alternativa, la cual se explicará con mayor detalle en capítulos posteriores, profundizando en las configuraciones necesarias para utilizar Maven de manera correcta con los *pipelines* declarativos y los Jenkinsfiles.



Cuestionario 25. *Plugins* necesarios para integrar Maven con Jenkins

Responde las siguientes preguntas para evaluar la lectura previa y verificar tu comprensión respecto a los *plugins* necesarios para utilizar Maven en Jenkins.

1. ¿Qué *plugins* son necesarios para integrar Maven con Jenkins?

- | | | | |
|---------------------|---------------------|---------------------|---------------------|
| A) Maven | B) Maven | C) Pipeline Maven | D) Pipeline Maven y |
| Integration y Maven | Integration y Maven | Integration y Maven | Maven Integration. |
| Integration Plus. | Pipelines. | Integration. | |

2. ¿Para qué sirve el plugin Maven Integration?

- | | | | |
|---|---|---|--------------------------|
| A) Contiene la funcionalidad básica para comunicar Maven con Jenkins. | B) Contiene funciones y características extras para integrar Maven con Jenkins. | C) Contiene las funcionalidades necesarias para utilizar Maven con los <i>pipelines</i> de Jenkins. | D) Todas las anteriores. |
|---|---|---|--------------------------|

3. Verdadero o falso: el plugin Pipeline Maven Integration es solo una extensión de Maven Integration, que únicamente agrega características opcionales extras.

- A) Verdadero. B) Falso.

4. Verdadero o falso: al instalar el plugin Maven Integration, también se instala la herramienta Maven dentro de Jenkins

- A) Verdadero. B) Falso.

5. Verdadero o falso: el *plugin* Maven Integration solo funciona como un puente entre Jenkins y la herramienta Maven.

- A) Verdadero. B) Falso.

6. Verdadero o falso: es mejor utilizar *jobs* de tipo *Maven-style* para crear proyectos basados en Maven.

- A) Verdadero. B) Falso



Capítulo 11

11 Espacio de trabajo de los pipelines

Contenido del capítulo

- 11.1. ¿Qué es el espacio de trabajo en el contexto de los *pipelines* en Jenkins?
- 11.2. Comportamiento del espacio de trabajo entre ejecuciones de un *pipeline*
- 11.3. ¿Cómo acceder al espacio de trabajo de un *pipeline*?
- 11.4. GitHub y el espacio de trabajo

11.1 ¿Qué es el espacio de trabajo en el contexto de los *pipeline* en Jenkins?



Lectura 29. Espacio de trabajo en Jenkins

Nivel básico

Objetivo: el estudiante reconoce la definición de espacio de trabajo en *pipelines* declarativos de Jenkins.

Descripción

La lectura aborda el concepto de espacio de trabajo en el contexto de *pipelines* de Jenkins, enfatizando su importancia y su objetivo principal, así como la forma en que este se “traduce” ya en la implementación del *pipeline*.

Un concepto importante dentro de Jenkins, específicamente dentro de los *pipelines* declarativos, es el de espacio de trabajo o *workspace*. De forma general, puedes imaginar este como ese espacio que Jenkins crea para gestionar todos los archivos procesados por un *pipeline*; ese donde se clona el repositorio, se realizan los *builds*, las pruebas, entre otras actividades. Al final del día, es una carpeta que Jenkins crea para cada *pipeline*, en la que se encuentran todos los archivos que estas manejan.

Como se describe en la documentación oficial de la CD Foundation (2025m), el *workspace* es el directorio de trabajo que Jenkins asigna a cada *pipeline* para almacenar los archivos del código fuente y ejecutar en él las tareas de compilación, prueba y despliegue. Su importancia recae en que proporciona un entorno aislado en el que cada ejecución del *pipeline* dispone del contexto completo de archivos necesario para que los *steps* se ejecuten correctamente sobre el código.

En un *pipeline* declarativo, la asignación del *workspace* está controlada por la directiva *agent*. Al definir un *agent* a nivel de *pipeline* (por ejemplo *agent any*), Jenkins crea un *workspace* único para todo el *pipeline* dentro del agente especificado; así, todos los *stages* del *pipeline* comparten ese mismo directorio de trabajo para ejecutar sus actividades (CD Foundation, 2025m).

En resumen, el *workspace* en Jenkins es el directorio de trabajo asignado por el agente a un *pipeline*. Sirve para ubicar el código fuente y realizar todas las tareas definidas en el *pipeline*.



Cuestionario 26. Espacio de trabajo

Responde las siguientes preguntas para evaluar la lectura anterior y verificar tu comprensión en el concepto de espacio de trabajo.

1. ¿Qué es el *workspace* en un *pipeline* declarativo de Jenkins?

- | | | | |
|--|--|--|-------------------------------|
| A) Es un directorio de trabajo que Jenkins asigna a cada <i>pipeline</i> . | B) Es el espacio donde el <i>pipeline</i> despliega el software. | C) Es una carpeta que se crea por <i>build</i> . | D) Ninguna de las anteriores. |
|--|--|--|-------------------------------|

2. Verdadero o falso: el *workspace* se comparte entre todos los *jobs* de Jenkins.

- A) Verdadero. B) Falso.

3. ¿Por qué es importante que cada *pipeline* tenga su propio *workspace*?

- | | | | |
|---|---|--|--------------------------|
| A) Porque así el <i>agent</i> del <i>pipeline</i> tiene dónde trabajar. | B) Porque así se organizan mejor los archivos que Jenkins maneja. | C) Porque proporciona un entorno aislado para la ejecución del <i>pipeline</i> . | D) Todas las anteriores. |
|---|---|--|--------------------------|

4. ¿Cuál es la relación entre la directiva *agent* y el *workspace* en un *pipeline* declarativo de Jenkins?

- | | | | |
|---|---|---|---|
| A) El <i>agent</i> se ejecuta dentro del <i>workspace</i> . | B) El <i>workspace</i> se crea dentro del <i>agent</i> definido para el <i>pipeline</i> . | C) El <i>agent</i> y el <i>workspace</i> no tienen relación alguna. | D) El <i>agent</i> crea múltiples <i>workspaces</i> con cada ejecución. |
|---|---|---|---|

11.2 Comportamiento del espacio de trabajo entre ejecuciones de un *pipeline*



Lectura 30. ¿Cómo se comporta el espacio de trabajo?

Nivel básico

Objetivo: el estudiante reconoce el comportamiento del espacio de trabajo entre ejecuciones de un *pipeline*.

Descripción

La lectura explica el comportamiento del espacio de trabajo entre ejecuciones de un *pipeline*. Se responden preguntas clave como la persistencia de archivos entre ejecuciones, los riesgos de reutilizar un espacio de trabajo y las configuraciones disponibles para su limpieza.

Ahora que ya conoces qué es el espacio de trabajo o *workspace* en el contexto de Jenkins, es momento de entenderlo mejor y de saber cómo es que se comporta.

Para comenzar, debes saber que todos estos espacios de trabajo se crean en una sola carpeta. Esta carpeta comúnmente es **`/var/lib/jenkins/workspace`** (al menos en entornos Linux). Dentro de esta carpeta, se van creando los espacios de trabajo para cada *pipeline*. Si por ejemplo, ejecutas un *pipeline* llamado *project-deployment*, se creará el espacio de trabajo **`/var/lib/jenkins/workspace/project-deployment`**. Observa que es dentro de la carpeta **`/var/lib/jenkins/workspace`** donde se crean los espacios de trabajo individuales para cada *pipeline*, aunque esto pueda sonar confuso.

¿Y qué pasa si hay dos ejecuciones simultáneas de un mismo *pipeline*? En ese caso, Jenkins optará por comenzar a crear carpetas como **`/project-deployment@2`**, **`/project-deployment@3`**, sucesivamente. Si no hay ejecuciones concurrentes, Jenkins reutilizará la misma carpeta para hacer el procesamiento del *pipeline*. Esto puede comprobarlo usted mismo ejecutando un *pipeline* y accediendo al *workspace*. El cómo acceder a los *workspaces* se mencionó de forma superficial en la práctica [Tu primer pipeline paso a paso](#), aunque se detalla en la sección [¿Cómo acceder al espacio de trabajo de un pipeline?](#).

Con esto anterior, podemos entender que por defecto, Jenkins no borra el contenido del *workspace* al finalizar la ejecución de un *pipeline*. El directorio permanece en el disco del agente y mantendrá los archivos descargados o generados, a menos que se realice una limpieza explícita. Además, reutilizará el mismo espacio de trabajo en caso de que sea posible hacerlo (no haya otra ejecución utilizándolo).

Ahora bien, hasta ahora solo hemos hablado de espacios de trabajo en *pipelines* declarativos, sin embargo, el mecanismo es idéntico en *pipelines scripted*. No hay comportamiento especial distinto, pues en ambos casos el agente asignado a la ejecución del *pipeline* crea el *workspace* y lo maneja de forma idéntica (CD Foundation, 2025s).

Este enfoque que adopta Jenkins es muy bueno para aislar las ejecuciones de *pipelines*, sin embargo, puede representar problemas serios cuando hablamos de muchas ejecuciones dentro de nuestras instancias de Jenkins. Dado que Jenkins no borra estas carpetas y los archivos dentro, estos podrían interferir en ejecuciones posteriores o incluso quedar como archivos residuales y comenzar a utilizar espacio en disco de forma innecesaria.



Nota - ¿Cómo mantener el espacio de trabajo en condiciones óptimas?

Los expertos tienen dos recomendaciones estrechamente relacionadas que pueden responder esta pregunta: limpiar el entorno entre ejecuciones del *pipeline* y evitar compartir espacios de trabajo entre múltiples ejecuciones de *pipelines*.

Siguiendo esta idea previa, existen algunos mecanismos que permiten limpiar el *workspace* después de cada ejecución de un *pipeline*. El más directo y que se menciona en la documentación de CD Foundation (2025n) es el paso `deleteDir()`, que elimina recursivamente el directorio actual dentro del *workspace*. Sin embargo este no es el único; *plugins* como [Workspace Cleanup](#) proporcionan alternativas para limpiar el espacio de trabajo después de cada ejecución.

La pregunta aquí es: ¿Se debería limpiar el espacio de trabajo tras cada ejecución? La respuesta corta es sí. Si se necesitan almacenar algunos artefactos resultado de la ejecución del *pipeline*, lo más recomendable es utilizar repositorios externos para ello.

Estas ideas sobre limpiar el espacio de trabajo y almacenar artefactos generados por los *pipelines*, se desarrollan con mayor detalle en capítulos posteriores, con ejemplos prácticos que ilustran cómo se realizan ya en la implementación. Por ahora, quédese con esta idea general sobre el *workspace* y el comportamiento que tiene entre ejecuciones de los *pipelines*.



Cuestionario 27. Comportamiento del espacio de trabajo

Responde las siguientes preguntas para evaluar la lectura previa y verificar tu comprensión sobre el comportamiento entre ejecuciones del espacio de trabajo.

1. ¿Qué sucede con el contenido del *workspace* por defecto entre dos ejecuciones consecutivas de un *pipeline*?

- | | | | |
|--|---|--|--|
| A) Se elimina después de cada ejecución. | B) Permanece entre ejecuciones a menos que se elimine explícitamente. | C) Se guarda en el repositorio de Git conectado al <i>pipeline</i> . | D) Se encripta y almacena de forma aislada para su posterior consulta. |
|--|---|--|--|

2. Verdadero o falso: el espacio de trabajo nunca se elimina automáticamente al finalizar la ejecución de un *pipeline*.

- A) Verdadero. B) Falso.

3. ¿Cuál **no** es una posible ruta al *workspace* del *pipeline* que lleva el nombre "nodejs-project-deploy" en un entorno Linux?

- | | | | |
|--|--|--|---|
| A) <code>/var/lib/jenkins/workspace</code> | B) <code>/var/lib/jenkins/workspace/nodejs-project-deploy</code> | C) <code>/var/lib/jenkins/workspace/nodejs-project-deploy@2</code> | D) Todas las anteriores son rutas válidas posibles. |
|--|--|--|---|

4. Verdadero o falso: el espacio de trabajo se comporta igual para *pipelines* declarativos y *scripted*.

A) Verdadero. B) Falso.

5. Verdadero o falso: el uso del comando `cleanWorkspace()` dentro de un *pipeline* permite limpiar el espacio de trabajo.

A) Verdadero. B) Falso.

6. ¿Cuál de las siguientes **no** es una consecuencia negativa de no limpiar el espacio de trabajo?

- | | | | |
|--|--|--|--|
| A) Interferencia con ejecuciones posteriores del <i>pipeline</i> . | B) Acumulación de archivos residuales en el disco. | C) Ahorro de espacio en disco y mejor rendimiento del sistema. | D) Consumo innecesario de espacio en el almacenamiento del agente. |
|--|--|--|--|

7. Verdadero o falso: se debería limpiar el espacio de trabajo tras cada ejecución del *pipeline*.

A) Verdadero B) Falso

11.3 ¿Cómo acceder al espacio de trabajo de un pipeline?



Lectura 31. ¿Cómo acceder al espacio de trabajo de un *pipeline*?

Nivel intermedio

Objetivo: el estudiante explica cómo acceder al espacio de trabajo de *pipelines* declarativos en Jenkins.

Descripción

La lectura aborda dos enfoques para acceder al espacio de trabajo de un *pipeline* declarativo en Jenkins. El primero, desde la propia definición del *pipeline*, presentando cómo visualizar la ruta y navegar

dentro de ella; el segundo, desde la interfaz gráfica, explicando cómo a partir de un *build* se puede ver la ruta del espacio de trabajo.

Con todo el contexto con el que cuentas hasta ahora sobre el espacio de trabajo y el primer acercamiento que tuviste hacia este en el capítulo [Tu primer pipeline paso a paso](#), estás más que listo para entender los detalles sobre cómo acceder al espacio de trabajo de un *pipeline* en Jenkins.

Para realizar lo anterior, asumiremos que cuentas con un *pipeline* creado en tu instancia de Jenkins y seguiremos los pasos descritos a continuación:

1. Abre tu instancia de Jenkins y entra a la ventana de gestión del *pipeline*.
2. Si no tienes ninguna ejecución del *pipeline*, inicia una manualmente y espera a que esta finalice.
3. En la ventana de gestión del *pipeline* y con al menos un *build* generado, selecciona alguno de ellos desde la parte inferior de la barra lateral izquierda.

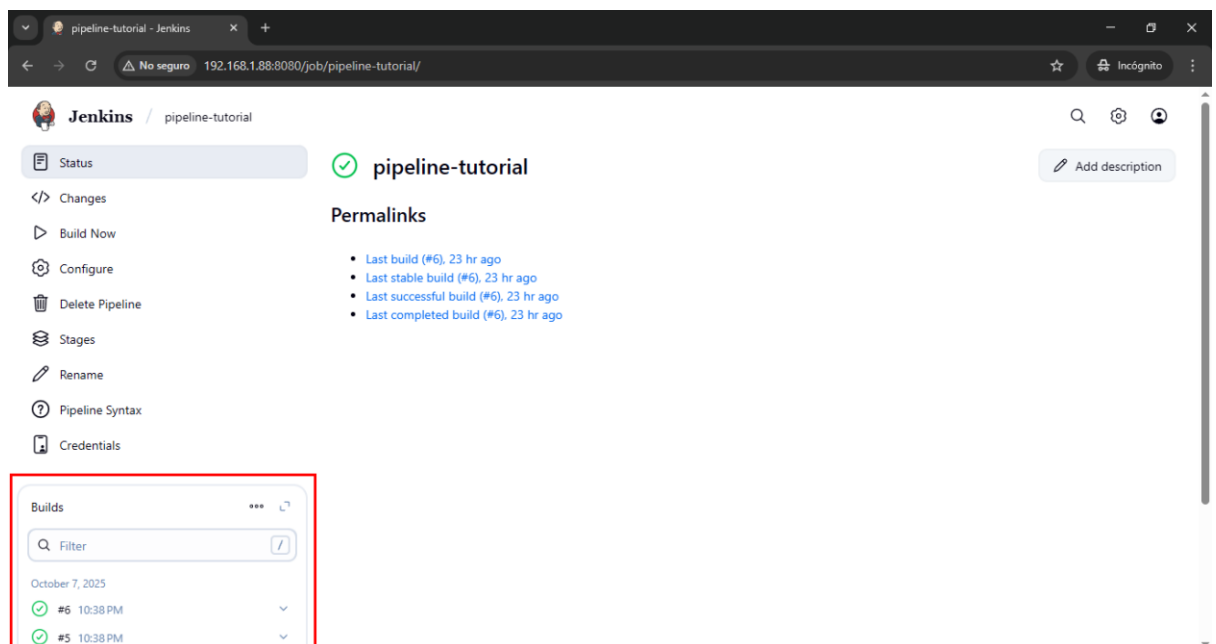


Figura 148. Captura de pantalla donde se resaltan los builds de un pipeline desde la ventana de gestión de este

4. En la ventana de detalle del *build*, selecciona la opción *workspaces* desde la barra lateral.

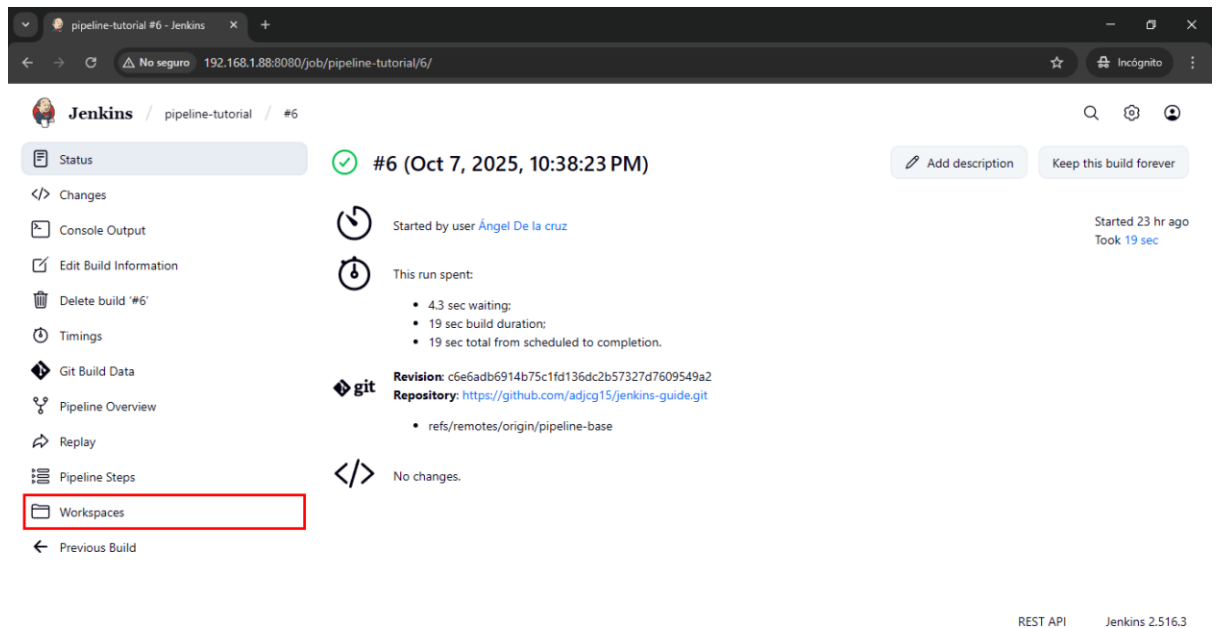


Figura 149. Ventana de detalles del build donde se resalta el apartado del espacio de trabajo para acceder a este

5. El paso previo hará que se muestre la ventana de espacios de trabajo del *build* seleccionado. Dentro se mostrará la ruta al espacio de trabajo (algo similar a `/var/lib/jenkins/workspace/pipeline-tutorial@2`) en forma de enlace. Haz clic en este enlace.

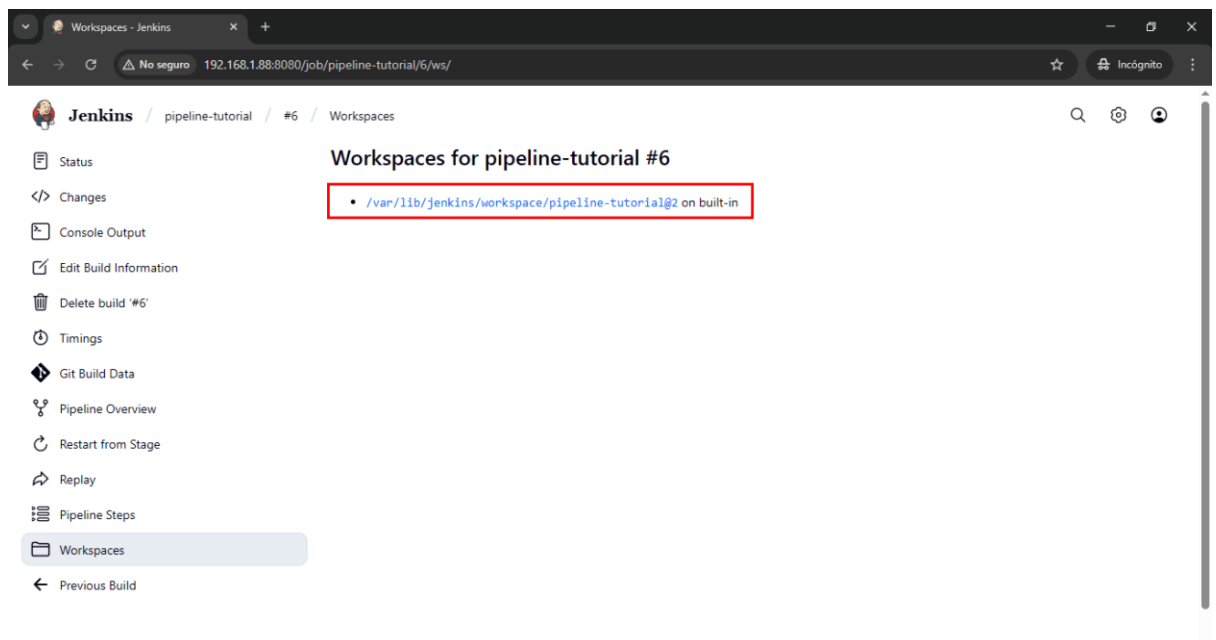


Figura 150. Pantalla de detalles del espacio de trabajo donde se resalta el enlace para acceder a la ruta de este

Este último paso te llevará a una representación visual dentro de Jenkins del espacio de trabajo (la carpeta), con todos los archivos que viven en él listos para descargarse,

ya sea de forma individual (dando clic en ellos) o como un zip a través de la opción *all files in zip*.

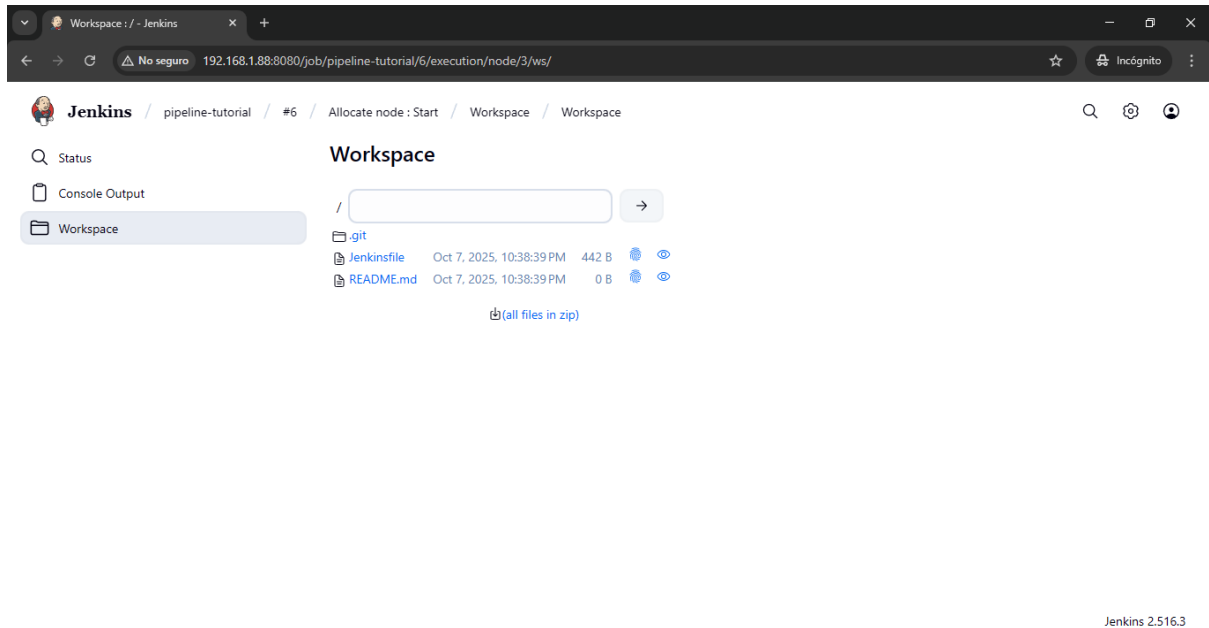


Figura 151. Pantalla de detalles del workspace en formato de carpeta

En caso de que requieras este acceso desde línea de comandos, abre una terminal en el servidor donde vive tu instancia de Jenkins, recupera la ruta hasta el paso 5 previamente descrito y después utiliza el comando **cd** para moverte hacia el directorio (*cd [ruta]*).

Si por alguna razón no puedes acceder a la interfaz gráfica para conocer la ruta del espacio de trabajo, puedes realizar el siguiente proceso a través de línea de comandos pura:

- Ejecuta el comando **sudo systemctl show jenkins | grep JENKINS_HOME**. Este comando busca, dentro de la configuración del servicio Jenkins, la variable de entorno **JENKINS_HOME**, que indica dónde está guardada toda la información y configuración de Jenkins, e imprime algo similar a lo siguiente:

```
adjcg15@jenkins-server:~$ sudo systemctl show jenkins | grep JENKINS_HOME
Environment=JENKINS_HOME=/var/lib/jenkins
```

Figura 152. Salida en consola de la búsqueda de JENKINS_HOME en los archivos de configuración

- Con la carpeta mostrada, ejecuta **cd [ruta]/workspace** para moverte hacia ese directorio. Recuerda que ese directorio es el que almacena los espacios de trabajo de todos los *pipelines* ejecutados en ese agente.

```
adjcg15@jenkins-server:~$ cd /var/lib/jenkins/workspace
```

Figura 153. Comando para moverse a la carpeta con los espacios de trabajo

- Una vez ahí, ejecuta el comando **ls** para listar todas las carpetas y archivos.

```
adjcg15@jenkins-server:/var/lib/jenkins/workspace$ ls  
pipeline-tutorial pipeline-tutorial@2 pipeline-tutorial@2@tmp pipeline-tutorial@tmp
```

Figura 154. Comando para listar los espacios de trabajo, junto a los resultados

Con este paso final, obtienes la lista de espacios de trabajo (directorios) de la instancia de Jenkins sobre la que estás trabajando; así puedes acceder al que sea de tu interés.

Antes de cerrar este capítulo, debes saber que también se puede acceder a esta información sobre la ruta del espacio de trabajo desde la definición de un *pipeline* declarativo. La documentación oficial de CD Foundation (2025s) presenta la variable de entorno *WORKSPACE*, que almacena la ruta absoluta del espacio de trabajo, por lo que dentro de un *pipeline* declarativo puede referenciarla como **env.WORKSPACE** (por ejemplo `echo "Workspace: ${env.WORKSPACE}"`).



Cuestionario 28. ¿Cómo acceder al espacio de trabajo de un *pipeline*?

Responde las siguientes preguntas para evaluar la lectura previa y verificar tu comprensión sobre cómo acceder al espacio de trabajo de un *pipeline*.

1. Verdadero o falso: solamente desde la interfaz gráfica de Jenkins se puede conocer la ruta del espacio de trabajo de un *pipeline*.

A) Verdadero.

B) Falso.

2. ¿Cuál es el orden correcto de los siguientes pasos para visualizar el espacio de trabajo desde la interfaz gráfica de Jenkins?
 1. Seleccionar la opción *workspaces* en la barra lateral.
 2. Seleccionar una ejecución del *pipeline*.
 3. Entrar a la ventana de gestión del *pipeline*.
 4. Abrir el enlace hacia el espacio de trabajo.

- A) $4 \rightarrow 2 \rightarrow 3 \rightarrow 1$. B) $1 \rightarrow 4 \rightarrow 3 \rightarrow 2$. C) $3 \rightarrow 2 \rightarrow 1 \rightarrow 4$. D) $2 \rightarrow 1 \rightarrow 3 \rightarrow 4$.
3. Verdadero o falso: en un *pipeline* declarativo, la variable `WORKSPACE` se puede usar para acceder directamente al directorio donde se encuentran los archivos del *job*.
- A) Verdadero. B) Falso.
4. ¿Cuál de las siguientes variables se utiliza en un *pipeline* declarativo para acceder a la ruta del *workspace*?
- A) `PIPELINE_DIR`. B) `WORKSPACE`. C) `AGENT_PATH`. D) `JENKINS_HOME`.
5. Cuando se accede a la ruta del espacio de trabajo a través de línea de comandos, ¿Cuál es la variable que busca el comando para descubrir la ruta?
- A) `PIPELINE_DIR`. B) `WORKSPACE`. C) `AGENT_PATH`. D) `JENKINS_HOME`.

11.4 GitHub y el espacio de trabajo



Lectura 32. Clonando repositorios en el espacio de trabajo

Nivel básico

Objetivo: el estudiante lista los *plugins* necesarios para clonar repositorios de GitHub en el espacio de trabajo de *pipelines* declarativos de Jenkins.

Descripción

La lectura explica tres *plugins* que trabajan en conjunto para permitir la conexión entre Jenkins y GitHub y clonar repositorios en el espacio de trabajo de los *pipelines*. Además aborda qué pasa si alguno de estos *plugins* no se encuentra instalado.

Una parte imprescindible de todo *pipeline* de despliegue, son los sistemas de control de versiones. Estas herramientas son la base fundamental para que los equipos de desarrollo puedan adoptar correctamente la Integración Continua, y posteriormente, evolucionar al Despliegue Continuo. En estas herramientas se almacenan todos los componentes necesarios (código, configuraciones y datos) para que una aplicación pueda funcionar, y en caso de realizar un cambio a alguno de estos elementos, sea posible verificarlo lo antes posible (Humble & Farley, 2010, pp. 13, 416).

Como el objetivo primordial de la Integración Continua es unificar el código de todos los desarrolladores con la mayor frecuencia posible, de modo que los cambios en el software se integren de manera constante, los desarrolladores pueden obtener retroalimentación temprana, detectar errores y realizar correcciones en las etapas más prontas del desarrollo. Bajo esta premisa, los sistemas de control de versiones se convierten en la única herramienta completamente indispensable para la adopción de esta práctica y la implementación de *pipelines* de despliegue (Humble & Farley, 2010, pp. 32-38, 57-59).

Tal como se ha explicado en capítulos previos, existen muchos tipos de herramientas que se pueden integrar con Jenkins. Como te puedes imaginar, una de las más populares son los sistemas de controles de versiones como Git. Hasta este punto en la guía, el lector ha logrado integrar GitHub con Jenkins para recuperar la definición de un *pipeline* declarativo (Jenkinsfile) alojado dentro de un repositorio de GitHub. Esto se realizó con detalle en la práctica [Tu primer pipeline paso a paso](#), durante la cual, al crear el *pipeline* y registrar la información del repositorio donde se encuentra el Jenkinsfile, se dejó marcada la opción *Lightweight checkout*; esta funcionalidad provoca que Jenkins únicamente revise el repositorio en busca del Jenkinsfile, pero que no recupera los demás elementos del proyecto.

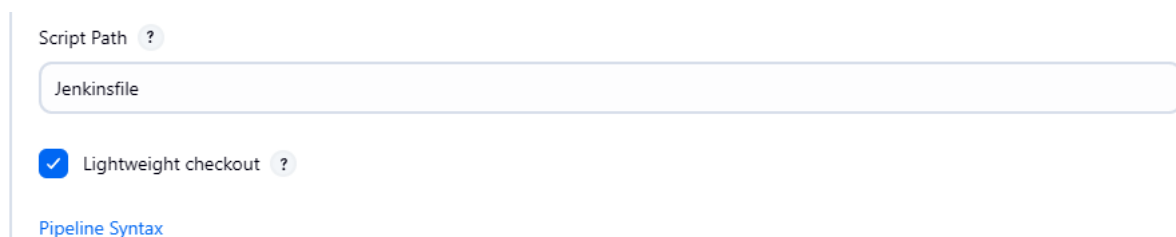


Figura 155. Opción en la configuración de un *pipeline* que limita el recuperado desde GitHub para solo traer el Jenkinsfile

Para clonar el proyecto, primero hay que subirlo al repositorio junto con todos los elementos necesarios para su funcionamiento. Después, se debe modificar el

Jenkinsfile para agregar un *stage* para clonar el proyecto en el espacio de trabajo del *pipeline*. Al clonar un proyecto en Jenkins ocurre lo siguiente:

1. Primero, se crea un directorio dentro del espacio de trabajo del *pipeline*, y en caso de que sea el primer *build* del *pipeline*, también se crea el espacio de trabajo automáticamente.
2. Después, internamente Jenkins ejecuta los comandos equivalentes a **git init**, **"git fetch ..."** y **"git checkout ..."** para descargar todos los archivos del repositorio y guardarlos dentro del directorio creado previamente.
3. En este punto, la clonación finaliza y todos los archivos, hasta el propio proyecto, están disponibles localmente en Jenkins dentro del directorio en el espacio de trabajo del *build* del *pipeline*, para que puedan ser utilizados durante la ejecución de las etapas (*stages*) posteriores declaradas en el Jenkinsfile.

Para poder clonar un proyecto de un repositorio de GitHub hacia un *pipeline* de Jenkins, se requiere únicamente de un *plugin* esencial: **Git Plugin**. Como estás pensando, sí, es el mismo *plugin* que se explica en el capítulo [Jenkins y GitHub](#).

Recordando un poco, este *plugin* habilita la comunicación de Jenkins con Git, proporcionando una serie de funcionalidades y configuraciones para utilizar Git dentro de Jenkins. Entre las características que proporciona, se encuentran: la configuración de credenciales y ramas, el *step* git para usarlo en *pipelines*, y la clonación y gestión de repositorios hacia Jenkins. Esta última característica es la que nos interesa y por la cual este *plugin* es indispensable para clonar los repositorios hacia Jenkins. Para esto se debe declarar explícitamente en el Jenkinsfile la URL, la rama y las credenciales del repositorio que se quiere clonar (CD Foundation, 2025d).

```
stage('Checkout') {  
  steps {  
    echo 'Recueprando Código Fuente...'  
    git branch: 'main',  
      url: 'https://github.com/TristanSantiago01/Proyecto_Guia_de_Jenkins.git',  
      credentialsId: 'GitHub_token_access'  
    echo 'Repositorio recuperado correctamente...'  
  }  
}
```

Figura 156. Código para llevar a cabo la descarga del repositorio del proyecto desde GitHub

Por otra parte, existen otros *plugins* que expanden las funcionalidades de los *pipelines*. Tal es el caso del **Pipeline: SCM Step Plugin**, que permite clonar repositorios de otra forma mediante la directiva nueva llamada ***checkout scm***. Aunque este *plugin* no es indispensable para utilizar Git, sí aporta un enfoque distinto y más simplificado para clonar los repositorios, haciendo innecesario declarar la URL del repositorio, la rama, o las credenciales directamente en el Jenkinsfile, ya que permite reusar la información del SCV registrada al crear el *pipeline*. Básicamente funciona como un intermediario entre el Git Plugin y la sintaxis declarativa del *pipeline* para clonar el repositorio (CD Foundation, 2025p).



```
stage('Checkout') {  
  steps {  
    // Clona el repositorio completo  
    checkout scm  
  }  
}
```

Figura 157. Código para llevar a cabo la descarga de un proyecto desde GitHub desde un pipeline, utilizando el plugin Pipeline: SCM Step Plugin

Adicionalmente, se puede instalar el *plugin* **GitHub Plugin**; aunque no es indispensable para realizar la clonación, sí tiene funciones y características que complementan al *plugin* de Git, permitiendo detectar cambios en ramas, recibir *webhooks*, y habilita más información e GitHub en la Interfaz Gráfica de Jenkins (CD Foundation, 2025e).

En general, únicamente el *plugin* Git es indispensable, ya que sin este no es posible clonar repositorios de Git en *jobs* o *pipelines* de Jenkins, ni tampoco es posible utilizar el step “**git**” dentro de la declaración de los *pipelines*.



Cuestionario 29. Clonando repositorios en el espacio de trabajo

Responda las siguientes preguntas para evaluar la lectura anterior y verificar tu comprensión sobre cómo acceder al espacio de trabajo de un *pipeline*.

1. ¿Cuáles son los *plugins* esenciales para que Jenkins pueda clonar un repositorio Git en un *pipeline* declarativo?

R = _____

2. Verdadero o falso: el repositorio clonado desde GitHub se almacena directamente en el espacio de trabajo del *pipeline*.

A) Verdadero. B) Falso.

3. Verdadero o falso: el plugin Git por sí solo es suficiente para clonar repositorios en *pipelines* declarativos.

A) Verdadero. B) Falso.

4. Verdadero o falso: se debe clonar el repositorio de GitHub cada vez que se ejecuta un *stage* para recuperar los datos y archivos necesarios.

A) Verdadero. B) Falso.

5. Cuando Jenkins clona un repositorio en un *pipeline* declarativo, ¿Qué sucede en el espacio de trabajo?

A) Se descargan los archivos versionados del repositorio.	B) Se ejecutan automáticamente todos los <i>tests</i> del proyecto.	C) Se eliminan todos los archivos previos del <i>workspace</i> sin excepción.	D) Se crean nuevos <i>jobs</i> en Jenkins automáticamente.
---	---	---	--

6. ¿Qué problema podría surgir si se configura un *pipeline* declarativo y se omite instalar el plugin Git?

A) Ninguno, no ocurren problema.	B) El <i>pipeline</i> no puede recuperar el repositorio.	C) El <i>pipeline</i> no se puede crear.	D) El <i>pipeline</i> solo clona el Jenkinsfile.
----------------------------------	--	--	--

7. Ordena los siguientes pasos en el proceso de clonar un repositorio de GitHub en Jenkins:

a. El código se descarga desde GitHub.

- b. Los archivos se guardan en el espacio de trabajo.
- c. El pipeline puede acceder a los archivos en etapas posteriores.
- d. Jenkins usa el plugin Git para ejecutar el comando de clonado.

A) $a \rightarrow d \rightarrow b \rightarrow c$ B) $a \rightarrow d \rightarrow c \rightarrow b$ C) $d \rightarrow a \rightarrow b \rightarrow c$ D) $d \rightarrow a \rightarrow c \rightarrow b$

8. Relaciona cada *plugin* con su función principal:

A) Git Plugin	(1) Permite utilizar repositorios Git dentro de pipelines declarativos.
B) Pipelin: SCM Step Plugin	(2) Simplifica las instrucciones para clonar repositorio de Git.
C) GitHub Plugin	(3) Esencial para realizar operaciones de clonado desde repositorios Git.
	(4) Se enfoca en la integración de Jenkins con GitHub.
	(5) Permite el manejo de webhooks
	(6) Utiliza la directiva "checkout scm" en lugar del step "git"



Capítulo 12

12 Instalación de herramientas en Jenkins

Contenido del capítulo

- 12.1. ¿Cómo instalar herramientas en un servidor de Jenkins?
- 12.2. Acceso a herramientas desde *pipelines* declarativos

12.1 ¿Cómo instalar herramientas en un servidor de Jenkins?



Lectura 33. Instalando herramientas en Jenkins

Nivel básico

Objetivo: el estudiante recuerda los pasos necesarios para instalar herramientas en un servidor de Jenkins.

Descripción

La lectura aborda las dos formas principales que existen para instalar herramientas complementarias a Jenkins: manualmente en el servidor que contiene a Jenkins o utilizando Jenkins.

A través de la interfaz gráfica de Jenkins, hay un apartado con el objetivo específico de gestionar las herramientas a las que tiene acceso la instancia de Jenkins que estamos configurando (revisa el capítulo [Tools](#) para conocer más al respecto).

Esta sección de gestión de herramientas es el primer punto de entrada para instalar herramientas en el servidor de Jenkins. Desde aquí se pueden gestionar diversas versiones de herramientas como Java Development Kit (JDK), Git, Gradle, Ant, Maven y más.

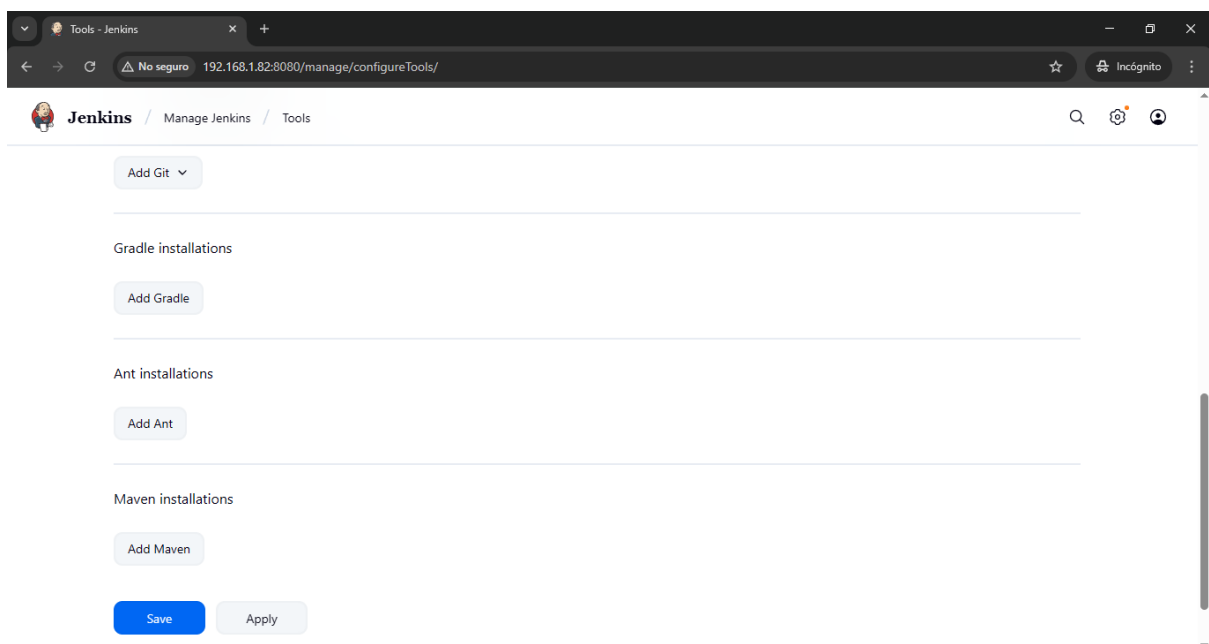


Figura 158. Pantalla de gestión de herramientas en Jenkins

Como ejemplo, veamos cómo es que se instala Maven desde la interfaz gráfica. A continuación se describen los pasos para ello:

1. Como primer paso, selecciona la opción *add Maven* desde la interfaz gráfica, dentro de la ventana de gestión de herramientas. Esto abrirá la configuración para registrar la herramienta.

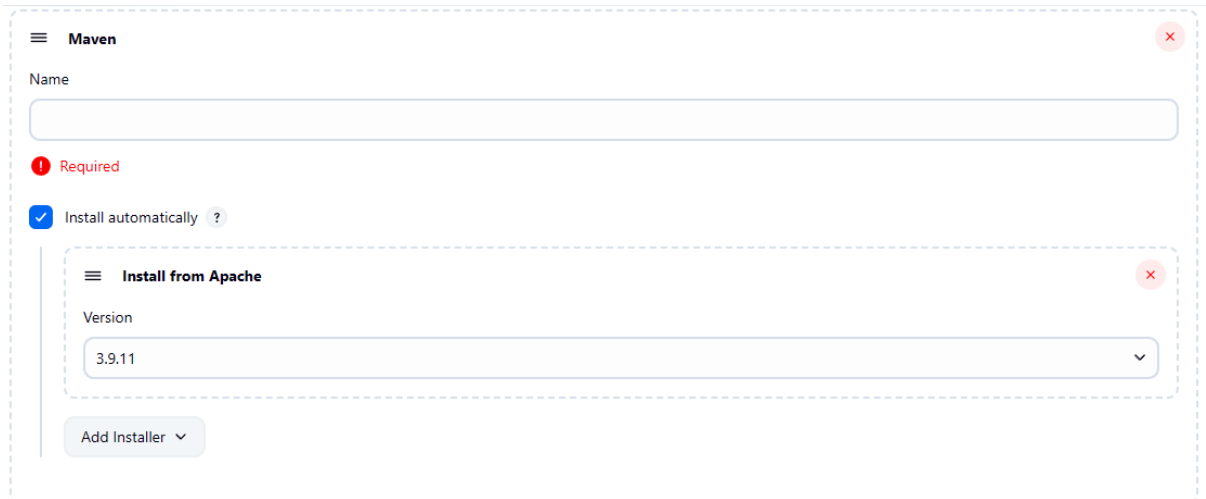


Figura 159. Sección para el registro de la herramienta Maven desde la pantalla de gestión de herramientas

Como puedes ver, la opción *install automatically* está seleccionada por defecto, indicando que se realizará una instalación de la herramienta automáticamente cuando esta se necesite. ¿Desde dónde se instalará? Desde el instalador oficial de Apache, tal como lo indica el campo *install from Apache*. En caso de que desees cambiar esto, seleccione la “X” en la parte superior derecha de esta opción y después elige otra opción desde la opción *add installer*.

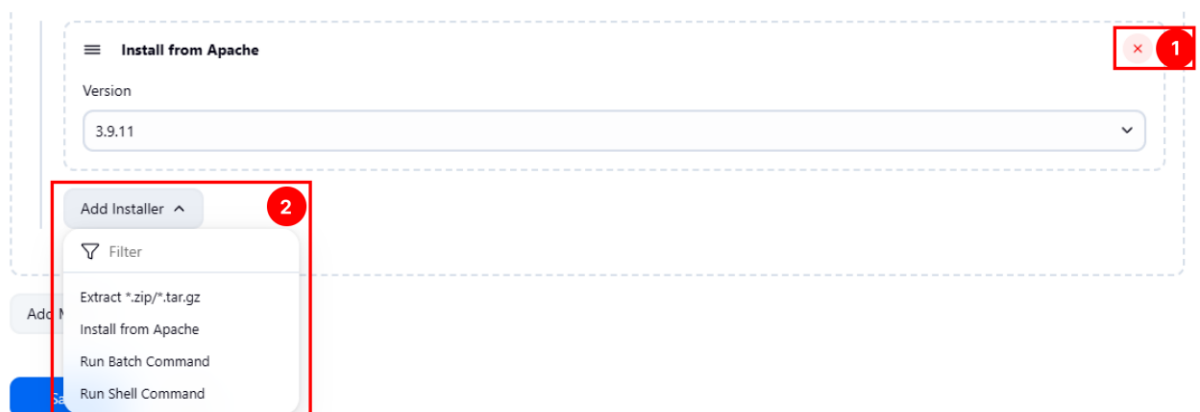


Figura 160. Sección de configuración de herramienta Maven donde se resalta la opción para descartar la instalación desde Apache (1) y cambiar el instalador para utilizar (2)

2. Si quisiéramos mantener esta configuración previamente descrita, solamente deberíamos asignar un nombre a la herramienta en el campo *name* y

seleccionar una versión del campo *version*, para finalmente dar clic en el botón *save* o *apply*.

Figura 161. Sección de configuración de herramienta Maven, donde se resalta la asignación de un nombre (1), la versión de Maven (2) y los botones para guardar/aplicar cambios

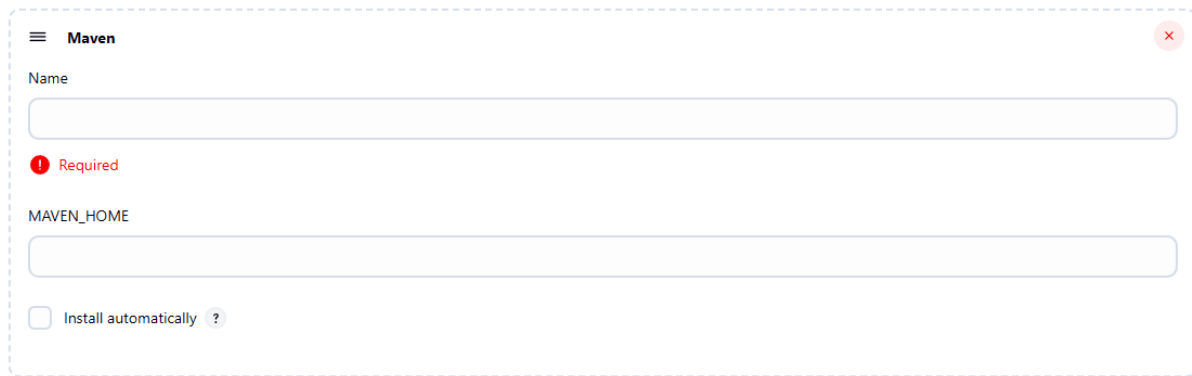
Esto anterior registra la herramienta para que pueda ser accedida y usada desde los *pipelines*; el enfoque es muy bueno si queremos que Jenkins gestione las instalaciones de las herramientas por nosotros (o de múltiples versiones de ellas), pues lo hace de forma interna dentro de una carpeta en el directorio de Jenkins, lo que termina facilitando la portabilidad de la configuración hacia otros entornos (CloudBeesTV, 2021).



Nota - ¡Prepara tus armas!

Antes de “aventarte” a configurar los *pipelines* y generar sus declaraciones, los expertos recomiendan configurar las variables de entorno y herramientas necesarias. Toma esto en cuenta para la próxima vez que configures un *pipeline* en Jenkins.

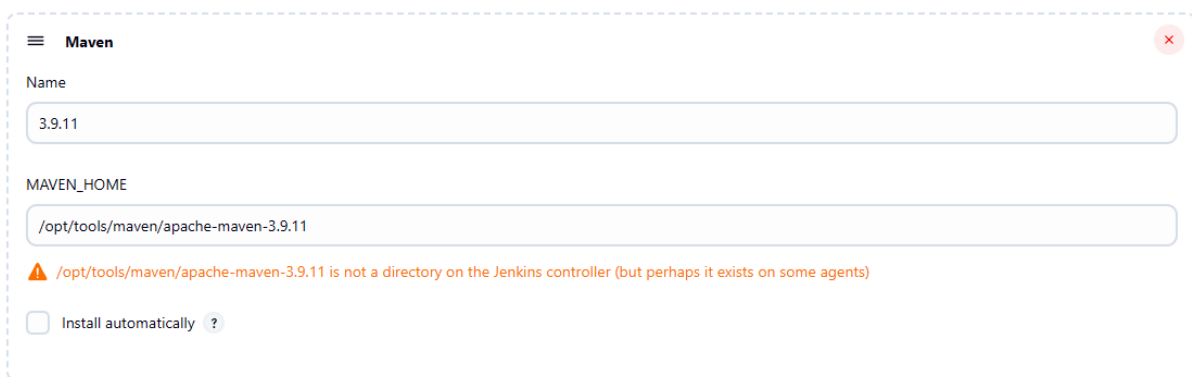
¿Pero qué pasa si ya tenemos una versión de la herramienta instalada localmente dentro del servidor donde corre la instancia de Jenkins? Estos casos también pueden ser manejados desde la interfaz gráfica de Jenkins, siguiendo los mismos pasos previamente explicados, pero quitando la selección la opción *install automatically*.



The screenshot shows the Jenkins configuration interface for the 'Maven' tool. At the top, there is a title bar with a hamburger menu icon and the word 'Maven'. Below this, there are two input fields: 'Name' and 'MAVEN_HOME'. The 'Name' field is currently empty, and a red error message 'Required' is displayed below it. The 'MAVEN_HOME' field is also empty. At the bottom, there is a checkbox labeled 'Install automatically' which is unchecked. A red 'X' icon is visible in the top right corner of the dialog box.

Figura 162. Sección de configuración de Maven habiendo quitado la selección de la opción "Install automatically"

En este caso, en lugar de especificar la versión y el instalador, deberemos colocar la ruta donde se encuentra instalada la herramienta, en este caso Maven (MAVEN_HOME). Y asignarle un nombre tal como en el caso previo. Toma en cuenta que, en el caso de que la herramienta no sea encontrada en el directorio, Jenkins lanzará un error al intentar usar la herramienta (CloudBeesTV, 2021).



The screenshot shows the Jenkins configuration interface for the 'Maven' tool. At the top, there is a title bar with a hamburger menu icon and the word 'Maven'. Below this, there are two input fields: 'Name' and 'MAVEN_HOME'. The 'Name' field contains the value '3.9.11'. The 'MAVEN_HOME' field contains the value '/opt/tools/maven/apache-maven-3.9.11'. A red error message is displayed below the 'MAVEN_HOME' field: '/opt/tools/maven/apache-maven-3.9.11 is not a directory on the Jenkins controller (but perhaps it exists on some agents)'. At the bottom, there is a checkbox labeled 'Install automatically' which is unchecked. A red 'X' icon is visible in the top right corner of the dialog box.

Figura 163. Sección de configuración de la herramienta Maven habiendo ingresado un valor para la ruta de la herramienta

Ahora bien, esto funciona muy bien para las herramientas ya mencionadas: JDKs, Ant, Maven, Gradle, Git, entre otras. ¿Pero qué pasa con otras herramientas? Por ejemplo, si queremos usar python, npm o incluso herramientas no disponibles para instalar con el gestor de paquetes de nuestro sistema operativo. Para estos casos, Jenkins también ofrece alternativas; ya sea que necesitemos una herramienta instalada con el gestor de paquetes del sistema operativo o instalada a través de la descarga directa de los binarios.

En estos casos particulares, es suficiente con instalar la herramienta (o descargar los binarios) dentro del servidor donde se encuentra nuestra instancia de Jenkins en la que queremos utilizar las herramientas. Esto lo logramos, por ejemplo, con `sudo apt install` en caso de que la herramienta esté disponible desde el gestor de paquetes, o descargando la carpeta con los binarios.

Por ahora, sepa que esto anterior es suficiente para que Jenkins pueda utilizar estas otras herramientas. En el capítulo siguiente profundizaremos en el cómo Jenkins accede a estas herramientas y las utiliza dentro de los *pipelines*.



Cuestionario 30. Instalando herramientas en Jenkins

Responde las siguientes preguntas para evaluar la lectura 1 y verificar tu comprensión respecto a las formas de instalar herramientas complementarias en Jenkins.

1. Verdadero o falso: Jenkins solo puede usar herramientas si están instaladas manualmente en el servidor donde se encuentra la instancia de Jenkins.

A) Verdadero. B) Falso.
2. Verdadero o falso: una desventaja de instalar herramientas directamente en el servidor donde se encuentra la instancia de Jenkins, es que dificulta la portabilidad de la configuración hacia otros entornos.

A) Verdadero. B) Falso.
3. ¿Cuál de las siguientes no es una de las formas de instalar herramientas para un servidor Jenkins?

A) A través de la interfaz gráfica de Jenkins.

B) Directamente con el gestor de paquetes del nodo que aloja la instancia de Jenkins.

C) Desde el *pipeline* que utiliza la herramienta.

D) Descargando los binarios de la herramienta en el nodo que aloja la instancia de Jenkins.
4. ¿Qué apartado de Jenkins permite instalar y administrar herramientas directamente desde la interfaz gráfica?

- A) El apartado de gestión de *tools* dentro de la configuración.
B) El apartado de *tools* desde la ventana de gestión de *pipelines*.
C) El apartado de *tools* en el menú principal.
D) Ninguno de los anteriores.
5. ¿Cuál es una ventaja de instalar las herramientas directamente desde Jenkins?
- A) Proveer una sola versión de la herramienta de forma global.
B) Aprovechar los recursos de Jenkins para la instalación.
C) Disminuir la complejidad de asociar las herramientas.
D) Delegar la gestión de las instalaciones a Jenkins.
6. Verdadero o falso: si la herramienta que queremos utilizar solamente se puede instalar a través de la descarga de sus binarios, Jenkins impide su uso dentro de los *pipelines*.
- A) Verdadero. B) Falso.

12.2 Acceso a herramientas desde *pipelines* declarativos



Lectura 34. Accediendo a herramientas en Jenkins

Nivel intermedio

Objetivo: el estudiante describe cómo acceder a herramientas externas desde un *pipeline* declarativo de Jenkins.

Descripción

La lectura describe como acceder y utilizar herramientas desde la definición de un *pipeline* declarativo en un *Jenkinsfile*, dependiendo de la forma en que fueron instaladas las herramientas.

En el capítulo previo se exploraron las diversas formas que tenemos para instalar una herramienta en Jenkins. En este capítulo, abordaremos las tres opciones que disponemos para utilizar esas herramientas desde un *pipeline* declarativo, según la forma en que fue instalada la herramienta de interés.

12.2.1 Herramientas instaladas desde la interfaz gráfica

Recordemos que, cuando instalamos una herramienta a través de la interfaz gráfica de Jenkins, tenemos dos opciones: delegar la gestión de la instalación a Jenkins o instalar la herramienta manualmente y referenciar la ruta de la herramienta.

En ambos casos, su uso desde el *pipeline* declarativo es exactamente el mismo (CloudBeesTV, 2024b). Supongamos que tenemos configuradas las tools Maven y Git en nuestra instancia de Jenkins, la primera referenciada a través de su ruta y la segunda instalada desde Jenkins, pero ambas configuradas desde la interfaz gráfica.

El acceso a estas herramientas desde el *pipeline* es igual para ambas y se realiza de la siguiente forma:

```
1 pipeline {
2   agent any
3   tools {
4     maven '3.9.11'
5     git '2.51.1'
6   }
7   stages {
8     stage('Build') {
9       steps {
10        sh 'mvn -v'
11        sh 'git -version'
12      }
13    }
14  }
15 }
```

Figura 164. Código de ejemplo donde se referencian las herramientas Maven y Git desde la definición de un pipeline

Lo que se está realizando en el código anterior, es una declaración global de las herramientas Maven y Git en las líneas 4 y 5. En pocas palabras, se está indicando que, de las herramientas instaladas, se utilicen las que llevan por nombre “3.9.11” para Maven y “2.52.1” para Git (los nombres asignados durante la configuración de las herramientas). Así, cuando en las líneas 10 y 11 se hace uso de los comandos *mvn* y *git*, Jenkins sabe exactamente qué herramientas utilizar.



Nota - ¿De dónde salen *tools*, *maven*, *git*, etc.?

Algunas características que hemos usado hasta el momento, no son nativas de Jenkins, se integran gracias a *plugins*. El uso de *tools* {}, *git* y *maven* en la definición del *pipeline* están disponibles gracias a *plugins* como [pipeline: declarative](#), [git](#) y [maven integration](#). Estos vienen ya integrados por defecto en los *plugins* recomendados y es por esto por lo que podemos utilizar estas características, pero considere este aspecto si en algún momento no encuentra disponibles las características.

12.2.2 Herramientas instaladas con el gestor de paquetes

Cuando la herramienta queda instalada a nivel de sistema a través del gestor de paquetes del sistema operativo, la herramienta ya está disponible para su uso y entonces es más fácil acceder a ella desde el *pipeline* declarativo:

```
1 pipeline {
2   agent any
3   stages {
4     stage('Build') {
5       steps {
6         sh 'mvn -v' // usa la instalación del sistema
7       }
8     }
9   }
10 }
```

Figura 165. Ejemplo de código que utiliza la instalación a nivel SO de la herramienta Maven, sin referenciarla en la directiva *tools*

En este caso, en la línea 6 se invoca directamente a la herramienta Maven y Jenkins buscará por ella en el PATH del servidor donde se encuentra el agente de Jenkins que ejecuta el *pipeline*. Aquí solamente asegúrate de tener la herramienta instalada y accesible por el PATH.

12.2.3 2.3 Herramientas instaladas con binarios

Cuando has descargado los binarios de una herramienta, dado que no se ha podido instalar con el gestor de paquetes del sistema operativo, el uso de dicha herramienta desde un *pipeline* declarativo cambia.

```
1 pipeline {
2   agent any
3   environment {
4     HUGO_BIN='/home/tools/hugo/0.151.2'
5   }
6   stages {
7     stage('Build') {
8       steps {
9         sh '$HUGO_BIN/hugo version'
10      }
11    }
12  }
13 }
```

Figura 166. Código que muestra la referencia y uso de una herramienta (Hugo), utilizando directamente la referencia a sus binarios

En el código mostrado previamente, en la línea 4 se declara una variable de entorno que apunta a la carpeta donde se encuentran los binarios de la herramienta, para que en la línea 9 se ejecute el comando `hugo version` accedido directamente desde la carpeta donde se encuentran los binarios.

Esta lógica anteriormente presentada es la forma más simple para acceder herramientas desde sus binarios: referenciar la ruta de los binarios a través de una variable de entorno y ejecutar los comandos con ayuda de esa ruta.



Cuestionario 31. Accediendo a herramientas en Jenkins

Responde las siguientes preguntas para evaluar la lectura previa y verificar tu comprensión respecto a los métodos disponibles para acceder a las herramientas complementarias en Jenkins.

1. ¿Qué directiva de un *pipeline* declarativo se utiliza para habilitar herramientas configuradas en Jenkins?

- A) *tools*. B) *stage*. C) *environment*. D) *agent*.

2. Verdadero o falso: antes de utilizar una herramienta desde un *pipeline* declarativo, esta debe instalarse ya sea a través de la interfaz gráfica, con el gestor de paquetes del sistema operativo o directamente con los binarios.

A) Verdadero. B) Falso.

3. Verdadero o falso: en la declaración `tools { maven '3.9.5' }` de un *pipeline* declarativo, "3.9.5" le indica **directamente** a Jenkins que esa es la versión que debe utilizar.

A) Verdadero. B) Falso.

4. Verdadero o falso: al registrar una nueva herramienta como Git desde la ventana de gestión de herramientas de la interfaz gráfica de Jenkins, esto permite que la herramienta (en este ejemplo el comando git) esté disponible para utilizarse desde cualquier *pipeline* sin la necesidad de otra configuración.

A) Verdadero. B) Falso.

5. Dentro de un equipo de desarrollo te han asignado la tarea de configurar un *pipeline*. Te han comentado que ya hay una configuración del JDK de Java v13.0.2 en el servidor, que lleva por nombre *Java-Global*. ¿Qué configuración dentro de la directiva `tools` es la correcta para utilizar esta herramienta ya configurada?

A) `jdk '13.0.2'`. B) `jdk 'java-13.0.2'`. C) `jdk 'Java-Global'`. D) Todas las anteriores.

6. Te han comentado que en el agente de Jenkins que ejecutará un *pipeline* hay una versión de Java instalada. Suponiendo que el comando `java` está expuesto en el PATH, verdadero o falso: puedes utilizar el comando `java` dentro del *pipeline* sin mayor configuración y este funcionará correctamente.

A) Verdadero B) Falso

7. Tu líder de proyecto te ha comentado que tienen una herramienta interna llamada *executor*; necesitas utilizarla en la ejecución de un *pipeline* para hacer el *build* del proyecto. Te dijo que ya tienen los binarios de la herramienta en el servidor donde se ejecutará el *pipeline* y te ha solicitado que construyas la definición del *pipeline* utilizando el comando *build* de esta herramienta para construir el proyecto. ¿Necesitas más información para generar la definición del *pipeline*?

- | | | | |
|---|--|---|--|
| A) Sí, el nombre de la <i>tool</i> registrado en Jenkins. | B) Sí, la ruta donde están almacenados los binarios. | C) No, pues no es posible utilizar este tipo de herramientas. | D) No, toda la información necesaria ya ha sido proporcionada. |
|---|--|---|--|

8. Verdadero o falso: el uso de todas las herramientas desde un *pipeline* declarativo es idéntico sin importar la forma en que dichas herramientas fueron instaladas.

- A) Verdadero. B) Falso.



Capítulo 13

13 Integración de Maven en *pipelines* declarativos

Contenido del capítulo

- 13.1. Integración de Maven en *pipelines* declarativos paso a paso
 - 13.1.1. Preparativos para la práctica
 - 13.1.2. Herramientas para ejecutar el proyecto
 - 13.1.3. Configuración de Git en Jenkins
 - 13.1.4. Trabajando con el proyecto en GitHub
 - 13.1.5. Snippet Generator
 - 13.1.6. Configurando Maven en Jenkins
 - 13.1.7. Utilizando Maven en el *pipeline*

13.1 Integración de Maven en *pipelines* declarativos paso a paso



Práctica 3. Integración de Maven en un *pipeline* declarativo paso a paso

Avanzado

Objetivo: el estudiante integra Maven con *pipelines* declarativos en Jenkins para compilar y empaquetar proyectos de Java.

Prerrequisitos: servidor de Jenkins instalado y funcionando correctamente. Cuenta activa en GitHub, editor de texto (Visual Studio Code idealmente). Java y Maven instalados en el entorno local, y repositorio de GitHub utilizado en la práctica anterior.

Descripción: la siguiente práctica guía paso a paso en la extensión de un *pipeline* declarativo base, para integrar Maven como herramienta para la construcción y empaquetado de una aplicación web sumamente básica en Spring Boot (Java). Se ponen en práctica conceptos importantes relacionados con la integración de Jenkins con herramientas, repositorios para recuperar el código fuente de un proyecto y la correcta gestión del espacio de trabajo de los *pipelines*.

Hasta este punto de la guía, el lector ha creado *Jobs* de tipo *pipeline* en Jenkins exitosamente, y ha logrado establecer comunicación entre GitHub y Jenkins para recuperar la definición de un *pipeline* declarativo. Sin embargo, no se han recuperado ningún tipo de proyecto desde el repositorio de GitHub, ni tampoco se ha realizado la compilación del código. En esta práctica se extenderá el *pipeline* para realizar la compilación de un proyecto escrito en Java.

13.1.1 Preparativos para la práctica

Si has llegado hasta este capítulo de la guía, deberías de contar con un servidor de Jenkins funcional, instalado con algunos *plugins* generales, y deberías de tener una cuenta de GitHub activa, estos elementos son indispensables para realizar esta práctica.

Antes de comenzar, es adecuado que verifiques que tu servidor de Jenkins cuenta con los *plugins*: Pipeline, Git, GitHub y Pipeline: SCM Step, que ya se han explicado, y algunos se han usado en prácticas previas. Además, serán necesarios los *plugins* Maven Integration y Pipeline Maven Integration, los cuales se explicaron más a fondo en el capítulo [Integración Maven con Jenkins](#).

De la misma forma, para este punto ya deberías de tener tus credenciales de GitHub registradas en Jenkins, si no es así, puedes revisar la sección [Configurar Credenciales de acceso a GitHub](#) para más información al respecto.

Como en la práctica previa [Tu primer pipeline paso a paso](#), se utilizará la misma definición del *pipeline* ubicada en el [repositorio público de esta guía](#), sin embargo, en esta ocasión se utilizarán las ramas “*checkout-scm-pipeline*” y “*mvn-build-pipeline*” como referencia.

El objetivo principal de esta práctica es que, a partir de la definición de un *pipeline* declarativo, el estudiante pueda recuperar un proyecto completo de un repositorio de GitHub, clonarlo localmente en el espacio de trabajo del *pipeline* de Jenkins, y ejecutar la compilación y empaquetado del código de manera automatizada mediante un simple clic.

13.1.2 Herramientas para ejecutar el proyecto

Aunque se podría realizar toda esta práctica sin siquiera abrir el proyecto en el repositorio de GitHub, motivamos al lector a descargar y tratar de ejecutar el proyecto localmente de forma manual (en su equipo, no en la máquina virtual), con el fin de verificar que el proyecto se ejecuta correctamente y no existe algún conflicto de dependencias o alguna actualización de Java o Maven que pudieran afectar la ejecución del proyecto.

Para poder ejecutar el proyecto, es necesario que la máquina cuente con una instalación de Java Temurin JDK 21 y de Maven 3.8 o mayor. Si requiere ayuda para instalar estas herramientas en máquinas Windows, Linux y MacOS, puedes consultar los siguientes recursos: [Tutorial: Instalar Java](#) y [Tutorial: Instalar Maven](#). Al finalizar las instalaciones, puede verificar que estas se hayan realizado correctamente abriendo una terminal y escribiendo los comandos **java --version** y **mvn --version**.

Además, recomendamos instalar el editor de código Visual Studio Code o cualquier otro editor de código o IDE de desarrollo de su preferencia con el que pueda abrir el

proyecto y ejecutarlo. En caso de usar Visual Studio Code, simplemente debe abrir una terminal y escribir el comando ***mvn spring-boot:run*** para ejecutar la aplicación en el puerto 8081, con esto podrá verificar que el proyecto funciona correctamente antes de automatizar la compilación mediante Jenkins.

13.1.3 Configuración de Git en Jenkins

En la última práctica utilizamos Git para recuperar la definición del *pipeline*, sin embargo, no se configuró a Git como herramienta de Jenkins, debido a que Jenkins automáticamente detecta y utiliza la instalación de Git del servidor Ubuntu (si es que existe). Sin embargo, cuando se trabaja con varios agentes o nodos de Jenkins, es posible necesitar más de una instalación de Git. Por ejemplo, si se tiene un agente Windows, sería necesario configurar una instalación de Git con la ruta (PATH) de Windows. Así que es recomendable configurar la herramienta Git que queremos usar en nuestro *pipeline*, para que no surjan errores inesperados en Jenkins.

Ahora que ya sabes instalar y configurar herramientas en Jenkins (revisa el capítulo [Instalación de herramientas en Jenkins](#) para más detalles), podemos realizar algunas configuraciones para indicarle a Jenkins qué instalación de Git debe usar.

Primero, hay que ir a la sección *Manage Jenkins / Tools*. Dentro de esta sección hay que buscar el apartado *Git installations* y hacer clic en el botón *Add Git*, lo cual abrirá un formulario muy sencillo. Aquí se debe ingresar el nombre que se desea asignar a esta herramienta, por ejemplo *Linux*. En el campo *executable PATH* es posible que ya tenga el valor *Git* por defecto; se puede dejar ese valor o ingresar la ruta completa */usr/bin/git*, y dejar desmarcada la casilla *install automatically*. Esta última opción se suele usar con agentes o nodos de Jenkins para instalar Git en los nodos automáticamente. Sin embargo, en esta práctica no se usará esta opción, ya que si solo se tiene una instancia de Jenkins y estás usando Jenkinsfiles, es necesario instalar Git manualmente para poder recuperar el Jenkinsfile del repositorio.

The figure shows two examples of the Jenkins 'Git' tool configuration. The top example is for 'Linux' with the path '/usr/bin/git'. The bottom example is for 'Windows' with the path 'C:\Program Files\Git\cmd\git.exe'. A red error message is displayed below the Windows path, stating: 'There's no such executable C:\Program Files\Git\cmd\git.exe in PATH: /usr/local/sbin, /usr/local/bin, /usr/sbin, /usr/bin, /snap/bin.'

Figura 167. Ejemplos de configuración de la herramienta Git para los entornos Linux y Windows

Finalmente, en la configuración del *pipeline*, se debe seleccionar la herramienta Git que se desea usar para este *job*. En este caso, al ser un servidor Ubuntu, se selecciona la herramienta “Linux”. Y en un supuesto, si este *pipeline* se ejecutara en un agente Windows, entonces usaríamos la herramienta Git para Windows.

The figure shows the Jenkins configuration page for a pipeline. The 'Pipeline' tab is selected. Under the 'Git executable' section, a dropdown menu is open, showing the following options: 'Linux', 'Linux' (highlighted), 'Windows', and '(Auto)'.

Figura 168. Configuración de pipeline para asociar el ejecutable de Git

A diferencia de otras herramientas como Maven, no es necesario declarar la herramienta Git dentro del Jenkinsfile en el bloque *tools*, ya que Jenkins siempre busca automáticamente la instalación del servidor o, como en este ejemplo, se hace referencia a la herramienta directamente desde la configuración del *pipeline*.

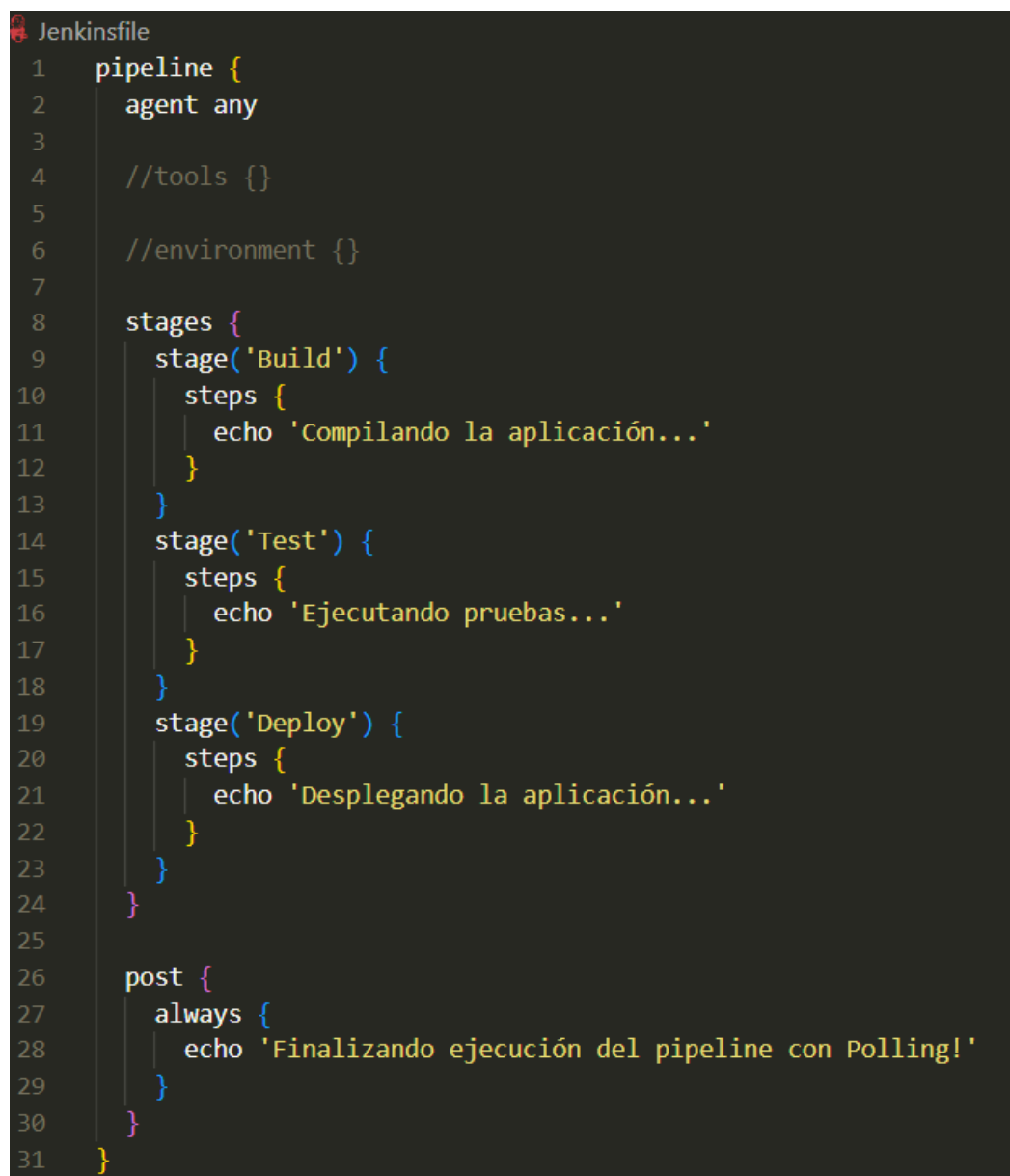
13.1.4 Trabajando con el proyecto en GitHub

Con todas las configuraciones realizadas, es momento de clonar el proyecto de GitHub a un *pipeline* de Jenkins. En la práctica [Tu primer pipeline paso a paso](#), se estableció la conexión entre GitHub y Jenkins para recuperar la definición del *pipeline*.

Sin embargo, como se explicó en el capítulo [GitHub y el espacio de trabajo](#), todavía no se ha clonado el proyecto a Jenkins, solo se recuperó la definición del *pipeline*.

Antes de continuar, nuevamente sugerimos que hagas un *fork* del [repositorio público de esta guía](#), si aún no lo has realizado, para que puedas trabajar en tu propio espacio sin ningún inconveniente y puedas modificar el Jenkinsfile conforme vas realizando cada práctica; esto anterior se explica en la sección [Creación de un Fork del repositorio](#).

A partir de la práctica previa, actualmente el Jenkinsfile que está consumiendo el *pipeline* de Jenkins se ve así:



```
Jenkinsfile
1 pipeline {
2   agent any
3
4   //tools {}
5
6   //environment {}
7
8   stages {
9     stage('Build') {
10      steps {
11        echo 'Compilando la aplicación...'
12      }
13    }
14    stage('Test') {
15      steps {
16        echo 'Ejecutando pruebas...'
17      }
18    }
19    stage('Deploy') {
20      steps {
21        echo 'Desplegando la aplicación...'
22      }
23    }
24  }
25
26  post {
27    always {
28      echo 'Finalizando ejecución del pipeline con Polling!'
29    }
30  }
31 }
```

Figura 169. Código con la definición base del pipeline para esta práctica

Como se puede observar en la imagen previa, no hay un *stage* que recupere los elementos del repositorio y los clone al espacio de trabajo del *pipeline*, es necesario agregar este *stage* antes del *build* para que todos los archivos y código fuente se encuentren disponibles para hacer la compilación mediante Maven. Para esto, se debe especificar la URL del repositorio, la rama que se quiere clonar y las credenciales de acceso dentro de una directiva *step*.

En la práctica anterior, trabajamos sobre la rama *pipeline-base*. Sin embargo, esta práctica seguirá lo cargado en la rama *checkout-scm-pipeline*. Ahí ya se encuentra el proyecto en Java que construiremos, así como el Jenkinsfile actualizado para realizar el *checkout*.

Aun con lo anterior, pon mucha atención a la sección siguiente, pues en ella utilizaremos una herramienta de gran utilidad para generar código de los *pipelines*. Esto te ayudará mucho para actualizar la etapa de *checkout* del *pipeline*, adaptándola a que esta funcione en tu entorno local, con tus propias configuraciones, trabajando con tu *fork* del proyecto.

13.1.5 Snippet Generator

Debido a que Jenkins utiliza los comandos directos de cada herramienta con la que se integra, es muy complicado conocer todos los disponibles para una herramienta. Por ello, desde la versión 2.0 de Jenkins se introdujo una herramienta muy útil para escribir los *steps* individuales del *pipeline*, provistos por los *plugins* instalados (CD Foundation, 2025m).

La herramienta se llama *Snippet Generator* y para utilizarla, hay que seguir los siguientes pasos:

- Ingresar al menú de un *job* de tipo *pipeline*.
- Navegar a la sección *Pipeline Syntax*, y la primera ventana que se abre es la de *Snippet Generator*.
- En esta ventana se debe seleccionar el *step* deseado en el menú *Sample Step*.
- Llenar el formulario dinámico que aparece; los campos pueden variar para cada *step*.
- Y finalmente, hacer clic en el botón *Generate Pipeline Script*.

En este caso, para crear un *step* que clone el repositorio, se pueden seguir los mismos pasos:

1. Ingresa al menú del *pipeline* con el que se ha estado trabajando y haz clic en la opción *Pipeline Syntax*.

Se abrirá otro menú, donde la primera opción es *Snippet Generator*.

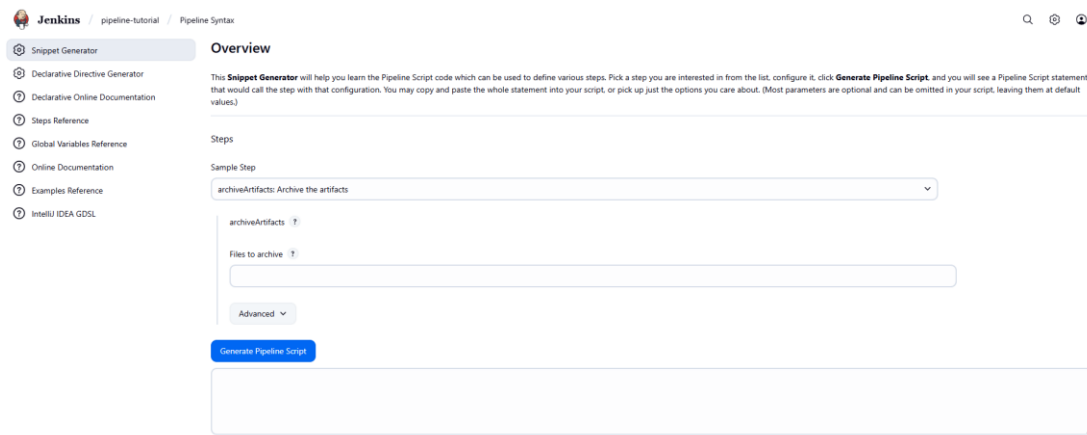


Figura 170. Ventana del Snippet Generator en Jenkins

2. Para el primer campo, selecciona la opción "git: Git"

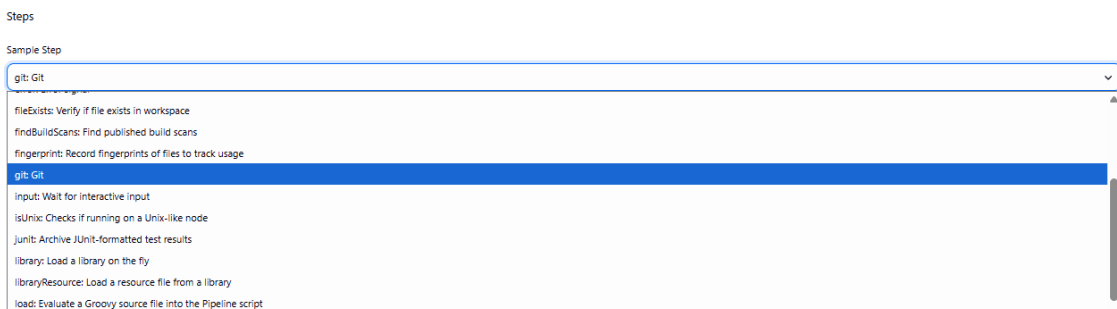


Figura 171. Selección de opción "git: Git" en la ventana del Snippet Generator

3. Esto habilitará un formulario que pedirá información del repositorio de origen, la rama específica y las credenciales necesarias. Muy similar al formulario que se llenó al crear el *job* para recuperar la definición del *pipeline*.
4. Primero, en el campo *Repository URL*, ingresa la URL del repositorio de origen (Tu repositorio privado o el repositorio público de esta guía).

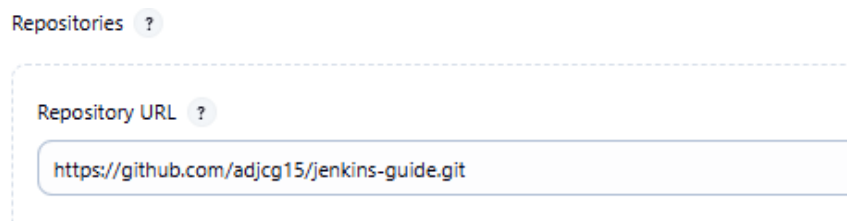


Figura 172. Captura parcial del campo donde se ingresa la URL de GitHub en el proceso de uso del Snippet Generator

- Posteriormente, en el apartado *Branch*, ingresa el nombre de la rama que se desea clonar. En el caso del repositorio público de la guía, la rama *checkout-scm-pipeline* es la que contiene el código final para realizar el *build*.

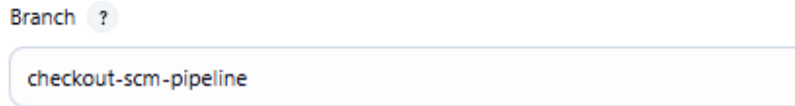


Figura 173. Captura parcial del campo donde se ingresa el nombre de la rama del repositorio GitHub en el proceso de uso del Snippet Generator

- Finalmente, en el campo *Credentials*, selecciona las credenciales que registraste previamente y haz clic en el botón *Generate Pipeline Script*.

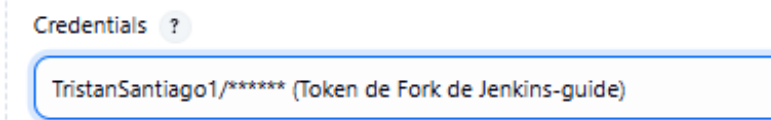


Figura 174. Captura parcial del campo donde se asignan las credenciales de GitHub en el proceso de uso del Snippet Generator

- Esto generará en la parte inferior del formulario el *step* para clonar el repositorio.



Figura 175. Botón para generar el *step* de forma automática, acompañado del *step* generado dadas las configuraciones indicadas en pasos previos

Esto te servirá para actualizar el código en el Jenkinsfile, adaptándolo a tu propio entorno. Tú Jenkinsfile debería tener algo similar a lo siguiente:

```
8   stages {
9
10      stage('Checkout') {
11          steps {
12              git branch: 'checkout-scm-pipeline',
13                  credentialsId: 'github-credential',
14                  url: 'https://github.com/adjcg15/jenkins-guide.git'
15          }
16      }
17
18      stage('Build') {
19          steps {
```

Figura 176. Código con el nuevo stage para realizar el Checkout del proyecto desde GitHub hacia Jenkins

Además, debes modificar la rama de origen en tu *pipeline* de Jenkins (En *Configuration/Pipeline/branches to build*), para que en lugar de usar la rama *pipeline-base*, Jenkins busque el *Jenkinsfile* en la rama *checkout-scm-pipeline*.

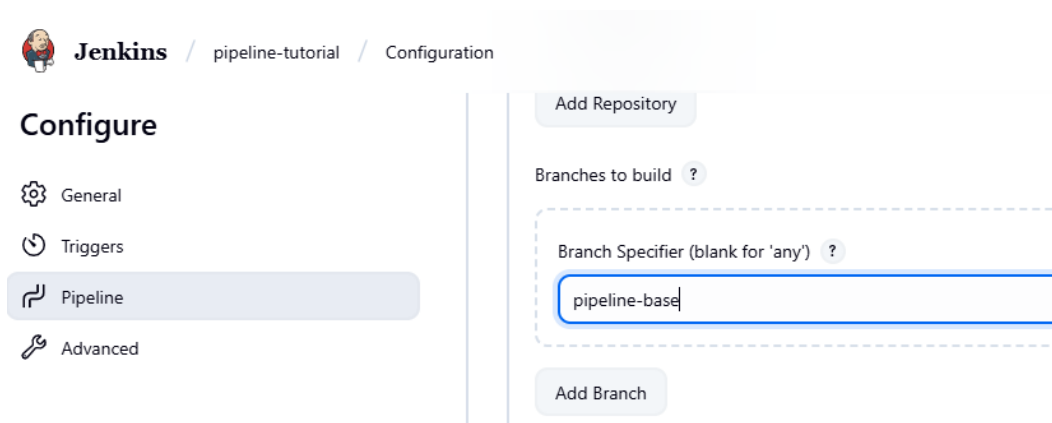


Figura 177. Captura parcial de la configuración del pipeline, referenciando al campo de la rama de GitHub, donde se muestra el valor previo

Es importante que verifiques los parámetros en el *Jenkinsfile* tras realizar el *fork* del proyecto, para que coincidan con los de tu repositorio privado y con el ID de las credenciales que registraste en Jenkins, utilizando la herramienta *Snippet Generator*.

De igual forma, asegúrate de actualizar la rama a la que apunta tu *job* de tipo *pipeline*. Aquí asumimos que seguimos trabajando sobre el *job* que creamos en la práctica previa, solamente adaptándolo a este nuevo enfoque.

Una vez realizadas estas modificaciones, puedes guardar los cambios (*commit* y *push*) y ejecutar el *pipeline* nuevamente.

1. Una vez se ejecuta, en la sección de *Builds* haz clic en el último *build* realizado.

2. En el siguiente menú, haz clic en el botón *Pipeline Overview* para ver las salidas obtenidas de la ejecución.
3. En este apartado que se abre, puedes revisar cada *stage* ejecutado, y observar cómo se utilizan los parámetros declarados en el Jenkinsfile para clonar el repositorio.

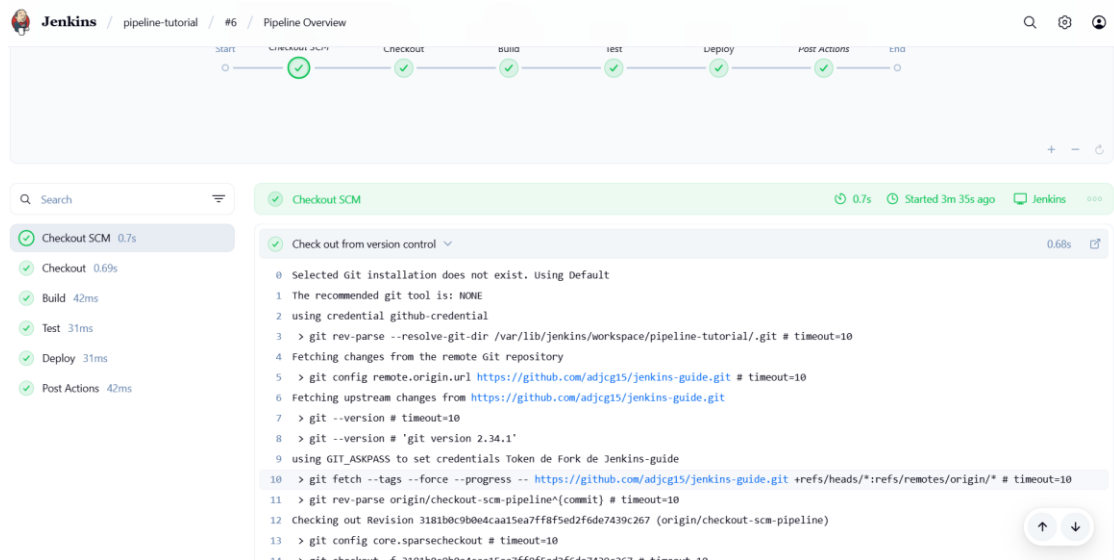


Figura 178. Pantalla de "Pipeline Overview", donde se pueden verificar los detalles sobre los pasos del pipeline junto con los parámetros utilizados

4. Si regresas al menú anterior y haces clic en el botón *Workspace*, se muestra el enlace para ir a la carpeta donde se encuentra el *workspace* del *pipeline* (esto ya se ha mostrado a lo largo de la guía).

Al hacer clic en el enlace del *workspace*, se pueden ver todos los archivos que se clonaron del repositorio a Jenkins. Para este punto ya debería ser visible la carpeta que contiene el Proyecto escrito en Java denominado "Jenkins-Guide_To-Do-List".

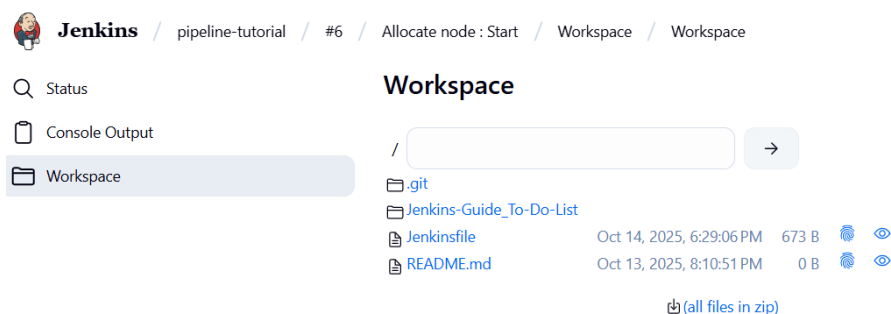


Figura 179. Pantalla del espacio de trabajo del pipeline tras realizar el step de checkout

5. Dentro del proyecto se encuentra un API y aplicación web escritas en Java Spring Boot, así como el archivo pom.xml que utilizará Maven para compilar el proyecto.

Workspace

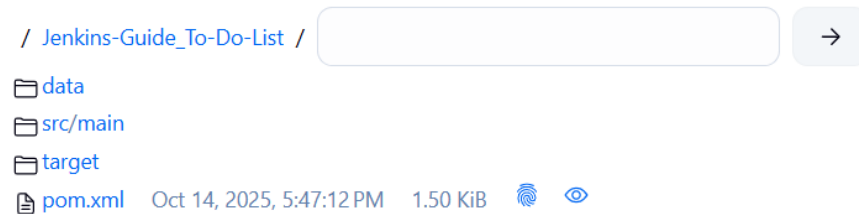


Figura 180. Archivos dentro del proyecto Java

Hasta este punto has clonado exitosamente un proyecto completo desde un repositorio hasta un *pipeline* de Jenkins, lo siguiente es compilar el código fuente utilizando Maven.

13.1.6 Configurando Maven en Jenkins

Tal como se explicó previamente, es indispensable que se encuentren instalados en Jenkins los *plugins* Maven Integration y Pipeline Maven Integration. Una vez instalados estos *plugins*, es posible configurar las herramientas correspondientes dentro de Jenkins (revisa el capítulo [Instalación de herramientas en Jenkins](#) para más detalles).

En primera instancia, es necesario configurar una instalación de Maven dentro de Jenkins. Para esto hay que ir a la sección: *Manage Jenkins/Tools*; dentro de esta sección, hay que buscar el apartado *Maven installations* y hacer clic en el botón *Add Maven*, lo cual abrirá un formulario muy sencillo.

En este formulario se debe ingresar el nombre que se desea asignar a esta herramienta (sugerimos “Maven3”), así como marcar la casilla *Install automatically*, seleccionar la versión de Maven correspondiente (sugerimos v3.9.x), y hacer clic en el botón *Save*. Estas configuraciones le delegaran a Jenkins la instalación de esta herramienta.

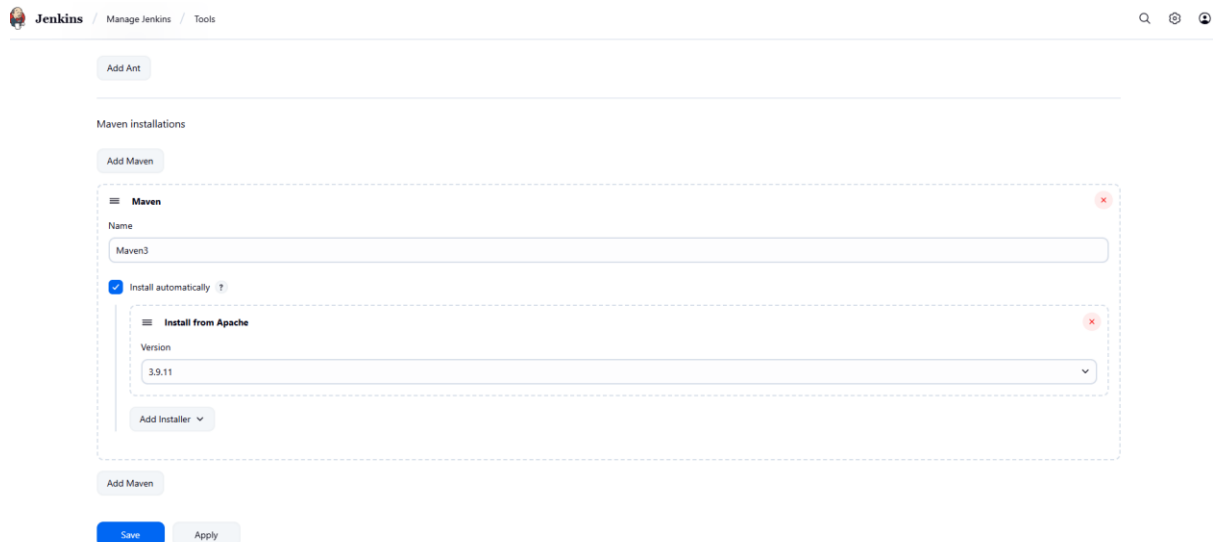


Figura 181. Configuración sugerida para la instalación de la herramienta Maven

El integrar herramientas con Jenkins de esta forma permite una mayor portabilidad y replicabilidad de los *pipelines*, ya que estos no dependerán de las herramientas instaladas en el ambiente local y configuraciones particulares de cada máquina que hospede al servidor de Jenkins. En su lugar, todas las herramientas serán instaladas y gestionadas por el propio Jenkins.

También puedes optar por instalar Maven manualmente en el servidor (como con Git) y simplemente pasar como parámetro la ruta (PATH) del MAVEN_HOME, aunque esto no lo recomendamos por las razones antes dadas.

Con estas configuraciones, ya es posible utilizar Maven dentro de los Jobs de Jenkins.

13.1.7 Utilizando Maven en el *pipeline*

Ahora que ya has clonado el proyecto y realizado las configuraciones correspondientes en Jenkins, es posible utilizar Maven para compilar el código fuente y ejecutar el proyecto de manera local (por ahora).

Aunque también es posible utilizar el *Snippet Generator* para crear el código de Maven, en este caso específico no lo usaremos porque el código que genera es muy simple. Si quisieras probarlo, vuelve a ingresar al *Snippet Generator* y busca la instrucción *withMaven*, completa el formulario y haz clic en el botón *Generate Pipeline Script*, pero esta opción generará un bloque de código como el siguiente.

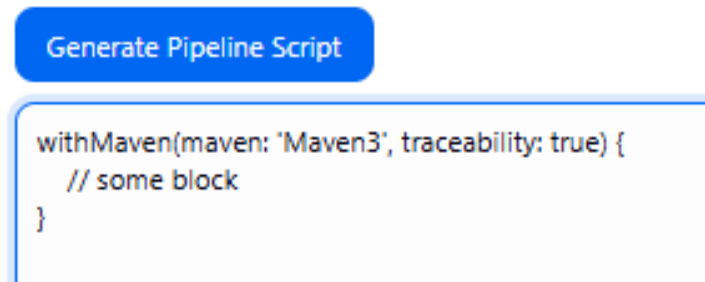


Figura 182. Código generado por el Snippet Generator para ejecutar Maven en el pipeline de Jenkins

La única ventaja de este código es que al declarar la herramienta “Maven3” en ese bloque, ya no es necesario declararla de manera global en la directiva *tools*. Sin embargo, si se usa Maven en distintos bloques, es mejor declararla de manera global, así como se desarrolla en esta guía. Para ello, sigue los siguientes pasos:

1. Primero, hay que declarar la herramienta Maven dentro de la directiva *tools* en el Jenkinsfile. Para esto, solo hay que agregar la línea de código **maven 'Maven3'**; esta simple línea le indica a Jenkins que se quiere utilizar la instalación de Maven previamente configurada.

```
4  tools {  
5    |   maven 'Maven3'  
6  }
```

Figura 183. Configuración de la herramienta Maven en la directiva *tools* dentro del pipeline de Jenkins

2. De manera general, para utilizar Maven solo se requiere un bloque *steps* y dentro un *step* de tipo **sh**, seguido del comando específico de Maven para ejecutar alguna de las fases de la herramienta. Puedes revisar las fases disponibles en el capítulo [Comandos Comunes de Maven](#). En este caso, puede seleccionar cualquier fase hasta *install* e ingresarla entre comillas simples.

Es importante mencionar que, debido a que el proyecto por compilar se encuentra dentro de una carpeta interna del repositorio, es necesario indicar la ruta relativa del archivo *pom.xml*. Esto se hace agregando el parámetro **-f Jenkins-Guide_To-Do-List/pom.xml**, este comando solo indica que busque dentro de la carpeta “Jenkins-Guide_To-Do-List” y no en la raíz del repositorio.

```
stage('Build') {  
  steps {  
    sh 'mvn -f Jenkins-Guide_To-Do-List/pom.xml clean install'  
    echo 'Compilación finalizada!!'  
  }  
}
```

Figura 184. Cambios en la etapa de build para incluir la construcción del proyecto utilizando Maven

Adicionalmente, si se ha prestado atención a los capítulos previos y seguido los consejos dados a lo largo de la guía, su servidor de Jenkins ya debería tener instalado el *plugin* Pipeline: SCM Step. Con este *plugin* se pueden simplificar los pasos para clonar el repositorio, reemplazando todas las instrucciones en el stage “Checkout” por el comando **checkout scm**, tal como se muestra en la siguiente imagen:

```
stage('Checkout') {  
  steps {  
    checkout scm  
  }  
}
```

Figura 185. Código para el checkout simplificado utilizando el plugin Pipeline: SCM Step

Así, Jenkins utilizará las credenciales, URL y rama registradas al crear el *pipeline* y con esto se puede omitir la declaración explícita de dichos parámetros en el Jenkinsfile, y el uso del *Snippet Generator*.

Ahora que ya se tienen las modificaciones necesarios, actualiza los cambios en la rama correspondiente de tu repositorio privado. Por un lado, deberías tener el stage *Build* alineado con lo mostrado anteriormente, así como el stage *Checkout* actualizado con el nuevo enfoque mostrado. No olvides subir tus cambios a tu repositorio cuando finalices. Si tienes alguna duda, la rama [mvn-build-pipeline](#) contiene el Jenkinsfile esperado al final de esta práctica, para tomarlo como referencia.

Una vez realizadas todas las configuraciones y actualizaciones en el Jenkinsfile, ejecuta el *pipeline* nuevamente y corrobora que se realizó la compilación correctamente en la sección *Pipeline overview*.

Al ejecutar el *pipeline* por primera vez después de configurar la instalación automática de Maven, es posible que tarde más de lo normal en finalizar la ejecución, ya que Jenkins necesita descargar e instalar todos los paquetes de Maven por primera vez. En ejecuciones futuras, esto ya no tomará tanto tiempo.

En el espacio de trabajo del *pipeline*, se encuentra la carpeta del proyecto “Jenkins-Guide-To-Do-List”, dentro hay una carpeta nueva llamada *target*, que se generó al hacer el *build* y que contiene el código compilado y el archivo *todolist-1.0.0.jar*, que se generó al finalizar la compilación y empaquetado con Maven.

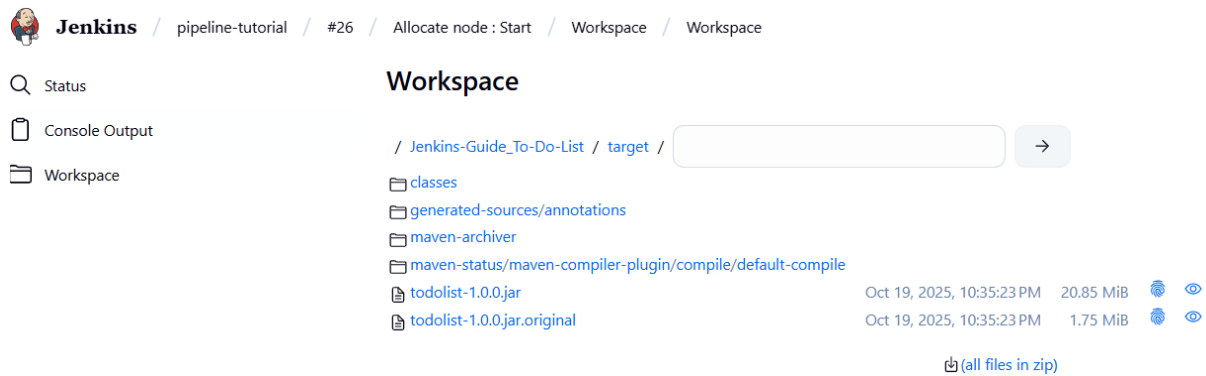


Figura 186. Archivos en el workspace del pipeline tras incluir el build del proyecto

Haciendo clic en el elemento *todolist-1.0.0.jar*, es posible descargarlo para probarlo. Para ello:

1. Descarga el archivo .jar
2. Abre una terminal en el equipo donde descargaste el archivo .jar y navega a la carpeta donde se descargó.
3. Ejecute el comando **java -jar todolist-1.0.0.jar** (asegúrate de tener Java JDK 17+ instalado en su equipo).
4. En el navegador web de tu preferencia, ingresa a la dirección localhost:8081 para ver la aplicación ejecutándose.



Figura 187. Captura de pantalla de la aplicación ejecutándose en el puerto 8081 gracias al archivo .jar

Con esto finaliza la tercera práctica de esta guía. Ahora ya puedes clonar repositorios en Jenkins, utilizar el Snippet Generator para crear *steps* en los Jenkinsfiles, e integrar en Jenkins herramientas como Maven para construir y empaquetar software de manera automatizada. En las siguientes prácticas se expandirá el *pipeline* para ejecutar

pruebas unitarias y de integración mediante el uso de herramientas como Junit y NewMan, así como Docker para desplegar automáticamente la aplicación compilada.



Rúbrica 3. Evaluación práctica "Integración de Maven en un *pipeline* declarativo paso a paso"

En la siguiente rúbrica, marca "Sí" o "No" en cada aspecto de evaluación para determinar si tu implementación cumple con los criterios requerido en la integración de Maven en *pipelines* declarativos para realizar el proceso de *build* para proyectos basados en Java.

Rubrica de Evaluación		
Aspecto por evaluar	Registro de Cumplimiento	
	Si	No
Ejecuté el proyecto escrito en Java en mi máquina local y revisé el proyecto previo a implementar los cambios en el <i>pipeline</i> .		
Revisé que el servidor de Jenkins contara con los <i>plugins</i> necesarios: Pipeline, Git, GitHub, Pipeline: SCM Step, Maven Integration y Pipeline Maven Integration, correctamente instalados y habilitados.		
Utilicé la herramienta <i>Snippet Generator</i> para generar el bloque de código para clonar el repositorio de GitHub.		
Utilicé las directivas <i>steps</i> y <i>stage</i> para actualizar el <i>Jenkinsfile</i> y clonar el repositorio de GitHub dentro del espacio de trabajo de Jenkins.		
Verifiqué que se descargó correctamente el proyecto en Java dentro del espacio de trabajo del <i>pipeline</i> .		

Realicé la configuración de la herramienta Maven en la sección <i>Tools</i> de Jenkins.		
Utilicé las directivas <i>steps</i> y <i>stage</i> para actualizar el <i>Jenkinsfile</i> para compilar con Maven el proyecto escrito en Java.		
Sustituí el bloque de código para clonar el repositorio por el comando checkout scm .		
Verifiqué que el archivo todolist-1.0.0.jar se encontraba dentro del espacio de trabajo después de ejecutar el <i>pipeline</i> final.		
Verifique al menos una vez la sección <i>Pipeline Overview</i> para revisar que el <i>pipeline</i> se ejecutó exitosamente.		
Ejecuté el archivo todolist-1.0.0.jar e ingreso a la dirección localhost:8081 para comprobar su correcta ejecución .		



Capítulo 14

14 Pruebas y pipelines

Contenido del capítulo

- 14.1. Pruebas de software y *pipelines* de Jenkins
- 14.2. Herramientas de prueba de software
- 14.3. Integración Junit con Jenkins
 - 14.3.1. Uso de JUnit como herramienta de prueba
 - 14.3.2. *Plugins* de Jenkins para integrar Junit
 - 14.3.3. Comando de Junit para recordar
 - 14.3.4. Acceso a Junit desde *pipelines* declarativos en Jenkins
- 14.4. Integración de Postman y Newman con Jenkins
 - 14.4.1. Uso de Postman y Newman como herramientas de prueba
 - 14.4.2. Newman en los *pipelines* declarativos de Jenkins
- 14.5. Integración de herramientas de prueba en *pipelines* declarativos paso a paso
 - 14.5.1. Prerrequisitos
 - 14.5.2. Herramientas para ejecutar el proyecto
 - 14.5.3. Agregando las pruebas unitarias al proyecto
 - 14.5.4. Agregando las pruebas de integración al proyecto
 - 14.5.5. Configuración de *plugins* necesarios en Jenkins
 - 14.5.6. Modificación de la declaración del *pipeline*
 - 14.5.7. *Pipeline* desde Jenkins

14.1 Pruebas de software y *pipelines* de Jenkins



Lectura 35. La prueba de Software

Nivel básico

Objetivo: el estudiante identifica los conceptos de prueba de software, niveles y tipos de pruebas de software.

Descripción

La lectura explica el concepto de prueba de software en el contexto del ciclo de vida del desarrollo de software, así mismo se abordan los tres niveles de prueba de software (unitarias, integración y de sistema), y los tipos de prueba de software que se suelen automatizar en Jenkins.

La prueba de software es un concepto muy amplio. La ISO/IEC/IEEE (2022), dentro del estándar 29119 (parte 1), define la prueba de software como un conjunto de actividades realizadas para facilitar el descubrimiento y/o evaluación de propiedades de uno o más elementos de prueba, como sistemas, documentos de requisitos o especificaciones de diseño. Actividades como la planeación, preparación, ejecución, informe y gestión son algunas de estas actividades.

Dentro de estas actividades, destacamos la ejecución de pruebas, que es el proceso de ejecutar una prueba sobre el elemento de prueba, produciendo resultados reales (ISO/IEC/IEEE, 2022).

El hecho de resaltar esto anterior, se debe a que la ejecución de pruebas resulta ser la actividad más automatizada con Jenkins, seguido de actividades como la comparación de las ejecuciones de pruebas, el registro de dichas ejecuciones y el preparado del entorno de prueba, así como de datos de prueba.

En este contexto, debemos añadir que existen niveles de prueba y tipos de prueba. Los niveles de prueba son una secuencia de etapas de prueba, cada una asociada con el logro de objetivos específicos y con el fin de tratar riesgos particulares; por ejemplo, pruebas de componentes (unitarias), de integración, sistema, integración de sistemas o aceptación. Por otra parte, los tipos de prueba hacen referencia a grupos de actividades de prueba que están enfocadas en características específicas de calidad, como pruebas de seguridad, funcionales, de usabilidad o de rendimiento (ISO/IEC/IEEE, 2022).

Aunque el estándar 29119 no proporciona una definición directa de estos niveles de prueba, sí lo hace la IEEE Computer Society (2024) dentro del SWEBOK:

- Prueba unitaria: pruebas que verifican la funcionalidad de elementos del sistema bajo prueba de forma aislada, que pueden ser subprogramas, subsistemas o componentes.
- Prueba de integración: pruebas que verifican la interacción entre elementos del sistema bajo prueba, como componentes, módulos o subsistemas.
- Prueba de sistema: pruebas que tienen como objetivo verificar el comportamiento del sistema bajo prueba (como un todo).
- Prueba de aceptación: pruebas que tienen como objetivo principal verificar que el sistema bajo prueba satisface los requisitos y expectativas de los usuarios finales.

Las pruebas unitarias son por mucho el nivel de prueba más automatizado con Jenkins, seguido por las pruebas de integración y de sistema en ese orden. Mientras que los tipos de pruebas más automatizados con Jenkins son pruebas de seguridad y funcionales.

Para resumir, debe quedar claro que la prueba de software es un concepto amplio, que involucra múltiples actividades las cuales pueden ser automatizadas; y que dentro de este contexto, destacan tanto niveles de prueba, como tipos de prueba, dos conceptos separados pero altamente relacionados. Esto último es importante, pues debes entender que son conceptos separados: puede haber pruebas unitarias funcionales, de integración de rendimiento o similar.



Lectura 36. Actividades de prueba de software dentro en Jenkins

Nivel básico

Objetivo: el estudiante localiza las actividades de prueba de software dentro de la etapas típicas de un *pipeline* de despliegue en Jenkins.

Descripción

La lectura aborda las actividades de prueba de software que suelen automatizar e integrar en los *pipelines* de despliegue en Jenkins. Así

mismo, se resalta en qué etapa del *pipeline* de despliegue ocurren estas actividades de prueba.

El estándar de pruebas 29119 (parte 2) reconoce tres niveles de procesos de prueba, definidos como conjuntos de actividades relacionadas las cuales transforman entradas en salidas y que son: procesos organizacionales, de gestión y dinámicos (ISO/IEC/IEEE, 2021).

Hablando particularmente de los procesos dinámicos de prueba, el estándar ya mencionado resalta que estos son utilizados para llevar a cabo pruebas dentro de un nivel de prueba o tipo de prueba; reconoce cuatro procesos dentro de este nivel: diseño de pruebas e implementación, gestión de ambiente y datos de prueba, ejecución de pruebas y reporte de incidentes de pruebas (ISO/IEC/IEEE, 2021).

Dentro de estos procesos se desarrollan actividades, las cuales pueden ser automatizadas utilizando herramientas. En el contexto de *pipelines* de despliegue con Jenkins, podemos destacar la automatización de actividades como:

1. Ejecutar los procedimientos de prueba (ejecución de pruebas)
2. Comparar los resultados de pruebas (ejecución de pruebas),
3. Registrar la ejecución de pruebas (ejecución de pruebas),
4. Establecer el entorno de pruebas (gestión de ambiente y datos de prueba) y
5. Preparar datos de prueba (gestión de ambiente y datos de prueba).

Las actividades previamente mencionadas no son todas las actividades de prueba reconocidas por el estándar 29119 dentro de los cuatro procesos dinámicos de prueba. Sin embargo, sí son las actividades más automatizadas en *pipelines* de despliegue con Jenkins. Esto no quiere decir que Jenkins automatice dichas actividades; comúnmente Jenkins integra otras herramientas de automatización de pruebas para lograr este nivel de automatización.

Para entender cómo es que se automatizan estas actividades dentro de los *pipelines* de despliegue, recordemos el capítulo [Etapas comunes de los pipelines de despliegue](#) de esta guía. En este capítulo se presentan cuatro etapas para describir el *pipeline* de despliegue, donde dos de ellas (aceptación y pruebas manuales) abarcan la mayor parte de las pruebas.

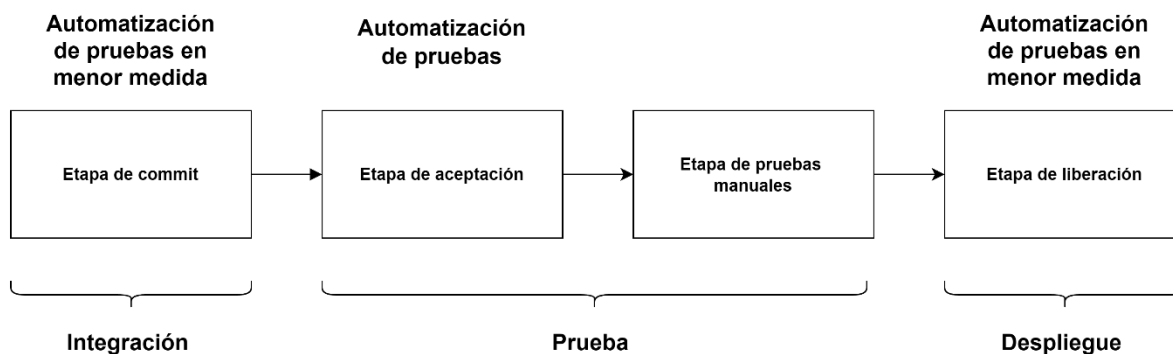


Figura 188. Esquema de las etapas de un pipeline de despliegue alineadas con la automatización de pruebas que ocurre en cada una de ellas

Sin embargo, que estas etapas previamente mencionadas abarquen la mayor parte de las pruebas, no significa que no haya pruebas en las otras etapas. Como lo muestra el diagrama anterior, las etapas de *commit* y liberación también incluyen pruebas, pero en menor cantidad y con enfoques distintos.

Lo importante aquí, es entender que la automatización de las actividades de prueba puede presentarse en cualquiera de las etapas del *pipeline* de despliegue, con tipos y niveles de prueba distintos según el objetivo de la prueba y considerando la automatización solamente de aquellas actividades necesarias.

Por ejemplo, en la etapa de *commit* se puede adoptar un enfoque de pruebas unitarias funcionales que aseguren la calidad del sistema en un primer nivel, descartando la automatización de establecer el entorno de pruebas (actividad 4 previamente mencionada), pues muy probablemente no sea necesario para este tipo de pruebas.

Aun con esto, hay que tener en cuenta que para fines prácticos de esta guía, el *pipeline* que hemos estado desarrollando hasta ahora, tiene etapas específicas para pruebas, considerando pruebas en un nivel de componente (unitarias) y de integración, ambas de tipo funcional.



Cuestionario 32. La prueba de software en los pipelines de Jenkins

Responde las siguientes preguntas para evaluar las dos lecturas anteriores y así verificar tu comprensión respecto a la prueba de software y su relación con los *pipelines* de despliegue.

1. ¿Qué es la prueba de software?

- | | | | |
|---|---|---|---|
| A) Conjunto de actividades para evaluar propiedades de elementos de prueba. | B) Proceso de corrección de errores detectados durante el desarrollo. | C) Documento que describe los requisitos del sistema. | D) Actividad enfocada en mejorar la calidad del software. |
|---|---|---|---|

2. Verdadero o falso: los niveles de prueba y los tipos de prueba son dos formas utilizados para referirse al mismo concepto.

- | | |
|---------------|-----------|
| A) Verdadero. | B) Falso. |
|---------------|-----------|

3. ¿Cuáles son los niveles de prueba de software reconocidos dentro del estándar 29119 de la ISO/IEC/IEEE?

- | | | | |
|---|---|---|--|
| A) Pruebas funcionales, de seguridad, de rendimiento y de usabilidad. | B) Pruebas unitarias, de integración, de sistema y de aceptación. | C) Pruebas manuales, automatizadas, híbridas y exploratorias. | D) Pruebas de diseño, de código, de interfaz y de usuario. |
|---|---|---|--|

4. ¿Cuáles son ejemplos de tipos de prueba según el estándar 29119 de la ISO/IEC/IEEE?

- | | | | |
|---|--|--|---|
| A) Pruebas unitarias, de integración, de sistema y de aceptación. | B) Pruebas de código, de componentes, de interfaz y de aceptación. | C) Pruebas automáticas, manuales, híbridas y de regresión. | D) Pruebas funcionales, de seguridad, de usabilidad y de rendimiento. |
|---|--|--|---|

5. ¿Cuál de las siguientes opciones **no** es un tipo de prueba de software?

- | | | | |
|-------------------------|----------------------|---------------------|--------------------------|
| A) Prueba de seguridad. | B) Prueba funcional. | C) Prueba unitaria. | D) Prueba de usabilidad. |
|-------------------------|----------------------|---------------------|--------------------------|

6. ¿Cuál es el nivel de prueba más automatizado en *pipelines* de despliegue de Jenkins?

- A) De integración. B) De sistema. C) De componente. D) De aceptación.

7. Selecciona la opción que **no** figura entre los tipos de prueba más automatizados con Jenkins.

- A) Prueba funcional. B) Prueba de seguridad. C) Prueba de rendimiento. D) Prueba de usabilidad.

8. Verdadero o falso: un nivel de prueba es una secuencia de etapas de prueba, cada una asociada con el logro de objetivos específicos y con el fin de tratar riesgos particulares.

- A) Verdadero. B) Falso.

9. ¿Qué es un tipo de prueba?

- | | | | |
|---|---|--|---|
| A) Fase que evalúa la integración de componentes. | B) Grupo de pruebas centradas en una característica de calidad. | C) Conjunto de herramientas de ejecución automática. | D) Actividad de comparación de resultados esperados y reales. |
|---|---|--|---|

10. ¿Cuáles son los niveles de procesos de prueba reconocidos dentro del estándar 29119 de la ISO/IEC/IEEE?

- | | | | |
|--|---|--|--------------------------------------|
| A) Organizacionales, de gestión y dinámicos. | B) Funcionales, de aceptación y de integración. | C) Unitarios, de integración y de sistema. | D) Manuales, automatizados y mixtos. |
|--|---|--|--------------------------------------|

11. Selecciona las actividades de prueba que comúnmente se automatizan con Jenkins.

- | | | | |
|---|---|---|---|
| A) Escribir código, compilar y documentar requisitos. | B) Configurar el hardware, crear repositorios y asignar tareas. | C) Revisar código, aprobar <i>merges</i> y liberar versiones. | D) Ejecutar pruebas, comparar resultados y registrar ejecuciones. |
|---|---|---|---|

12. Verdadero o falso: Jenkins automatiza las actividades de prueba sin necesidad de herramientas externas.

- A) Verdadero. B) Falso.

13. ¿En qué etapas de los *pipelines* de despliegue se suele automatizar la mayor parte de las pruebas?

- | | | | |
|--|--|---|--|
| A) Etapas de <i>commit</i> y liberación. | B) Etapas de integración continua y monitoreo. | C) Etapas de aceptación y pruebas manuales. | D) Etapas de despliegue y mantenimiento. |
|--|--|---|--|

14. Verdadero o falso: la automatización de las actividades de prueba puede presentarse en cualquiera de las etapas del *pipeline* de despliegue.

- A) Verdadero. B) Falso.

15. Verdadero o falso: el alcance de la automatización de las actividades de prueba depende de la etapa del *pipeline*.

- A) Verdadero. B) Falso.

14.2 Herramientas de prueba de software



Lectura 37. Herramientas para automatizar la prueba de software

Nivel básico

Objetivo: el estudiante recuerda el concepto de herramienta de prueba de software.

Descripción

La lectura explica el concepto de herramienta de prueba de software. Así mismo, se mencionan distintas herramientas de prueba que se pueden integrar con Jenkins según el tipo de prueba a automatizar.

El proceso de prueba de software involucra una amplia variedad de tareas que pueden ir desde lo más simple, como la generación de datos, hasta actividades más complejas, como la gestión de ambientes de prueba o ejecución de cientos de casos de prueba. Bajo este contexto, la automatización en la prueba de software se vuelve indispensable en los flujos de trabajo de Integración y Despliegue Continuo (IEEE Computer Society, 2024, p. 143).

Para lograr la automatización de la prueba de software, se requiere del uso de programas o aplicaciones especializadas, normalmente conocidas como herramientas de prueba. Las herramientas de prueba son software que da soporte al proceso de prueba, y permite a los desarrolladores y *testers* automatizar gran parte de las actividades y tareas de la prueba de software (ISO/IEC/IEEE, 2013).

Aunque tradicionalmente la automatización del proceso de prueba de software se asocia únicamente con la ejecución automática de casos o *scripts* de prueba, en la actualidad existen muchas otras tareas y actividades que pueden automatizarse mediante las herramientas de prueba. Entre ellas, se incluyen la gestión de los casos de prueba, el monitoreo y control de las pruebas, la generación de datos de prueba, los análisis estáticos, la generación de casos de prueba, así como la creación y mantenimiento de entornos de prueba, entre otras más (ISO/IEC/IEEE, 2013).

Entre las principales herramientas de prueba, podemos destacar el uso de Junit, PHPUnit o ZUnit para la ejecución pruebas unitarias; Selenium, WebDriver, SoapUI y

Postman para ejecutar pruebas funcionales y de aceptación; Gamekins para la generación automática de casos de prueba; JMeter para la realización pruebas de carga, entre otras más con diversos propósitos dentro de los *pipelines* de CI/CD. En términos generales, cualquier herramienta de prueba, ya sea comercial, de código abierto o desarrollada internamente en las organizaciones, puede integrarse con Jenkins para automatizar distintas partes del proceso de prueba de software.



Lectura 38. ¿Por qué usar las herramientas de prueba?

Nivel básico

Objetivo: el estudiante identifica la utilidad del uso de herramientas de prueba de software en los *pipelines* de despliegue.

Descripción

La lectura explica los beneficios y ventajas de utilizar las herramientas de prueba de software, así mismo, se resalta la importancia del papel que juega dentro de los flujos de trabajo de los *pipelines* de despliegue.

En el contexto de CI/CD, la prueba de software se centra en establecer ciclos cortos de retroalimentación que guían el desarrollo, diseño y despliegue del software. Estos ciclos se logran mediante la automatización de las pruebas que se ejecutan con cada cambio en el sistema. Así, para desarrollar software de calidad es necesario establecer pruebas automáticas en los distintos niveles: unitarias, de integración, y de aceptación, así como ejecutarlas como parte de los *pipelines* de despliegue (Humble & Farley, 2010, p. 83-110).

En los proyectos donde se practican la integración y despliegue continuos, la ejecución de las pruebas es más frecuente que en otras metodología ágiles. Tanto las pruebas de regresión, como los nuevos casos de prueba, se ejecutan al menos una vez cada que se realiza una compilación (*build*), fomentando un enfoque de prueba continua. Bajo esta práctica, las pruebas se ejecutan automáticamente bajo demanda, como después de un *commit* al repositorio o durante generación de un *build* para desplegar en producción, garantizando una validación constante del software durante el desarrollo (ISO/IEC/IEEE, 2022).

El utilizar las herramientas de prueba adecuadas ayuda a contrarrestar las carga de las tediosas operaciones administrativas relacionadas con la prueba de software, y las hacen menos propensa a errores, influyendo directamente en la efectividad del proceso de prueba y desarrollo de software (IEEE Computer Society, 2024, p.143).

Entre los principales beneficios que se suelen obtener al integrar herramientas de prueba dentro de los *pipelines* de CI/CD y del propio proceso de desarrollo de software, GeeksforGeeks (2025a) destacan que:

- Se reduce el esfuerzo, se ahorra tiempo, se obtiene una mayor cobertura y mejora la estimación del tiempo de ejecución de las prueba.
- Las herramientas de prueba permiten procesos más confiables y consistentes que las ejecuciones manuales, ofreciendo mayor flexibilidad para ejecutar pruebas de regresión, cambiando los datos de prueba.
- Algunas herramientas permiten realizar pruebas que no son posibles manualmente, como las pruebas de carga o de rendimiento.
- Cada ejecución de pruebas automática puede generar reportes detallados de los resultados obtenidos.

Bajo este contexto, es lógico que el uso de herramientas de prueba también conlleve algunos inconvenientes: no todas son gratuitas, algunas requieren de una licencia costosa, es necesario buscar documentación y adquirir experiencia para poder utilizarlas de manera correcta y eficiente, pueden variar las configuraciones del entorno y la infraestructura necesarias para su utilización, entre otros. Por ello, se deben de tener en consideración estos aspectos antes de seleccionar una herramienta de prueba de software, con el fin de obtener los mayores beneficios al menor costo (GeeksforGeeks, 2025a).



Cuestionario 33. Herramientas para la prueba de software

Responde las siguientes preguntas para evaluar las dos lecturas previas y verificar tu comprensión respecto a las herramientas de prueba de software y su importancia en los *pipelines* de despliegue.

1. De forma general, podemos decir que las herramientas de prueba de software sirven para:

- | | | | |
|---|---|--|---|
| A) Ejecutar <i>scripts</i> y casos de prueba automáticamente. | B) Automatizar actividades y tareas de la prueba de software. | C) Automatizar las actividades más redundantes de la prueba de software. | D) Generar Reportes de resultados de las pruebas. |
|---|---|--|---|

2. De las siguientes herramientas, ¿Cuál no es una herramienta de prueba?

- | | | | |
|----------|-------------|----------|------------|
| A) JUnit | B) Selenium | C) Maven | D) PostMan |
|----------|-------------|----------|------------|

3. ¿Qué tipo de pruebas automatiza la herramienta de prueba JMeter?

- | | | | |
|-------------|--------------|------------------|-----------------|
| A) De carga | B) Unitarias | C) De aceptación | D) De Seguridad |
|-------------|--------------|------------------|-----------------|

4. ¿Cuál de las siguientes opciones **no** es un beneficio de utilizar herramientas de prueba?

- | | | | |
|---|--|-----------------------------------|--------------------------|
| A) Algunas requieren licencias con costo. | B) Algunas requieren configuraciones de entorno e infraestructura. | C) Algunas son complejas de usar. | D) Todas las anteriores. |
|---|--|-----------------------------------|--------------------------|

5. Verdadero o falso: el utilizar herramientas de prueba de software hace más propensa la introducción de errores al proceso de prueba.

- | | |
|---------------|-----------|
| A) Verdadero. | B) Falso. |
|---------------|-----------|

6. Verdadero o falso: el utilizar herramientas de prueba de software mejora la cobertura de pruebas y disminuye el esfuerzo.

- | | |
|---------------|-----------|
| A) Verdadero. | B) Falso. |
|---------------|-----------|

7. Verdadero o falso: el utilizar herramientas de prueba de software permite a los equipos de desarrollo enfocarse en la generación de casos de prueba y refinar los datos de entrada.

A) Verdadero.

B) Falso.

8. Verdadero o falso: el utilizar herramientas de prueba de software permite a los equipos de desarrollo automatizar algunos tipos de pruebas que no se pueden realizar manualmente.

A) Verdadero.

B) Falso.

14.3 Integración de JUnit con Jenkins

14.3.1 Uso de JUnit como herramienta de prueba



Lectura 39. Introducción a las pruebas unitarias con JUnit

Nivel básico

Objetivo: el estudiante recuerda los aspectos clave para utilizar JUnit en la creación de pruebas unitarias de software.

Descripción

La lectura proporciona una breve introducción hacia las pruebas unitarias y su propósito en el desarrollo de software. Además, explica el rol de JUnit en este contexto, abordando cómo permite generar y automatizar las pruebas unitarias.

Como se mencionó en un capítulo previo, las pruebas unitarias (o de componentes) son pruebas que ayudan a verificar la funcionalidad de elementos de forma aislada, que pueden ser subprogramas, subsistemas o componentes (IEEE Computer Society, 2024).

Este tipo de pruebas ofrecen múltiples ventajas en el desarrollo de software, ya que permiten detectar de forma temprana errores de entrada, salida o lógica antes de que lleguen a producción, reduciendo el tiempo dedicado a la depuración y facilitando la

identificación precisa del código defectuoso. Además, funcionan como una forma de documentación, ya que muestran claramente el comportamiento esperado del código, lo que ayuda a otros desarrolladores a comprenderlo, modificarlo o refactorizarlo con mayor confianza y a verificar que los cambios no introduzcan nuevos errores (Amazon Web Services, s/f).

JUnit es un *framework* de pruebas unitarias para Java, diseñado para facilitar la creación y ejecución automatizada de pruebas. Su arquitectura actual (JUnit 6.0.1) se compone de tres subproyectos principales: JUnit Platform, JUnit Jupiter y JUnit Vintage, que trabajan juntos para ofrecer compatibilidad y flexibilidad en la ejecución de pruebas: JUnit Platform permite ejecutar las pruebas en la Java Virtual Machine (JVM); JUnit Jupiter proporciona el modelo para escribir las pruebas; y JUnit Vintage provee un motor para correr pruebas de versiones anteriores del *framework* (Bechtold et al., 2024).

JUnit organiza las pruebas en clases de prueba y métodos de prueba; un método se marca como prueba con la anotación `@Test` y usa aserciones (por ejemplo `Assertions.assertEquals`) para comprobar resultados (Bechtold et al., 2024). Estos son los componentes principales que permiten la implementación de pruebas dentro de JUnit: anotaciones y aserciones.

A continuación se muestra un ejemplo de la estructura general para la definición de pruebas unitarias en JUnit:

```
1  import static org.junit.jupiter.api.Assertions.assertEquals;
2
3  import example.util.Calculator;
4
5  import org.junit.jupiter.api.Test;
6
7  class MyFirstJUnitJupiterTests {
8
9      private final Calculator calculator = new Calculator();
10
11     @Test
12     void addition() {
13         assertEquals(2, calculator.add(1, 1));
14     }
15
16 }
```

Figura 189. Código con la estructura general para la definición de pruebas unitarias en JUnit

Para el lector familiarizado con Java, este código no debe resultar ajeno. Básicamente se define una clase de prueba llamada *MyFirstJUnitJupiterTests* y se define un método de prueba con la anotación *@Test* llamado *addition*, donde se utiliza la aserción *assertEquals* para verificar que un resultado de un método es el que se espera.

Aunque JUnit ofrece muchas anotaciones, algunas de las más comunes son: *@BeforeEach* y *@AfterEach* que se utilizan para ejecutar métodos antes y después de cada prueba individual, respectivamente; permiten preparar o limpiar el entorno de prueba antes y después de cada ejecución. También *@BeforeAll* y *@AfterAll*, que se ejecutan una sola vez antes y después de todas las pruebas de la clase de prueba (Bechtold et al., 2024).

Así, JUnit proporciona el entorno necesario para la automatización de pruebas unitarias dentro de proyectos con Java, asegurándose de incluir los componentes necesarios dentro del proyecto (cuestión que abordaremos en capítulos posteriores) e implementando pruebas de forma tan simple como se mostró en el ejemplo dentro de este capítulo.



Cuestionario 34. Introducción a las pruebas unitarias con JUnit

Responde las siguientes preguntas para evaluar la lectura anterior y verificar tu comprensión respecto al uso de JUnit para la generación y automatización de pruebas unitarias.

1. ¿Qué es una prueba unitaria?

- | | | | |
|--|--|---|---|
| A) Prueba que evalúa la integración entre varios sistemas. | B) Prueba que verifica la funcionalidad de un componente de forma aislada. | C) Prueba que valida el rendimiento general del software. | D) Prueba manual aplicada después del despliegue. |
|--|--|---|---|

2. ¿Cuál de las siguientes afirmaciones describe mejor el propósito de una prueba unitaria?

- | | | | |
|--|--|--|--|
| A) Detectar errores de lógica en componentes antes de llegar a producción. | B) Probar la comunicación entre módulos del sistema. | C) Medir la velocidad de ejecución del código. | D) Detectar errores de lógica en componentes después de desplegar. |
|--|--|--|--|

3. ¿Cuáles de los siguientes son beneficios de ejecutar pruebas unitarias?

- | | | | |
|---|--|--|---|
| A) Incremento del tamaño del código fuente y documentar el código indirectamente. | B) Reducir la complejidad del despliegue y verificar que los cambios no metan errores. | C) Detección temprana de errores y reducción del tiempo de depuración. | D) Mayor comprensión en el comportamiento del código y facilidad de prueba. |
|---|--|--|---|

4. ¿En qué entorno se utiliza JUnit?

- | | | | |
|--|----------------------------------|---|---|
| A) En aplicaciones escritas en Python. | B) En bases de datos SQL Server. | C) En pruebas dentro de <i>pipelines</i> con Jenkins. | D) En proyectos desarrollados con Java. |
|--|----------------------------------|---|---|

5. Verdadero o falso: los componentes principales que permiten la implementación de pruebas dentro de JUnit son las aserciones y los métodos de prueba.

- A) Verdadero. B) Falso.

6. ¿Qué anotación se utiliza para indicar que un método debe ejecutarse antes de todas las pruebas en una clase de JUnit?

- A) @BeforeEach B) @Test C) @BeforeAll D) @BeforeEvery

7. Verdadero o falso: las aserciones permiten verificar si el resultado de una prueba coincide con el resultado esperado.

- A) Verdadero. B) Falso.

8. Relaciona la anotación con su función en JUnit:

- | | |
|------------------------------------|---|
| A) <code>@Test</code> | (1) _____ Ejecuta un método antes de cada prueba. |
| B) <code>@BeforeEach</code> | (2) _____ Marca un método como prueba unitaria. |
| C) <code>@AfterEach</code> | (3) _____ Ejecuta un método una vez después de todas las pruebas. |
| D) <code>@AfterAll</code> | (4) _____ Ejecuta un método después de cada prueba. |

14.3.2 Plugins de Jenkins para integrar JUnit



Lectura 40. *Plugins en Jenkins para usar JUnit*

Nivel básico

Objetivo: el estudiante recuerda el *plugin* y configuraciones necesarias para integrar JUnit con Jenkins.

Descripción

La lectura expone el plugin necesario dentro de Jenkins para integrar JUnit en *pipelines* declarativos, explicando su propósito principal y funcionalidades destacables.

El *plugin* principal que permite trabajar con JUnit dentro de Jenkins lleva el mismo nombre que esta herramienta. El plugin JUnit permite que Jenkins consuma informes XML de pruebas unitarias generados durante los *builds* y proporcione visualización gráfica de los resultados de las pruebas, fallos y tendencias. Así, ayuda a integrar los resultados de pruebas automatizadas dentro del proceso de CI/CD para facilitar el análisis de calidad del código (CD Foundation, 2025h).

El propósito principal del *plugin* previamente mencionado es recuperar los resultados de las pruebas unitarias en el formato JUnit XML, para que Jenkins pueda visualizar qué pruebas pasaron, fallaron o fueron omitidas, ver tendencias de resultados y mejorar la visibilidad de la calidad de la base de código (CD Foundation, 2025h).

En este sentido, durante la ejecución del *pipeline* debemos asegurarnos que los archivos JUnit en formato XML sean generados antes de llamar a este *plugin*, pues este solo se encarga de tomarlos desde la ruta donde se encuentren, para presentar los resultados gráficamente dentro de Jenkins y hacer las demás acciones ya mencionadas.

Destacando algunas otras de las funcionalidades principales de este *plugin*, tenemos (CD Foundation, 2025h):

1. Archivar archivos XML generados por las pruebas.
2. Marcar *builds* como *unstable* si hay fallos.
3. Mostrar gráficos de tendencia de pruebas e informes completos de cada ejecución.

4. Publicar resultados como *checks* en plataformas SCM (para indicar el estado del proyecto que despliega el *pipeline*).

Es importante resaltar que este *plugin* se instala como cualquier otro *plugin* en Jenkins y que no es necesario para ejecutar las pruebas dentro de Jenkins, tal como se explica en los capítulos siguientes. Sin embargo, al momento de instalarlo, este expone un *step* (**junit**) para que Jenkins pueda leer los archivos de resultados XML de las pruebas y con ellos generar los reportes, marcar el *build* como estable/inestable (inestable si hay algún fallo) y mostrar los resultados en la interfaz gráfica de Jenkins (CD Foundation, 2025i).

Así pues, Jenkins no necesita forzosamente de algún *plugin* específico o configuraciones adicionales (fuera de lo abordado hasta este momento dentro de la guía) para integrar JUnit en la automatización de pruebas unitarias. Sin embargo, el *plugin* que lleva el mismo nombre de la herramienta proporciona capacidades extra en la integración de estas dos herramientas.



Cuestionario 35. *Plugins* en Jenkins para usar JUnit

Responde las siguientes preguntas para evaluar la anterior y verificar tu comprensión respecto al uso de JUnit para la generación y automatización de pruebas unitarias.

-
1. ¿Cuál es el nombre del *plugin* que permite integrar resultados de JUnit con Jenkins?

A) Jenkins Test Plugin. B) JUnit Plugin. C) JUnit Runner Plugin. D) JUnit Test Plugin.
 2. ¿Cuál es el formato de los archivos que procesa el *plugin* JUnit?

A) JSON B) YAML C) TXT D) XML
 3. Verdadero o falso: el *plugin* JUnit es necesario para ejecutar pruebas unitarias con dicha herramienta dentro de Jenkins.

A) Verdadero. B) Falso.

4. ¿Qué acción realiza Jenkins con los resultados de pruebas JUnit?

A) Los elimina tras cada ejecución. B) Los muestra gráficamente en reportes. C) Los guarda dentro de un repositorio. D) Los persiste en archivos internos.

5. ¿Qué hace Jenkins si detecta fallos en las pruebas JUnit?

A) Detiene el *pipeline*. B) Elimina los archivos XML. C) Marca el build como unstable. D) Reintenta todas las pruebas.

6. Verdadero o falso: el *plugin* JUnit puede publicar resultados como *checks* en plataformas SCM.

A) Verdadero B) Falso

7. ¿Cómo se utiliza el *plugin* JUnit en un *pipeline*?

A) Con el step *junit*. B) Con el step *runJUnit*. C) Con la acción *buildReport()*. D) Con el comando *mvn junit*.

8. ¿Qué debe ocurrir antes de ejecutar el *step junit* en un *pipeline*?

A) Compilar el código fuente. B) Instalar los *plugins* de SCM. C) Limpiar el *workspace*. D) Generar los archivos XML de resultados.

9. Verdadero o falso: Jenkins necesita configuraciones adicionales complejas para usar el *plugin* JUnit.

A) Verdadero. B) Falso.

10. Verdadero o falso: el *step junit* permite ejecutar pruebas unitarias en JUnit dentro de *pipelines* de Jenkins.

A) Verdadero.

B) Falso.

14.3.3 Comandos de JUnit para recordar



Lectura 41. Principales comandos de JUnit

Nivel básico

Objetivo: el estudiante identifica los comandos clave de JUnit para ejecutar pruebas unitarias.

Descripción

La lectura explica los principales comandos que se utilizan para ejecutar pruebas unitaria utilizando JUnit en solitario, y Junit en integración con Maven.

Normalmente, JUnit se suele descargar e integrar directamente en los IDEs de desarrollo o con herramientas de construcción de código de terceros como Maven, Ant o Gradle, las cuales brindan soporte nativo para utilizar JUnit de manera más sencilla y, en algunas ocasiones, omitiendo su descarga y configuración manual. Aunque también es posible utilizar JUnit de manera independiente, como un *standalone*. Sin embargo, esta opción no es tan común, ya que puede resultar más compleja, además, la mayoría de los IDEs de desarrollo y herramientas de construcción de software tienen soporte para JUnit (Stefan Bechtold et al., s/f).

En el caso particular de los *pipelines* de CI/CD, se busca crear *pipelines* lo más ligeros posible sin descuidar el flujo de trabajo, por lo que utilizar JUnit *standalone* es una opción válida para evitar instalar otras herramientas y dependencias.

De manera general, existen dos instrucciones esenciales para utilizar JUnit *standalone*. La primera: **`javac -d target -cp target:junit-platform-console-standalone-6.0.0.jar <PATH/ArchvioConPruebas.java>`**, tiene el objetivo de compilar los archivos .java que contienen las pruebas; en esta instrucción, el parámetro `-d target` indica el directorio donde guardaran los archivos compilados y `-cp` indica dónde buscar las clases de java necesarias para utilizar JUnit (Stefan Bechtold et al., s/f).

La segunda instrucción: **`java -jar junit-platform-console-standalone-6.0.0.jar execute -class-path . -scan-class-path`**, sirve para ejecutar los archivos de clase de pruebas generados. Este es muy similar al primer comando, pero se debe cambiar el comando

javac por **java**, agregar el comando **execute**, e indicar los archivos de clase que se quieren ejecutar; en este ejemplo particular, los parámetros **-class-path** y **scan-class-path**, indican a JUnit que debe buscar todos los archivos de pruebas en todo el proyecto y ejecutar los que encuentre (Stefan Bechtold et al., s/f).

Por otra parte, y en el caso particular de esta guía, como ya se está utilizando Maven dentro del *pipeline*, es mucho mejor opción aprovechar esta herramienta y ejecutar las pruebas unitarias de JUnit mediante Maven. Los comandos de Maven simplifican el uso de JUnit, pues solo se necesita escribir el comando **mvn test** para que Maven gestione el escaneo de los archivos de pruebas, los compile y los ejecute automáticamente.

El comando **mvn test** ejecuta todos los archivos de prueba en un proyecto. Para compilar y ejecutar archivos o *suits* de prueba específicas, se debe utilizar el parámetro **-Dtest=**, seguido del nombre de los archivos de prueba que se quieren ejecutar, separados por coma: **mvn test -Dtest=FirstUnitTest,SecondUnitTest** (Stefan Bechtold et al., s/f).

Para que Maven pueda utilizar JUnit, es necesario agregar en el archivo pom.xml la herramienta JUnit y el *plugin surefire*, con este *plugin* Maven puede escanear automáticamente el proyecto en busca de archivos o *suits* de pruebas. Stefan Bechtold et al. (s/f) mencionan que el *plugin surefire* únicamente puede identificar los archivos de prueba que tengan una de las siguientes nomenclaturas:

- ****/*Test*.java**.
- ****/*Test.java**.
- ****/*Tests.java**.
- ****/*TestCase.java**.

Al utilizar Maven para gestionar y ejecutar las pruebas unitarias de JUnit, se suele generar un reporte de resultados en la ruta **target/surefire-reports/*.xml**. Este archivo se puede consultar manualmente en el espacio de trabajo del *pipeline*, o mediante el *plugin* JUnit, se puede hacer que Jenkins lo interprete y muestre los resultados directamente en la interfaz gráfica, esto se hace colocando la instrucción **junit 'target/surefire-reports/*.xml'** en el Jenkinsfile, en una sección posterior a la ejecución de las pruebas (CD Foundation, 2025h).



Cuestionario 36. Comandos para usar JUnit

Responde las siguientes preguntas para evaluar la lectura anterior y verificar tu comprensión respecto a los comandos necesarios para utilizar JUnit.

1. Verdadero o falso: es necesario integrar la herramienta JUnit con una IDE o herramienta secundaria para poder utilizarla.
A) Verdadero. B) Falso.
2. ¿Cuál es comando para ejecutar pruebas con JUnit standalone?
A) *run* B) *execute* C) *javac* D) *Dtest*
3. ¿Cuál es el comando para ejecutar pruebas específicas con Maven/Junit?
A) *mvn clean -Dtest* B) *mvn -Dtest* C) *mvn clean test -Dtest* D) Ninguno de los anteriores.
4. ¿En qué se diferencia el comando **mvn test** del comando **mvn test -Dtest** para ejecutar pruebas unitarias con MAVEN/Junit?
A) Ejecuta únicamente pruebas compiladas. B) Ejecuta solamente las pruebas exitosas. C) Ejecuta todas las pruebas en el proyecto D) Todas las anteriores.
5. Verdadero o falso: ***/TestCases.java* es un nombre valido para un archivo de pruebas unitarias identificable por Maven.
A) Verdadero. B) Falso.

14.3.4 Acceso a JUnit desde *pipelines* declarativos en Jenkins



Lectura 42. ¿Cómo utilizar JUnit desde un *pipeline* declarativo en Jenkins?

Nivel Intermedio

Objetivo: el estudiante describe la forma de acceder a la herramienta Junit desde un *pipeline* declarativo de Jenkins.

Descripción

La lectura explica cómo utilizar la herramienta Junit desde los *pipelines* declarativos de Jenkins, así como ejecutar las *suits* de pruebas y generar reportes en formato XML.

En lecturas previas se ha mencionado que es posible utilizar JUnit de manera independiente o a través de una herramienta intermediaria como Maven. Al integrarlo con Jenkins ocurre algo similar, pues es posible interactuar con JUnit mediante *steps* declarados en el Jenkinsfile, que ejecutan las instrucciones de JUnit *standalone*. Pero para ello, es necesario contar con el archivo *.jar* de JUnit, disponible en alguna ubicación del proyecto o en el propio servidor de Linux, a fin de utilizarlo para ejecutar las pruebas. Por otra parte, cuando se emplea Maven con Jenkins, esto ya no es necesario, ya que Maven gestiona automáticamente la instalación y dependencias de JUnit. En este caso, la interacción con JUnit es a través de Maven, utilizando una directiva *steps* y el comando ***mvn test*** (Prashanth Punnam, 2025).

```
stage('Test') {  
  steps {  
    sh 'mvn clean test'  
    echo 'Ejecutando pruebas...'  
  }  
}
```

Figura 190. Código que muestra el uso de JUnit a través de Maven en la definición de un pipeline

Adicionalmente, gracias al *plugin* JUnit, en el Jenkinsfile se puede agregar un bloque de código que permite consultar los resultados de las pruebas y mostrarlos en la interfaz gráfica de Jenkins. Este bloque normalmente se implementa dentro de una sección *post*, junto con la directiva *always*, seguida de un *step junit*, de modo que siempre se muestren los resultados de las pruebas (CodeFresh, s/f).

```

post {
    always {
        junit '**/target/surefire-reports/*.xml'
    }
}

```

Figura 191. Código que muestra el uso de `junit` para guardar los resultados de pruebas unitarias en un pipeline de Jenkins

El `step` básico de JUnit es `junit 'target/test-results/*.xml'`. No obstante, se pueden agregar otros parámetros para controlar su comportamiento en casos especiales. Cuando se quiere utilizar varios parámetros, estos se deben separar mediante comas, siendo obligatorio indicar la ruta del archivo XML con los resultados mediante el parámetro `testResults`. A partir de este, se pueden agregar los demás parámetros según las necesidades del *pipeline* y del proyecto (CodeFresh, s/f).

CD Foundation (2025h) Resalta entre los parámetros más útiles:

Parámetro	Valor por defecto	Función
<code>testResults</code>	N/A (obligatorio)	Ruta de archivos XML con resultados
<code>allowEmptyResults</code>	false	Si se permite no encontrar reportes
<code>keepLongStdio</code>	false	Si mantener o truncar salidas largas
<code>healthScaleFactor</code>	1.0	Peso del resultado de pruebas en el health del job
<code>skipPublishingChecks</code>	false	Evita publicar resultados como "checks" en SCM
<code>testDataPublishers</code>	[]	Extensiones para análisis avanzado

Combinando estos parámetros, se pueden generar *steps* más personalizados que se adhieran a las necesidades específicas de un proyecto. Por ejemplo, un *step* que publique los resultados sin importar como haya terminado el *build*, que no marque error si no se encontró un archivo XML, que mantenga los logs completos sin truncar, y que las pruebas fallidas no afecten tanto el *build health* que Jenkins muestra como porcentaje.


```
post {  
    always {  
        junit testResults: 'target/test-results/**/*.xml',  
              allowEmptyResults: true,  
              keepLongStdio: true,  
              healthScaleFactor: 0.8  
    }  
}
```

Figura 192. Uso de junit en pipeline de Jenkins, de forma personalizada, utilizando múltiples parámetros

Debido a que el *plugin JUnit* modifica la variable de entorno **currentBuild.result** y le asigna valores como: SUCCESS, UNSATABLE O FAILURE, es posible consultar estos estados y dependiendo de los resultados de las pruebas tomar decisiones lógicas dentro del *pipeline*, como enviar notificaciones o detener la ejecución de etapas posteriores (CloudBees, s/f-a).

```
stage('Continuar solo si todas las pruebas pasaron') {  
    when {  
        expression { currentBuild.result != 'UNSTABLE' && currentBuild.result != 'FAILURE' }  
    }  
    steps {  
        echo "Todas las pruebas pasaron, continuando pipeline..."  
    }  
}
```

Figura 193. Ejemplo de código que toma una decisión basado en el resultado del build; en este caso, se imprime un mensaje exitoso cuando el resultado no es fallido ni inestable



Cuestionario 37. Acceso a Junit desde Jenkins

Responde las siguientes preguntas para evaluar la lectura anterior y verificar tu comprensión respecto a cómo utilizar JUnit desde los *pipelines* declarativos de Jenkins.

1. En un *pipeline* declarativo, ¿Qué parámetro del paso JUnit indica el patrón de ubicación de los archivos XML con los resultados de las pruebas?

R = _____

2. ¿Cuál de las siguientes rutas es la más común para los reportes JUnit generados por Maven?

A) target/test-
results/*.xml

B) target/tests-
results/*.xml'

C) resultst/test-
results/*.xml'

D) 'target/*.xml'

3. El parámetro *allowEmptyResults* dentro del paso *junit* se utiliza para:

A) Ignorar los
errores de
compilación.

B) Permitir que el
pipeline continúe
aunque no se
encuentren archivos
de resultados.

C) Mostrar los
resultados de todas
las pruebas
pasadas.

D) Ejecutar las
pruebas en paralelo.

4. Verdadero o falso: el paso *junit* puede colocarse dentro del bloque *post* o *steps* de un *pipeline* declarativo.

A) Verdadero.

B) Falso.

5. Verdadero o falso: si no se encuentra ningún archivo XML con resultados de pruebas, Jenkins detiene el *pipeline* automáticamente y marca el *build* como fallido.

A) Verdadero.

B) Falso.

6. Un equipo de desarrollo que trabaja en una aplicación de banco, tiene inquietudes respecto a los resultados de la pruebas unitarias. Es imprescindible que todas las pruebas pasen sin ningún error, y que no se suban cambios ni actualizaciones en las que han fallado pruebas unitarias. ¿Qué elementos de JUnit puede ayudarles a controlar esto?

R = _____

14.4 Integración de Postman y Newman con Jenkins

14.4.1 Uso de Postman y Newman como herramientas de prueba



Lectura 43. Postman y Newman para la prueba de software.

Nivel básico

Objetivo: el estudiante recuerda los aspectos claves para utilizar PostMan y NewMan.

Descripción

La lectura explica qué son Postman y Newman, así como su objetivo principal y los tipos de pruebas que pueden automatizar. Además, se profundiza en los elementos y requerimientos necesarios para utilizar Postman y Newman como herramientas de prueba de software.

Como se ha descrito en los capítulos previos, JUnit se centra en dar soporte para la realización de pruebas unitarias. Sin embargo, existen diversidad de herramientas destinadas a otros tipos de pruebas, como las funcionales, de carga, rendimiento, de seguridad, entre otras más.

Dado que el proyecto de ejemplo expuesto en esta guía consiste en una aplicación web con un *backend* API REST construido en Spring Boot, resulta conveniente emplear una herramienta para realizar pruebas de integración y funcionales a la API. Una de las herramientas más utilizadas es Postman, una plataforma integral que permite gestionar todo el ciclo de vida desarrollo de un API, abarcando desde su diseño, construcción y prueba, hasta su documentación, despliegue y monitoreo (Postman, s/f).

Se puede entender a Postman como un plataforma de soporte para desarrollar APIs, la cual cuenta con varias herramientas internas especializadas en alguna etapa del desarrollo de APIs. Una de las más útiles es Newman, una herramienta de línea de comandos enfocada exclusivamente en ejecutar colecciones de prueba de Postman. De manera práctica, un desarrollador puede utilizar Postman para crear pruebas funcionales, de integración, de regresión, entre otras, y utilizar Newman para ejecutarlas (Postman, s/f-b).

Aunque Postman dentro de sus capacidades permite ejecutar pruebas, esto solo se puede realizar manualmente. Por otro lado, Newman fue creado con la intención de automatizar la ejecución de colecciones de pruebas de Postman en entornos de integración y despliegue continuo. En otras palabra, Postman es una plataforma más general para el desarrollo de APIs, mientras que Newman se enfoca en su totalidad en la automatización de pruebas de APIs y provee de una mayor facilidad para integrarse con entornos de CI/CD, como los *pipelines* de Jenkins (Apidog, s/f).

Postman se puede utilizar de tres formas: mediante su interfaz de línea de comandos, desde la aplicación web, o descargando la aplicación de escritorio. Por otra parte, Newman solo está disponible por línea de comandos. Aunque ambas herramientas se pueden utilizar de manera independientemente (no es necesario instalar Postman para ejecutar pruebas con Newman), Newman únicamente puede ejecutar colecciones de pruebas de Postman, por lo que es necesario al menos utilizar la versión web de Postman para generar dichas colecciones (Postman, s/f-a).

Cabe destacar que Newman está desarrollado sobre Node.js, por lo que para instalarlo y utilizarlo se necesita contar con una instalación de Node.js, así como con una colección de pruebas de Postman, ya sea en formato JSON o mediante una URL de referencia (Postman, s/f-a).



Lectura 44. Principales comandos para utilizar Newman.

Nivel básico

Objetivo: el estudiante identifica los comandos clave de Newman para ejecutar pruebas en Jenkins.

Descripción

La lectura explica los comandos que se deben utilizar para ejecutar colecciones de pruebas de Postman dentro de *pipelines* declarativos de Jenkins.

Como se menciona en el capítulo previo, Newman es una herramienta de línea de comandos, sin interfaz, gráfica diseñada para integrarse en *pipelines* de CI/CD. Debido a esto, su instalación es bastante simple, pues solo se necesita ejecutar el comando ***npm install -g Newman*** dentro del *pipeline* de Jenkins y esta estará disponible para usarse a partir de ese momento (Postman, s/f-a).

Por otro lado, Newman únicamente cuenta con dos versiones de un mismo comando: Para ejecutar pruebas, uno de estos es ***newman run <collection-json-file>***, que sirve para ejecutar una colección de pruebas en formato JSON, mientras que el comando ***newman run <collection-url>***, sirve para ejecutar las pruebas a partir de una URL en la que se encuentra la colección de pruebas (Postman, s/f-a).

Adicionalmente, se pueden agregar varios parámetros o argumentos para modificar y personalizar el comportamiento de las pruebas. Uno de los argumentos que se utilizan recurrentemente son las variables de entorno, las cuales se deben proporcionar mediante el parámetro ***-e***, seguido de una archivo JSON o la dirección URL que contenga dichas variables de entorno: ***newman run <collection> -e dev-environment.json*** (Postman, s/f-b).

En la siguiente Tabla, se describen los principales argumentos que se pueden utilizar con Newman, y que pueden resultar útiles al usarlos en Jenkins:

Parámetro	Función
<i>-v, --version</i>	Muestra el número de versión de Newman.
<i>-g, --globals [file URL]</i>	Especifica un archivo JSON o URL de las variables globales.
<i>-n, --iteration-count [number]</i>	Especifica el número de veces que se van a ejecutar las pruebas, útil para realizar iteraciones con distintos datos (Requiere de archivo de datos).

<code>-d, --iteration-data [file]</code>	Especifica un archivo de datos JSON o CSV a usar para cada iteración de ejecución de pruebas.
<code>--working-dir [path]</code>	Indicar en qué folder se desea trabajar, por defecto esta seleccionado el folder actual.
<code>--timeout [number]</code>	Establece un límite de tiempo (en milisegundos) para terminar de ejecutar la colección de pruebas.
<code>-r [reporter-name], --reporters [reporter-name]</code>	Genera un reporte de la ejecución de la colección de pruebas en formato: cli, json, junit, progress, y emoji train.
<code>--disable-unicode</code>	Desactiva los caracteres Unicode especiales que aparecen al ejecutar las pruebas (Permite una mayor compatibilidad con distintos entornos.)
<code>--bail [optionalModifiers]</code>	Detiene la ejecución si una prueba falla.

Aunque estos son los argumentos que consideramos más útiles, y que se suelen usar recurrentemente al ejecutar pruebas con Newman, existen más argumentos con otros objetivos más especializados. Si te interesa, revisa la página de [Newman Flags](#).



Cuestionario 38. Newman como herramienta para prueba de software

Responde las siguientes preguntas para evaluar las dos lecturas previas y así verificar tu comprensión respecto la herramienta de prueba Newman y los aspectos importantes para considerar al utilizarla como herramienta de prueba de software.

1. ¿Cuál es el objetivo principal de la herramienta Postman?

- | | | | |
|--|---------------------------------------|---|-------------------------------|
| A) Gestionar y automatizar la ejecución de | B) Automatizar el desarrollo de APIs. | C) Gestionar el ciclo de vida desarrollo de un API. | D) Ninguna de las anteriores. |
|--|---------------------------------------|---|-------------------------------|

pruebas de
software.

2. ¿Cuál es la diferencia principal entre Postman y Newman?

- | | | | |
|--|---|--|---|
| <p>A) Postman cuenta con más comandos y herramientas para ejecutar pruebas, Newman únicamente tiene una pocas funciones.</p> | <p>B) Postman es una plataforma general para desarrollar APIs, Newman se enfoca en automatizar las pruebas.</p> | <p>C) Postman es más sencillo de usar por su interfaz gráfica, Newman es más complicado porque solo tiene línea de comandos.</p> | <p>D) No hay diferencia alguna, son la misma herramienta.</p> |
|--|---|--|---|

3. De las siguientes opciones, ¿Qué tipo de pruebas de software se pueden realizar con Newman?

- | | | | |
|--------------------|-----------------|------------------|--------------------------|
| A) De integración. | B) Funcionales. | C) De regresión. | D) Todas las anteriores. |
|--------------------|-----------------|------------------|--------------------------|

4. ¿Cuál es el comando de Newman que sirve para ejecutar una colección de pruebas?

- | | | | |
|--------------------------|------------------------|---------------|------------------------|
| A) npm install -g newman | B) newman npm run test | C) newman run | D) newman test execute |
|--------------------------|------------------------|---------------|------------------------|

5. ¿Qué formatos admite Newman para ejecutar una colección de pruebas?

- | | | | |
|---------------|---------------|--------------|---------------|
| A) XML y JSON | B) JSON y URL | C) XML y URL | D) URL Y XAML |
|---------------|---------------|--------------|---------------|

6. ¿Cuál es el argumento de Newman que sirve para declarar variables de entorno al ejecutar una colección de pruebas?

- | | | | |
|-------|-------|-------|---------|
| A) -e | B) -r | C) -t | D) -env |
|-------|-------|-------|---------|

7. Si se requiere generar un reporte de pruebas, ¿Qué parámetro de Newman se debe usar?
- A) -reporters B) -rep C) -t D) -e
8. Verdadero o falso: para utilizar Newman es necesario contar con una instalación de Java activa.
- A) Verdadero. B) Falso.
9. Verdadero o falso: para utilizar Newman es necesario contar con una instalación de NodeJS.
- A) Verdadero. B) Falso.
10. Verdadero o falso: es posible utilizar Newman desde línea de comandos o desde la interfaz gráfica.
- A) Verdadero. B) Falso.
11. Verdadero o falso: Newman únicamente puede interpretar y ejecutar colecciones de pruebas de Postman.
- A) Verdadero. B) Falso.

14.4.2 Newman en los *pipelines* declarativos de Jenkins



Lectura 45. *Plugins* de Jenkins para utilizar Newman

Nivel básico

Objetivo: el estudiante recuerda los *plugins* y configuraciones necesarias para utilizar Newman en Jenkins.

Descripción

La lectura explica los *plugins* necesarios para integrar la herramienta Newman dentro de los *pipelines* declarativos de Jenkins, así como las configuraciones necesarias para que los *plugins* funcionen correctamente.

A diferencia de otras herramientas de prueba, Newman no cuenta con un *plugin* concreto para integrarse con Jenkins, en cambio, este utiliza la herramienta y el *plugin* de **NodeJS** para gestionar su instalación. Mediante este *plugin* es posible establecer una instalación de NodeJS dentro del servidor de Jenkins y configurar los paquetes npm globales necesarios para que se instale Newman automáticamente (Postman, s/f-b).

Como todas las herramientas que se han integrado con Jenkins, NodeJS también requiere ciertas configuraciones para su instalación. Se debe seleccionar una versión de NodeJS, así como declarar explícitamente las dependencias npm que se quieren instalar. De manera general, este *plugin* permite ejecutar scripts de NodeJS; en este caso concreto, permite utilizar la herramienta Newman a través de los comandos descritos previamente (CD Foundation, 2025w).

Como se mencionó en la lectura previa, es posible generar reportes de pruebas con Newman. Estos se pueden crear en formato: HTML JSON, XML, como barra de progreso, o con emojis gráficos. Para que Jenkins pueda interpretar los reportes generados con Newman en formato HTML, es necesario instalar el *plugin* **HTML Publisher** y especificar datos relacionados a la ubicación del archivo de reportes (Harish Rajora, 2021).

De manera resumida, para utilizar Newman con Jenkins se requieren de tres cosas: instalar los *plugins* NodeJS y HTML Publisher, configurar el *plugin* NodeJS para que descargue los paquetes y dependencias de Newman automáticamente, y una colección de pruebas de Postman para ser ejecutadas mediante Newman.



Lectura 46. Utilizando Newman desde los *pipelines* de Jenkins.

Nivel intermedio

Objetivo: el estudiante comprende como acceder a la herramienta Newman desde un *pipeline* declarativo.

Descripción

La lectura explica qué acciones son necesarias para utilizar e interactuar con la herramienta Newman desde un *pipeline* declarativo en Jenkins, cómo ejecutar una colección de pruebas y generar reportes de ejecución.

En las últimas lecturas se ha explicado qué es Newman, cuál es su utilidad en la ejecución de pruebas de software, como se utiliza y qué se necesita para integrarlo con Jenkins. En esta lectura se profundiza en cómo utilizar Newman dentro de los *pipelines* declarativos de Jenkins, los *steps* y comandos concretos que son necesarios para ejecutar pruebas y generar reportes automáticamente.

Recapitulando lo que se explicó en lecturas previas, para utilizar Newman es necesario contar con una colección de pruebas de Postman, ya sea en formato JSON o directamente la URL de la colección. Además, es necesario instalar los *plugins* NodeJS y HTML Publisher. Finalmente, se debe configurar la instalación de NodeJS en la sección *Tools* e indicar los paquetes npm que se quieren descargar, entre ellos los de Newman.

Cuando se configura la instalación de NodeJS, normalmente se suelen configurar los paquetes npm que se van a utilizar, aunque también es posible instalar las herramientas en tiempo de ejecución del *pipeline*. Para instalar Newman en tiempo de ejecución, se requiere de un bloque *stage* acompañado de la directiva *steps*, con las instrucciones ***sh 'npm install -g Newman'*** y ***sh 'npm install -g newman-reporter-htmlextra'***, para descargar las dependencias necesarias (Abubakkar Sithik, 2025).

```
stage('Install Newman') {  
  steps {  
    sh 'npm install -g newman'  
    sh 'npm install -g newman-reporter-htmlextra'  
  }  
}
```

Figura 194. Código para instalar Newman en tiempo de ejecución del pipeline

Para ejecutar las pruebas utilizando Newman, se requiere de otro bloque *stage*, igualmente acompañado de una directiva *steps*, y un *step* de tipo *sh* con el comando ***newman run <colección-postman>***. Adicionalmente, recomendamos utilizar los argumentos ***--disable-unicode*** y ***--color off*** para desactivar los caracteres Unicode

(como ✓, ✗, →) y tabular los resultados. Dentro del mismo *step*, se deben ingresar los comandos `--r htmlextra` y `--reporter-htmlextra-export <ruta-exportacion>`, para crear y exportar los reportes de resultados en formato HTML (Harish Rajora, 2021).

```
stage('Integration tests') {
  steps {
    sh '''
      newman run Jenkins-Guide_To-Do-List/to_do_app_api_tests.postman_collection.json \
        --disable-unicode \
        --color off \
        -r htmlextra \
        --reporter-htmlextra-export Jenkins-Guide_To-Do-List/target/integration-tests-report.html
    '''
  }
}
```

Figura 195. Ejemplo de uso de Newman en pipeline de Jenkins

Finalmente, se puede usar otro bloque *stage* o *post* para que Jenkins muestre el reporte HTML directamente en la interfaz web dentro del panel del *pipeline*. Recomendamos utilizar el bloque *post* para que esta acción se realice hasta el final de la ejecución del *pipeline*. Dentro de una directiva *always*, se debe agregar un *step* de tipo **publishHTML**, indicando el nombre de la carpeta donde se encuentra el reporte, el nombre del archivo HTML, y el nombre que se le va a mostrar al cargar el reporte en la interfaz gráfica de Jenkins.

```
post {
  always {
    publishHTML(target: [
      reportDir: 'newman',
      reportFiles: 'html-report.html',
      reportName: 'Newman HTML Report'
    ])
  }
}
```

Figura 196. Ejemplo de uso del plugin HTML Publisher en pipeline de Jenkins

Con este conjunto de directivas e instrucciones, puedes utilizar Newman para ejecutar colecciones de pruebas de Postman y generar reportes de los resultados de manera automática. En la siguiente práctica llevaremos a un *pipeline* toda la teoría antes explicada, actualizarás tu *pipeline* para integrar JUnit y Newman para realizar pruebas unitarias y de integración como parte de tu proceso de CI/CD.



Cuestionario 39. Integrando Newman con Jenkins

Responde las siguientes preguntas para evaluar las dos lecturas previas y verificar tu comprensión respecto a integrar la herramienta de pruebas Newman con los *pipelines* declarativos de Jenkins.

1. ¿Cuál es el *plugin* indispensable para utilizar Newman en Jenkins?

- A) Newman *plugin* B) NodeJS *plugin* C) HTML Publisher *plugin* D) Todos los anteriores

2. Para instalar NodeJS y Newman directamente en el espacio de trabajo de un *pipeline* de Jenkins, ¿Qué se debe configurar dentro del servidor de Jenkins?

- | | | | |
|---|---|---|--|
| A) Una instalación de NodeJS con los paquetes npm necesarios. | B) Una instalación de Newman con las dependencias necesarias. | C) Un nodo secundario que contenga la herramienta Newman. | D) Un <i>job</i> de tipo <i>pipeline</i> que contenga una instalación de Newman. |
|---|---|---|--|

3. Verdadero o falso: el *plugin* Newman Integration sirve para integrar las funcionalidades básicas de Newman con Jenkins.

- A) Verdadero. B) Falso.

4. Verdadero o falso: el *plugin* HTML Publisher sirve para mostrar los reportes de pruebas en la interfaz gráfica de Jenkins

- A) Verdadero. B) Falso.

5. Verdadero o falso: solo es posible utilizar Newman si se instala localmente en el servidor donde se aloja Jenkins.

A) Verdadero. B) Falso.

6. ¿Qué tipo de bloque declarativo de Jenkins se puede usar para encapsular las instrucciones para ejecutar pruebas con Newman?

A) *pipeline* B) *stage* C) *post* D) *test*

7. ¿Qué tipo de directiva se usa dentro de los bloques *stage* para declarar las instrucciones que se van a ejecutar con Newman?

A) *agent* B) *step* C) *steps* D) *stages*

8. ¿Qué tipo de *step* se debe usar para ejecutar las instrucciones y comando de Newman?

A) *batch* B) *sh* C) *with-newman* D) *with-npm*

9. ¿Qué tipo de *step* se debe usar para mostrar el reporte de pruebas en la interfaz gráfica de Jenkins?

A) *publih-html* B) *sh* C) *batch* D) *publishHTML*

10. Enumera los tres prerequisites que se deben cumplir para utilizar Newman en Jenkins.

R = _____

11. Un grupo de ingenieros de software están desarrollando un *pipeline* de Jenkins, pero al ejecutarlo falla. Los logs muestran el mensaje "Newman no se reconoce como un comando". ¿Qué posible causa podría provocar este error?

R = _____

12. Con base en el problema antes planteado, ¿Qué hace falta para resolver el problema?

R = _____

13. El mismo equipo de desarrollo tiene un archivo JSON con la colección de pruebas, pero debido a que han integrado más equipos y personas, han decidido centralizar la gestión de colecciones en Postman, accediendo a ellas con sus URLs. ¿Cómo cambia el comando *newman run* para adaptarse a este escenario?

R = _____

14.5 Integración de herramientas de prueba en *pipelines* declarativos paso a paso



Práctica 4. Integración de herramientas de prueba en *pipelines* declarativos paso a paso

Avanzado

Objetivo: el estudiante integra JUnit y Newman con *pipelines* declarativos en Jenkins para la ejecución automatizada de pruebas de software unitarias y de integración en proyectos de Java.

Prerrequisitos: servidor de Jenkins instalado y funcionando correctamente. Cuenta activa en GitHub, editor de texto (idealmente Visual Studio Code). Java instalado en el entorno local. Repositorio de GitHub utilizado en la práctica.

Descripción: la siguiente práctica guía paso a paso en la extensión de un *pipeline* declarativo que realiza el *build* de un proyecto en Java, para automatizar la prueba de software unitaria (JUnit) y de integración (Newman). Explica paso a paso cómo descargar el proyecto base en Java, cómo instalar las herramientas necesarias para realizar las pruebas de integración, los *plugins* de Jenkins necesarios para ejecutar las pruebas unitarias y la automatización del proceso de prueba a través de la definición del *pipeline* declarativo. Finalmente explica cómo persistir los resultados de las pruebas y comunicar problemas en caso de tener resultados de prueba insatisfactorios.

Para este momento, confiamos en que el lector ya tiene un conocimiento general sobre Jenkins y los conceptos fundamentales detrás de la creación de *pipelines*, como el de *job*, *pipeline* de despliegue o Jenkinsfile. En la práctica anterior, el lector pudo extender el *pipeline* que se ha venido desarrollando a lo largo de la guía, automatizando la compilación (o *build*) del proyecto.

Ahora, el lector enfrentará un nuevo reto: integrar pruebas automatizadas dentro del *pipeline* de Jenkins que hemos venido desarrollando hasta ahora. Con base en los fundamentos sobre pruebas automatizadas que se han venido abordando a lo largo de este capítulo, agregaremos pruebas unitarias y de integración (automatizadas) dentro del *pipeline* de Jenkins para probar el proyecto que se presentó en la práctica previa.

14.5.1 Prerrequisitos

Como primer prerrequisito, motivamos encarecidamente al lector a desarrollar las prácticas previas a esta. Partiremos con el proyecto y el *pipeline* desde donde lo dejamos en la práctica anterior y entender bien el contexto en el que se desarrollan ambos será de gran ayuda para seguir esta práctica más fácilmente. Sin embargo, explicaremos con detalle el paso a paso de esta práctica y referiremos a capítulos previos en caso de ser necesario.

Se espera que, para este punto, el lector cuente con un servidor de Jenkins funcional, con los *plugins* Pipeline, Git, GitHub, Maven Integration, Pipeline Maven Integration y Pipeline: SCM Step debidamente instalados y configurados; si ha seguido la guía capítulo a capítulo, estos *plugins* ya estarán instalados dentro de su servidor.

Además, esperamos que el lector disponga de una cuenta de GitHub activa, con las credenciales de esta registradas en Jenkins. Si no es así, revise la sección [Configurar Credenciales de acceso a GitHub](#) para más información al respecto.

Tal como en las prácticas previas, estaremos trabajando sobre la definición del *pipeline* y el proyecto ubicados en el [repositorio público de esta guía](#). Sin embargo, en esta ocasión se utilizará la rama “*mvn-build-pipeline*” como punto de partida para el desarrollo de esta práctica. Por ello, se espera que el lector ya tenga un *fork* del proyecto, listo para trabajar localmente sobre la rama ya mencionada.

El objetivo de esta práctica es que el lector, a partir de la definición de un *pipeline* declarativo que ya realiza el *build* del proyecto (en Java con Spring Boot), comprenda cómo implementar pruebas unitarias con JUnit y de integración utilizando Newman,

para posteriormente automatizar su ejecución desde el *pipeline* de Jenkins, extendiendo así la funcionalidad de este último.

14.5.2 Herramientas para ejecutar el proyecto

Para poder ejecutar el proyecto, es necesario que la máquina donde se encuentra trabajando el lector cuente con una instalación de Java Temurin JDK 21 y de Maven 3.8 o mayor. Si requiere ayuda para instalar estas herramientas en máquinas Windows, Linux y MacOS, puedes consultar los siguientes recursos: [Tutorial: Instalar Java](#) y [Tutorial: Instalar Maven](#). Al finalizar las instalaciones puede verificar que estas se hayan realizado correctamente abriendo una terminal y escribiendo los comandos ***java --version*** y ***mvn --version***.

Además, recomendamos instalar el editor de código Visual Studio Code o cualquier otro editor de código o IDE de desarrollo de su preferencia con el que pueda abrir el proyecto y ejecutarlo. En caso de usar Visual Studio Code, simplemente debe abrir una terminal y escribir el comando ***mvn spring-boot:run*** para ejecutar la aplicación en el puerto 8081. Con esto podrá verificar que el proyecto funciona correctamente antes de automatizar las pruebas mediante Jenkins.

14.5.3 Agregando las pruebas unitarias al proyecto

Como punto de partida y como ya fue mencionado previamente, asumiremos que el lector ya ha realizado un *fork* del proyecto y cuenta con una versión local de la rama [mvn-build-pipeline](#).

Partiendo de este supuesto, seguiremos los siguientes pasos para agregar pruebas unitarias al proyecto:

Etapas 1: modificar la implementación del servicio encargado de hacer el CRUD de las tareas (TO-DOs). Esto, dado que para este punto del proyecto Java, el CRUD se realiza únicamente con un archivo local; para desacoplar este comportamiento y poderlo probar unitariamente de forma más simple, realizaremos algunas modificaciones sobre el archivo.

1. Ubica el archivo *TodoService.java* dentro del proyecto Java, que se encuentra en la carpeta de servicios.

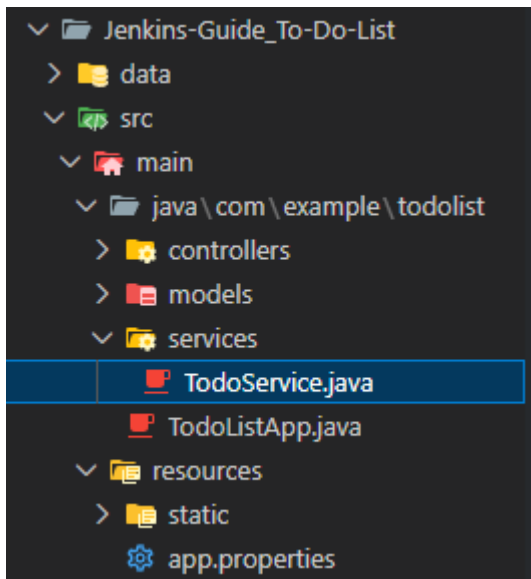


Figura 197. Captura de pantalla de la ubicación del servicio *TodoService.java* dentro del proyecto Java

2. Declara un atributo privado de tipo *String* llamado *filePath* y elimina el atributo *FILE_PATH*.

```
1 private String filePath;
```

Figura 198. Código con la declaración del atributo privado *filePath*

3. Sustituye las referencias a *FILE_PATH* en las líneas 20 y 45 del archivo original por la referencia al atributo *filePath*.

```
1 File file = new File(this.filePath); // Line 20
2
3 mapper.writerWithDefaultPrettyPrinter()
4     .writeValue(new File(this.filePath), todos); // Line 45
```

Figura 199. Nuevas líneas para sustituir las antiguas líneas 20 y 45 del archivo de servicios

4. Declara dos constructores: el primero no toma argumentos y asigna el valor "data/to_do.json" al atributo previamente creado; el segundo toma un argumento de tipo *String* y lo asigna al atributo previamente creado.

```
1 public TodoService() {  
2     this.filePath = "data/to_do.json";  
3 }  
4  
5 public TodoService(String filePath) {  
6     this.filePath = filePath;  
7 }  
8
```

Figura 200. Código de los nuevos constructores para la clase del servicio

5. Guarda los cambios en el archivo.

Con estos simples cambios sobre la clase del servicio, desacoplamos el archivo destino de la lógica del CRUD y, con ello, facilitamos la implementación de las pruebas unitarias. Es decir, preparamos el terreno para implementarlas.

Puedes ver un [ejemplo del archivo final en el repositorio de GitHub](#).

Etapla 2: habilitar el proyecto de Spring Boot para agregar las pruebas unitarias.

Una vez modificada la clase que probaremos, debemos indicarle a Spring Boot que implementaremos pruebas en JUnit. Para ello, sigue los siguientes pasos:

1. Ubica el archivo *pom.xml* en la raíz del proyecto Java.

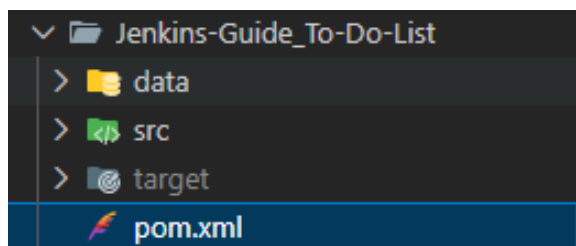


Figura 201. Captura de pantalla donde se muestra el archivo *pom.xml* dentro del proyecto de Spring Boot

2. Agrega la siguiente dependencia dentro del archivo, entre la última etiqueta de `</dependency>` y la etiqueta de cierre `</dependencies>`.

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-test</artifactId>
4   <scope>test</scope>
5 </dependency>
```

Figura 202. Código para agregar como dependencia dentro del archivo pom.xml

La línea 2 indica que la librería viene del proyecto Spring Boot, la 3 especifica el nombre de la librería (*starter* de Spring Boot) y la 4 indica que solo se usa al ejecutar pruebas.

3. Guarda los cambios en el archivo.

Con esto, habremos configurado el proyecto Java para implementar las pruebas unitarias con toda la seguridad.

Etapas 3: implementar las pruebas unitarias.

Una vez que ya tenemos preparado todo para las pruebas, solo queda agregarlas al proyecto. Puedes guiarte del [archivo de pruebas](#) o de los siguientes pasos para realizarlo:

1. Crea la clase `TodoServiceTest.java` dentro de la carpeta `src\test\java\com\example\todolist\services`.

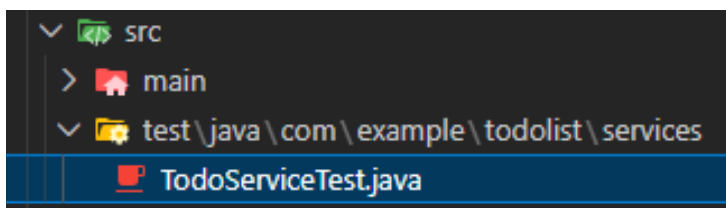


Figura 203. Archivo `TodoServiceTest.java` dentro de la estructura de carpetas del proyecto

2. Agrega el siguiente código al archivo. Este código también lo puedes consultar en el [archivo de pruebas](#), donde se incluyen las referencias a los paquetes necesarios.

```
15 class TodoServiceTest {
16
17     private TodoService todoService;
18
19     @TempDir
20     Path tempDir;
21
22     private File testFile;
23
24     @BeforeEach
25     void setUp() {
26         testFile = tempDir.resolve("to_do.json").toFile();
27
28         todoService = new TodoService(testFile.getPath());
29     }
30
31     @Test
32     void testGetAllWhenFileDoesNotExist() {
33         List<TodoModel> result = todoService.getAll();
34         assertTrue(result.isEmpty());
35     }
36
37     @Test
38     void testAddCreatesNewTodo() {
39         TodoModel todo = new TodoModel();
40         todo.setTitle("Estudiar Spring");
41         todo.setCompleted(false);
42
43         TodoModel saved = todoService.add(todo);
44
45         assertEquals(1, saved.getId());
46
47         List<TodoModel> todos = todoService.getAll();
48         assertEquals(1, todos.size());
49         assertEquals("Estudiar Spring", todos.get(0).getTitle());
50     }
51
52     @Test
53     void testDeleteRemovesTodo() {
54         todoService.add(new TodoModel(1, "Tarea 1", false));
55         todoService.add(new TodoModel(2, "Tarea 2", false));
56
57         todoService.delete(1);
58
59         List<TodoModel> todos = todoService.getAll();
60         assertEquals(1, todos.size());
61         assertEquals(2, todos.get(0).getId());
62     }
63 }
```

Figura 204. Código del archivo de pruebas, que incluye todas las pruebas unitarias sobre el servicio de la aplicación

En las líneas 17 a 22, se declaran los atributos principales de las pruebas: el servicio que se va a probar, el directorio temporal (*mock*) anotado con

`@TempDir` y el archivo que se usará como almacenamiento de prueba, estableciendo el contexto controlado en el que se ejecutarán las pruebas.

En las líneas 25 a 29, el método `setUp`, anotado con `@BeforeEach`, prepara el entorno antes de cada prueba, creando el archivo temporal e inicializando el servicio, lo que asegura independencia entre pruebas. A partir de ahí, las líneas 32 a 35 validan el comportamiento del servicio cuando no existen datos, las líneas 38 a 50 prueban la creación y persistencia de una nueva tarea, y finalmente las líneas 53 a 62 verifican que la operación de eliminación funcione correctamente.

En conjunto, esta clase de pruebas expone pruebas unitarias enfocadas en validar comportamientos clave del servicio de forma aislada y reproducible.

3. Guarda los cambios en el archivo.

Aunque las pruebas unitarias mostradas no resultan muy complejas o extensas, sirven como punto de partida general para comprender cómo preparar estas dentro de proyectos Java, con el fin de automatizar su ejecución en un *pipeline* de Jenkins. Sin embargo, antes de llegar ahí, pasemos a agregar las pruebas de integración dentro del proyecto.

14.5.4 Agregando las pruebas de integración al proyecto

Este paso es más fácil que el anterior, pues el lector no debe preocuparse de crear las pruebas desde cero. Solamente sigue los siguientes pasos:

1. Descarga la [colección de postman](#) con las pruebas de integración para el proyecto Java.
2. Copia el archivo descargado en la carpeta raíz del proyecto Java.

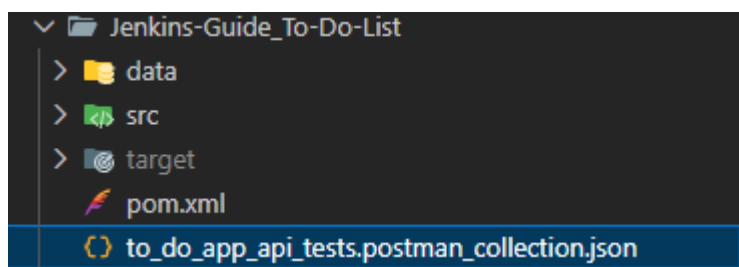


Figura 205. Ejemplo de ubicación final para la colección Postman dentro del proyecto Java

La colección de Postman que acabas de descargar, valida el flujo básico de la API del proyecto Java, cubriendo las operaciones principales de consulta, creación y eliminación.

Si consultas el archivo, verás lo siguiente:

1. Primero, la prueba “Get TO-DOs list” verifica que el endpoint **GET /todos** responda correctamente (código 200), que el resultado sea una lista, y que cada elemento tenga los campos esperados (*id*, *title*, *completed*), contemplando también el caso de una lista vacía. Esto asegura que la API expone correctamente los datos y mantiene un contrato estable de respuesta.
2. Luego, la prueba “Create TO-DO entry” valida la creación de una nueva tarea mediante **POST /todos**, comprobando que la respuesta sea exitosa, que el objeto creado tenga un identificador y que los datos devueltos coincidan con los enviados en la solicitud. El identificador generado se guarda como variable de colección para reutilizarse más adelante.
3. Finalmente, la prueba “Delete TO-DO entry” elimina la tarea creada con **DELETE /todos/{id}** y confirma, mediante una consulta posterior, que el elemento ya no existe en la lista.

En este sentido, se espera que el lector entienda el objetivo de las pruebas de integración y no los detalles de su implementación desde cero, como lo son crear la colección en Postman, configurar los detalles detrás, comprender a detalle el JSON de la colección, entre otros.

Lo único que el lector debe asegurar es que el archivo con las pruebas sea idéntico al proporcionado y se encuentre en la ruta indicada dentro de su proyecto local, listo para ejecutarse con Newman en pasos posteriores para obtener los resultados de prueba.

14.5.5 Configuración de *plugins* necesarios en Jenkins

Para esta práctica, debemos tener tres *plugins* en radar: JUnit, NodeJS y HTML Publisher. Puedes conocer más detalles sobre ellos en el capítulo [Configuración de *plugins* necesarios en Jenkins](#). También puedes conocer detalles sobre la gestión e instalación de *plugins* en el capítulo dedicado a ellos: [Plugins](#).

Como primer paso, debes asegurarte de que estos tres *plugins* previamente mencionados se encuentren instalados en tu servidor de Jenkins.

Fuera de ello, solamente debes asegurarte de que el *plugin* de NodeJS está correctamente configurado para acceder a los paquetes de Newman asociados. Para ello:

1. Desde el apartado de configuración de Jenkins, abre la sección de *tools*.
2. Si tienes correctamente instalado el *plugin* de NodeJS, ahí encontrarás una sección relacionada llamada *NodeJS Installations*. Haz clic en el botón *Add NodeJS* de esa sección.
3. El paso anterior abrirá un formulario para registrar una nueva instalación de NodeJS. Como primer paso, introduce en el campo *Name* la versión LTS de NodeJS; al momento en el que se escribe la guía, esta es “v24.12.0”, pero puede cambiar.
4. Marca la casilla *Install automatically*.
5. Selecciona la misma versión indicada en el nombre para el campo *Version*.
6. Ingresa el valor “*newman newman-reporter-htmlextra*” en el campo *Global npm packages to install*. Esto, para que más adelante podamos utilizarlos durante la ejecución del *pipeline*.
7. En el campo *Global npm packages refresh hours* deja el valor por defecto (72).
8. Finalmente, verifica los valores ingresados.

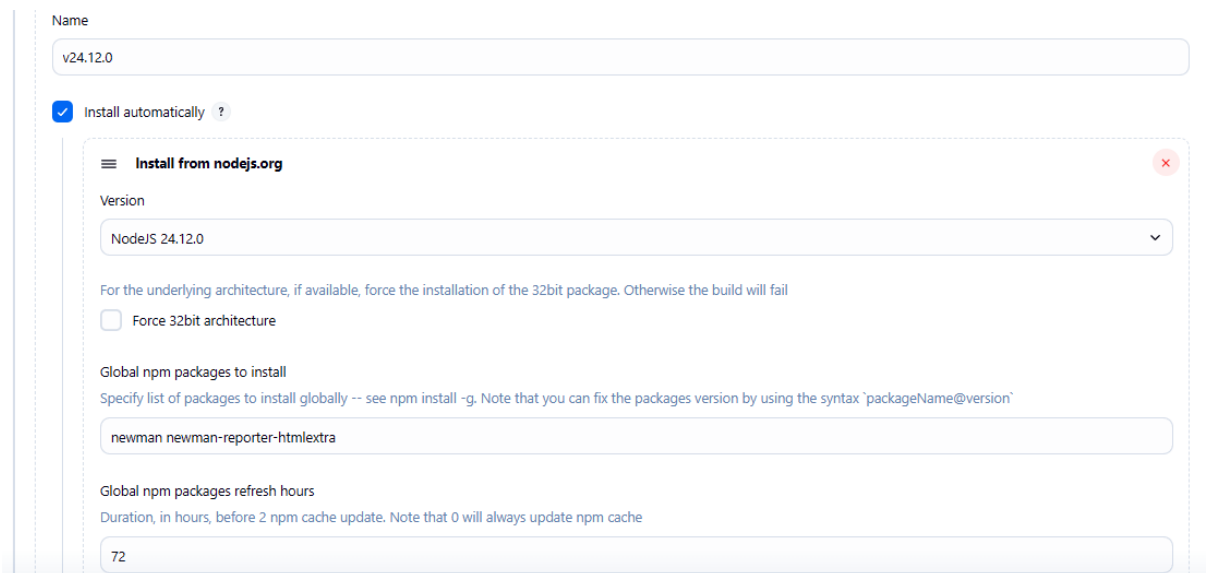
The image shows a screenshot of the Jenkins configuration page for adding a new NodeJS installation. The form is titled "Name" and has a text input field containing "v24.12.0". Below this is a checkbox labeled "Install automatically" which is checked. A dashed box contains a sub-form titled "Install from nodejs.org". Inside this sub-form, there is a "Version" dropdown menu set to "NodeJS 24.12.0". Below the dropdown is a checkbox for "Force 32bit architecture" which is unchecked. Further down is a text input field for "Global npm packages to install" containing the text "newman newman-reporter-htmlextra". At the bottom of the sub-form is another text input field for "Global npm packages refresh hours" set to "72".

Figura 206. Configuración del plugin NodeJS para establecer una instalación con lo necesario para utilizar Newman

Con esta configuración y asegurando que los otros dos *plugins* están instalados en su servidor de Jenkins, está preparado para pasar de lleno a los cambios en el *pipeline* que permitirán automatizar la ejecución de las pruebas.

14.5.6 Modificación de la declaración del *pipeline*

Entremos de lleno a los cambios sobre el *Jenkinsfile* sobre la rama que ya veníamos trabajando en esta práctica:

1. Si no lo has hecho hasta ahora, sube los cambios realizados sobre el proyecto hacia tu repositorio de GitHub.
2. Agrega la herramienta *nodejs* a la directiva *tools* del *pipeline*. Esto permitirá llamar comandos de paquetes asociados (como Newman) desde el *pipeline*. Asegúrate que el valor entre comillas de la herramienta coincida con el nombre dado en la configuración del *plugin* de Node.js.

```
tools {  
    maven 'Maven3'  
    nodejs 'v24.12.0'  
}
```

Figura 207. Inclusión de herramienta *nodejs* en la directiva *tools* del *pipeline*

Aquí también es importante asegurarse de que la herramienta *maven* está correctamente configurada con el nombre adecuado. Aunque esto se realizó en la práctica previa, puedes hacerlo desde la misma pantalla donde configuraste el *plugin* de Node.js.

3. Elimina el *stage* vacío de “Test” y cree dos nuevos: uno para “Unit tests” y otro para “Integration tests”.
4. Agrega el siguiente comando en el *stage* de pruebas unitarias:

```
stage('Unit tests') {  
    steps {  
        sh 'mvn -f Jenkins-Guide-To-Do-List/pom.xml clean test'  
    }  
}
```

Figura 208. Código del *stage* para las pruebas unitarias en el *pipeline* de Jenkins

Este *step* llama a la ejecución de pruebas unitarias con Maven. Muy similar al *step* que se realiza durante el *build* del proyecto.

5. Agrega el siguiente comando en el *stage* de pruebas de integración:


```
stage('Integration tests') {
    steps {
        sh '''
            set -e

            echo "Starting Spring Boot app..."
            mvn -f Jenkins-Guide_To-Do-List/pom.xml spring-boot:run &
            APP_PID=$!

            trap "echo Stopping Spring Boot...; kill $APP_PID || true" EXIT

            echo "Waiting for app to start..."
            sleep 15

            echo "Running Postman integration tests..."
            newman run Jenkins-Guide_To-Do-List/to_do_app_api_tests.postman_collection.json \
                --disable-unicode \
                --color off \
                -r htmlextra \
                --reporter-htmlextra-export Jenkins-Guide_To-Do-List/target/integration-tests-report.html
            ...
        '''
    }
}
```

Figura 209. Código del stage para las pruebas de integración dentro del pipeline de Jenkins

Este *stage*: primero levanta la aplicación Spring Boot usando Maven en segundo plano y guarda su ID de proceso (PID) para poder detenerla al final del proceso. Se define una trampa para asegurar que la aplicación se cierre correctamente cuando el *stage* termine, incluso si ocurre un error (comando *trap*).

La razón de levantar la aplicación, es que la colección de Postman, la de las pruebas de integración, requiere hacer las llamadas a la API. Y se realiza en segundo plano, para que las demás instrucciones se ejecuten.

Posteriormente, el *pipeline* espera 15 segundos para que la aplicación esté disponible y, una vez pasado este tiempo, ejecuta la colección de pruebas de Postman usando Newman. Finalmente los resultados de estas pruebas se almacenan en la carpeta *target* del proyecto Java, con el nombre *integration-tests-report.html*, para su posterior consulta.

6. Modifica la directiva *post* del *pipeline* para incluir lo siguiente:

```
post {
    always {
        junit '**/target/surefire-reports/*.xml'

        publishHTML(target: [
            reportDir: 'Jenkins-Guide_To-Do-List/target',
            reportFiles: 'integration-tests-report.html',
            reportName: 'Newman Integration Tests Report',
            alwaysLinkToLastBuild: true,
            keepAll: true
        ])

        echo 'Finalizando ejecución del pipeline con pruebas!'
    }
}
```

Figura 210. Código de la directiva *post* en el pipeline para guardar los resultados de las pruebas

Este bloque *post* se ejecuta siempre al finalizar el *pipeline*, sin importar si la ejecución fue exitosa o fallida. Su objetivo es recolectar y publicar los resultados de las pruebas realizadas.

Aquí se hace uso del *plugin* de JUnit para recopilar los reportes de pruebas unitarias generados por Maven, procesarlos y mostrarlos. Luego, se utiliza el *plugin* de HTML Publisher para publicar el reporte HTML de las pruebas de integración ejecutadas con Newman, manteniendo el historial de reportes y enlazando siempre al último *build*. Esto anterior (*keepAll: true*) permite que cada *build* guarde los resultados de las pruebas de integración, pues por defecto solamente se mantendría el último (a nivel de *pipeline*, no de *build*).

7. Con esto, el último y único paso es sincronizar los cambios locales sobre el *Jenkinsfile* con el repositorio GitHub, para llamarlos desde un *pipeline* de Jenkins y ejecutar todo el flujo.

Puede ver la versión final del *Jenkinsfile* de esta práctica [aquí](#).

14.5.7 Pipeline desde Jenkins

Antes de continuar, asegúrate de tener sincronizados tus cambios locales con el repositorio GitHub, pues a partir de aquí trabajaremos puramente con Jenkins y este recuperará los archivos desde ahí.

Si aún no ha configurado un *pipeline* en Jenkins, te recomendamos revisar la práctica "[Tu primer pipeline paso a paso](#)", pues aquí solo se darán los requisitos del *pipeline* con el que probaremos que todo lo que hemos hecho hasta ahora funcione correctamente. Puedes reutilizar el *pipeline* de la práctica previa, pero asegúrate de hacer referencia a la nueva rama sobre la que trabajaste.

Dicho lo anterior, asegúrate de tener un *pipeline* con la siguiente configuración. Si seguiste las prácticas previas, como lo acabamos de mencionar, ya deberías tener uno casi idéntico:

Configuración del Job

- *Enter an item name:* **tests-pipeline**
- *Select an item type:* **Pipeline**

Configuración General del pipeline

- Discard old builds: **Check**
 - *Max # of builds to keep:* **5**
- Do not allow concurrent builds: **Check**
 - *Abort previous builds:* **Check**

Configuración Triggers del pipeline

- *Poll SCM:* **Check**
 - *Schedule:* **H 12 * * * (todos los días a las 12am)**

Configuración Pipeline del pipeline

- Definition: Pipeline script from SCM
 - SCM: **Git**
 - *Repository URL:* **la URL de tu repositorio en GitHub** (como al realizar un clone)
 - *Credentials:* **las mismas credenciales que se configuraron en prácticas previas.**
 - *Branch Specifier (blank for 'any'):* ***/tests-pipeline** (o cualquiera sobre la que estuvieras trabajando)
 - *Script Path:* **Jenkinsfile**

- *Lightweight checkout*: **Check**

Con esta configuración, podemos verificar el correcto funcionamiento de nuestro *pipeline*:

1. En la ventana de gestión del *pipeline*, ejecuta un *build* manual.

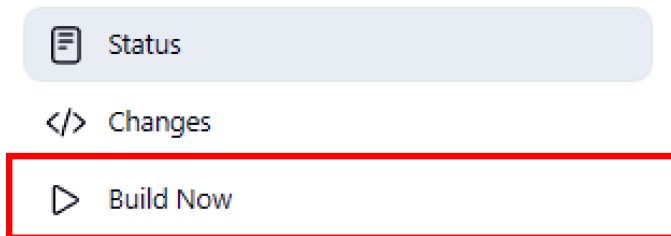


Figura 211. Captura de pantalla de la ventana de gestión del pipeline, donde se resalta el botón para lanzar un build manual

2. Cuando el *build* se termine de ejecutar, ve a la ventana de gestión del *build*. Ahí podrás identificar dos secciones interesantes.

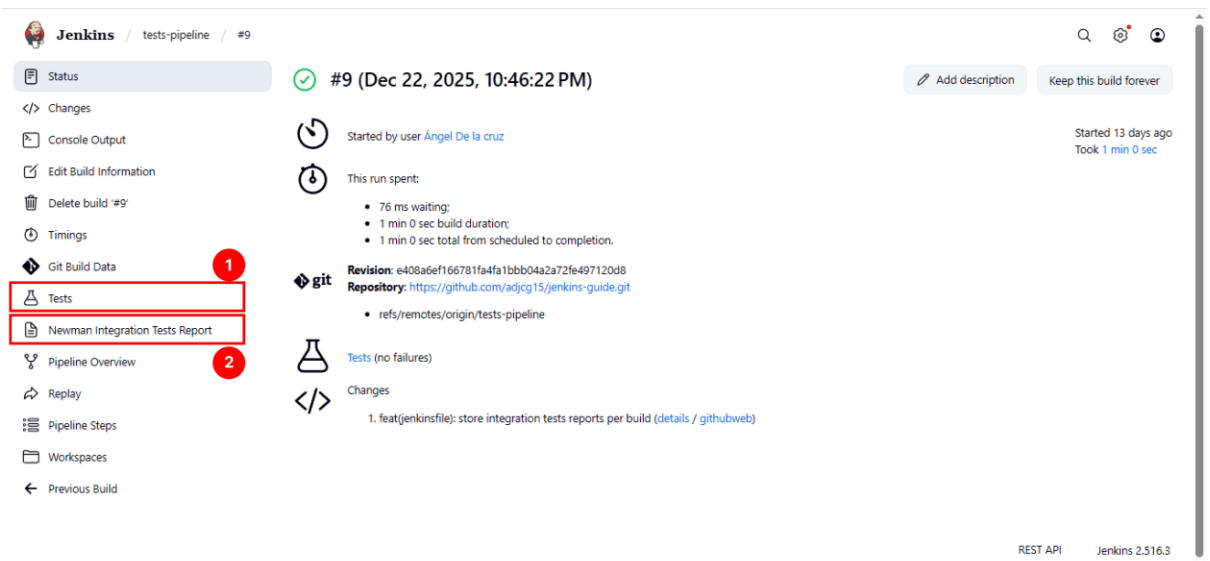


Figura 212. Secciones de tests y newman resaltadas dentro del build del pipeline

3. La primera de ellas, la etiquetada con el número 1 en la imagen anterior, contiene detalles sobre las pruebas unitarias en JUnit. Haz clic sobre ella para verlos.

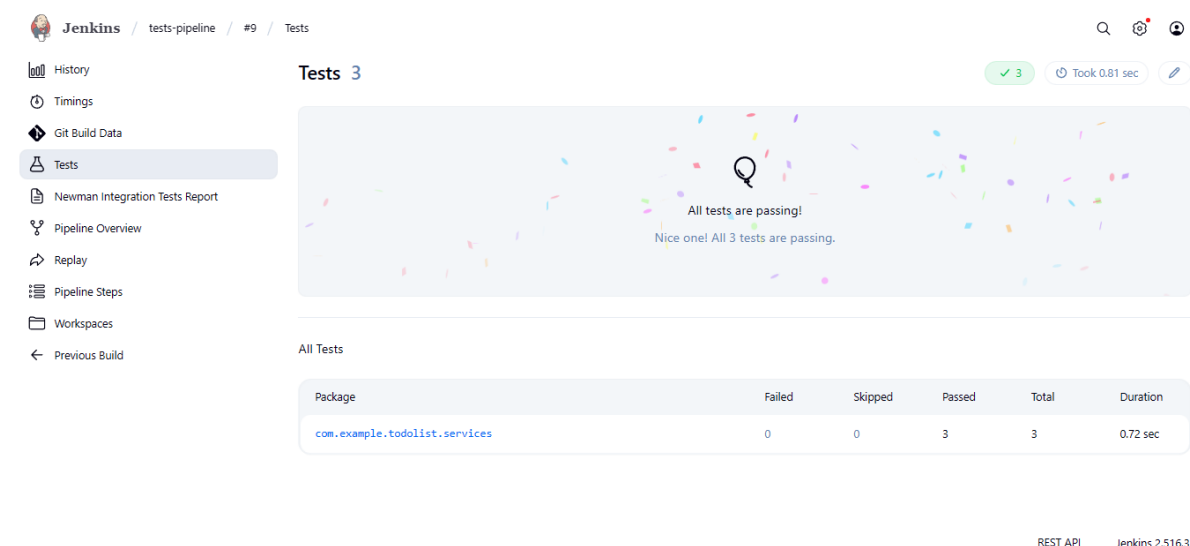


Figura 213. Sección de resultados de las pruebas del pipeline (JUnit)

Esta sección nos dice lo que hasta ahora ya conocemos: tres pruebas unitarias sobre el servicio del proyecto. Además, nos dice que todas las pruebas pasaron. En caso de haber pruebas fallidas, aquí obtendríamos un resumen de ello y, en ese caso y como comportamiento por defecto del *plugin* de JUnit, el *pipeline* fallaría durante el *build*.

- La segunda de las secciones resulta un poco más interesante, pues lleva por título *Newman Integration Tests Report*. Este nombre fue asignado desde que llamamos al *plugin* HTML Publisher desde el Jenkinsfile. Ahí se asignó el valor *reportName: Newman Integration Tests Report* y fue entonces que asignamos este nombre.

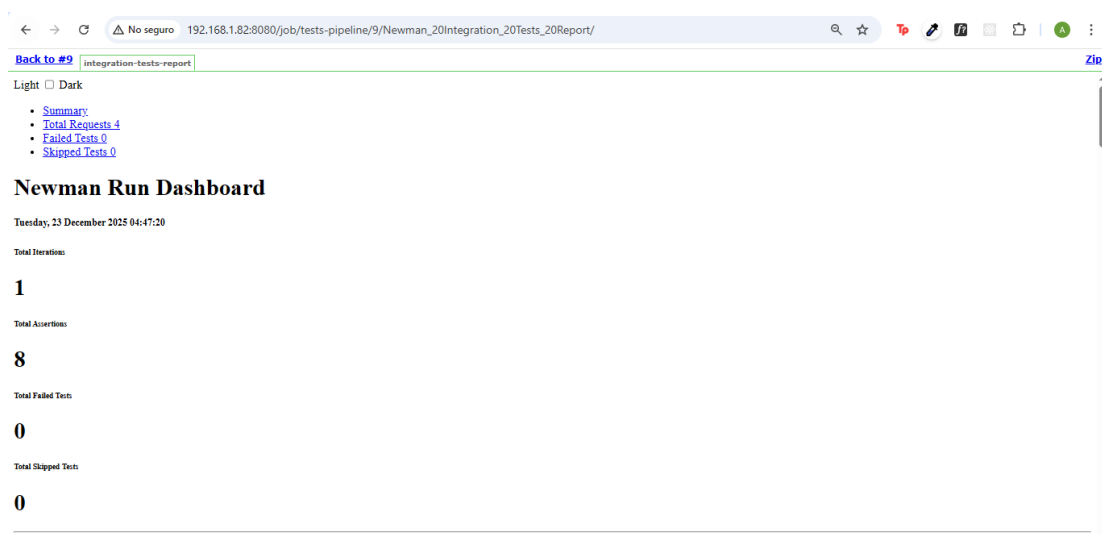


Figura 214. Sección de resultados asociados a Newman tras la ejecución del pipeline

Esta sección también da detalles sobre las pruebas, pero en este caso las de integración. Nos muestra los resultados de Newman almacenados en HTML con ayuda de los paquetes que instalamos desde el *plugin* de Node.js y que llamamos desde el Jenkinsfile.

Así, cerramos esta práctica de automatización de pruebas con Jenkins. Con ella, extendimos un poco más el funcionamiento del *pipeline*: ya no solo hace el *build* del proyecto, ahora incluye la ejecución de pruebas unitarias y de integración, junto con la publicación de los resultados en Jenkins. Con esto, nos acercamos más al cierre de la guía y a los últimos pasos en nuestro *pipeline*: el despliegue del proyecto.



Rúbrica 4. Evaluación de práctica " Integración de herramientas de prueba en *pipelines* declarativos paso a paso "

En la siguiente rúbrica, marca "Sí" o "No" en cada aspecto de evaluación para determinar si tu implementación cumple con los criterios requeridos en la integración de JUnit y Newman en *pipelines* declarativos para realizar pruebas unitarias y de integración en proyectos basados en Java.

Rubrica de Evaluación		
Aspecto por evaluar	Registro de Cumplimiento	
	Sí	No
Revisé los prerrequisitos y herramientas para seguir la práctica.		
Configuré mi ambiente local a partir del proyecto público y la rama indicada por la práctica.		

Modifiqué la implementación original del servicio en el proyecto Java para prepararlo para la implementación de pruebas unitarias.		
Implementé las pruebas unitarias desde cero, siguiendo lo especificado en la práctica.		
Agregué la colección de Postman indicada a mi proyecto local para las pruebas de integración.		
Me aseguré de que los <i>plugins</i> necesarios estuvieran instalados en Jenkins previo a la implementación del <i>pipeline</i> .		
Configuré exitosamente el plugin de Node.JS en Jenkins para la ejecución de pruebas de integración utilizando Newman.		
Modifiqué el Jenkinsfile siguiendo el paso a paso para agregar las pruebas unitarias y de integración.		
Comprendí cada modificación realizada sobre el Jenkinsfile para integrar la automatización de las pruebas.		
Entendí el rol de cada uno de los tres <i>plugins</i> mencionados para la ejecución de pruebas unitarias y de integración.		
Me aseguré de contar con un <i>pipeline</i> con las configuraciones especificadas en la práctica.		
Logré identificar los recursos de la interfaz gráfica de Jenkins que me permiten acceder a los resultados de las pruebas.		



Capítulo 15

15 Docker y *pipelines*

Contenido del capítulo

- 15.1. Uso de Docker en *pipelines* de despliegue.
- 15.2. Componentes clave de un Dockerfile.
- 15.3. *Plugins* en Jenkins para la integración con Docker.
- 15.4. Comandos de Docker para recordar.
 - 15.4.1. Comandos para gestionar imágenes de Docker.
 - 15.4.2. Comandos para gestionar contenedores de Docker.
 - 15.4.3. Comandos para utilizar DockerHub.
- 15.5. ¿Cómo configurar Docker en Jenkins?

15.1 Uso de Docker en *pipelines* de despliegue



Lectura 47. Introducción a Docker en el contexto de los *pipelines*

Nivel básico

Objetivo: el estudiante reconoce el uso de Docker en los *pipelines* de despliegue.

Descripción

La lectura proporciona una breve introducción hacia Docker en el contexto de los *pipelines* de despliegue, recordando qué es Docker, para qué se utilizar en este contexto y las ventajas de utilizarlo.

Docker es una plataforma para desarrollar, subir y ejecutar aplicaciones. Esta plataforma permite separar las aplicaciones que creamos de la infraestructura que necesitamos, con el fin de entregar software rápidamente. En cierto sentido, permite gestionar la infraestructura de las aplicaciones de la misma forma que gestionamos las aplicaciones, lo que principalmente permite reducir el tiempo entre la creación del código de las aplicaciones y su llegada a un entorno productivo (Docker Inc, 2025i).

En este contexto, surge un concepto importante: el de contenedor. Docker Inc (2025i) menciona que los contenedores son la forma en que Docker empaqueta y corre aplicaciones de forma aislada; estos contenedores incluyen todo lo necesario para ejecutar aplicaciones, manteniéndose ligeros y sin depender de las máquinas donde se ejecutan, permitiendo compartirlos dentro del equipo, asegurándose que el mismo contenedor funciona para todos de la misma forma.

Este concepto de contenedor previamente abordado, se acompaña de otro igualmente importante: el de imagen. Docker Inc (2025i) resalta que una imagen es una plantilla de solo lectura con las instrucciones necesarias para crear un contenedor Docker, mientras que estos últimos resultan ser instancias ejecutables de las imágenes, las cuales se pueden crear, levantar, detener, mover o eliminar.

Con esta idea de imágenes y contenedores de Docker, comenzamos a visualizar la potencia de la plataforma, sin embargo, Cotton (2023) resume muy bien los principales beneficios de Docker:

- Rendimiento de software consistente y confiable: dada la estandarización que ofrece sobre el ambiente sobre el que se trabaja, asegurando consistencia sobre todo el proceso de desarrollo.
- Disminución de *bugs*: pues los ambientes de producción, desarrollo, u otros, permanecen constantes, reduciendo la posibilidad de encontrar disparidades.
- Incorporación de desarrolladores más rápida: pues ya no se tienen que preocupar por la configuración local de los ambientes.
- Cadenas de suministro más seguras: pues se pueden trazar las dependencias y detectar problemas más rápido gracias a las imágenes de Docker.
- Productividad mejorada: dadas todas las ventajas anteriores, un impacto directo es la disminución del tiempo de entrega de software.

Esta capacidad de permitir desarrollos en ambientes estandarizados utilizando contenedores locales, los hace candidatos excelentes para flujos de trabajo de integración y despliegue continuo (Docker Inc, 2025i).

Dado que los contenedores proporcionan la capacidad de generar ambientes aislados y reproducibles, construir y probar aplicaciones en contenedores de Docker hace más fácil prevenir comportamientos inesperados; a la vez que se definen los requisitos específicos para el ambiente de construcción, se incluyen los sistemas operativos, binarios y más (Docker Inc, 2024j).

Junto a Jenkins, que funciona como el orquestador, Docker define y encapsula cómo se debe construir y ejecutar la aplicación. Jenkins es el sistema que activa automáticamente cada uno de los pasos del *pipeline* CI/CD (como obtener el código, construir la imagen Docker, ejecutar pruebas dentro de contenedores, o empujar esa imagen a un repositorio) cada vez que hay un cambio en el código fuente. En esta relación, Jenkins funciona como un trabajador incansable que, al detectar un nuevo cambio en el repositorio, coordina la construcción de imágenes Docker, ejecuta pruebas dentro de contenedores y puede automatizar despliegues posteriores (Mikhalev, 2025).

La integración entre Docker y Jenkins se basa en que Jenkins usa contenedores Docker como agentes o entornos aislados para ejecutar tareas específicas de la *pipeline*. Esto significa que cada etapa del proceso puede ejecutarse en un contenedor con todas las herramientas y configuraciones necesarias, lo que evita interferencias y produce resultados consistentes sin importar dónde se ejecute. Además, al

construir imágenes Docker ligeras y reproducibles, Jenkins puede manejar despliegues rápidos, revertir versiones si es necesario y escalar pruebas en múltiples contenedores simultáneamente, acelerando así todo el ciclo de CI/CD (Mikhalev, 2025).

Así pues, piensa en Docker como esta herramienta capaz de gestionar entornos aislados y reproducibles para ejecutar tus aplicaciones; que se puede integrar con Jenkins en procesos de CI/CD para proveer entornos estandarizados donde se construye y prueba la aplicación, mientras que Jenkins se encarga de automatizar y orquestar todo ese flujo.



Cuestionario 40. Introducción a Docker en el contexto de los *pipelines*

Responde las siguientes preguntas para evaluar la lectura previa y verificar tu comprensión respecto al uso de Docker en el contexto de los *pipelines* de Jenkins.

1. ¿Cuál es el objetivo principal de Docker?

A) Permitir la creación y publicación de imágenes en DockerHub.

B) Gestionar aplicaciones.

C) Proporcionar ambientes aislados y reproducibles a través de contenedores.

D) La creación de imágenes a partir de contenedores.

2. ¿Cuál de las siguientes afirmaciones describe mejor el propósito de usar Docker en un *pipeline* de Jenkins?

A) Permitir que Jenkins se ejecute más rápido en el servidor.

B) Aislar los entornos de construcción y despliegue en contenedores reproducibles.

C) Sustituir la necesidad de agentes en Jenkins.

D) Incrementar el tamaño de los archivos generados en el *build*.

3. Verdadero o falso: Jenkins puede ejecutar pasos específicos de un *pipeline* dentro de contenedores Docker.

A) Verdadero. B) Falso.

4. Verdadero o falso: el uso de Docker como agente permite que cada ejecución del *pipeline* tenga un entorno limpio y consistente.

A) Verdadero. B) Falso.

5. Verdadero o falso: los dos casos de uso más comunes de Docker con Jenkins son utilizar Docker como agentes y la creación de imágenes de aplicaciones.

A) Verdadero. B) Falso.

15.2 Componentes clave de un Dockerfile



Lectura 48. Estructura de los *Dockerfiles*

Nivel básico

Objetivo: el estudiante recuerda los componentes clave de un Dockerfile.

Descripción

La lectura presenta una introducción a los Dockerfiles, su sintaxis general, relación con las imágenes y contenedores en Docker y los componentes clave que lo componen, acompañados de una breve explicación y ejemplos para recordarlos.

Un Dockerfile es un archivo de texto que contiene instrucciones para construir el código fuente de aplicaciones. Docker construye imágenes a partir de estos archivos, por lo que resultan cruciales en este contexto. El nombre por defecto de este tipo de archivos es *Dockerfile*, sin una extensión de archivo específica (Docker Inc, 2025h).

En cuanto a la sintaxis de este archivo, Docker Inc (2025h) resalta múltiples directivas con propósitos distintos, pero alineadas hacia un mismo lugar: generar imágenes de Docker. Las directivas más relevantes que mencionan:

1. **FROM**: esta directiva permite establecer la imagen base sobre la que se generará la imagen que estamos configurando. Estas imágenes base se encuentran en [Docker Hub](https://hub.docker.com/), un repositorio de Docker para imágenes.

```
FROM ubuntu:22.04
```

Figura 215. Ejemplo de uso de directiva FROM en archivo Dockerfile

En este ejemplo previo, se muestra el uso de la directiva FROM. Ahí, se establece como imagen base *ubuntu*, en su versión 22.04. Esta notación con dos puntos es un estándar al nombrar las imágenes en Docker.

2. **WORKDIR**: permite asignar un directorio de trabajo para otras directivas como *RUN*, *CMD*, *ENTRYPOINT*, *COPY* y *ADD*. Si el directorio de trabajo no existe, Docker lo crea.

```
WORKDIR /path/to/workdir
```

Figura 216. Ejemplo de uso de la directiva WORKDIR en el archivo Dockerfile

3. **COPY**: copia archivos o directorios desde una ruta base en el host (donde se ejecuta Docker) y las añade hacia una ruta destino en el sistema de archivos de la imagen.

```
COPY hello.py /
```

Figura 217. Ejemplo de uso de la directiva COPY en el archivo Dockerfile

En este ejemplo anterior, se copia el archivo *hello.py* de la máquina host hacia la ruta *root* dentro de la imagen.

4. **EXPOSE**: esta directiva sirve para decirle a Docker que nuestra imagen final tiene un servicio escuchando en un puerto dado. En el ejemplo siguiente, en el puerto 8081.

```
EXPOSE 8081
```

Figura 218. Ejemplo de uso de la directiva EXPOSE en el archivo Dockerfile

5. **ENTRYPOINT**: esta directiva permite configurar un contenedor que se ejecutará como un ejecutable. Es decir, que cada vez que se levante un contenedor de la imagen, se ejecute un comando específico.

```
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Figura 219. Ejemplo de uso de la directiva ENTRYPOINT en el archivo Dockerfile

En este ejemplo previo, se ejecuta el comando *java* con los parámetros *-jar* y *app.jar* cada vez que el contenedor de la imagen se ejecuta. Estos aspectos sobre la gestión de imágenes y contenedores se abordan en un capítulo posterior.

Como nota final, los comentarios en los archivos Dockerfile se agregan anteponiendo el símbolo *#* a la línea que deseamos comentar.

```
# Comentario en Dockerfile
```

Figura 220. Ejemplo de comentario en archivo Dockerfile

Aunque este capítulo puede parecer breve, será más que suficiente para comprender lo que vendrá más adelante al integrar Jenkins y Docker. Entiéndase que la guía no pretende profundizar en aspectos de Docker, pero sí retomar aquellos relevantes que facilitarán el seguimiento del uso de Jenkins. Por ello, dejamos este capítulo hasta aquí en cuando a los Dockerfiles; sin embargo, si el lector se encuentra interesado en aprender más al respecto, puede revisar la [referencia de los Dockerfiles](#).



Cuestionario 41. Dockerfiles

Responde las siguientes preguntas para evaluar la lectura previa y verificar tu comprensión respecto a la estructura de los Dockerfiles.

1. ¿Qué es lo que representan los archivos Dockerfile?

- A) Imágenes. B) Contenedores. C) Aplicaciones. D) Ninguna de las anteriores.

2. ¿Cuál es la extensión de los archivos Dockerfile?

- A) .yaml B) .json C) .docker D) Ninguna.

3. ¿Cuál es el nombre que llevan por defecto los archivos Dockerfile?

- A) Docker.yaml B) Dockerfile.json C) Dockerfile D) dockerfile

4. Verdadero o falso: los archivos Dockerfile contienen código que sirve para crear contenedores de Docker.

- A) Verdadero. B) Falso.

5. Relaciona cada una de las directivas con su propósito dentro de los Dockerfiles.

A) FROM

(1) Copia archivos desde el host hacia la imagen.

B) WORKDIR

(2) Hace que el contenedor de la imagen se levante como ejecutable.

C) COPY

(3) Permite establecer la imagen base.

D) EXPOSE

(4) Indica que hay un servicio en un puerto específico en la imagen.

E) ENTRYPOINT

(5) Establece el directorio base para otras directivas e instrucciones.

15.3 *Plugins* en Jenkins para la integración con Docker



Lectura 49. *Plugins* necesarios para usar Docker con Jenkins

Nivel básico

Objetivo: el estudiante identifica los *plugins* necesarios para realizar una integración de Docker en un *pipeline* de Jenkins.

Descripción

La lectura aborda los *plugins* necesarios para que se pueda acceder y utilizar Docker desde los *pipelines* de Jenkins, incluyendo sus objetivos principales, las características destacables de cada uno y aquello que los diferencia.

Para trabajar con Docker dentro de Jenkins, hay tres *plugins* principales que nos permiten hacerlo: **Docker Plugin**, **Docker Pipeline Plugin** y **CloudBees Docker Build and Publish Plugin**. A continuación abordaremos cada uno de ellos, dejando en claro su funcionalidad en el entorno de Jenkins (Vladimir Mikhalev, 2025).

Docker Plugin es el primero que abordaremos. Como lo menciona la CD Foundation (s/f-a), su objetivo principal es permitir que Jenkins use Docker como un proveedor de *agentes* (nodos de ejecución): Jenkins genera agentes dinámicamente, ejecuta un *build* en un contenedor Docker previamente configurado y luego elimina el agente automáticamente.

Otras capacidades que ofrece Docker Plugin según CD Foundation (s/f-b):

- Provisión dinámica de agentes en contenedores.
- Configuración de múltiples *hosts* Docker e imágenes.
- Integración con etiquetas para selección de agentes.

Pasando al segundo *plugin* mencionado, CloudBees (s/f-a) menciona que su objetivo principal es proporcionar un lenguaje específico de dominio (DSL) dentro de Jenkins Pipeline, para ejecutar operaciones Docker comunes directamente desde un Jenkinsfile, como ejecutar pasos de *build* dentro de contenedores, construir y publicar

imágenes, ejecutar y probar contenedores, así como personalizar uso de servidores y registros Docker.

El tercer plugin, CloudBees Docker Build and Publish Plugin, tiene como objetivo principal construir una imagen Docker a partir de un Dockerfile en el espacio de trabajo y luego publicarla en un registro Docker (como DockerHub), gestionando credenciales y tags (CD Foundation, s/f-a).

En conjunto, estos *plugins* integran Docker en distintas capas del *pipeline* de Jenkins: Docker Plugin permite que las etapas del *pipeline* se ejecuten sobre agentes basados en contenedores, proporcionando aislamiento y escalabilidad; Docker Pipeline Plugin habilita el uso directo de Docker dentro del Jenkinsfile, permitiendo ejecutar pasos en contenedores, construir, probar y publicar imágenes; finalmente, CloudBees Docker Build and Publish Plugin, simplifica específicamente la compilación y publicación de imágenes Docker como parte del flujo de CI/CD, gestionando etiquetas y credenciales sin requerir *scripting* manual de Docker.



Cuestionario 42. *Plugins* necesarios para usar Docker con Jenkins

Responde las siguientes preguntas para evaluar la lectura anterior y verificar tu comprensión respecto a los *plugins* que permiten integrar Jenkins y Docker.

-
1. ¿Cuál de los siguientes *plugins* permite usar instrucciones Docker directamente en un Jenkinsfile?

A) Docker Plugin	B) Docker Pipeline Plugin	C) CloudBees Docker Build and Publish Plugin	D) No es necesario un plugin.
------------------	---------------------------	--	-------------------------------
 2. ¿Qué *plugin* permite que Jenkins cree agentes efímeros basados en contenedores Docker?

- A) Docker Plugin B) Docker Pipeline Plugin C) CloudBees Docker Build and Publish Plugin D) No es necesario un plugin.

3. Verdadero o falso: el *plugin* Docker Pipeline depende del Docker Plugin para poder ejecutar contenedores.

- A) Verdadero. B) Falso.

4. Verdadero o falso: el *plugin* CloudBees Docker Build and Publish Plugin tiene como objetivo crear agentes basados en contenedores Docker.

- A) Verdadero. B) Falso.

5. Para poder ejecutar contenedores Docker dentro de un *pipeline* declarativo, el *plugin* más importante que debes instalarse es:

- A) Docker Plugin B) Docker Pipeline Plugin C) CloudBees Docker Build and Publish Plugin D) No es necesario un plugin.

15.4 Comandos de Docker para recordar



Lectura 50. Comando básicos necesarios para usar Docker

Nivel básico

Objetivo: el estudiante identifica los comandos clave para desplegar un contenedor de Docker.

Descripción

La lectura recopila los comandos necesarios para crear exitosamente un contenedor de Docker a partir de una imagen. Así mismo, se explican otros comandos esenciales en el uso de Docker que pueden ser de

utilidad para el lector a la hora de gestionar contenedores e imágenes de Docker.

En Docker hay esencialmente tres elementos que vas a utilizar en algún punto de tu desarrollo: las imágenes, los contenedores y DockerHub. Cada uno de estos elementos tiene comandos específicos asociados que sirven para gestionarlos en general (Docker Inc, s/f).

15.4.1 Comandos para gestionar imágenes de Docker

El primero es “***docker build -t <nombre_imagen> .***”. Este sirve para construir una imagen (normalmente a partir de un Dockerfile); el parámetro ***-t*** permite agregar una etiqueta para identificar la imagen (***<nombre_imagen>*** refiere a este tag); el punto indica que la ruta en donde se encuentra el contexto de construcción de la imagen es la carpeta donde se ejecuta el comando. Este punto se puede cambiar por una ruta y es ahí donde Docker buscará el Dockerfile y otros archivos de referencia (Docker Inc, 2024i).

Para eliminar una imagen de manera local, se utiliza el comando ***docker rmi <nombre_imagen>***. Adicionalmente se puede usar el parámetro ***-f*** para forzar la eliminación de imágenes que se estén ocupando en algún contenedor activo (Docker Inc, 2024h).

Mediante el comando ***docker images*** o ***docker image ls*** se pueden listar todas las imágenes guardadas localmente, incluyendo el repositorio al que pertenecen, su etiqueta y el espacio de disco que ocupan (Docker Inc, 2025).

Con estos tres comandos básicos puedes crear y gestionar las imágenes en tu entorno local.

15.4.2 Comandos para gestionar contenedores de Docker

Los contenedores son más complejos, así que se suelen utilizar más comandos para gestionarlos. El comando principal es ***docker run -name <nombre_contenedor> <nombre_imagen>***, que sirve para crear un contenedor, desde cero, a partir de una imagen; por eso se debe indicar el nombre de la imagen que se quiere utilizar y, opcionalmente, se le puede asignar un nombre al contenedor mediante el parámetro ***-name***. Adicionalmente, se suelen agregar los argumentos ***-p <puerto_local:puerto_contenedor>*** para configurar un puerto de comunicación entre el

host y el contenedor, y el argumento *-d* para ejecutar el contenedor en segundo plano (Docker Inc, 2024k).

Al igual que con las imágenes de Docker, también es posible eliminar contenedores enteros mediante el comando ***docker rm <nombre_contenedor>***, sin embargo, el contenedor debe estar inactivo al momento de ejecutar el comando. También es posible utilizar el argumento *-f* para forzar la eliminación de un contenedor activo (Docker Inc, 2024f).

Para detener o inicializar un contenedor que ya fue creado previamente se utiliza el comando ***docker start | stop <nombre_contenedor> | <id_contenedor>***. Es importante resaltar que *docker start* no es lo mismo que *docker run*: el primero únicamente inicializa un contenedor que hayas detenido previamente, mientras que el segundo comando crea un contenedor a partir de una imagen (Docker Inc, 2023).

Cuando se crea un contenedor, este se ejecuta en un espacio aislado, lo que complica la comunicación con el entorno interno del contenedor. Una forma de interactuar con él es mediante el comando ***docker exec -it <nombre_contenedor> sh***. El comando *docker exec* permite ejecutar un comando nuevo dentro del contenedor, mientras que la parte *sh* abre una conexión *Shell* con el contenedor (es el comando para ejecutar, en este caso). Sin embargo, para utilizar el comando *exec*, es necesario que el contenedor no se encuentre suspendido (Docker Inc, 2025a).

Finalmente, el comando ***docker ps*** lista todos los contenedores activos. Para ver también los contenedores que se encuentran inactivos, se puede agregar el parámetro *--all* (Docker Inc, 2025b).

15.4.3 Comandos para utilizar DockerHub

Todos los comandos previamente explicados sirven para gestionar imágenes y contenedores de manera local. Sin embargo, si se quiere utilizar un servicio de nube como DockerHub para la gestión de imágenes, se deben aprender otros comandos extra para subir y descargar imágenes de la nube.

El primero comando es el más sencillo: ***docker login -u <nombre_usuario>***, que sirve para iniciar sesión en el servicio de DockerHub. Este paso es necesario para subir las imágenes de Docker posteriormente (Docker Inc, 2024g).

Una vez autenticado, se puede utilizar el comando ***docker push nombre_usuario/<nombre_imagen>*** para subir una imagen directamente a

DockerHub o a un repositorio privado, si es que se tiene configurado (Docker Inc, 2025e).

Finalmente, para recuperar una imagen de DockerHub se utiliza el comando ***docker pull <nombre_imagen>***. Debido a que una imagen puede variar dependiendo de sus etiquetas, si no se indica una específica, entonces se descarga por defecto la que tenga la etiqueta *latest* (Docker Inc, 2025d).

Adicionalmente, cada uno de los comandos explicados previamente cuenta con varios parámetros o argumentos extra para modificar su comportamiento. Sin embargo, en esta guía no se profundiza al respecto y se limita a recordar los comandos básicos para su uso en Jenkins. Si quieres leer más al respecto, puedes revisar el [Glosario de comandos de Docker](#).



Cuestionario 43. Comando básicos necesarios para usar Docker

Responde las siguientes preguntas para evaluar la lectura anterior y verificar tu comprensión respecto a los comandos esenciales para utilizar la herramienta Docker.

1. ¿Cuál de los siguientes comandos permite crear una imagen de Docker?
A) *docker run* B) *docker build* C) *docker up* D) *docker img*
2. ¿Cuál de los siguiente parámetros permite agregar una etiqueta a una imagen de Docker?
A) *-t* B) *-e* C) *-note* D) Ninguna de las anteriores.
3. ¿Cuál de los siguientes comando permite crear y levantar un contenedor a partir de una imagen de Docker?

- A) *docker build* B) *docker create* C) *docker run* D) *docker up*
4. El parámetro *-p 8080:80* se usa en conjunto con el comando:
- A) *docker build* B) *docker run* C) *docker stop* D) *docker login*
5. ¿Cuál comando permite ejecutar scripts y comandos en los contenedores internamente?
- A) *docker shell* B) *docekr sh* C) *docker run* D) *docker exec*
6. ¿Qué parámetro se suele utilizar en conjunto con el comando *docker login* para indicar el usuario?
- A) *-credentials* B) *-c* C) *-u* D) Ninguno de los anteriores.
7. El comando *docker push jenkins/guide_image* fuerza la creación de una imagen de Docker a partir de un Dockerfile.
- A) Verdadero. B) Falso.
8. El comando *docker rmi app_image* permite editar una imagen de Docker a partir de un Docekrfile.
- A) Verdadero. B) Falso.
9. El argumento *-f* se suele usar para crear o eliminar imágenes de manera segura.
- A) Verdadero. B) Falso.
10. El comando *docker start app_image* permite crear un contenedor desde cero a partir de un Dockerfile.

A) Verdadero.

B) Falso.

15.5 ¿Cómo configurar Docker en Jenkins?



Lectura 51. Integrando Docker con Jenkins

Nivel Intermedio

Objetivo: el estudiante describe los componentes y requisitos necesarios para integrar Docker en un *pipeline* de Jenkins.

Descripción

La lectura explica las acciones necesarias que el estudiante debe realizar para instalar e integrar la herramienta Docker con Jenkins y poder usarla dentro de los *pipelines* declarativos.

Primero, se deben instalar los *plugins* recomendados: Docker Plugin, Docker Pipeline Plugin y CloudBees Docker Build and Publish. Estos *plugins* permiten continuar con las configuraciones posteriores (Vladimir Mikhalev, 2025).

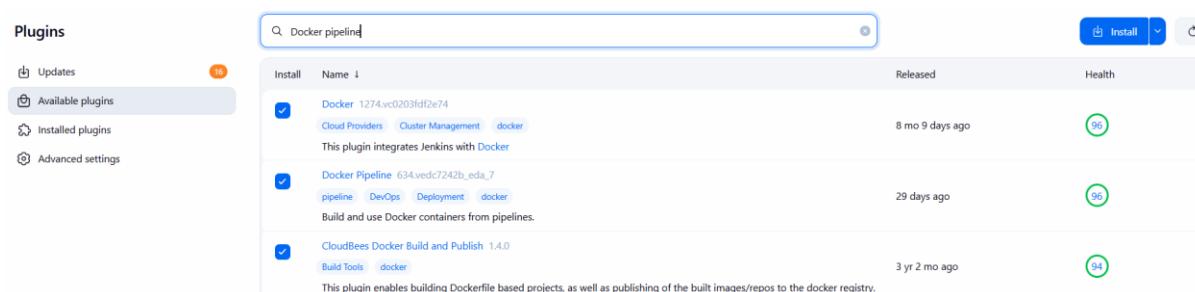


Figura 221. Captura de pantalla de ventana de gestión de plugins en Jenkins donde se muestran los tres requeridos para la integración con Docker

En segunda instancia, se debe configurar una instalación de Docker. Para esto existen dos métodos: instalar Docker directamente en el servidor o delegar la instalación a Jenkins. Tal como se ha recomendado a lo largo de esta guía, es preferible delegar la instalación a Jenkins. Sin embargo, en caso de optar por instalar Docker manualmente en el servidor, es necesario asignarle los permisos al usuario de Jenkins para interactuar con el servicio de Docker; esto se hace mediante el comando **`sudo usermod -a -G docker jenkins`** (DevOps Shack, 2022).

Para delegar la instalación de Docker y como se ha visto en otros capítulos con otros *plugins*, hay que ir a la sección *Manage/Tools* de Jenkins, buscar la herramienta Docker, asignarle un nombre y marcar la casilla para habilitar la instalación automática.

Además, dependiendo de tus necesidades puedes o no utilizar el servicio de DockerHub para guardar las imágenes generadas y tener un mayor organización. En caso de necesitarlo, entonces es necesario tener una cuenta de DockerHub y registrar las mismas credenciales en Jenkins en la sección *Manage/Credentials* (CloudBeesTV, 2021a).



Figura 222. Captura de pantalla de la ventana de gestión de credenciales en Jenkins

Tal como se explicó en lecturas previas, la manera que Docker ofrece para crear imágenes, es mediante un archivo Dockerfile. Para que Jenkins pueda utilizar este archivo para generar una imagen automáticamente, es necesario que el Dockerfile se encuentre en el mismo directorio de trabajo que el Jenkinsfile. En el caso de esta guía, el Dockerfile y el Jenkinsfile se encuentran al mismo nivel, en el mismo repositorio de GitHub. Es importante entender que Jenkins no construye la imagen como tal, simplemente ubica el Dockerfile y le delega a la instalación de Docker (antes configurada) la construcción de la imagen (CloudBeesTV, 2020).

En la lectura previa se explicaron los comandos básicos de Docker para crear y gestionar imágenes y contenedores. Al utilizar el *plugin* Docker Pipeline, se simplifican estos comandos. En lugar de utilizar *steps* de tipo *sh* para ejecutar los comandos, el *plugin* Docker Pipeline agrega una capa de abstracción que permite utilizar funciones para ejecutar los mismos comandos a través de una variable global llamada *docker*. Para utilizar una función o método, primero se debe invocar a la variable *docker*, seguido de la función que se quiere realizar.

Bajo el contexto del *plugin* Docker Pipeline, para crear una imagen de Docker no se utiliza la instrucción ***sh "docker build -t nombre_imagen"***; en su lugar, se utiliza la instrucción ***def nombre_variable = docker.build "nombre_imagen"***. Esta instrucción

crea una imagen de Docker a partir de un Dockerfile, y retorna el resultado para poder utilizarlo posteriormente (CloudBees, s/f-a).

Para subir una imagen recién creada a un servidor, se puede utilizar el `step docker.withRegistry`, junto con la función `docker.push`; se deben indicar las credenciales registrada previamente y la URL del repositorio, el cual puede ser una nube privada o pública como DockerHub. Normalmente se utiliza la nube pública de DockerHub con la dirección <https://index.docker.io/v1/> (CloudBees, s/f-a).

```
docker.withRegistry('https://docker.mycorp.com/', 'docker-login') {  
    git '...'  
    docker.build('myapp').push('latest')  
}
```

Figura 223. Ejemplo del uso de `docker.withRegistry` y `docker.build` en un pipeline declarativo de Jenkins

Finalmente, para crear y levantar un contenedor a partir de una imagen, se pueden utilizar tres funciones distintas que dependiendo de tus necesidades.

La primera es **`docker.image(<nombre_imagen>).run`**, la cual ejecuta el contenedor en segundo plano.

```
def app = docker.image('nginx:latest').run('-p 8080:80')  
sh 'sleep 5'  
sh 'curl http://localhost:8080'  
app.stop()
```

Figura 224. Ejemplo de uso de `docker.image(...).run` en pipeline declarativo de Jenkins

La segunda es **`docker.image(<nombre_imagen>).inside{}`**, la cual genera un contenedor temporal aislado.

```
docker.image('alpine:latest').inside {  
    sh 'echo "Estoy dentro del contenedor Alpine"'  
    sh 'cat /etc/os-release'  
}
```

Figura 225. Ejemplo de uso de `docker.image(...).inside` en pipeline declarativo de Jenkins

La tercera opción es utilizar la función **`docker.image(<nombre_imagen>).withRun{}`**, la cual ejecuta el contenedor y automáticamente lo detiene al final del bloque.

```
docker.image('mysql').withRun {c ->
    sh './test-with-local-db'
}
```

Figura 226. Ejemplo de uso de `docker.image(...).withRun` en pipeline declarativo de Jenkins

Tanto dentro de los corchetes de *inside*, como de *withRun*, se pueden ejecutar comandos dentro del contenedor. Cada una se puede usar a voluntad dependiendo las necesidades de tu proyecto. Por ejemplo, *run* es útil para desplegar la aplicación en segundo plano y probarla manualmente, mientras que las otras dos opciones son más útiles para realizar pruebas automáticas en un contenedor y que este se elimine al finalizarlas (CloudBees, s/f-a).

Es importante mencionar que para usar la sintaxis declarativa de los *pipelines* de Jenkins, estas funciones se deben englobar dentro de un *step* de tipo *script* para que Jenkins pueda acceder al motor de *scripting* y poder llamar a las funciones y métodos declarados. En caso de usar una sintaxis de tipo *scripted*, esto no es necesario (CD Foundation, 2025r).

Por último, aunque el *plugin* Docker Pipeline facilita el uso de Docker, todavía es posible utilizar los comandos de Docker directamente en lugar de las funciones. Para utilizar los comandos directamente en el Jenkinsfile, se necesita declarar un *step* de tipo *sh*, seguido de los comandos a utilizar (Vladimir Mikhalev, 2025).

```
sh '''
docker build -t ${IMAGE_NAME}:${IMAGE_TAG} .
'''

withCredentials([usernamePassword(credentialsId: 'dockerhub-credentials', usernameVariable:
'DOCKER_USER', passwordVariable: 'DOCKER_PASS')]) {
    sh '''
        echo "$DOCKER_PASS" | docker login -u "$DOCKER_USER" --password-stdin
    '''
}
sh '''
docker push ${IMAGE_NAME}:${IMAGE_TAG}
docker run -d --name ${CONTAINER_NAME} ${IMAGE_NAME}:${IMAGE_TAG}
docker exec ${CONTAINER_NAME} echo "¡Hola desde dentro del contenedor!"
docker stop ${CONTAINER_NAME} || true
docker rm ${CONTAINER_NAME} || true
docker logout
'''
sh 'docker system prune -f || true'
```

Figura 227. Ejemplo de comandos de Docker llamados directamente desde la línea de comandos en pipeline declarativo de Jenkins

Ahora ya sabes cómo utilizar Docker en conjunto con Jenkins para crear imágenes, publicarlas en DockerHub y alzar contenedores automáticamente. En capítulos posteriores aprenderás a integrar Docker-Compose y realizarás una última práctica para poner todos estos conocimientos en acción.



Cuestionario 44. Integrando Docker con Jenkins

Responde las siguientes preguntas para evaluar la lectura anterior y verificar tu comprensión respecto a los pasos necesarios para integrar Docker con Jenkins.

1. ¿Cuántos y cuáles son los *plugins* existentes para utilizar Docker dentro de los *pipelines* declarativos de Jenkins?

R = _____

2. Un equipo de desarrollo necesita configurar Docker con Jenkins, pero no quieren instalar Docker manualmente en el servidor. ¿En qué sección de Jenkins deben configurar esta instalación? Y ¿Qué opción deben seleccionar para no instalar Docker manualmente?

R = _____

3. Para utilizar DockerHub con Jenkins y poder subir imágenes de Docker a la nube desde un *pipeline* de Jenkins, ¿Qué paso previo en Jenkins es necesario?

R = _____

4. Al utilizar el *plugin* Docker Pipeline es posible utilizar funciones y métodos dentro de los *pipelines* declarativos, ¿Qué *step* se debe utilizar para declarar dichos métodos?

R = _____

5. Con el objetivo de automatizar las pruebas unitarias y funcionales de una aplicación, un equipo de desarrollo utiliza Docker para aislar su aplicación y realizar las pruebas. El equipo no quieren tener basura ni contenedores huérfanos al finalizar la ejecución del *pipeline*, ¿Qué instrucción de un *pipeline* declarativo podrían usar para lograr este objetivo?

R = _____

6. Si se utiliza el *plugin* Docker Pipeline, ya no es necesario utilizar el comando *docker exec* dentro de los *pipelines* declarativos, ¿De qué manera se pueden ejecutar comandos dentro de un contenedor?

R = _____

7. Un equipo de desarrollo que utiliza Docker dentro de su *pipeline* declarativo de Jenkins decidió dejar de utilizar permanentemente el *plugin* Docker Pipeline, ya que no creen que sea necesario. Al eliminarlo, el *pipeline* comenzó a fallar y los mensaje de error dicen que no se encuentran algunas funciones. ¿Qué modificaciones debe realizar el equipo de desarrollo para que el *pipeline* funcione nuevamente?

R = _____

8. Relaciona cada función o método de Docker con su descripción más acertada.

A) <i>docker.build</i>	(1) Ejecuta un contenedor mientras ejecuta comandos dentro del bloque {}
B) <i>docker.push</i>	_____
C) <i>docker.image().run</i>	(2) Construye una imagen e Docker a partir de un Dockerfile.
D) <i>docker.image().Inside{}</i>	_____
E) <i>docker.image().withRun{}</i>	(3) Realiza un login automático a un repositorio de Docker.
F) <i>docker.withRegistry</i>	_____
	(4) Sube una imagen a un repositorio de Docker.

	(5) Ejecuta comandos dentro del bloque {}, se elimina al finalizar.

	(6) Ejecuta un contenedor en segundo plano.



Capítulo 16

16 Docker Compose y *pipelines*

Contenido del capítulo

- 16.1. Uso de Docker Compose en *pipelines* de despliegue.
- 16.2. Componentes claves de un archivo YAML.
- 16.3. Comandos de Docker Compose para recordar.
- 16.4. Uso de Docker Compose en *pipelines* de despliegue.

16.1 Uso de Docker Compose en *pipelines* de despliegue



Lectura 52. Introducción a Docker Compose en el contexto de los *pipelines*

Nivel básico

Objetivo: el estudiante reconoce el uso de Docker Compose en los *pipelines* de despliegue.

Descripción

La lectura proporciona una breve introducción hacia Docker Compose en el contexto de los *pipelines* de despliegue, recordando qué es Docker Compose, para qué se utilizar en este contexto y las ventajas de utilizarlo.

Docker Compose es una herramienta para definir y ejecutar aplicaciones que constan de múltiples contenedores. Esta herramienta simplifica el control de aplicaciones enteras, facilitando la gestión de servicios, redes y volúmenes a través de archivos de configuración YAML (Docker Inc, 2026b).

Esta herramienta trabaja en todos los ambientes, lo que la hace perfecta para flujos de trabajo CI/CD. Además, ofrece comandos para gestionar el ciclo de vida de las aplicaciones: iniciar y detener servicios, monitorear el estado de los servicios, facilitar la ejecución de los servicios, entre otros (Docker Inc, 2026b).

Siguiendo esta línea, es claro que Docker Compose trae muchos beneficios a la mesa. En este sentido, Docker Inc (2025f) resume muy bien los principales beneficios de la herramienta:

1. Control simplificado: agilizando la orquestación y replicación a través de los archivos YAML.
2. Colaboración eficiente: dada la centralización de la gestión con los archivos YAML.
3. Desarrollo rápido de aplicaciones: pues reutiliza la configuración de los contenedores siempre que su definición no cambie, lo que permite cambios rápidos en el ambiente.

4. Portabilidad entre ambientes: dada la capacidad de definir variables en los archivos YAML.

Con lo anterior, es claro que Docker Compose es una herramienta útil, sin embargo, toda esta utilidad gira en torno a los archivos YAML tan mencionados, pues la mayoría de los beneficios los mencionan.

Los archivos YAML, o simplemente archivos Compose, sirven para configurar los servicios de la aplicación. Estos archivos llevan por nombre *compose* o *docker-compose* (preferentemente la primera opción), con la extensión de archivo *.yaml* o *.yml*. Y siguen la [especificación Compose](#), que declara las reglas sobre cómo definir aplicaciones multi contenedor (Docker Inc, 2026a).

Es justamente este conjunto de capacidades y el enfoque a través de los archivos *compose* lo que hace que la herramienta se alinee tanto con prácticas de CI/CD. Como lo menciona GeeksForGeeks (2025b), Docker Compose habilita el desarrollo de una configuración consistente y reproducible en los niveles de producción, pruebas y desarrollo; reduciendo la intervención manual, mientras que se decrementan los posibles errores.

Con esto, Docker Compose promueve un proceso de trabajo más efectivo, así como ciclos de entrega de software más rápidos; y al final del día, un proceso de entrega de software más confiable (GeeksForGeeks, 2025b).



Cuestionario 45. Introducción a Docker Compose en el contexto de los *pipelines*

Responde las siguientes preguntas para evaluar la lectura anterior y verificar tu comprensión respecto al uso de Docker Compose en el contexto de los *pipelines* de Jenkins.

-
1. ¿Cuál es el objetivo principal de Docker Compose?

- | | | | |
|--|-----------------------------|--|-----------------------------------|
| A) Definir y ejecutar aplicaciones multi contenedor. | B) Gestionar archivos YAML. | C) Proporcionar ambientes aislados y reproducibles a | D) La creación de imágenes muchas |
|--|-----------------------------|--|-----------------------------------|

través de
contenedores.

imágenes con
YAML.

2. ¿Cuál de las siguientes afirmaciones describe mejor el propósito de usar Docker Compose en un *pipeline* de Jenkins?

A) Permitir que Jenkins se ejecute de forma más confiable en el servidor.

B) Ejecutar aplicaciones multi contenedor en Jenkins.

C) Sustituir la necesidad de Docker en Jenkins.

D) Habilitar el desarrollo de una configuración consistente y reproducible en diversos ambientes.

3. Verdadero o falso: los archivos YAML son el estándar para definir aplicaciones multi contenedor con Docker Compose.

A) Verdadero.

B) Falso.

4. Verdadero o falso: la gestión uniforme de aplicaciones completas en diversos ambientes de Docker Compose y su gestión a través de archivos *compose* lo hacen una herramienta sumamente útil en el contexto de CI/CD.

A) Verdadero.

B) Falso.

5. Verdadero o falso: integrar Docker Compose en *pipelines* de Jenkins reduce la intervención manual, mientras que incrementa los posibles errores.

A) Verdadero.

B) Falso.

16.2 Componentes clave de un archivo YAML



Lectura 53.

Estructura de los archivos *compose*

Nivel básico

Objetivo: el estudiante recuerda los componentes clave de un archivo YAML.

Descripción

La lectura presenta una introducción a los archivos *compose* (YAML), su sintaxis general y los componentes clave que lo componen, acompañados de una breve explicación y ejemplos para recordarlos.

Como se abordó en la sección anterior, los archivos *compose* o YAML cobran mucha relevancia al trabajar con Docker Compose. Por esta razón, en este capítulo se abordan los elementos más importantes que te permitirán comprender los archivos que se estarán desarrollando a lo largo de la última práctica.

Si bien este capítulo no pretende desarrollar una referencia completa sobre dichos archivos y sus elementos, pues ello está fuera del alcance de esta guía, se motiva al lector a seguir aprendiendo sobre Docker Compose y sobre los archivos *compose*. Para ello, puedes consultar la [Referencia de los archivos *compose*](#).

Los archivos *compose* se representan a través de llaves y valores, separados por el símbolo de dos puntos. Cada línea representa un par llave-valor (en su mayoría), siendo las llaves los elementos definidos por la propia sintaxis. Entre los diversos elementos, las tabulaciones representan niveles, tal como en los lenguajes de programación. Dicho esto, a continuación presentamos algunos de los elementos más relevantes para esta guía:

El elemento de nivel superior ***version*** es el primero que abordamos. Este elemento permite definir explícitamente la versión de archivos Compose que estamos utilizando. Docker Compose utiliza siempre la última versión, a menos que se especifique una explícitamente con este elemento al inicio del archivo (Docker Inc, 2025g).

El segundo elemento de nivel superior es ***name***. Este es útil para definir un nombre de proyecto, es decir, para decirle a Compose cómo identificar el proyecto (Docker Inc, 2025g).

```
version: "3.9"
name: tienda-online
```

Figura 228. Ejemplo de uso de elementos `version` y `name` en archivo `Compose`. Mientras `version` indica cómo interpretar el archivo, `name` indica cómo identificar al proyecto

Dicho lo anterior, un concepto relevante en este contexto es el de **servicio**. Un servicio es una definición abstracta de un recurso de cómputo dentro de una aplicación, que puede ser reemplazado de forma independiente de otros componentes. El elemento de nivel superior **services** permite definir la configuración aplicada a cada servicio (Docker Inc, 2026b).

```
services:
  proxy:
    image: nginx
    container_name: proxy
    restart: unless-stopped
    ports:
      - 80:80
    volumes:
      - ./proxy/nginx.conf:/etc/nginx/conf.d/default.conf:ro

  backend:
    image: node:18
    container_name: backend
    restart: unless-stopped
```

Figura 229. Ejemplo general de código de archivo `Compose`, donde se muestra el uso del elemento `services`

En este ejemplo previo, se definen dos servicios: uno que lleva por nombre *proxy* y otro *backend*. Esta es la sintaxis general del elemento `services`, que justamente nos permite definir la configuración de cada servicio. ¿Pero qué hay de la configuración de los servicios? ¿Qué significa todo lo demás? Vayamos paso a paso.

Partiendo con el elemento **image**, como su nombre ya da pistas, permite indicar a partir de qué imagen se quiere crear el contenedor para el servicio (*proxy* o *backend* en el ejemplo). Este elemento debe acompañarse de un *addressable image format*, como el nombre de la imagen, su registro en DockerHub, el registro privado, o similar (Docker Inc, 2026b).

El elemento **container_name** es muy descriptivo por sí solo, tal como *image*. Este sirve para especificar un nombre personalizado para el contenedor del servicio. Este nombre puede incluir caracteres alfanuméricos, guiones, guiones bajos y puntos (Docker Inc, 2026b).

Pasando al elemento **ports**, este sirve para definir el mapeo de puertos entre la máquina *host* y el contenedor. Primero indicando el puerto del *host*, y posteriormente el puerto del contenedor (Docker Inc, 2026b).

El elemento **volumes** se usa para compartir o conservar datos, permitiendo que un contenedor acceda a archivos que están fuera de él, ya sea en el equipo anfitrión o en un volumen administrado por Docker; con *volumes* puedes montar carpetas del *host* o volúmenes dentro de los contenedores (Docker Inc, 2026b).

El elemento anteriormente mencionado se puede definir usando una sola cadena separada por dos puntos, donde primero se indica el volumen o ruta del *host*, luego la ruta dentro del contenedor y, de forma opcional, el modo de acceso; si no se especifica este último, se asume lectura y escritura (*rw*), *ro* para solo lectura, y en sistemas con SELinux se pueden usar las opciones *z* o *Z* para controlar si ese contenido se comparte entre contenedores o se mantiene privado (Docker Inc, 2026b).

Finalmente, el elemento **restart** define qué debe hacer Docker cuando un contenedor se detiene. Por ejemplo, con **no** el contenedor no se reinicia, con **always** se reinicia siempre sin importar la causa, con **on-failure** solo se reinicia cuando ocurre un error (y se puede limitar cuántos intentos hace), y con **unless-stopped** se reinicia automáticamente excepto cuando el usuario lo detiene o elimina de forma manual (Docker Inc, 2026b).

Todo esto permite gestionar la infraestructura de aplicaciones a través de archivos YAML, utilizando Docker por detrás. Si bien aquí no se profundiza en los detalles de cada elemento, ni en todos los que existen en los archivos Compose, esperamos proporcionar una vista general sobre el significado de aquellos que estaremos utilizando más adelante.



Cuestionario 46. Estructura de los archivos *compose*

Responde las siguientes preguntas para evaluar la lectura previa y verificar tu comprensión respecto a la estructura general de los archivos *compose* o YAML.

1. Verdadero o falso: un archivo *compose* está definido, de forma general, por llaves y valores separados por dos puntos y que tienen niveles para representar bloques, como en un lenguaje de programación.

A) Verdadero. B) Falso.
2. ¿Qué elemento dentro de los archivos *compose* permite declarar la versión de Docker para utilizar en los contenedores?

A) *version* B) *image* C) *container-version* D) Ninguno de los anteriores.
3. ¿Qué elemento dentro de los archivos *compose* permite establecer un nombre de proyecto?

A) *version* B) *name* C) *image* D) *volumes*
4. Verdadero o falso: mientras que el elemento *version* indica cómo interpretar el archivo, *name* establece cómo identificar el proyecto.

A) Verdadero. B) Falso.
5. ¿Qué elemento dentro de los archivos *compose* permite definir la configuración aplicada a cada servicio?

A) *service* B) *docker-services* C) *services* D) *compose-services*
6. Verdadero o falso: existe un elemento *service* que permite indicar el nombre de cada servicio de la forma *services: <nombre-servicio>*.

A) Verdadero. B) Falso.

7. ¿Qué elemento dentro de los archivos *compose* permite indicar a partir de qué imagen se quiere crear el contenedor para un servicio?

A) No existe un elemento para ello. B) *container_name* C) *volumes* D) *image*

8. ¿Qué elemento dentro de los archivos *compose* permite especificar un nombre personalizado para el contenedor de un servicio?

A) *volumnes* B) *container_name* C) *image* D) No existe un elemento para ello.

9. Verdadero o falso: el elemento *volumes* sirve para definir el mapeo de puertos entre la máquina *host* y el contenedor.

A) Verdadero. B) Falso.

10. ¿Qué elemento dentro de los archivos *compose* se usa para que un contenedor acceda a archivos que están fuera de él?

A) *image* B) *volumes* C) *volume* D) *images*

11. Verdadero o falso: el elemento *restart* define qué debe hacer Docker cuando un contenedor se detiene.

A) Verdadero. B) Falso.

16.3 Comandos de Docker Compose para recordar



Lectura 54. Comandos esenciales de Docker Compose

Nivel básico

Objetivo: el estudiante identifica los comandos clave para orquestar contenedores de Docker con la herramienta Docker Compose.

Descripción

La lectura lista los principales comandos que se suelen utilizar con Docker Compose para gestionar y orquestar contenedores. Así mismo, se explica en qué consiste cada comando y los principales argumentos que se suelen utilizar en conjunto.

Al igual que todas las herramientas antes vistas, Docker Compose también tiene su propio conjunto de comandos. Tal como es de esperarse, muchos de los comandos son muy similares a los de Docker en cuanto a sintaxis y propósito. De manera general, los comandos más utilizados para gestionar los contenedores y servicios sirven para: crearlos, levantarlos, consultarlos o eliminarlos.

El principal comando de Docker Compose es ***docker compose up***, el cual sirve para compilar, crear, ejecutar y vincular todos los contenedores (servicios) definidos en el archivo *compose.yaml*. Docker maneja automáticamente las dependencias entre los servicios, si es que existen. También es posible alzar un único servicio o contendor, indicando el nombre del servicio ***up <nombre_servicio>*** (Docker Inc, 2024e).

Entre los principales parámetros que se suelen utilizar con el comando ***docker compose up*** se encuentran: ***--detach***, para ejecutar todos los contenedores en segundo plano; ***--no-recreate***, para no volver a crear aquellos contenedores que ya se estén ejecutando; ***--force-recreate***, para forzar la creación de todos los contenedores nuevamente (Docker Inc, 2024e).

Para listar los servicios de Docker Compose que se están ejecutando, se utiliza el comando ***docker compose ls***. Adicionalmente, también se pueden listar los contenedores de Docker Compose que estén detenidos, mediante el argumento ***-all*** (Docker Inc, 2024c).

Una vez que se levantan uno o varios contenedores mediante Docker Compose, es posible detenerlos mediante el comando ***docker compose stop***, y volverlos a iniciar mediante el comando ***docker compose start*** o ***docker compose restart***. A estos tres comandos es posible agregarle el parámetro ***--timeout*** para establecer un lapso específico (Docker Inc, 2024d).

El comando **`docker compose rm`** sirve para eliminar permanentemente todos los contenedores detenidos al alcance del `compose.yaml`. Debido a que es necesario que los contenedores se encuentren detenidos, se puede combinar con parámetro **`--stop`** y **`--volume`**, para eliminar también los volúmenes anónimos relacionados (DockerInc, 2024).

Por otro lado, también es posible utilizar el comando **`docker compose down`** para detener y remover todos los contenedores, redes, volúmenes e imágenes creadas con el comando `up`. Aunque ambos comandos sirven esencialmente para lo mismo, es más común encontrar implementaciones con el comando `down` que el comando `rm` (Docker Inc, 2024a).

Aunque se utilice Docker Compose en lugar Docker solo, los contenedores que se levantan siguen siendo esencialmente contenedores de Docker, por lo que también es posible ejecutar comandos dentro de cada contenedor. Para eso se utiliza el comando **`docker compose exec -it <nombre_servicio> <comando>`**, que es el equivalente a **`docker exec`**, pero enfocado en servicios `compose` (Docker Inc, 2024b).

Finalmente, es importante mencionar que estos no son los únicos comandos disponibles para Docker Compose, ni los únicos parámetros con los que se pueden combinar. Si quieres aprender más al respecto, puedes revisar la [lista oficial de comandos de Docker Compose](#).



Cuestionario 47. Comandos esenciales de Docker Compose

Responde las siguientes preguntas para evaluar la lectura anterior y verificar tus conocimientos respecto a los comandos básicos de Docker Compose.

-
1. ¿Cuál es el comando de Docker Compose que permite lanzar todos los servicios definidos en un archivo `compose.yaml`?
A) `run` B) `up` C) `compose` D) `launch`
 2. Si se quiere ejecutar un conjunto de contenedores en segundo plano, ¿Qué opción se debe agregar al comando?

- A) *--background* B) *--attached* C) *--detach* D) *--no-recreate*

3. Si se quiere ejecutar un único contenedor de un archivo *compose.yaml*, excluyendo los demás, ¿Qué se debe agregar al comando?

- A) El nombre de los servicios excluidos. B) La cantidad y el ID de los servicios a ejecutar. C) La ruta completa donde se ubica el servicio concreto. D) El nombre del servicio a ejecutar.

4. ¿Para qué sirve el comando *docker start*?

- A) Crea y alza los contenedores a partir de un archivo .yaml. B) Inicia contenedores previamente creados. C) Inicia únicamente contenedores que no han sido creado. D) Ninguna de las anteriores.

5. ¿Para qué sirve el comando *docker compose rm*?

- A) Eliminar contenedores. B) Detener todos los contenedores activos. C) Detener y reiniciar los contenedores activos. D) Todas las anteriores.

6. De las siguientes opciones, ¿Cuál es una restricción de utilizar el comando *docker compose rm*?

- A) No debe haber más de 5 servicios activos. B) Los contenedores deben ser independientes unos de otros. C) Solo se puede usar con servicios individuales. D) Los contenedores deben estar detenidos.

7. ¿Cuál es la diferencia esencial entre utilizar el comando *docker compose rm* y *docker compose down*?

R = _____

8. Verdadero o falso: **no** es posible detener varios contenedores a la vez.

A) Verdadero. B) Falso.

9. Verdadero o falso: **no** es posible eliminar varios contenedores a la vez.

A) Verdadero. B) Falso.

10. Verdadero o falso: si se usa el comando *docker compose down*, únicamente se eliminan los contenedores dentro del alcance del archivo *compose.yaml*.

A) Verdadero. B) Falso.

16.4 Uso de Docker Compose en *pipelines* de despliegue



Lectura 55. Integrando Docker Compose con Jenkins

Nivel Intermedio

Objetivo: el estudiante describe los componentes y requerimientos necesarios para integrar Docker Compose en un *pipeline* de Jenkins

Descripción

La lectura explica los requisitos y pasos necesarios para integrar Docker Compose con Jenkins. Así mismo, se explican las instrucciones y directivas que se deben utilizar para poder ejecutar los comandos de Docker Compose dentro de los *pipelines* declarativos.

En términos generales, para utilizar Docker Compose con Jenkins no existe un *plugin* que facilite la integración o permita configurar una instalación. Por ello, es necesario instalar manualmente Docker Compose directamente en el servidor donde se aloja Jenkins. GeeksForGeeks (2025a) indican que esto se logra ejecutando los comandos:

1. **`sudo curl -L`**

`"https://github.com/docker/compose/releases/download/1.29.2/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose`

2. **`sudo chmod +x /usr/local/bin/docker-compose`**

- Además de brindarle a Jenkins los permisos necesarios para operar Docker, mediante el comando **`sudo usermod -aG docker Jenkins`**.

Al utilizar Docker Compose se debe crear un documento `compose.yml` que defina todos los contenedores que se van a crear y levantar. Es importante que este documento se encuentre al alcance de Jenkins. Normalmente se guarda en el mismo repositorio en el que se encuentra el `Jenkinsfile` (CloudBeesTV, 2022).

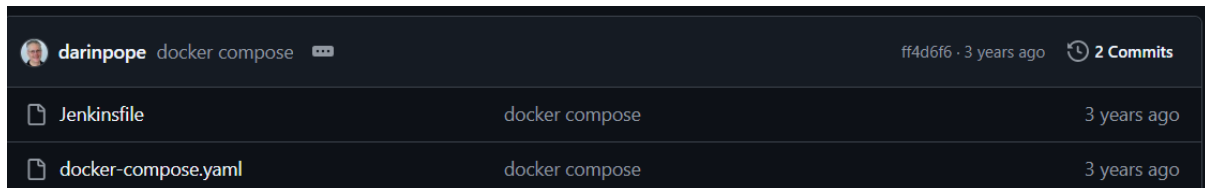


Figura 230. Ejemplo de repositorio con archivos `Jenkinsfile` y `docker-compose.yml` al mismo nivel

Debido a que no existe un *plugin* que facilite el uso de Docker Compose dentro de los *pipelines* de Jenkins, es necesario ejecutar los comandos mediante un *step* de línea de comandos: `sh` en el caso de Linux y `bat` en el caso de Windows. Normalmente estos *step* se colocan dentro de la directiva *steps*, la cual a su vez se forma parte de un bloque *stage*. Por ejemplo, para levantar un conjunto de contenedores, listarlos, ejecutar alguna prueba y posteriormente desmontarlos, se requerirían cuatro *steps sh* distintos, uno para cada acción (GeeksForGeeks, 2025a).

```
stage('Docker Up') {
  steps {
    sh 'docker compose up -d'
    sh 'docker compose ps'
  }
}
stage('Docker Test') {
  steps {
    sh 'docker-compose exec -T wordpress /bin/bash -c "apt-get update && apt-get install -y maven && mvn test"'
  }
}
stage('Docker Down') {
  steps {
    sh 'docker compose down'
  }
}
```

Figura 231. Ejemplo de comandos de Docker Compose ejecutados desde Jenkins

Jenkins no gestiona el ciclo de vida de los contenedores automáticamente, así que si se levanta un contenedor dentro de un *pipeline*, el contenedor continuará activo al finalizar el flujo de trabajo del *pipeline*. En las siguiente y última práctica desplegarás paso a paso una aplicación mediante contenedores.



Cuestionario 48. Integrando Docker Compose con Jenkins

Responde las siguientes preguntas para evaluar la lectura 4 y verificar tus conocimientos respecto a los pasos para integrar Docker Compose con Jenkins.

1. ¿Cuál es el *plugin* indispensable para integrar Docker Compose con los *pipelines* declarativos de Jenkins?

R = _____

2. Durante la ejecución de un *pipeline* declarativo de Jenkins, la consola de salida del *pipeline* muestra el siguiente mensaje: "Got permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock:". ¿Cuál es el problema que está generando este error?

R = _____

3. ¿De qué forma se puede solucionar el problema descrito en la pregunta previa?

R = _____

4. Comúnmente, ¿Dónde debe ubicarse el archivo *docker-compose.yml* para que un *pipeline* de Jenkins lo encuentre sin rutas absolutas? Escribe la ruta relativa que usarías dentro del *pipeline*.

R = _____

5. Un equipo de desarrollo ha generado un *pipeline* declarativo. En uno de los *stages* dedicados a ejecutar los contenedores, el *pipeline* falla y muestra un error que dice que el comando declarado no existe, ¿Cuál puede ser el problema?

- | | | | |
|-----------------------------------|---|---|--|
| A) El usuario de Jenkins no tiene | B) Jenkins no clonó correctamente el repositorio. | C) Docker Compose no está instalado en el servidor. | D) Falta un <i>sleep</i> antes de usar el comando. |
|-----------------------------------|---|---|--|

permisos
suficientes.

6. En un *pipeline* declarativo, ¿Qué *step* es obligatorio para ejecutar comandos de Docker Compose?

A) *dockerStep* B) *script* C) *sh* D) *pipelineDocker*
7. Verdadero o falso: es posible instalar Docker Compose como herramienta desde la sección *Global Tools* en Jenkins, de la misma forma que Maven o NodeJS.

A) Verdadero. B) Falso.
8. Verdadero o falso: para que Jenkins pueda ejecutar Docker Compose, basta con que Docker esté instalado; Docker Compose no necesita instalarse por separado.

A) Verdadero. B) Falso.



Capítulo 17

17 Integración de Docker y Docker Compose en *pipelines* declarativos paso a paso

Contenido del capítulo

- 17.1. Integración de Docker y Docker Compose en *pipelines* declarativos paso a paso.
 - 17.1.1. Prerrequisitos de la práctica.
 - 17.1.2. Preparación del entorno de trabajo.
 - 17.1.3. Creación del Dockerfile del proyecto.
 - 17.1.4. Creación y publicación de la imagen del proyecto.
 - 17.1.5. Usando Docker Compose para levantar un contenedor.
 - 17.1.6. Conexión SSH de Jenkins hacia la máquina anfitriona.

17.1 Integración de Docker y Docker Compose en *pipelines* declarativos paso a paso



Práctica 5. Docker y Docker Compose en un *pipeline* declarativo

Nivel avanzado

Objetivo: el estudiante integra Docker y Docker Compose con Jenkins para el despliegue de software mediante contenedores.

Prerrequisitos: servidor de Jenkins instalado y funcionando correctamente, cuenta activa en GitHub, editor de texto (Recomendación: Visual Studio Code), java instalado en el entorno local, repositorio de GitHub utilizado en la práctica, cuenta activa de DockerHub.

Descripción

La siguiente práctica guía paso a paso al lector para extender el *pipeline* declarativo de Jenkins, que actualmente realiza el *build* y la ejecución de pruebas, para automatizar el despliegue del proyecto mediante contenedores Docker. Se explica paso a paso el uso de las herramientas y los *plugins* que se deben utilizar en Jenkins para integrar Docker y Docker Compose con el *pipeline* declarativo.

En los capítulos previos de esta guía, se han explicado con detalle conceptos clave de Jenkins y de su uso en la creación de *pipelines* de despliegue. Para este punto, el lector ya debería tener los conocimientos necesarios para crear un *pipeline* declarativo en Jenkins e integrarlo con diversas herramientas de automatización. A continuación, en este capítulo se detalla paso a paso cómo utilizar Docker y Docker Compose dentro de los *pipelines* declarativos de Jenkins para crear, publicar, y desplegar un contenedor de manera automatizada.

17.1.1 Prerrequisitos de la práctica

Como con todas las herramientas que se utilizan con Jenkins, es necesario realizar algunas configuraciones e instalaciones previo a integrar una herramienta directamente en un *pipeline*.

En primera instancia, se deben instalar los servicios Docker y Docker Compose directamente en la máquina que aloja al servidor de Jenkins. Aunque es posible delegarle a Jenkins la instalación de Docker, gracias al Docker Plugin, no existe un *plugin* para Docker Compose que permita lo mismo. Para evitar errores de replicación durante la instalación, es preferible instalar manualmente una única vez las herramientas de Docker, y configurar en Jenkins la ruta del ejecutable de Docker (Vladimir Mikhalev, 2025).

Para instalar Docker en un servidor Linux de distribución Ubuntu, se deben ejecutar una serie de comandos en el siguiente orden:

1. **`sudo apt update`** para actualizar la lista local de paquetes disponibles desde los repositorios configurados.
2. **`sudo apt install ca-certificates curl`** para descargar la clave GPG de Docker de forma segura.
3. **`sudo install -m 0755 -d /etc/apt/keyrings`** para crear un directorio con los permisos necesarios para guardar claves GPG de repositorios.
4. **`sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc`** para descargar la clave GPG oficial de Docker y guardarla como un archivo `.asc` en el directorio previamente creado.
5. **`sudo chmod a+r /etc/apt/keyrings/docker.asc`** para dar permisos de lectura a todos los usuarios sobre la clave.
6. **`sudo tee /etc/apt/sources.list.d/docker.sources <<EOF`** para crear un archivo dentro de la lista de fuentes de descarga e ingresar los datos de descarga del repositorio oficial. El comando **`EOF`** inicia un *here-document*, es decir, todo lo que sigue se escribe dentro del archivo.
7. Dentro del archivo se coloca la siguiente información (cada instrucción en un renglón independiente):

Types: deb

URLs: `https://download.docker.com/linux/ubuntu`

Suites: `$(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")`

Components: `stable`

Signed-By: `/etc/apt/keyrings/docker.asc`

Esto, para indicar el tipo de archivo, la URL del repositorio, seleccionar la rama de descarga, y la ubicación de la clave GPG para verificar el repositorio.

8. Luego, se debe ingresar nuevamente el comando **EOF** para finalizar el bloque *here-document*.
9. Posteriormente, hay que actualizar los repositorios nuevamente con el comando **sudo apt update**.
10. Finalmente, con el comando **sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin** se puede instalar Docker y sus componente principales desde el repositorio oficial.
11. Para verificar la instalación se puede ejecutar el comando **sudo docker run hello-world**. Para más información respecto a la instalación, revisa el [tutorial sobre cómo instalar Docker y Docker Compose en Ubuntu](#).
12. Una vez instalado, ejecuta el comando **sudo usermod -aG docker jenkins** para otorgar los permisos necesarios a Jenkins para utilizar el servicio de Docker, y reinicia el servidor de Jenkins.

A pesar de que se va a instalar Docker manualmente, todavía es necesario descargar e instalar los *plugins* necesarios para utilizar Docker dentro de los *pipelines* declarativos de Jenkins. Tal como se explica en el capítulo [plugins en Jenkins para la integración con Docker](#), los *plugins* esenciales para utilizar Docker con Jenkins son Docker Plugin, Docker pipeline Plugin y CloudBees Docker Build and Publish Plugin (Vladimir Mikhalev, 2025).

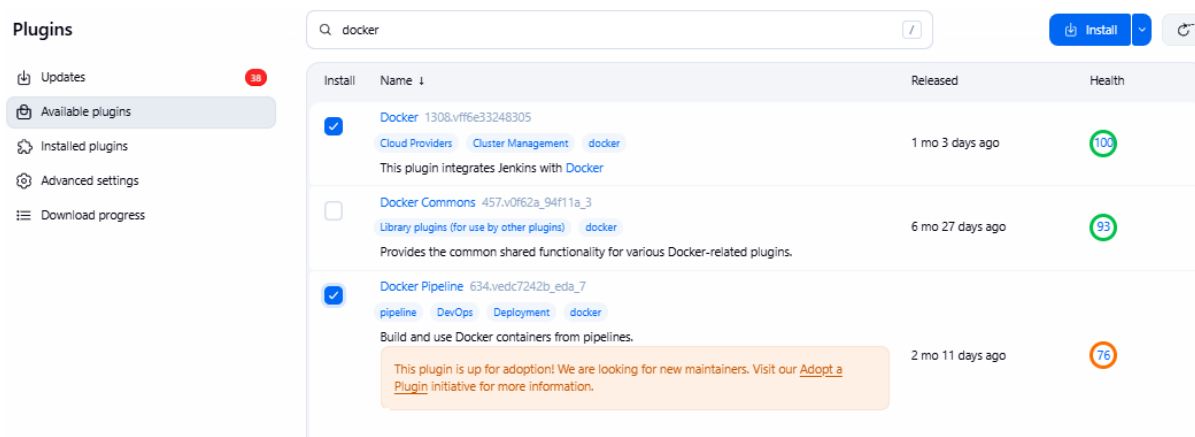


Figura 232. Pantalla de gestión de plugins mostrando aquellos asociados a Docker

Además, debido a que esta práctica es una continuación directa de las cuatro prácticas previas de esta guía, es recomendable haberlas realizado antes de empezar con este capítulo. En las prácticas previas se desarrolló un *pipeline* declarativo para automatizar la construcción y prueba de software de un proyecto en Java, por lo que es importante que el lector cuente con el Jenkinsfile correspondiente y un *fork* del repositorio de esta guía (en esta práctica se utiliza específicamente la rama [deployment-pipeline](#)).

17.1.2 Preparación del entorno de trabajo

Una vez se encuentran Docker y Docker Compose correctamente instalado en el servidor que hospeda Jenkins (tu máquina virtual), y se han descargado los *plugins* necesarios, es momento de configurar el acceso a la herramienta. En la sección *Tools* de Jenkins hay que buscar el apartado *Docker* e ingresar la ruta (*path*) del ejecutable de Docker (puedes consultarlo por línea de comandos con el comando **which docker**), y guardar los cambios haciendo clic en el botón Save (CloudBeesTV, 2020).

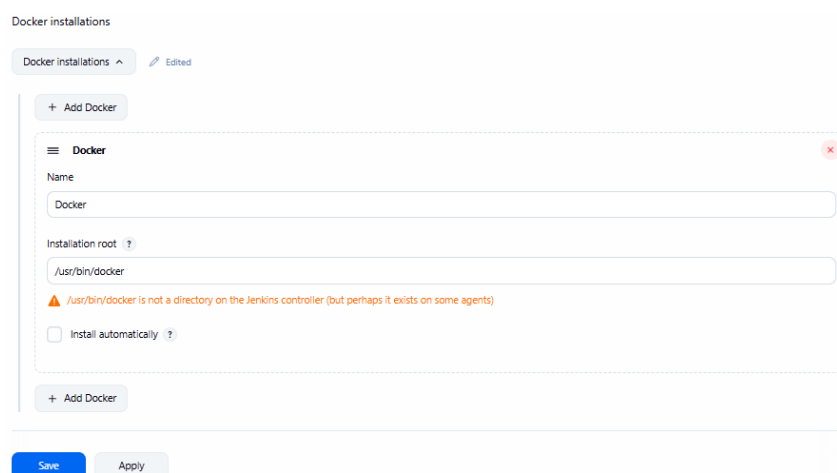
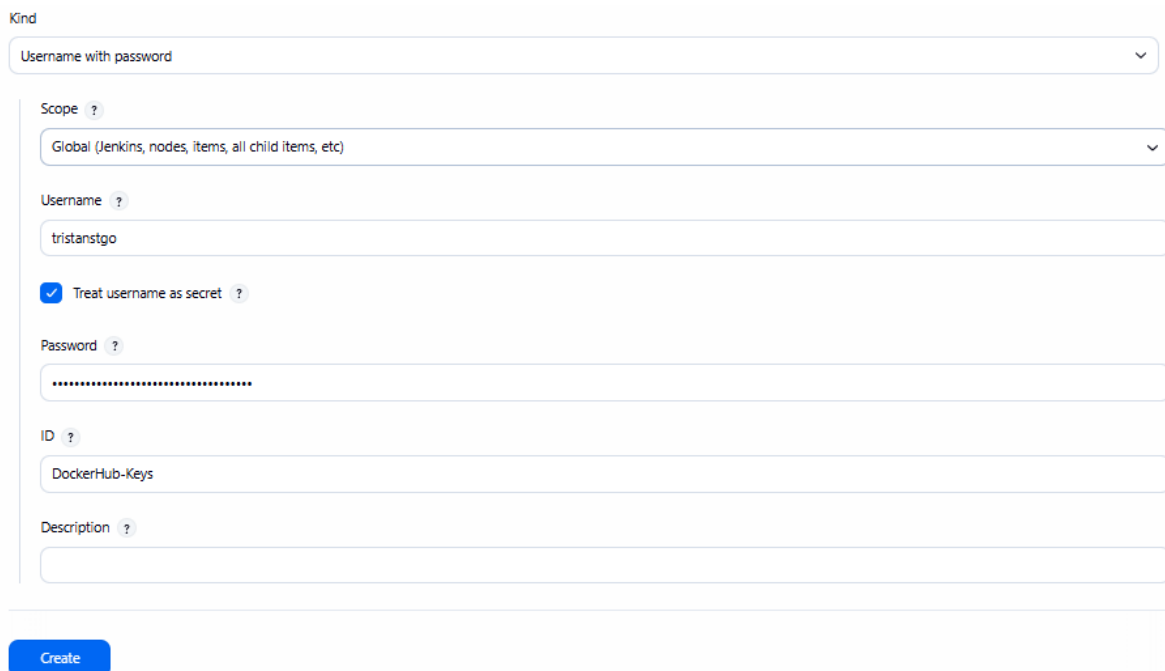


Figura 233. Pantalla de configuración de tools donde se muestra la instalación de Docker, apuntando hacia la instalación local en el servidor

Una parte del *pipeline* que construiremos en unos momentos subirá la imagen generada del contenedor a DockerHub, por lo que es necesario dar de alta las credenciales de usuario de DockerHub en Jenkins. Si tienes dudas respecto a cómo realizar esto, puedes revisar el capítulo [Uso de Credenciales en Jenkins](#) (CloudBeesTV, 2021a).

Dentro del formulario de registro, selecciona las credenciales de tipo *Username with Password*, deja el alcance global por defecto y captura tu nombre de usuario y contraseña. Opcionalmente, también puedes utilizar un token de acceso en lugar de tu contraseña, estos se pueden generar el apartado “Configuración de cuenta” de DockerHub. Si necesitas ayuda para generar un token, revisa el siguiente material: [Tutorial-Generar token de acceso](#) (CloudBeesTV, 2021a).



The screenshot shows the 'Add Credentials' form in Jenkins. The 'Kind' dropdown is set to 'Username with password'. The 'Scope' dropdown is set to 'Global (Jenkins, nodes, items, all child items, etc)'. The 'Username' field contains 'tristanstgo'. The 'Treat username as secret' checkbox is checked. The 'Password' field is masked with dots. The 'ID' field contains 'DockerHub-Keys'. The 'Description' field is empty. A blue 'Create' button is at the bottom left.

Figura 234. Pantalla de configuración de credenciales donde se muestra el ejemplo con DockerHub

17.1.3 Creación del Dockerfile del proyecto

Actualmente, el *pipeline* de Jenkins compila el proyecto, y realiza pruebas unitarias y de integración, por lo que no es necesario compilar ni probar el proyecto nuevamente dentro de Docker. Esto simplifica el Dockerfile, ya que no se necesita copiar todo el proyecto, solo el archivo JAR. Para este proyecto, únicamente se requiere una imagen con Java, establecer un directorio de trabajo, copiar el archivo JAR generado por Jenkins y ejecutarlo dentro del contenedor. Estas instrucciones se pueden consultar en la siguiente figura.

```
1 FROM eclipse-temurin:21-jre
2
3 WORKDIR /app
4
5 # Copiamos el JAR ya compilado por Jenkins
6 COPY target/todolist-1.0.0.jar app.jar
7
8 # Puerto de la aplicación
9 EXPOSE 8081
10
11 # Ejecutar la app
12 ENTRYPOINT ["java", "-jar", "app.jar"]
```

Figura 235. Configuración necesaria para que el Dockerfile copie la aplicación al contenedor

Adicionalmente, se recomienda agregar un archivo `.dockerignore` para evitar copiar basura dentro del contenedor y hacerlo lo más ligero posible. Tal como se muestra en la siguiente figura.

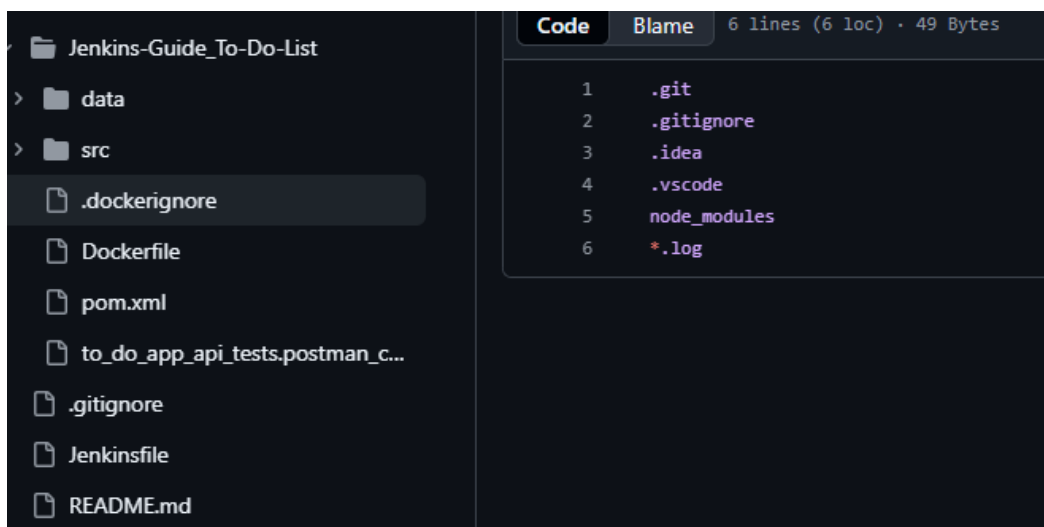
The image shows a code editor interface. On the left, a file explorer shows a project structure with folders 'data' and 'src', and files like '.dockerignore', 'Dockerfile', 'pom.xml', 'to_do_app_api_tests.postman_c...', '.gitignore', 'Jenkinsfile', and 'README.md'. The '.dockerignore' file is selected. On the right, the code editor shows the content of '.dockerignore' with 6 lines: 1. '.git', 2. '.gitignore', 3. '.idea', 4. '.vscode', 5. 'node_modules', and 6. '*.log'. The editor has tabs for 'Code' and 'Blame', and a status bar indicating '6 lines (6 loc) · 49 Bytes'.

Figura 236. Ejemplo del archivo `.dockerignore`

En este caso, puede no hacerlo. Sin embargo, se considera una buena práctica agregar este archivo. Para nuestro caso específico, este archivo se puede ignorar y no agregarlo al proyecto, pues estamos copiando únicamente el archivo `.jar` hacia el contenedor. Si hubiéramos copiado una carpeta completa, ahí cobraría sentido el archivo `.dockerignore`, para descartar todos aquellos archivos no necesarios.

17.1.4 Creación y publicación de la imagen del proyecto

La última versión funcional del Jenkinsfile de esta guía es la utilizada en la práctica anterior: [Práctica 3 - Integración de herramientas de prueba en pipelines declarativos](#).

Hasta ese punto, el Jenkinsfile cuenta con cinco etapas: *checkout*, *Build*, *Unit Test*, *Integration Test* y una directiva *Post* para publicar los resultados de las pruebas. Solo queda pendiente realizar la fase de *Deploy*, la cual se aborda a continuación.

En primera instancia, hay que realizar las modificaciones necesarias para que el archivo JAR del proyecto se genere justo después de la ejecución de pruebas. En el stage *"Build"*, se recomienda cambiar el comando ***install*** por ***compile*** (se había utilizado ***install*** previamente para que el lector pudiera verificar el funcionamiento de la API).

```
19     stage('Build') {
20         steps {
21             sh 'mvn -f Jenkins-Guide_To-Do-List/pom.xml clean compile'
22             echo 'Compilación finalizada!!'
23         }
24     }
```

Figura 237. Código del Jenkinsfile con el cambio aplicado al stage *Build* para cambiar el comando *"install"* por *"compile"*

Con este cambio, Maven ya no ejecutara las etapas de validación, pruebas, empaquetado y verificación de manera implícita, ya que estas se realizan de manera explícita en *stages* posteriores del Jenkinsfile. Adicionalmente, hay que agregar otro stage que ejecute el comando ***mvn -f Jenkins-Guide_To-Do-List/pom.xml package -DskipTests*** para generar el archivo JAR, que será usado por Docker para crear la imagen del proyecto. El empaquetado debe ocurrir justo después de las pruebas de integración y antes de la creación de la imagen.

```
56     stage('Package') {
57         steps {
58             sh 'mvn -f Jenkins-Guide_To-Do-List/pom.xml package -DskipTests'
59             echo 'JAR generado correctamente!!'
60         }
61     }
```

Figura 238. Código del Jenkinsfile para agregar el stage de empaquetado de la aplicación Java

Después de crear el archivo JAR, se debe utilizar el Dockerfile del repositorio para generar la imagen del proyecto mediante la instrucción ***def dockerImage = docker.build(("usuario_DockerHub/todolist-app"))***, dentro de una directiva *script* en un stage nuevo. Posteriormente, se puede usar la instrucción ***docker.withRegistry('https://index.docker.io/v1/', 'docker_credentials') { dockerImage.push() }*** para subir la imagen recién generada a DockerHub. Adicionalmente, se recomienda agregar la

directiva *dir* para indicar a Jenkins en que carpeta se ubica el Dockerfile (Vladimir Mikhalev, 2025). En la siguiente figura se muestra cómo se encuentra estructurada la etapa de creación de la imagen con Docker en el Jenkinsfile, según lo descrito previamente.

```
64     stage('Create Docker image'){
65         steps {
66             script {
67                 dir('Jenkins-Guide_To-Do-List') {
68                     def dockerImage = docker.build("tristanstgo/todolist-app")
69                     docker.withRegistry('https://index.docker.io/v1/', 'docker_credential') {
70                         dockerImage.push() }
71                 }
72             }
73         }
74     }
75     stage('Deploy') {
```

Figura 239. Código del Jenkinsfile para crear y publicar la imagen de docker

Es importante que escribas correctamente tu nombre de usuario de DockerHub al crear la imagen con la función ***docker.build(...)*** y al ingresar las credenciales de autenticación en la función ***docker.withRegistry(...)***, ya que estos serán los datos que Jenkins utilizará para publicar la imagen en tu repositorio de DockerHub. Además, que cambies *'docker_credential'* por el id de tus credenciales de DockerHub previamente registradas.

17.1.5 Usando Docker Compose para levantar un contendor

Para este punto es posible descargar la imagen y levantar un contenedor con únicamente funciones de Docker. Sin embargo, debido a que parte del objetivo de este capítulo es integrar Docker Compose con Jenkins, a continuación se explica paso a paso qué instrucciones son necesarias para levantar un contendor con un archivo *docker-compose.yml*.

Como primer paso, se debe crear un archivo *docker-compose.yml* dentro de */Jenkins-Guide_To-Do-List*. Este archivo tendrá las instrucciones necesarias para descargar la imagen del proyecto desde DockerHub, asignar un puerto de exposición, montar un volumen de persistencia de datos y levantar el contendor correspondiente. Las instrucciones necesarias se muestran en la siguiente figura.

```
1  version: "3.8"
2
3  services:
4    todoist:
5      image: tristanstgo/todolist-app
6      container_name: todolist-container
7      ports:
8        - "8081:8081"
9      volumes:
10       - ./data:/app/data
11      restart: unless-stopped
```

Figura 240. Código del archivo `docker-compose.yml` para levantar la app con Docker Compose

Posteriormente, se debe modificar nuevamente el `Jenkinsfile`. Pero ahora, en lugar de utilizar las funciones del *plugin* de Docker, se deben utilizar *steps* de tipo `sh` para ejecutar comandos nativos de Docker Compose. Dentro una bloque `dir()`, para indicar la ruta del archivo `.yml`, hay que agregar el comando `sh 'docker compose up -d'` para levantar el contenedor en segundo plano. Después, esperar alrededor de 15 segundos y verificar el estatus del contenedor mediante el comando `sh 'docker compose ps'`, el cual mostrará todos los contenedores activos (CloudBeesTV, 2022).

```
77  stage('Deploy') {
78    steps {
79      dir('Jenkins-Guide_To-Do-List'){
80        sh 'docker compose up -d'
81        sleep 15
82        sh 'docker compose ps'
83      }
84    }
85    echo 'Aplicación desplegada en la dirección localhost:8081'
86  }
87 }
88 }
```

Figura 241. Código del `Jenkinsfile` con el stage de Deploy

Si el resultado del comando `sh 'docker compose ps'` es nulo o vacío, significa que ocurrió un error, esto se puede verificar en la propia salida de la consola de Jenkins al ejecutar el *pipeline*. En la siguiente figura se muestra una salida de consola exitosa al ejecutar el *pipeline*.

```
[Pipeline] sleep
Sleeping for 15 sec
[Pipeline] sh
+ docker compose ps
time="2026-01-08T19:44:11-06:00" level=warning msg="/var/lib/jenkins/workspace/pipeline-tutorial/Jenkins-Guide_To-Do-List/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME                IMAGE                                COMMAND                                SERVICE  CREATED        STATUS        PORTS
todolist-container  tristanstgo/todolist-app            "java -jar app.jar"                  todolist  15 seconds ago Up 15 seconds  0.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp
```

Figura 242. Ejecución exitosa en consola tras ejecutar el pipeline con el stage de Deploy



Nota - Limpiar tu lugar de trabajo es buena práctica

Al finalizar cada ejecución, Jenkins guarda diversos archivos en el *workspace* que se van generando durante la ejecución del *pipeline*. Es recomendable limpiar este espacio de trabajo para liberar espacio y evitar problemas en futuras ejecuciones.

Para limpiar el *workspace* de un *pipeline*, se puede utilizar el *plugin* [WorkSpace CleanUp](#). Este *plugin* proporciona el step `cleanWs()`, que permite limpiar el espacio de trabajo en un *pipeline* automáticamente. Simplemente hay que descargar el *plugin* y, dentro del bloque `post{}` y `always{}`, agregar este paso antes de finalizar la ejecución (CD Foundation, 2024b).

El utilizar esta instrucción por sí sola supone un problema, ya que limpia todo el *workspace* indiscriminadamente. Sin embargo, se pueden utilizar patrones para excluir algunos archivos que se desean conservar, como los reportes de pruebas o el `.jar` del proyecto (Amr Essam, 2024). En la siguiente figura se muestra cómo utilizar estos patrones para conservar los reportes de pruebas unitarias y de integración que genera el *pipeline*.

```
110     cleanWs(
111         patterns: [
112             [pattern: "**/target/surefire-reports/**", type: "EXCLUDE"],
113             [pattern: "**/target/**/*.xml", type: "EXCLUDE"],
114             [pattern: "**/target/**/*.html", type: "EXCLUDE"],
115             [pattern: "**/target/*.jar", type: "EXCLUDE"]
116         ]
117     )
118
119     echo 'Finalizando ejecución del pipeline!'
```

Figura 243. Ejemplo de llamada a `cleanWs` en pipeline de Jenkins, manteniendo ciertos archivos de resultados.

Finalmente, antes de poder ejecutar el *pipeline* nuevamente, cambia el campo *Branch Specifier* dentro de la configuración del *pipeline*. Aquí debes colocar la rama sobre la que has estado realizando los cambios que se han venido describiendo hasta ahora.

Recuerda, esta configuración del *pipeline* ya se ha venido explicado en los capítulos previos (prácticas anteriores), por lo que ya deberías tener configurado este *pipeline* y solo deberías cambiar la referencia a la rama sobre la que trabajará.

En nuestro caso, la rama en la que se encuentra el proyecto actualizado, con los archivos de Docker nuevos (Dockerfile y docker-compose.yml) y el Jenkinsfile actualizado, lleva por nombre *deployment-pipeline*.

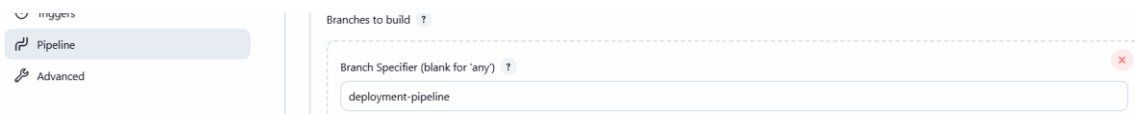


Figura 244. Cambio de la configuración del *pipeline* para asignar la rama con los nuevos cambios

Hasta este punto se han automatizado todas las etapas primarias del *pipeline* de despliegue, integrando herramientas de construcción, prueba y despliegue de software. Así mismo, puedes ejecutar el *pipeline* sin ningún problema e ingresar desde el navegador de tu máquina física a la aplicación desplegada, ingresando a la dirección `<IP de tú máquina virtual>:8081`. Sin embargo, el contenedor se está desplegando de manera local dentro del mismo servidor donde se encuentra Jenkins. Esto no suele ser lo más común, un escenario más real sería desplegar el contenedor en otro servidor dedicado.



Nota - No todo dura para siempre

Debes tener muy en cuenta que desplegar servicios de forma indefinida dentro de un *pipeline* no es una práctica recomendada. Contenedores (u otros servicios) ejecutándose dentro del *pipeline* de forma indefinida, como se mostró hace un momento, no debería realizarse. Como práctica recomendada por los expertos, *pipelines* deben ejecutar tareas con un inicio y fin bien definido.

Así pues, como parte final de la práctica, se realizará una conexión SSH desde Jenkins (tú máquina virtual Ubuntu) hacia la máquina anfitriona (tú máquina física Windows) para desplegar la aplicación.

17.1.6 Conexión SSH de Jenkins hacia la máquina anfitriona

Como primer requisito, vamos a crear un usuario local en la máquina física para que Jenkins se pueda conectar. Para crear un usuario, abre una terminal de PowerShell como administrador y ejecuta los siguientes comandos, tal como lo menciona Elizabet J (2025):

1. **`net user jenkins * /add`** para crear el usuario. Asigna una contraseña segura, pero que puedas recordar.
2. Ejecuta el comando **`net localgroup Administradores jenkins /add`** para asignar permisos de administrador al nuevo usuario.
3. Finalmente, ejecuta el comando **`net localgroup docker-users jenkins /add`** para agregarlo al grupo docker y que pueda usar el servicio.

Es importante que recuerdes el usuario y contraseña nuevos, porque hay que darlos de alta en la credenciales de Jenkins.

El siguiente paso es dar de alta el usuario y contraseña en Jenkins para la conexión SSH. Tal como se ha realizado previamente con otras credenciales, en la sección *Credentials*, hay que agregar una nueva credencial de tipo *Username with password*. En el campo *Username* ingresa el usuario **jenkins** y en el campo *password*, la **contraseña que asignaste** y el ID **ssh_credentials** (preferentemente) para referenciarlo en el Jenkinsfile.

Además, hay que instalar en Jenkins el *plugin* [SSH Pipeline Steps](#). Este *plugin* facilita la conexión de Jenkins mediante SSH y la ejecución de comandos remotos (CD Foundation, 2026).

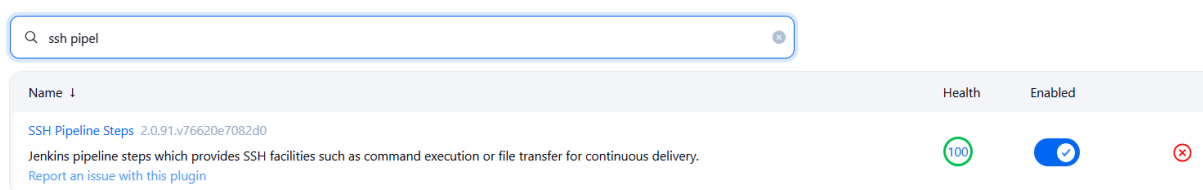
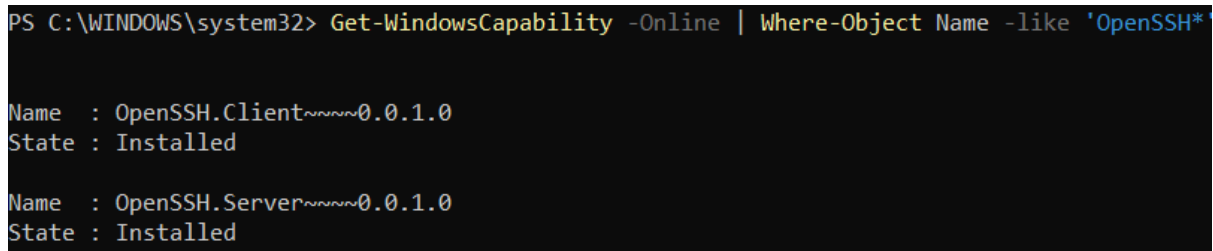


Figura 245. Captura de pantalla de ventana de configuración de pipelines, donde se muestra el plugin SSH Pipeline Steps

Por otra parte, regresando a tu máquina física, es momento de cambiar temporalmente al nuevo usuario en tu máquina anfitriona. Inicia sesión con el usuario *jenkins* y verifica que este tenga un servidor SSH activo. En Windows normalmente viene instalado OpenSSH Server por defecto; puedes verificar si está instalado ejecutando el comando **`Get-WindowsCapability -Online | Where-Object Name -like`**

'**OpenSSH.Server***' dentro de una terminal PowerShell, o instalarlo directamente ejecutando el comando **Add-WindowsCapability -Online -Name OpenSSH.Server~~~~0.0.1.0**. Finalmente, para iniciar el servicio se debe ejecutar el comando **Start-Service sshd** (Microsoft, 2025). Si tienes dudas de como instalar OpenSSH en Windows puedes revisar el siguiente material: [Instalar OpenSSH Windows](#).



```
PS C:\WINDOWS\system32> Get-WindowsCapability -Online | Where-Object Name -like 'OpenSSH*'

Name : OpenSSH.Client~~~~0.0.1.0
State : Installed

Name : OpenSSH.Server~~~~0.0.1.0
State : Installed
```

Figura 246. Comando en Windows para verificar la instalación del servicio SSH

Es posible que el *firewall* de Windows bloquee las conexiones SSH externas, así que es necesario crear una regla para permitir la conexión. En una terminal de PowerShell ejecuta el siguiente comando:

```
New-NetFirewallRule `
-Name sshd-inbound `
-DisplayName "OpenSSH Server (Inbound)" `
-Enabled True `
-Direction Inbound `
-Protocol TCP `
-Action Allow `
-LocalPort 22
```

El siguiente paso es verificar que el usuario nuevo tenga acceso al servicio Docker mediante el comando **docker ps**, o abriendo la aplicación Docker Desktop. En caso de que exista un error, verifica que sí hayas agregado el usuario *jenkins* al grupo Docker para que pueda usar el servicio. En caso de que no tengas Docker instalado todavía, debes ir a la página de descarga oficial de [Docker Desktop](#), seleccionar el instalador de Windows y ejecutarlo.

Además, es posible que sea necesario editar el archivo de configuración de Docker **C:\Users\jenkins\.docker\config.json** y eliminar el campo **"credsStore": "desktop"**. En algunos casos este campo provocara que Jenkins no pueda vincularse con Docker para ejecutar los comandos de Docker Compose (Oscar Zhou, 2025).

Por último, crea una carpeta en la ruta **C:/deploy/todolist**. En esta ruta se va a descargar el archivo YAML para que lo lea Docker Compose. Una vez verificados estos requisitos, puedes volver a cambiar a tu usuario principal para continuar editando el Jenkinsfile.

Ahora es momento de volver a editar el Jenkinsfile. Para utilizar las funcionalidades del *plugin* [SSH Pipeline Step](#), se usa una variable llamada **remote**, la cual mapea las configuraciones de conexión como el usuario, contraseña, dominio entre otras. Dentro de la etapa de *Deploy*, hay que instanciar esta variable con la instrucción **def remote=**, posteriormente se le asignará un nombre y un *host* (dirección de tu máquina física, no de la máquina virtual) (CD Foundation, 2026).

```
82     steps {
83         script {
84             def remote = [
85                 name: 'SSHWin',
86                 host: '192.168.1.71',
87                 allowAnyHosts: true
88             ]
```

Figura 247. Configuración relacionada al comando *remote* en el Jenkinsfile

Como se puede observar, todavía no se asigna un usuario y contraseña a la conexión. Para esto hay que consultar las credenciales de *ssh* previamente registradas. Para acceder al nombre de usuario y contraseña por separado, se debe usar una variable de entorno como intermediario. Dentro de un bloque *environment* instancia una variable con el valor de las credenciales *ssh* (CloudBeesTV, 2024a).

```
77     stage('Deploy') {
78         environment {
79             SSH_KEYS = credentials('ssh_credentials')
80         }
```

Figura 248. Asignación de credenciales *SSH* a variable de entorno de Jenkins

A través de esta nueva variable de entorno es posible acceder al usuario y contraseña por separado mediante los campos **USR** y **PSW**, y asignarlos a la variable **remote** (CloudBeesTV, 2024a).

```

82     steps {
83         script {
84             def remote = [
85                 name: 'SSHWin',
86                 host: '192.168.1.71',
87                 allowAnyHosts: true
88             ]
89             remote.user=env.SSH_KEYS_USR
90             remote.password=env.SSH_KEYS_PSW

```

Figura 249. Asignación de usuario y contraseña SSH en Jenkinsfile

Finalmente, hay que utilizar la función `sshCommand()` para ejecutar los comandos **`docker compose up -d`** y **`docker compose ps`**. Adicionalmente, se agrega un comando para ir a una carpeta en específico de Windows y descargar el archivo `docker-compose.yml` desde el repositorio de GitHub mediante una llamada `curl`. Los comandos deben ir seguidos, separados por `&&` para que se ejecuten correctamente. Según CD Foundation (2026), la instrucción final luce así:

```

sshCommand (remote: remote, command:"cd /d C:\\deploy\\todolist && curl -L -o
docker-compose.yml https://raw.githubusercontent.com/adjcg15/jenkins-
guide/deployment-pipeline/Jenkins-Guide_To-Do-List/docker-compose.yml &&
docker compose up -d && docker compose ps")

```

Sin espacios ni saltos de línea. La etapa de *Deploy* de tu Jenkinsfile debería lucir similar a lo que se muestra en la figura a continuación.

```

78 stage('Deploy') {
79     environment {
80         SSH_KEYS = credentials('ssh_credentials')
81     }
82     steps {
83         script {
84             def remote = [
85                 name: 'SSHWin',
86                 host: '192.168.1.71',
87                 allowAnyHosts: true
88             ]
89             remote.user=env.SSH_KEYS_USR
90             remote.password=env.SSH_KEYS_PSW
91             sshCommand (remote: remote, command:"cd /d C:\\deploy\\todolist && curl -L -o docker-compose.yml https://raw.githubusercontent.com/adjcg15/
jenkins-guide/deployment-pipeline/Jenkins-Guide_To-Do-List/docker-compose.yml && docker compose up -d && docker compose ps")
92         }
93         echo 'Aplicación desplegada en la dirección localhost:8081'
94     }
95 }
96

```

Figura 250. Código final de stage Deploy en Jenkinsfile

Recuerda cambiar la dirección host a la IP de tu computadora. Guarda los cambios realizados y vuelve a ejecutar el *pipeline*, ahora deberías de poder acceder a la

aplicación a través de tu máquina física Windows, ingresando a la dirección localhost:8081. En futuras ejecuciones, verifica que el servicio *sshd* de Windows y el servicio de Docker están corriendo correctamente antes de ejecutar el *pipeline*.

Esta quinta y última práctica también marca el fin de esta guía. Esperamos que cada uno de los capítulos haya sido de ayuda para el lector durante su aprendizaje en el uso de Jenkins, para conocer las capacidades y características de la herramienta, y para crear *pipelines* de despliegue completos, integrando diferentes tecnologías de desarrollo de software.



Rúbrica 5. Evaluación Práctica “Docker y Docker-Compose en un *pipeline* declarativo”

En la siguiente rúbrica, marca “Sí” o “No” en cada aspecto de evaluación para determinar si tu implementación cumple con los criterios requeridos en la integración de Docker y Docker-Compose en *pipelines* declarativos para desplegar un proyecto en Java mediante contenedores de manera automatizada.

Rubrica de Evaluación		
Aspecto por evaluar	Registro de Cumplimiento	
	Sí	No
Revisé que cumpliera con los prerrequisitos de la práctica antes de comenzar a realizarla.		
Instalé los plugin recomendados para la práctica: Docker Plugin, Docker Pipeline Plugin, CloudBees Docker Build and Publish Plugin y SSH Pipeline Step Plugin.		
Registré mi nombre de usuario y contraseña (o token de autenticación) de DockerHub en la sección <i>Credentials</i> de Jenkins.		

Configuré una instalación de Docker en mi máquina virtual que hospeda Jenkins, así como en mi máquina física .		
Utilicé las funciones del plugin de Docker para crear una imagen y subirla a DockerHub.		
Al ejecutar el <i>pipeline</i> verifiqué que se genera una imagen del proyecto correctamente.		
Al ejecutar el <i>pipeline</i> verifiqué que la imagen generada se sube correctamente a mi repositorio de DockerHub .		
Utilicé comandos nativos de Docker Compose para crear y levantar un contenedor utilizando el archivo .yml del repositorio y la imagen publicada en DockerHub.		
Al desplegar el contenedor de manera local en la máquina virtual, logré acceder a la aplicación a través de la dirección [IP de tu máquina virtual]:8081 .		
Creé un usuario local nuevo de Windows para conectarme por SSH en mi máquina física.		
Habilite el servicio OpenSSH-Server de Windows.		
Creé una carpeta nueva en la ruta C:/deploy/todolist para descargar el archivo YAML de Docker Compose.		
Modifiqué la variable remote para que el host objetivo fuera mi máquina física.		
Al ejecutar el <i>pipeline</i> verifiqué que se descargaba correctamente el archivo YAML en la ruta C:/deploy/todolist.		
Al ejecutar el <i>pipeline</i> verifiqué que se crea y levanta un contenedor con la aplicación en la dirección localhost:8081 .		



Capítulo 18

18 Respuestas de cuestionarios

Contenido del capítulo

- 18.1. Respuestas para el capítulo 2
- 18.2. Respuestas para el capítulo 3
- 18.3. Respuestas para el capítulo 4
- 18.4. Respuestas para el capítulo 5
- 18.5. Respuestas para el capítulo 6
- 18.6. Respuestas para el capítulo 7
- 18.7. Respuestas para el capítulo 8
- 18.8. Respuestas para el capítulo 9
- 18.9. Respuestas para el capítulo 10
- 18.10. Respuestas para el capítulo 11
- 18.11. Respuestas para el capítulo 12
- 18.12. Respuestas para el capítulo 13
- 18.13. Respuestas para el capítulo 14
- 18.14. Respuestas para el capítulo 15
- 18.15. Respuestas para el capítulo 16
- 18.16. Respuestas para el capítulo 17

18.1 Respuestas para el capítulo 2

Cuestionario 1

- | | | |
|-------------------------------|-----------|--------|
| 1 → B | 5 → C | 9 → B |
| 2 → C | 6 → D | 10 → D |
| 3 → C | 7 → A | 11 → C |
| 4 → GitHub, JUnit y Terraform | 8 → Falso | |

18.2 Respuestas para el capítulo 3

Cuestionario 2

- | | | |
|-----------|---------------|---------------|
| 1 → C | 4 → D | 7 → Verdadero |
| 2 → Falso | 5 → B, C, A | 8 → Falso |
| 3 → A | 6 → Verdadero | |

18.3 Respuestas para el capítulo 4

Cuestionario 3

- | | | |
|---------------|-----------|-------|
| 1 → B | 3 → Falso | 5 → D |
| 2 → Verdadero | 4 → D | |

Cuestionario 4

- | | | |
|-------|-------|-------|
| 1 → B | 3 → A | 5 → A |
| 2 → D | 4 → A | |

Cuestionario 5

- | | | |
|-------|-----------|----------------|
| 1 → D | 5 → C | 9 → Verdadero |
| 2 → D | 6 → Falso | 10 → C |
| 3 → A | 7 → B | 11 → Verdadero |
| 4 → A | 8 → D | 12 → B |

Cuestionario 6

- | | | |
|-----------|-------------------------------------|----------------|
| 1 → C | 5 → A | 8 → Falso |
| 2 → B | 6 → Interfaz gráfica y Jenkinsfiles | 9 → Falso |
| 3 → Falso | | 10 → Verdadero |
| 4 → Falso | 7 → C | |

Cuestionario 7

- | | | |
|-----------------------------------|---|--|
| 1 → <i>Scripted</i> y declarativo | 5 → B | 8 → Declarativo porque son más fáciles de comprender |
| 2 → Verdadero | 6 → A (3,4,5,7,8,10,11); B (1,2,6,9,12) | 9 → B |
| 3 → Falso | 7 → Falso | 10 → B |
| 4 → A | | |

Cuestionario 8

- | | | |
|------------------------|---------------|---|
| 1 → <i>pipeline {}</i> | 5 → C | 9 → Falso |
| 2 → <i>post {}</i> | 6 → B | 10 → Bloques, secciones, directivas y condicionales |
| 3 → B | 7 → Falso | |
| 4 → C | 8 → Verdadero | |

Cuestionario 9

- | | | |
|---------------|-------------------------------|--|
| 1 → Falso | 6 → A | 11 → <i>post</i> |
| 2 → Falso | 7 → D | 12 → <i>options</i> |
| 3 → Verdadero | 8 → B | 13 → <i>pipeline, agent, stages y step</i> |
| 4 → Verdadero | 9 → B | 14 → <i>environment, tools y post.</i> |
| 5 → B | 10 → <i>triggers con Cron</i> | |

18.4 Respuestas para el capítulo 5

Cuestionario 10

- | | | |
|---------------|-------|-------|
| 1 → A | 4 → D | 7 → B |
| 2 → Verdadero | 5 → A | |
| 3 → B | 6 → C | |

Cuestionario 11

- | | | |
|-------|---------------|-----------|
| 1 → B | 4 → Falso | 7 → Falso |
| 2 → C | 5 → C | |
| 3 → D | 6 → Verdadero | |

18.5 Respuestas para el capítulo 6

Cuestionario 12

- | | | |
|---------------|---------------|--------|
| 1 → A | 5 → Verdadero | 9 → C |
| 2 → Falso | 6 → D | 10 → A |
| 3 → Verdadero | 7 → B | |
| 4 → Verdadero | 8 → C | |

Cuestionario 13

- | | | |
|---------------|---------------|----------------|
| 1 → Verdadero | 5 → Verdadero | 9 → Falso |
| 2 → D | 6 → C | 10 → Verdadero |
| 3 → Falso | 7 → A | 11 → |
| 4 → B | 8 → D | 12 → |

18.6 Respuestas para el capítulo 7

Cuestionario 14

- | | | |
|---------------|-------|-----------|
| 1 → C | 3 → A | 5 → Falso |
| 2 → Verdadero | 4 → D | |

Cuestionario 15

- | | | |
|---------------|-----------|--------|
| 1 → D | 5 → B | 9 → C |
| 2 → Verdadero | 6 → Falso | 10 → B |
| 3 → D | 7 → D | |
| 4 → C | 8 → A | |

18.7 Respuestas para el capítulo 8

Cuestionario 16

- | | | |
|-------|-------|-----------|
| 1 → C | 4 → B | 7 → Falso |
|-------|-------|-----------|

2 → C
3 → D

5 → Verdadero
6 → Falso

8 → Verdadero

Cuestionario 17

1 → B
2 → C
3 → D
4 → B

5 → B
6 → Falso
7 → Verdadero
8 → Verdadero

9 → CLI e Interfaz Web (GUI)
10 → Es cuando un plugin depende de otro para funcionar. No se puede eliminar un plugin del cual dependen otros

Cuestionario 18

1 → C
2 → Falso

3 → Falso
4 → Verdadero

5 → A
6 → D

18.8 Respuestas para el capítulo 9**Cuestionario 19**

1 → Falso
2 → Falso
3 → D

4 → C
5 → D
6 → Falso

7 → C
8 → A

Cuestionario 20

1 → B
2 → D
3 → C
4 → B

5 → B
6 → B
7 → B
8 → Verdadero

9 → Falso
10 → Falso

18.9 Respuestas para el capítulo 10**Cuestionario 21**

1 → Falso
2 → B
3 → C

4 → Verdadero
5 → Verdadero
6 → Falso

7 → Falso
8 → D

Cuestionario 22

1 → A
2 → B
3 → B
4 → A

5 → B
6 → C
7 → Verdadero
8 → Falso

9 → Falso
10 → Falso

Cuestionario 23

1 → B
2 → C
3 → D

4 → C
5 → B
6 → Verdadero

7 → Falso
8 → Falso

Cuestionario 24

1 → C
2 → D
3 → D

5 → D
6 → A
7 → C

9 → B
10 → Falso

4 → B

8 → C

Cuestionario 25

1 → C

3 → Falso

5 → Verdadero

2 → A

4 → Falso

6 → Falso

18.10 Respuestas para el capítulo 11**Cuestionario 26**

1 → A

3 → C

2 → Falso

4 → B

Cuestionario 27

1 → B

4 → Verdadero

7 → Verdadero

2 → Verdadero

5 → Falso

3 → A

6 → C

Cuestionario 28

1 → Falso

3 → Verdadero

5 → D

2 → C

4 → B

Cuestionario 29

1 → Git Plugin

4 → Falso

7 → C

2 → Verdadero

5 → A

8 → A (1,3);

3 → Verdadero

6 → B

B (2,6);

C (4,5)

18.11 Respuestas para el capítulo 12**Cuestionario 30**

1 → Falso

3 → C

5 → D

2 → Verdadero

4 → A

6 → Falso

Cuestionario 31

1 → A

4 → Falso

7 → B

2 → Verdadero

5 → C

8 → Falso

3 → Falso

6 → Verdadero

18.12 Respuestas para el capítulo 13**No hay cuestionarios asociados a este capítulo.****18.13 Respuestas para el capítulo 14****Cuestionario 32**

1 → A

6 → C

11 → D

2 → Falso

7 → D

12 → Falso

3 → B

8 → Verdadero

13 → C

4 → D
5 → C

9 → B
10 → A

14 → Verdadero
15 → Verdadero

Cuestionario 33

1 → B
2 → C
3 → A

4 → D
5 → Falso
6 → Verdadero

7 → Falso
8 → Verdadero

Cuestionario 34

1 → B
2 → A
3 → C

4 → D
5 → Falso
6 → C

7 → Falso
8 → A-2; B-1; C-4; D-3

Cuestionario 35

1 → B
2 → D
3 → Falso
4 → B

5 → C
6 → Verdadero
7 → A
8 → D

9 → Falso
10 → Falso

Cuestionario 36

1 → Falso
2 → B

3 → C
4 → C

5 → Falso

Cuestionario 37

1 → testResults
2 → A

3 → B
4 → Verdadero

5 → Verdadero
6 → `healthScaleFactor` y `currentBuild.result`

Cuestionario 38

1 → C
2 → B
3 → D
4 → C

5 → B
6 → A
7 → A
8 → Falso

9 → Verdadero
10 → Falso
11 → Verdadero

Cuestionario 39

1 → B
2 → A
3 → Falso
4 → Verdadero
5 → Falso

6 → B
7 → C
8 → B
9 → D
10 → Tener una colección de Postman, instalar el *plugin* NodeJS y configurar los paquetes en el *plugin*.

11 → No se configuró el paquete en el *plugin* de NodeJS/No hay una instalación de Newman en el *pipeline*.
12 → Configurar el *plugin* / `npm install -g Newman`.
13 → No cambia en uso, solo en la referencia realizada a la colección.

18.14 Respuestas para el capítulo 15

Cuestionario 40

1 → C
2 → B

3 → Verdadero
4 → Verdadero

5 → Falso

Cuestionario 41

1 → A
2 → D

3 → C
4 → Falso

5 → A-3; B-5; C-1; D-4; E-2

Cuestionario 42

1 → B
2 → A

3 → Falso
4 → Falso

5 → B

Cuestionario 43

1 → B
2 → A
3 → C
4 → B

5 → D
6 → C
7 → Falso
8 → Falso

9 → Falso
10 → Falso

Cuestionario 44

1 → Docker Plugin, Docker Pipeline Plugin y CloudBees Docker Build and Publish Plugin
2 → *Manage/Tools* y *Install Automatically*
3 → Registrar las credenciales de DockerHub.

4 → *script*
5 → *docker.image(...).inside{}* o *docker.image(...).withRun{}*
6 → Cualquiera de las 3 funciones de *docker.image* abordada en el capítulo asociado.

7 → Cambiar las funciones por instrucciones con el *step sh*
8 → A-2; B-4; C-6; D-1; E-5; F-3

18.15 Respuestas para el capítulo 16

Cuestionario 45

1 → A
2 → D

3 → Verdadero
4 → Verdadero

5 → Falso

Cuestionario 46

1 → Verdadero
2 → D
3 → B
4 → Verdadero

5 → C
6 → Falso
7 → D
8 → B

9 → Falso
10 → B
11 → Verdadero

Cuestionario 47

1 → B
2 → C
3 → D
4 → B

5 → A
6 → D
7 → *rm* solo elimina contenedores detenidos; *down* detiene y elimina.
8 → Falso

9 → Falso
10 → Verdadero
11 →
12 →

Cuestionario 48

1 → No existe *plugin* para ello.
2 → El usuario de Jenkins no tiene los permisos para usar Docker.
3 → Ejecutando *sudo usermod -aG docker jenkins*

4 → *./docker-compose.yml*
5 → C
6 → C

7 → Falso
8 → Falso

18.16 Respuestas para el capítulo 17

No hay cuestionarios asociados a este capítulo.



Capítulo 19

19 Bibliografía y recursos

Contenido del capítulo

19.1. Fuentes actualizadas.

19.1 Fuentes actualizadas

Abubakkar Sithik. (2025, febrero 9). *Running Postman Collections in Jenkins*. <https://dev.to/abusithik/running-postman-collections-in-jenkins-7k>

Amazon Web Services. (s/f). *What is Unit Testing?* Recuperado el 2 de noviembre de 2025, de <https://aws.amazon.com/what-is/unit-testing/>

Amr Essam. (2024, julio 23). *Jenkins Pipeline Clean Workspace After Each Build*. <https://www.baeldung.com/ops/jenkins-pipeline-clean-workspace>

Apidog. (s/f). *Newman vs Postman: Know the Key Differences for API Testing*. Recuperado el 30 de octubre de 2025, de <https://apidog.com/articles/what-is-the-difference-between-newman-and-postman/>

Arun, S. (2024, febrero 26). *GitHub Integration with Jenkins Pipeline by Poll SCM method*. <https://medium.com/@sangeetv09/github-integration-with-jenkins-pipeline-by-poll-scm-method-444cdeca0205>

Beach, B., Shetty, P., Simsek, M., Dange, S., & Akarslan, B. (2023). Practicing Continuous Integration and Continuous Delivery on AWS. En *Amazon Web Services*.

Bechtold, S., Brannen, S., Link, J., Merdes, M., Philipp, M., de Rancourt, J., & Stein, C. (2024). *JUnit User Guide*. <https://docs.junit.org/current/user-guide>

Carter, C. (2024, septiembre 10). *Guía del Sistema Operativo Ubuntu Server: Configuración y Beneficios*. <https://contabo.com/blog/es/ubuntu-server-instalacion-configuracion-y-beneficios/>

CD Foundation. (s/f-a). *CloudBees Docker Build and Publish*. Recuperado el 12 de enero de 2026, de <https://plugins.jenkins.io/docker-build-publish/>

CD Foundation. (s/f-b). *Docker*. Recuperado el 12 de enero de 2026, de <https://plugins.jenkins.io/docker-plugin/>

CD Foundation. (s/f-c). *Jenkins Plugins*. Recuperado el 28 de noviembre de 2025, de <https://plugins.jenkins.io/>

CD Foundation. (2024a, agosto). *User guide: Credentials plugin*. <https://wiki.jenkins-ci.org/JENKINS/attachments/59511751/133957121.pdf>

- CD Foundation. (2024b, agosto). *Workspace Cleanup*. <https://plugins.jenkins.io/ws-cleanup/>
- CD Foundation. (2025a). *Branches and Pull Requests*. <https://www.jenkins.io/doc/book/pipeline/multibranch/>
- CD Foundation. (2025b). *Credentials*. <https://www.jenkins.io/doc/book/security/credentials/>
- CD Foundation. (2025c). *Getting started with the Guided Tour*. <https://www.jenkins.io/doc/pipeline/tour/getting-started/>
- CD Foundation. (2025d). *Git | Jenkins plugin*. <https://plugins.jenkins.io/git/>
- CD Foundation. (2025e). *GitHub | Jenkins plugin*. <https://plugins.jenkins.io/github/>
- CD Foundation. (2025f). *Glossary*. <https://www.jenkins.io/doc/book/glossary/>
- CD Foundation. (2025g). *Hardware Recommendations*. <https://www.jenkins.io/doc/book/scaling/hardware-recommendations/>
- CD Foundation. (2025h). *JUnit | Jenkins plugin*. <https://plugins.jenkins.io/junit/>
- CD Foundation. (2025i). *JUnit Plugin*. <https://www.jenkins.io/doc/pipeline/steps/junit/>
- CD Foundation. (2025j). *Linux*. <https://www.jenkins.io/doc/book/installing/linux/>
- CD Foundation. (2025k). *Managing Nodes*. <https://www.jenkins.io/doc/book/managing/nodes/>
- CD Foundation. (2025l). *Managing Plugins*. <https://www.jenkins.io/doc/book/managing/plugins/>
- CD Foundation. (2025m). *Maven Integration*. <https://plugins.jenkins.io/maven-plugin/>
- CD Foundation. (2025n). *Pipeline*. <https://www.jenkins.io/doc/book/pipeline/>
- CD Foundation. (2025o). *Pipeline: Basic Steps*. <https://www.jenkins.io/doc/pipeline/steps/workflow-basic-steps/>
- CD Foundation. (2025p). *Pipeline Maven Integration*. <https://plugins.jenkins.io/pipeline-maven/>

CD Foundation. (2025q). *Pipeline: SCM Step*. <https://plugins.jenkins.io/workflow-scm-step/>

CD Foundation. (2025r). *Pipeline Syntax*. <https://www.jenkins.io/doc/book/pipeline/syntax/>

CD Foundation. (2025s). *¿Qué es Jenkins?* <https://www.jenkins.io/doc/>

CD Foundation. (2025t). *Using a Jenkinsfile*. <https://www.jenkins.io/doc/book/pipeline/jenkinsfile/>

CD Foundation. (2025u). *Using credentials*. <https://www.jenkins.io/doc/book/using/using-credentials/>

CD Foundation. (2025v). *Working with projects*. <https://www.jenkins.io/doc/book/using/working-with-projects/>

CD Foundation. (2025w, junio). *NodeJS*. <https://plugins.jenkins.io/nodejs/>

CD Foundation. (2026, enero 7). *SSH Pipeline Steps*.

CloudBees. (s/f-a). *Docker Pipeline plugin*. Recuperado el 10 de noviembre de 2025, de <https://docs.cloudbees.com/docs/cloudbees-ci/latest/pipelines/docker-workflow>

CloudBees. (s/f-b). *How to abort a Pipeline build if JUnit tests fail?* Recuperado el 28 de octubre de 2025, de <https://docs.cloudbees.com/docs/cloudbees-ci-kb/latest/client-and-managed-controllers/how-to-abort-a-pipeline-build-if-junit-tests-fail>

CloudBees. (s/f-c). *How to deal with large Git repositories in Jenkins pipelines*. Recuperado el 18 de octubre de 2025, de <https://docs.cloudbees.com/docs/cloudbees-ci-kb/latest/training/how-to-deal-with-large-repositories-in-jenkins>

CloudBees. (s/f-d). *Using PATH in Git Configuration*. Recuperado el 18 de octubre de 2025, de <https://docs.cloudbees.com/docs/cloudbees-ci-kb/latest/best-practices/using-path-in-git-configuration>

CloudBeesTV. (2020, diciembre 3). *How to Use Jenkins to Build and Run an Image Using Docker*.

- CloudBeesTV. (2021a, abril 21). *How to Push a Docker Image to Docker Hub Using Jenkins*. https://youtu.be/alQQ84M4CYU?si=xAelC6tnlRlvh_U1
- CloudBeesTV. (2021b, julio 1). *How to Configure Maven in Jenkins*. https://www.youtube.com/watch?v=V-VFrYF_Z1Y
- CloudBeesTV. (2022). *How to Use Docker Compose With Jenkins*. https://www.youtube.com/watch?v=ZPD_PzGOvFM
- CloudBeesTV. (2024a). *How to SSH From Jenkins to Remote Server*. <https://www.youtube.com/watch?v=TTgcYNBhzb4>
- CloudBeesTV. (2024b, febrero 6). *How To Install and Configure Tools on a Jenkins Agent*. <https://www.youtube.com/watch?v=L400xHcWcsw>
- CodeFresh. (s/f). *JUnit Jenkins: The Basics and a Quick Tutorial*. Recuperado el 28 de octubre de 2025, de <https://codefresh.io/learn/jenkins/junit-jenkins/>
- Cotton, B. (2023, agosto 14). *5 Benefits of a Container-First Approach to Software Development*. <https://www.docker.com/blog/5-benefits-of-a-container-first-approach-to-software-development/>
- DevOps Shack. (2022, diciembre 28). *Docker Build & Publish Using Jenkins!* <https://youtu.be/BePJ1bBWk3E?si=4aR-7qtdpiyxhTpT>
- Dileepkumar, S. R., & Mathew, J. (2023). Transforming Software Development: Achieving Rapid Delivery, Quality, and Efficiency with Jenkins-Based CI/CD Pipelines. *2023 Annual International Conference on Emerging Research Areas: International Conference on Intelligent Systems, AICERA/ICIS 2023*. <https://doi.org/10.1109/AICERA/ICIS59538.2023.10420251>
- Docker Inc. (s/f). *CLI cheat sheet [PDF]*. Recuperado el 9 de noviembre de 2025, de https://docs.docker.com/get-started/docker_cheatsheet.pdf
- Docker Inc. (2023, febrero 9). *docker container stop*. <https://docs.docker.com/reference/cli/docker/container/stop/>
- Docker Inc. (2024a, febrero 9). *docker compose down*. <https://docs.docker.com/reference/cli/docker/compose/down/>

Docker Inc. (2024b, febrero 9). *docker compose exec*.
<https://docs.docker.com/reference/cli/docker/compose/exec/>

Docker Inc. (2024c, febrero 9). *docker compose ls*.
<https://docs.docker.com/reference/cli/docker/compose/ls/>

Docker Inc. (2024d, febrero 9). *docker compose stop*.
<https://docs.docker.com/reference/cli/docker/compose/stop/>

Docker Inc. (2024e, febrero 9). *docker compose up*.
<https://docs.docker.com/reference/cli/docker/compose/up/>

Docker Inc. (2024f, febrero 9). *docker container rm*.
<https://docs.docker.com/reference/cli/docker/container/rm/>

Docker Inc. (2024g, febrero 9). *docker login*.
<https://docs.docker.com/reference/cli/docker/login/>

Docker Inc. (2024h, febrero 21). *docker image rm*.
<https://docs.docker.com/reference/cli/docker/image/rm/>

Docker Inc. (2024i, agosto 27). *Build, tag, and publish an image*.

Docker Inc. (2024j, septiembre 3). *Continuous integration with Docker*.
<https://docs.docker.com/build/ci/>

Docker Inc. (2024k, octubre 22). *Publishing and exposing ports*.
<https://docs.docker.com/get-started/docker-concepts/running-containers/publishing-ports/>

Docker Inc. (2025a, enero 9). *docker container exec*.
<https://docs.docker.com/reference/cli/docker/container/exec/>

Docker Inc. (2025b, enero 9). *docker container ls*.
<https://docs.docker.com/reference/cli/docker/container/ls/>

Docker Inc. (2025c, enero 9). *docker image ls*.
<https://docs.docker.com/reference/cli/docker/image/ls/>

Docker Inc. (2025d, enero 9). *docker image pull*.
<https://docs.docker.com/reference/cli/docker/image/pull/>

- Docker Inc. (2025e, enero 9). *docker image push*.
<https://docs.docker.com/reference/cli/docker/image/push/>
- Docker Inc. (2025f, julio 1). *Why use Compose*.
<https://docs.docker.com/compose/intro/features-uses/>
- Docker Inc. (2025g, julio 9). *Version and name top-level elements*.
<https://docs.docker.com/reference/compose-file/version-and-name/>
- Docker Inc. (2025h, noviembre 3). *Dockerfile overview*.
<https://docs.docker.com/build/concepts/dockerfile/>
- Docker Inc. (2025i, diciembre 17). *What is Docker*. <https://docs.docker.com/get-started/docker-overview/>
- Docker Inc. (2026a, enero 6). *How Compose works*.
<https://docs.docker.com/compose/intro/compose-application-model/>
- Docker Inc. (2026b, enero 7). *Define services in Docker Compose*.
<https://docs.docker.com/reference/compose-file/services/>
- Docker Inc. (2026c, enero 7). *Docker Compose*. <https://docs.docker.com/compose/>
- DockerInc. (2024, febrero 9). *docker compose rm*.
<https://docs.docker.com/reference/cli/docker/compose/rm/>
- Elizabet J. (2025, agosto 10). *Create Local User with PowerShell*.
<https://monovm.com/blog/create-local-user-with-powershell/>
- Ferguson, J., & Hudson, S. (2011). *The Definitive Guide*. O'Reilly Media.
- GeeksforGeeks. (2024, abril 23). *Wget Command in Linux/Unix*.
<https://www.geeksforgeeks.org/linux-unix/wget-command-in-linux-unix/>
- GeeksForGeeks. (2025a, junio 23). *How to Use Docker Compose With Jenkins*.
<https://www.geeksforgeeks.org/devops/how-to-use-docker-compose-with-jenkins/>
- GeeksforGeeks. (2025a, junio 23). *Test Tools and Automation- Strategies , Costs, Risks and Benefits*. <https://www.geeksforgeeks.org/software-engineering/test-tools-and-automation-strategies-costs-risks-and-benefits/>

- GeeksforGeeks. (2025b, julio 11). *apt-get command in Linux with Examples*. <https://www.geeksforgeeks.org/linux-unix/apt-get-command-in-linux-with-examples/>
- GeeksForGeeks. (2025b, julio 23). *CI/CD Pipelines with Docker Compose: Building Efficient Workflows*. <https://www.geeksforgeeks.org/devops/ci-cd-pipelines-with-docker-compose/>
- GeeksforGeeks. (2025c, julio 23). *How to Start, Stop and Restart Services in Linux Using systemctl Command*. <https://www.geeksforgeeks.org/linux-unix/start-stop-restart-services-using-systemctl-in-linux/>
- GeeksforGeeks. (2025d, agosto 13). *Freestyle Projects In Jenkins*. <https://www.geeksforgeeks.org/devops/creating-and-configuring-freestyle-projects-in-jenkins/>
- Harish Rajora. (2021, noviembre 6). *Publish Jenkins HTML Reports for Newman*. <https://toolsqa.com/postman/publish-jenkins-html-reports-for-newman/>
- Horatiu Vlascel. (2022). *What is software packaging and why is it important?* <https://www.advancedinstaller.com/what-is-software-packaging.html>
- Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation* (Addison-Wesley Professional, Ed.; 1st ed.). Addison-Wesley Professional.
- IEEE. (2021). IEEE Standard for DevOps: Building Reliable and Secure Systems Including Application Build, Package, and Deployment. *ISO/IEC/IEEE Std 32675:2022*, 1–91. <https://doi.org/10.1109/IEEESTD.2021.9415476>
- IEEE Computer Society. (2024). *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0* (W. Hironori, Ed.). IEEE Computer Society. <https://www.computer.org/education/bodies-of-knowledge/software-engineering/v4>
- ISO/IEC/IEEE. (2013). 29119-1:2013 - ISO/IEC/IEEE International Standard for Software and systems engineering — Software testing — Part 1: Concepts and definitions. *ISO/IEC/IEEE 29119-1:2013(E)*, 2013.

- ISO/IEC/IEEE. (2021). ISO/IEC/IEEE International Standard - Software and systems engineering - Software testing – Part 2: Test processes. *ISO/IEC/IEEE 29119-2:2021(E)*, 1–64. <https://doi.org/10.1109/IEEESTD.2021.9591508>
- ISO/IEC/IEEE. (2022). ISO/IEC/IEEE International Standard - Software and systems engineering –Software testing –Part 1:General concepts. *ISO/IEC/IEEE 29119-1:2022(E)*, 1–60. <https://doi.org/10.1109/IEEESTD.2022.9698145>
- KodeKloud. (s/f). *Jenkins Installation Options*. Recuperado el 8 de septiembre de 2025, de <https://notes.kodekloud.com/docs/Jenkins-For-Beginners/Jenkins-Setup-and-Interface/Jenkins-Installation-Options>
- Laster, B. (2018). *Jenkins 2: Up and Running: Evolve Your Deployment Pipeline for Next Generation Automation*.
- Microsoft. (2025, septiembre 4). *Get started with OpenSSH for Windows*. https://learn.microsoft.com/en-us/windows-server/administration/openssh/openssh_install_firstuse?tabs=powershell&pivot=s=windows-11
- Mikhalev, V. (2025, enero 16). *Mastering Docker and Jenkins: Build Robust CI/CD Pipelines Efficiently*. <https://www.docker.com/blog/docker-and-jenkins-build-robust-ci-cd-pipelines/>
- Nandita Sahu. (2022, septiembre 28). *Jenkins Pipeline con Maven, SonarQube y Talisman*. <https://medium.com/@nanditasahu031/jenkins-pipeline-with-maven-sonarqube-and-talisman-fa9118910b98>
- Oscar Zhou. (2025, julio 30). *[Docker/Registry] Troubleshooting Image Pull Failures in Self-Built Container Management Tool that Uses Docker Compose SDK*. <https://oscarchou.com/posts/troubleshoot/deep-dive-docker-pull-flow/>
- Postman. (s/f-a). *Install and run Newman*. Recuperado el 30 de octubre de 2025, de <https://learning.postman.com/docs/collections/using-newman-cli/installing-running-newman/>
- Postman. (s/f-b). *Integrate your Postman tests with Jenkins using Newman*. Recuperado el 7 de noviembre de 2025, de <https://learning.postman.com/docs/collections/using-newman-cli/integration-with-jenkins/>

- Postman. (s/f-c). *Newman command reference*. Recuperado el 6 de noviembre de 2025, de <https://learning.postman.com/docs/collections/using-newman-cli/newman-options/>
- Postman. (s/f-d). *Tools*. Recuperado el 30 de octubre de 2025, de <https://www.postman.com/product/tools/>
- Postman. (s/f-e). *What is Postman?* Recuperado el 30 de octubre de 2025, de <https://www.postman.com/product/what-is-postman/>
- Prashanth Punnam. (2025, mayo 10). *CI/CD Pipeline with Jenkins: Build, Test & Deploy*. <https://www.accelq.com/blog/ci-cd-pipeline-with-jenkins/>
- Red Hat. (2025, mayo 10). *What is CI/CD?* <https://www.redhat.com/en/topics/devops/what-is-ci-cd#how-red-hat-can-help>.
- Sistla, S. (2021). The Evolution of Continuous Integration – Continuous Delivery . *International Journal of Innovative Research in Engineering & Multidisciplinary Physical Sciences* , 9(4). <https://doi.org/10.5281/zenodo.14261017>
- Stefan Bechtold, Christian Stein, Juliette de Rancourt, Marc Philipp, Matthias Merdes, Sam Brannen, & Johannes Link. (s/f). *JUnit User Guide*. Recuperado el 27 de octubre de 2025, de <https://docs.junit.org/current/user-guide/#running-tests-console-launcher>
- The Apache Software Foundation. (2025a, septiembre 27). *Apache Maven Project*. <https://maven.apache.org/>
- The Apache Software Foundation. (2025b, septiembre 27). *Maven Getting Started Guide*. <https://maven.apache.org/guides/introduction/introduction-to-the-pom.html>
- Tomas Fernandez. (2022). *Design an Effective Build Stage for Continuous Integration*. <https://semaphore.io/blog/build-stage>
- Vladimir Mikhalev. (2025, enero 16). *Mastering Docker and Jenkins: Build Robust CI/CD Pipelines Efficiently*. <https://www.docker.com/blog/docker-and-jenkins-build-robust-ci-cd-pipelines/>